



Compilation of real-time audio DSP on FPGA

Maxime Popoff

► To cite this version:

Maxime Popoff. Compilation of real-time audio DSP on FPGA. Embedded Systems. INSA de Lyon, 2024. English. NNT : 2024ISAL0128 . tel-05033875

HAL Id: tel-05033875

<https://theses.hal.science/tel-05033875v1>

Submitted on 14 Apr 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



N° d'ordre NNT : 2024ISAL0128

THÈSE DE DOCTORAT DE L'INSA LYON
Membre de l'Université de Lyon

École Doctorale 512
Infomaths

Spécialité / Discipline de doctorat :
Informatique

Soutenue publiquement le 13 Décembre 2024, par :
Maxime POPOFF

Compilation of Real-Time Audio DSP on FPGA

Devant le jury composé de :

CHOTIN	Roselyne	Professeure	Sorbonne Université	Rapporteuse
MCPHERSON	Andrew	Professeur	Imperial College London	Rapporteur
SOLNON	Christine	Professeure	INSA Lyon	Examinatrice
HÉLIE	Thomas	Directeur de Recherche	IRCAM, Paris	Examineur
SENTIEYS	Olivier	Professeur	Université de Renne	Président
RISSET	Tanguy	Professeur	INSA Lyon	Directeur de thèse
MICHON	Romain	Chargé de Recherche	Inria, Lyon	Co-encadrant

Référence : TH1158_POPOFF Maxime

L'INSA Lyon a mis en place une procédure de contrôle systématique via un outil de détection de similitudes (logiciel Compilatio). Après le dépôt du manuscrit de thèse, celui-ci est analysé par l'outil. Pour tout taux de similarité supérieur à 10%, le manuscrit est vérifié par l'équipe de FEDORA. Il s'agit notamment d'exclure les auto-citations, à condition qu'elles soient correctement référencées avec citation expresse dans le manuscrit.

Par ce document, il est attesté que ce manuscrit, dans la forme communiquée par la personne doctorante à l'INSA Lyon, satisfait aux exigences de l'Etablissement concernant le taux maximal de similitude admissible.

INSA LYON

Campus LyonTech La Doua

20, avenue Albert Einstein - 69621 Villeurbanne cedex - France

Tél. +33 [0]4 72 43 83 83 - Fax +33 [0]4 72 43 85 00

www.insa-lyon.fr



Département FEDORA – INSA Lyon - Ecoles Doctorales

SIGLE	ECOLE DOCTORALE	NOM ET COORDONNEES DU RESPONSABLE
ED 206 CHIMIE	CHIMIE DE LYON https://www.edchimie-lyon.fr Sec. : Renée EL MELHEM Bât. Blaise PASCAL, 3e étage secretariat@edchimie-lyon.fr	M. Stéphane DANIELE C2P2-CPE LYON-UMR 5265 Bâtiment F308, BP 2077 43 Boulevard du 11 novembre 1918 69616 Villeurbanne directeur@edchimie-lyon.fr
ED 341 E2M2	ÉVOLUTION, ÉCOSYSTÈME, MICROBIOLOGIE, MODÉLISATION http://e2m2.universite-lyon.fr Sec. : Bénédicte LANZA Bât. Atrium, UCB Lyon 1 Tél : 04.72.44.83.62 secretariat.e2m2@univ-lyon1.fr	Mme Sandrine CHARLES Université Claude Bernard Lyon 1 UFR Biosciences Bâtiment Mendel 43, boulevard du 11 Novembre 1918 69622 Villeurbanne CEDEX e2m2.codir@listes.univ-lyon1.fr
ED 205 EDISS	INTERDISCIPLINAIRE SCIENCES-SANTÉ http://ediss.universite-lyon.fr Sec. : Bénédicte LANZA Bât. Atrium, UCB Lyon 1 Tél : 04.72.44.83.62 secretariat.ediss@univ-lyon1.fr	Mme Sylvie RICARD-BLUM Laboratoire ICBMS - UMR 5246 CNRS - Université Lyon 1 Bâtiment Raulin - 2ème étage Nord 43 Boulevard du 11 novembre 1918 69622 Villeurbanne Cedex Tél : +33(0)4 72 44 82 32 sylvie.ricard-blum@univ-lyon1.fr
ED 34 EDML	MATÉRIAUX DE LYON http://ed34.universite-lyon.fr Sec. : Yann DE ORDENANA Tél : 04.72.18.62.44 yann.de-ordenana@ec-lyon.fr	M. Stéphane BENAYOUN Ecole Centrale de Lyon Laboratoire LTDS 36 avenue Guy de Collongue 69134 Ecully CEDEX Tél : 04.72.18.64.37 stephane.benayoun@ec-lyon.fr
ED 160 EEA	ÉLECTRONIQUE, ÉLECTROTECHNIQUE, AUTOMATIQUE https://edeea.universite-lyon.fr Sec. : Philomène TRE COURT Bâtiment Direction INSA Lyon Tél : 04.72.43.71.70 secretariat.edeea@insa-lyon.fr	M. Philippe DELACHARTRE INSA LYON Laboratoire CREATIS Bâtiment Blaise Pascal, 7 avenue Jean Capelle 69621 Villeurbanne CEDEX Tél : 04.72.43.88.63 philippe.delachartre@insa-lyon.fr
ED 512 INFOMATHS	INFORMATIQUE ET MATHÉMATIQUES http://edinfomaths.universite-lyon.fr Sec. : Renée EL MELHEM Bât. Blaise PASCAL, 3e étage Tél : 04.72.43.80.46 infomaths@univ-lyon1.fr	M. Hamamache KHEDDOUCI Université Claude Bernard Lyon 1 Bât. Nautibus 43, Boulevard du 11 novembre 1918 69 622 Villeurbanne Cedex France Tél : 04.72.44.83.69 direction.infomaths@listes.univ-lyon1.fr
ED 162 MEGA	MÉCANIQUE, ÉNERGÉTIQUE, GÉNIE CIVIL, ACOUSTIQUE http://edmega.universite-lyon.fr Sec. : Philomène TRE COURT Tél : 04.72.43.71.70 Bâtiment Direction INSA Lyon mega@insa-lyon.fr	M. Etienne PARIZET INSA Lyon Laboratoire LVA Bâtiment St. Exupéry 25 bis av. Jean Capelle 69621 Villeurbanne CEDEX etienne.parizet@insa-lyon.fr
ED 483 ScSo	ScSo¹ https://edsciencesociales.universite-lyon.fr Sec. : Mélina FAVETON Tél : 04.78.69.77.79 melina.faveton@univ-lyon2.fr	M. Bruno MILLY (INSA : J.Y. TOUSSAINT) Univ. Lyon 2 Campus Berges du Rhône 18, quai Claude Bernard 69365 LYON CEDEX 07 Bureau BEL 319 bruno.milly@univ-lyon2.fr

¹ScSo : Histoire, Géographie, Aménagement, Urbanisme, Archéologie, Science politique, Sociologie, Anthropologie

Remerciements

Le travail présenté dans cette thèse doit beaucoup au soutien et aux contributions de nombreuses personnes, dont l'accompagnement a joué un rôle essentiel dans sa réussite au cours de ces trois années.

Tout d'abord, je tiens à remercier le laboratoire CITI, l'INSA de Lyon et l'Inria de m'avoir accueilli, de m'avoir permis de réaliser cette thèse et de m'avoir donné les moyens de mener mes recherches dans de bonnes conditions, en m'offrant un environnement de travail adapté.

Je remercie mes collègues du CITI, et en particulier l'équipe EMERAUDE, qui, en plus de m'apporter de nombreuses connaissances, m'a véritablement permis de m'épanouir pendant ces trois années, grâce à une ambiance bienveillante dans laquelle je me suis rapidement senti à ma place. Un grand merci également à Pierre pour son travail, dont la contribution a été déterminante dans le développement de cette thèse.

Une reconnaissance particulière va à mes directeurs de thèse, Tanguy et Romain, pour avoir cru en mon potentiel et m'avoir fait bénéficier de leur expertise. Leur encadrement a été précieux, ponctué de sujets passionnants à explorer, tout en me laissant la liberté essentielle pour développer mes propres recherches.

Les commentaires constructifs de mes rapporteurs m'ont permis d'améliorer ce manuscrit, tout comme les questions pertinentes des membres du jury lors de la soutenance. Mes sincères remerciements leur sont adressés pour leur apport.

Je remercie particulièrement mes collègues de thèse, Alix, Aurélien, David, Guillaume, Léopold et Tommy, avec qui j'ai partagé ces trois années d'épreuves. Ils ont su me motiver dans mon travail et, parfois, m'ont convaincu de décrocher un peu plus tôt. Un remerciement spécial à Alix et son vélo, grâce auxquels j'ai pu préserver ma santé physique et mentale !

Je tiens à exprimer ma gratitude envers ma famille, qui m'a permis de faire des études et m'a soutenu dans mes choix, aussi bien quand j'ai décidé de poursuivre de longues études qu'au moment de reprendre une thèse après avoir travaillé.

Un grand merci à mes amis, présents malgré la distance, sans vous, tout cela n'aurait pas eu autant de sens. Alexia, Baptiste, Célestin, Gauthier, Jad, Marion, Martin, Morand, Moriceau, Raph, Rosinski, Seb : merci d'avoir été là depuis mes études et de continuer à l'être.

Enfin, cette thèse n'aurait pas été possible sans le soutien sans faille de Mégane. Merci de m'avoir soutenu dans les moments de doute et de découragement que ce travail a pu susciter, merci de m'avoir aidé à surmonter les défis rencontrés, merci de m'avoir encouragé sans relâche jusqu'au bout, et surtout, merci de me supporter chaque jour.

Maxime

Abstract

Field-Programmable Gate Arrays (FPGAs) are powerful embedded platforms that offer unique performance in terms of throughput, computational power, and a large number of inputs/outputs, making them highly suited to real-time audio processing. However, programming them presents significant challenges. High-Level Synthesis (HLS) bypasses complex hardware description languages (HDLs) traditionally used for FPGA programming, streamlining the compilation process and enabling high-level audio DSP specifications to be directly converted into FPGA bitstreams. Still, this approach remains difficult to generalize for various audio programs. This thesis presents the principles of real-time audio DSP implementation on FPGAs using HLS tools. It introduces a general methodology for generating optimized C++ code for audio DSP with HLS and presents the first “audio DSP compiler” specifically targeting FPGAs. This compilation flow leverages the use of FAUST, a high-level functional programming language designed for sound synthesis and audio processing, to program FPGAs. Two major technological breakthroughs are presented: achieving ultra-low latency for real-time audio DSP (on the microsecond scale) and the ability to easily deploy massive multichannel systems, opening new research perspectives, particularly in the fields of audio spatialization and active acoustic control applications.

Résumé

Les FPGA (Field-Programmable Gate Arrays) sont des plateformes programmables offrant des performances uniques en termes de débit, de nombre d'entrées/sorties et de puissance de calcul, les rendant particulièrement intéressantes pour des applications de traitement du signal audio numérique en temps réel. Cependant, leur programmation constitue un défi majeur. La synthèse de haut niveau (HLS) permet de s'affranchir des langages de description matérielle complexes traditionnellement utilisés pour la programmation des FPGA. Cela ouvre la voie à une véritable chaîne de compilation allant des spécifications DSP audio de haut niveau jusqu'au design d'architectures matérielles. Toutefois, cette approche reste difficile à généraliser pour des programmes audio variés. Cette thèse étudie la compilation de DSP audio en temps réel pour FPGA. Elle propose une méthodologie générale pour générer du code C++ optimisé pour la synthèse de haut niveau et introduit le premier « compilateur DSP audio » dédié aux FPGA. Cette chaîne de compilation permet d'utiliser FAUST, un langage de programmation fonctionnel de haut niveau, spécifiquement conçu pour la synthèse sonore et le traitement audio, afin de programmer les FPGA. Deux avancées technologiques majeures sont présentées : la réalisation d'une latence ultra-faible pour les DSP audio en temps réel (à l'échelle de la microseconde) et la capacité à déployer facilement et efficacement des systèmes multicanaux de grande capacité, ouvrant ainsi de nouvelles perspectives de recherche, notamment dans les domaines de la spatialisation audio et des applications de contrôle actif de l'acoustique.

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	General Structure of the Thesis	2
1.3	Contributions	3
2	Real-Time Audio DSP: Fundamental Concepts	5
2.1	Basics of Digital Signal Processing	6
2.1.1	DSP Principles	6
2.1.2	Dataflow Programming Languages	7
2.2	Converting Between Worlds: Digital and Analog Interfacing	7
2.2.1	Analog-to-Digital Conversion	8
2.2.2	Digital-to-Analog Conversion	8
2.3	Fundamentals of Audio Resolution	9
2.3.1	Quantization	9
2.3.2	Sampling Frequency	10
2.4	Real-Time Audio Constraints	11
2.4.1	Buffering	11
2.4.2	Latency	12
2.4.3	Throughput	12
2.4.4	Jitter	13
2.5	The Range of Latency: From Real-Time to Ultra-Low Latency	15
2.5.1	Real-Time Processing	15
2.5.2	Low Latency	15
2.5.3	Ultra-Low Latency	16
2.6	Control Mechanisms	17
2.6.1	Control Rate	17
2.6.2	Controllers	18
2.7	Memory Management	19
2.7.1	Compilation Time, Initialization Time	19
2.7.2	Memory Size and Accesses	19
2.8	Audio DSL: The FAUST Example	19
2.8.1	The FAUST Language	19
2.9	Conclusion	21
3	State of The Art	23
3.1	Audio Domain Specific Languages	24
3.1.1	Classical Programming Languages for Audio	24
3.1.2	Audio Synthesis and Composition Languages	24
3.1.3	Graphical Programming Languages	25
3.1.4	Transcompiler Languages	25
3.2	Platform for Real-Time Audio Processing	26

3.2.1	CPU and Digital Signal Processor Chip	26
3.2.2	Embedded Systems	27
3.2.3	GPU	28
3.3	Audio Processing on FPGA	29
3.3.1	Audio Hardware Design	29
3.3.2	FPGA in Audio Industry	30
3.3.3	FPGA Programming and Domain-Specific Compilation	31
3.4	Real-Time Audio Processing Challenges	32
3.4.1	Audio Latency	32
3.4.2	Multichannel	34
3.5	Synthesis and Insights	35
4	Hardware Design of Audio DSP	37
4.1	Basics of Hardware Design	38
4.1.1	SoC and FPGA	38
4.1.2	IP and Block-Based Architecture	40
4.1.3	Hardware Description Language	40
4.1.4	High Level Synthesis	41
4.1.5	Audio Compilation Flow	42
4.2	Implementation Guidelines	44
4.2.1	FPGA-Centric Design	44
4.2.2	Hardware/Software Partitioning	47
4.2.3	One-Sample and Multi-Sample Strategies	49
4.2.4	Memory Accesses	50
4.3	Interfacing with the Analog Domain: Audio Codecs	53
4.3.1	Audio Serial Interface	53
4.3.2	GPIO Usage of I ² S Protocols	56
4.3.3	Audio Codecs Architecture	57
4.3.4	Ultra-Low Latency Audio Codecs	60
4.4	Hardware and Software Audio Host Control	62
4.4.1	Simple Control with Baremetal	62
4.4.2	Embedded <i>Linux</i> and Interfacing With External World	66
4.5	FPGA Audio Compilation Scheme	66
4.5.1	From High Level DSP to Hardware and Software Components	67
4.5.2	Hardware Integration and User Interface	68
4.5.3	Our Audio DSP Compiler: Syfala	69
4.5.4	Additional Compiler Features	71
4.6	Conclusion	73
5	Optimizing Audio on FPGA with C++ High Level Synthesis	75
5.1	Introduction and Principles	77
5.1.1	Resources and Latency: The Key Metrics in FPGA Design	77
5.1.2	AMD/Xilinx HLS Tools Principle	79
5.2	Optimizing Hardware Design With Sequential Language	85
5.2.1	Guiding the Implementation with Directives and Pragmas	85
5.2.2	Unrolling loops: The Sequential/Parallel Trade-Off	86
5.2.3	Pipelining Loops to Minimize Latency and Maximize Throughput	88
5.3	Introduction to Pragmas	89
5.3.1	Trivial Program Example	89
5.3.2	Pragma Unroll: Sequential/Parallel Control	90
5.3.3	Interpreting Latency Reports	91
5.3.4	Pragma Array_partition and Bind_storage: Control Over Memory Layout	93

5.3.5	Pragma Pipeline: Optimizing the Design Throughput	95
5.3.6	Pragma Interface: Configuring the IP Interface	96
5.4	Methodology for Hardware Designing Audio DSP in C++	100
5.4.1	Default Implementation	100
5.4.2	Optimizing Pipeline	102
5.4.3	Nested Loops: A Key to Major Optimizations	104
5.4.4	Unrolling: Enhancing Parallelism When Pipelining is Insufficient	109
5.4.5	Multi-Sample Design Guidelines: Exploiting FPGA Parallelism	111
5.4.6	Multi-Sample Case Study	117
5.5	Interfacing DSP IP With CPU and Memory	121
5.5.1	IP Interface	122
5.5.2	Memory Access Optimization	124
5.6	Final Evaluation and Insights on HLS Optimization for Audio DSPs	126
5.6.1	Further Recommended Guidelines	127
5.6.2	Concluding Thoughts on Automating HLS for Audio Applications	130
6	Real-World Applications and Outcomes	131
6.1	Compiling from a DSL: FAUST to FPGA	133
6.1.1	FAUST Audio Program	133
6.1.2	FAUST Hardware/Software Dispatching	134
6.1.3	FAUST Compiler Modifications	134
6.1.4	Compiling Process Overview	135
6.1.5	Hardware and Software Integration	137
6.1.6	Control Binding	138
6.1.7	Experimental Results and Contributions	139
6.2	Ultra Low Latency Audio Path	140
6.2.1	I ² S Transceiver	141
6.2.2	Audio Codecs Configuration	142
6.2.3	External Audio Codec Latency Measurements	143
6.2.4	Troubleshooting SSM2603: Audio Distortion Due to Faulty Initialization	146
6.3	Multichannel Interfaces	147
6.3.1	Frugal Board	148
6.3.2	Ultra Low Latency Board	150
6.3.3	Syfala Toolchain Compatibility	154
6.4	Experimental Results	154
6.4.1	FIR Filter	155
6.4.2	Frugal WFS System	158
6.4.3	Active Noise Control: LMS and FxLMS	159
6.4.4	Modal Synthesis	162
6.4.5	Comparative Insights: Evaluating Our Approach Against Existing Works	163
7	Conclusions and Perspectives	165
	Appendix	168
A	Impact of Unsafe Math Optimization	169
B	One-Sample Case Study	173
C	LMS and FxLMS Case Study	179
D	Experimental Results	187

E	Figures	191
F	Programs	195
G	Syfala Command-Line Interface Reference	203

Publications

1. Maxime Popoff, Romain Michon, Tanguy Risset, Yann Orlarey and Stéphane Letz. Towards an FPGA-Based Compilation Flow for Ultra-Low Latency Audio Signal Processing. In Sound and Music Computing (SMC-22), Jun. 2022.
2. Maxime Popoff, Romain Michon, Tanguy Risset, Pierre Cochard, Yann Orlarey, Stéphane Letz and Florent de Dinechin. Audio DSP to FPGA Compilation. In IEEE International Conference on Application-specific Systems, Architectures and Processors (ASAP-23). Jul. 2023.
3. Maxime Popoff, Romain Michon, Tanguy Risset, Pierre Cochard, Stéphane Letz, Yann Orlarey and Florent de Dinechin. Audio DSP to FPGA Compilation : The Syfala Toolchain Approach. Research Report RR-9507. Univ Lyon, INSA Lyon, Inria, CITI, Grame, Emeraude, May 2023.
4. Maxime Popoff, Romain Michon and Tanguy Risset. Enabling Affordable and Scalable Audio Spatialization With Multichannel Audio Expansion Boards for FPGA. In Sound and Music Computing (SMC-24), Jul. 2024.
5. Loïc Alexandre, Pierre Lecomte, Marie-Annick Galland and Maxime Popoff. Feedback Acoustic Noise Control with Faust on FPGA : Application to Noise Reduction in Headphones. In International Faust Conference (IFC-22). Jun. 2022.
6. Pierre Cochard, Maxime Popoff, Romain Michon and Tanguy Risset. Programming FPGA Platforms for Real-Time Audio Signal Processing in C++. In Sound and Music Computing (SMC-24). Jul. 2024.
7. Romain Michon, Joseph Bizien, Maxime Popoff and Tanguy Risset. Making Frugal Spatial Audio Systems Using Field-Programmable Gate Array. In New Interfaces for Musical Expression (NIME-23). Jun. 2023.
8. Pierre Cochard, Maxime Popoff, Antoine Fraboulet, Tanguy Risset, Stéphane Letz and Romain Michon. A Programmable *Linux*-Based FPGA Platform for Audio DSP. In Sound and Music Computing (SMC-23). Jun. 2023.

1

CHAPTER 1

Introduction

1.1 Motivation and Context

In recent years, the demand for high-performance audio processing systems has grown significantly, driven by advancements in areas such as multimedia applications, real-time digital audio effects, and immersive sound environments. As audio technologies continue to evolve, so does the need for efficient and adaptable processing solutions capable of handling complex tasks with high throughput and minimal computational delay.

Audio Digital Signal Processing (DSP) is employed across a diverse range of devices, each with specific computational requirements. While some applications, such as audio and vibratory systems, require high throughput, others (like dynamic acoustic control, artificial reverberation, and noise cancellation) demand ultra-low latency (under 1 ms) to ensure optimal performance. Techniques such as spatial audio rendering and virtual soundscapes, including Ambisonics and Wave Field Synthesis (WFS), could involve thousands of speakers to create a truly immersive auditory experience. Similarly, active noise cancellation systems may need to process thousands of audio channels simultaneously for effective results. A notable example of such a high-demand application is the Sphere concert venue in Las Vegas, which incorporates almost 1 600 speaker modules, each containing 96 speakers, totaling an astounding 152 000 individual speakers. Each speaker processes its own audio stream using advanced spatial audio technologies, including 3D audio beamforming and WFS, aiming to enhance the realism and depth of the immersive sound experience.

At the same time, embedded systems are being increasingly used in artistic and cultural contexts, ranging from interactive installations and digital musical instruments to virtual and augmented reality, enabling new forms of creative expression. However, programming these systems poses a significant challenge for many artists, creators, and non-specialist engineers. Despite recent advancements in accessibility, a gap remains between the complexity of these tools and the flexibility they offer for creative digital audio work.

Historically, real-time audio systems have relied on CPUs or GPUs, but achieving sub-millisecond latency or managing massive multi-channel interfaces with these architectures is challenging. This has driven the exploration of specialized architectures such as Field-Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs). FPGAs present a promising solution to many of the challenges faced in real-time audio processing. Renowned for their high computational capabilities and extremely low latency, they provide a way to surpass the limitations of traditional computing platforms, particularly in real-time multi-channel audio processing. In fact, some industrial audio systems, including those deployed in the Sphere venue, have already begun

leveraging FPGA technology. However, until recently, developing FPGA-based systems required deep expertise in microelectronics, rendering them inaccessible to many audio DSP engineers and programmers.

Typically, DSP algorithms are developed at a high level (in languages like *MATLAB*, *Python*, or *C/C++*), but implementing them on FPGAs involves translating these algorithms into Hardware Description Language (HDL) such as *VHDL* or *Verilog*, a process that is both challenging and time-consuming. New hardware design methodologies, such as HLS, have progressively reduced the time and expertise required for FPGA design. These methodologies often rely on either block-based design or software-based languages. Block-based design assembles pre-existing hardware blocks whereas software-based HLS offers more direct control over the design, allowing algorithms to be written in sequential programming languages (typically *C/C++*) and mapped directly to hardware. Designers can use pragmas or directives to guide the HLS tools during the parallelization process.

Despite these advancements, programming FPGAs remains a complex task. Currently, there is no readily available solution that allows to program them using high-level audio DSP specifications in a straightforward manner, without requiring specialized hardware design knowledge. This thesis focuses on enabling FPGA programming using high-level programming approaches, specifically leveraging HLS for audio applications. Audio programs have unique characteristics, and many optimization decisions can be guided by the specific nature of the audio processing tasks. We demonstrate that using a Domain-Specific Language (DSL) tailored for audio applications can reduce the exploration space and assist in the automation of hardware compilation.

This work presents two major contributions to the field of audio DSP compilation on FPGA. First, by thoroughly analyzing and classifying audio program specifications, it aims to provide a general methodology to guide the FPGA implementation of any audio DSP without requiring deep knowledge of hardware design optimizations or FPGA programming. This is further enhanced by offering a general approach for writing efficient C++ code to achieve optimized HLS synthesis. Second, it synthesizes this methodology into the first audio DSP-to-FPGA compiler, allowing real-time audio DSPs to be developed from high-level descriptions written in either C++ or the FAUST language, an audio DSL designed for audio processing and sound synthesis.

Leveraging this tool, this thesis demonstrates a system capable of achieving an ultra-low latency of 11 μ s from analog input to analog output, using low-latency codecs. It also introduces multichannel physical interfaces, supporting up to 512 channels connected to a single FPGA. By scripting and automating not only the FPGA implementation of audio DSPs but also the entire system, including hardware/software co-design with CPU control (potentially running a *Linux* operating system), audio interfaces, and user interfaces, the tool enables straightforward compilation of audio DSPs on an System on Chip (SoC) with simple command lines.

1.2 General Structure of the Thesis

Chapter 2 presents the core concepts and principles of real-time audio processing, beginning with the fundamentals of digital audio signal representation and the principles of Analog-to-Digital Converter (ADC) and Digital-to-Analog Converter (DAC) conversion. The discussion covers essential notions such as latency and throughput, which are critical in real-time audio systems. and analyzes the specific latency requirements for interactive audio systems.

Chapter 3 reviews the state of the art in current research on real-time audio processing, providing an overview of existing methods, technologies and available platforms. It includes a survey of audio DSLs, analyzing their role in facilitating audio processing. It also explores the current state of FPGAs usage in both audio research and in the industry, the unique possibilities FPGAs offer

for audio applications, as well as the significant challenges they pose, including their complexity in programming and integration.

Chapter 4 proposes guidelines for effectively designing audio hardware by analyzing fundamental DSP concepts and applying them to SoC FPGA hardware-software co-design. It begins by detailing the basics of FPGA design and offers algorithmic-level implementation guidelines. The chapter then focuses on optimizing the audio analog interface for low-latency applications. It concludes by presenting two control system implementations (bare-metal and OS-based) for managing hardware audio with software, and integrates these elements into an audio compilation scheme, introducing our compilation flow: *Syfalala* [Emeb].

Chapter 5 delves into HLS optimization, applying the previously detailed implementation guidelines to HLS. It proposes a methodology for writing optimized C++ code for real-time audio applications. The chapter demonstrates the remarkable performance improvements achieved, with resource usage and latency reductions by more than a factor of 10 in some cases. It provides an in-depth explanation of the pragma directives essential for optimizing and offers guidance on generating proper hardware interfaces to further streamline audio processing systems.

Chapter 6 provides application examples of this thesis work. It first details the *Syfalala* toolchain with a practical example in FAUST, analyzing each step of the compilation process and the contributions we made to optimizing the FAUST compiler for FPGA implementation. It then presents the two PCB boards developed for this thesis, which facilitate interfacing with thousands of channels on an FPGA, focusing on low latency or ease of use. The chapter describes the work done on the audio interface and path, including specific configurations of external audio codecs¹ and custom I²S transceiver, allowing us to measure the lowest possible latency. Finally, it shows experimental results of our system, demonstrating the gains and contributions of this thesis, including both the FAUST approach and C++ optimization.

1.3 Contributions

The global contribution of this thesis is to facilitate the programming of FPGA platforms for real-time audio DSP applications through the use of high-level programming languages such as FAUST and C++. This takes the form of a toolchain: *SyFaLa*.

Chapter 2: Definition of latency ranges for the terms Real-Time, Low Latency, and Ultra-Low Latency in interactive audio systems.

Chapter 4: Implementation guidelines for hardware design of audio on FPGA.

Chapter 4: Study and analysis of ADC/DAC converter architectures for audio applications.

Chapter 4: Implementation of bare-metal and *Linux*-based systems on the SoC CPU to control an audio DSP on FPGA. Development of simple PCB for physical control.

Chapter 4: Introduction of *Syfalala*, the first open-source automatic compilation flow of audio programs on FPGA, offering many tunable parameters (e.g., number of output channels, hardware or software control interface, audio codec used, etc.).

Chapter 5: Generic design guidelines offering a methodology to systematically optimize C++ code for real-time audio DSP in *Vitis*.

Chapter 5: Loop interchanging: systematic optimization to improve the implementation of sequential audio programs with a tenfold performance increase.

¹In this thesis, “audio codec” always refer to a hardware component providing analog outputs and inputs (audio ADC/DAC): not an audio compression algorithm.

Chapter 5: Characterization by measuring external memory access times for audio programs to assist with scheduling.

Chapter 6: Latency measurement of ultra-low latency audio codecs and our complete system. Presentation of the first ultra-low latency real-time audio DSP implementation on FPGA, achieving a latency of 11 μ s from analog-to-analog.

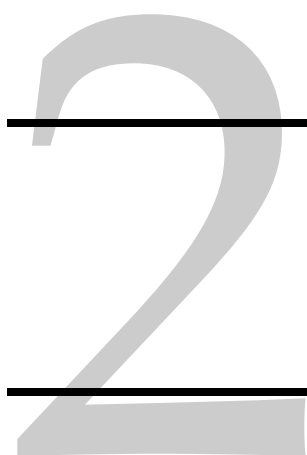
Chapter 6: Presentation of our custom configurable and scalable I²S transceiver , which enables ultra-low latency and simplifies the scalability of the design.

Chapter 6: Presentation of two boards for high-density multichannel audio systems (with applications in Wave Field Synthesis, Ambisonics, etc.). The first board is cost-effective and targets the maker community, while the second targets professional applications, offering ultra-low latency across up to 512 concurrent channels.

Chapter 6: Bug resolution in the initialization of the *SSM2603* codec.

Chapter 6: Evaluation of the performance of our modifications to the FAUST compiler for FPGA implementation, focusing on memory access and computation dispatching.

Chapter 6: Evaluation of the performance of our audio compilation toolchain and optimizations for various audio applications compiled on different *Xilinx*-based FPGA evaluation boards.



CHAPTER 2

Real-Time Audio DSP: Fundamental Concepts

Contents

2.1	Basics of Digital Signal Processing	6
2.1.1	DSP Principles	6
2.1.2	Dataflow Programming Languages	7
2.2	Converting Between Worlds: Digital and Analog Interfacing	7
2.2.1	Analog-to-Digital Conversion	8
2.2.2	Digital-to-Analog Conversion	8
2.3	Fundamentals of Audio Resolution	9
2.3.1	Quantization	9
2.3.2	Sampling Frequency	10
2.4	Real-Time Audio Constraints	11
2.4.1	Buffering	11
2.4.2	Latency	12
2.4.3	Throughput	12
2.4.4	Jitter	13
2.5	The Range of Latency: From Real-Time to Ultra-Low Latency	15
2.5.1	Real-Time Processing	15
2.5.2	Low Latency	15
2.5.3	Ultra-Low Latency	16
2.6	Control Mechanisms	17
2.6.1	Control Rate	17
2.6.2	Controllers	18
2.7	Memory Management	19
2.7.1	Compilation Time, Initialization Time	19
2.7.2	Memory Size and Accesses	19
2.8	Audio DSL: The FAUST Example	19
2.8.1	The FAUST Language	19
2.9	Conclusion	21

This chapter provides an overview of the foundational concepts underlying audio DSP. It covers the main principles that are crucial for comprehending the specificities of digital audio processing. By exploring these fundamental concepts, our objective is to establish an understanding of the theoretical framework that guides the design and implementation of audio processing systems on FPGA platforms.

A brief summary of these concepts is studied here. For those interested in delving deeper into audio DSP, the following books are recommended:

- *Digital Signal Processing A Practical Guide for Engineers and Scientists*, by Steven K Smith [Smi03].
- *Audio Signal Processing and Coding*, by Andreas Spanias, Ted Painter and Venkatraman Atti [SPA06].
- *The Computer Music Tutorial*, by Curtis Roads [Roa23].

The chapter is organized as follows:

- Section 2.1 provides a concise functional overview of audio DSP and discusses dataflow programming languages.
- Section 2.2 details how the conversion between analog and digital signals is done, describing ADC and DAC principles.
- Section 2.3 presents the basics of audio sampling, detailing the principles of sampling frequency and quantization principles.
- Section 2.4 explores four critical real-time-related concepts: buffering, latency, throughput, and jitter.
- Section 2.5 discusses the requirements for low latency based on a literature survey and analysis of existing systems.
- Section 2.6 examines control mechanisms commonly employed in audio DSP implementations to optimize processing load.
- Section 2.7 delve into memory management strategies for the implementation of audio DSP on embedded architectures.
- Section 2.8 introduces the FAUST programming language, a domain-specific language widely utilized in audio DSP, and referenced throughout this thesis.

2.1 Basics of Digital Signal Processing

2.1.1 DSP Principles

Digital Signal Processing (DSP) refers to the manipulation of signals such as audio, video, radio, temperature, pressure, or any other measurable phenomena after they have been converted into a digital format. The objective is to analyze, modify, or improve these signals to achieve desired outcomes, such as noise reduction, signal enhancement, or data compression.

Audio DSP consists of three sequential key steps: acquisition (or synthesis), processing, and output (see Fig. 2.1). The processing part can be seen as a functional block composed of many simple mathematical functions that form a complex, linear or non-linear processing function. Audio signals are continuous, infinite streams with no time dependencies, which makes them well-suited for dataflow architectures. These architectures operate without a program counter, allowing data to flow asynchronously through the program, aligning well with dataflow programming languages. The processing function can be controlled or modified with controller and have mem-

ory access. Input and output functions are managed by converters such as an Analog-to-Digital Converter and a Digital-to-Analog Converter.

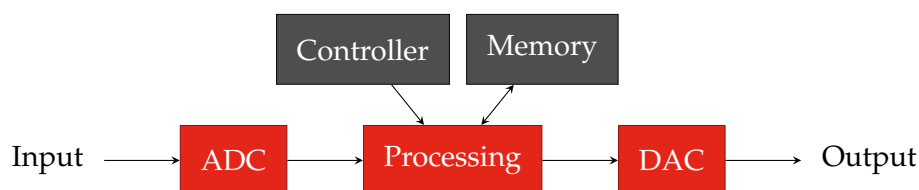


Figure 2.1: Basic audio DSP functional overview.

2.1.2 Dataflow Programming Languages

Dataflow languages use a Model of Computation (MoC) that prioritize the flow of data through a system rather than the sequence of commands or instructions. In traditional imperative programming, a series of steps that the computer should follow to achieve a result is specified. In contrast, dataflow languages are centered around how data moves and transforms through various stages of computation. The core concept of a dataflow language is that computation is triggered by the availability of data. This means that an operation is executed when all its required inputs are present, which naturally supports concurrent execution of operations. Because operations can run in parallel as soon as their inputs are ready, dataflow languages often excel in scenarios where tasks can be performed simultaneously, making them well-suited for real-time processing and complex data transformations such as audio processing.

Dataflow languages can be visual or textual. Visual dataflow languages often work with diagrams where blocks represent operations and arrows show the flow of data between these blocks. For instance, *LabVIEW* and *Simulink* use graphical interfaces to model and control various systems. These visual representations can be more intuitive for designing and understanding complex workflows compared to writing code in a traditional programming language. A survey on dataflow programming has been conducted by Wesley et al. [JVH⁺04].

The Synchronous Dataflow (SDF) MoC was introduced by Lee and Messerschmidt [LM87] to model DSP applications on parallel hardware. Dataflow scheduling can be broadly classified into two main categories: static dataflow scheduling, where production and consumption rates are determined at compile time, and parametric (dynamic and reconfigurable) dataflow, where the rates can vary during execution [DMR⁺16].

Based on dataflow MoC, first synchronous languages were developed in the 1980s with some well-known being *Esterel* [BG92], *Lustre* [BLGJ91], and *Signal* [HCRP91]. These languages are specifically designed to describe reactive systems with real-time behavior and parallel processing. They guarantee critical real-time characteristics, such as execution within bounded time and memory, as well as ensuring the absence of deadlocks. For audio, DSL like FAUST use a dataflow MoC.

2.2 Converting Between Worlds: Digital and Analog Interfacing

Historically, analog processing was the only way to process an audio signal. Analog audio processing is still very used, mostly because of the non-linear behaviors it can implement [DZ02] (such as guitar effect, tube amplifier, etc.) that are difficult to simulate on a digital system. However, most audio systems nowadays rely on digital technologies. They offer more flexibility and reliability than analog systems, with lots of advantages such as higher dynamic range, better frequency response, robustness, and signal preservation and restoration [SPA06].

A digital signal is a discrete-time (or sampled) signal with discrete (or quantized) amplitude values [Gro08]. The amplitude can take a defined value from a finite discrete set.

While most modern audio processing systems use a digital approach, most audio recording and rendering techniques are analog (microphone, speaker). In order to do the transition between the analog and digital domain analog-to-digital conversion (ADC) and digital-to-analog conversion (DAC) are used. The next section briefly explains these processes, for a more detailed explanation, refer to Chapter 3 of Smith's book [Smi03]. A comprehensive analysis of the ADC/DAC architecture is presented in Chapter 4.3.

2.2.1 Analog-to-Digital Conversion

The Analog-to-Digital Converter (ADC) converts an analog voltage variation (such as the voltage variation transduced by a microphone from air variations) to a digital signal. Going from an analog signal that is continuous to a digital signal that is discrete in time and amplitude necessitates quantization and sampling.

The sampling step consists of periodically measuring the instantaneous value of the analog signal at discrete time intervals, defined by the sampling rate (see Fig. 2.2). This process converts the signal from a continuous-time representation to a discrete-time sequence. After sampling, the signal undergoes quantization, where each sampled value is approximated to the nearest level within a finite discrete set of values (as explained in Section 2.3.1). The sampling rate is expressed in samples per second; for example, a sampling rate of 48 kHz means that the signal is measured 48,000 times per second. According to the Nyquist-Shannon sampling theorem, if the sampling rate is at least twice the highest frequency component of the signal, the original analog signal can be perfectly reconstructed, assuming ideal conditions. In practice, increasing the sampling rate beyond the Nyquist rate can help mitigate aliasing and reduce quantization noise in oversampled systems, but does not inherently improve fidelity beyond a certain point.

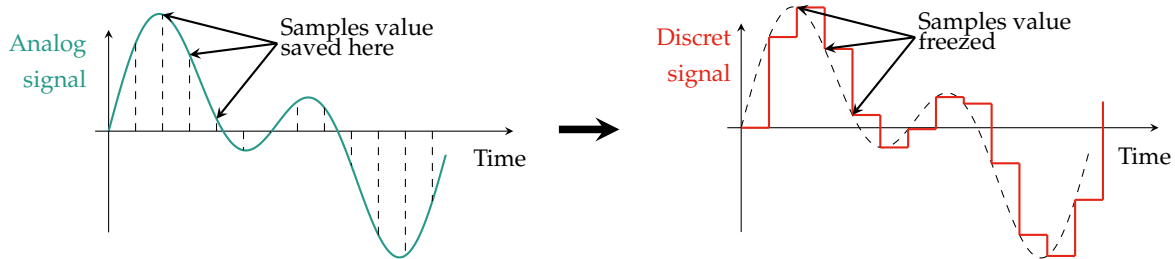


Figure 2.2: Sampling principle (ADC).

2.2.2 Digital-to-Analog Conversion

In contrast to an ADC, a Digital-to-Analog Converter (DAC) converts digital signals into analog signals that can, for example, be sent to amplified speakers.

Theoretically, one can model an ideal DAC as generating an infinitely narrow impulse (Dirac delta) at each sampling instant, with amplitude proportional to the digital sample value. When these impulses pass through an ideal low-pass filter whose cutoff is half the sampling rate, the original bandlimited analog signal can be perfectly reconstructed. In practice, however, most DACs do not produce impulses. A common method is the zero-order hold (ZOH): each sample value is held until the next one, creating a staircase output that is then smoothed by a low-pass filter (see Fig. 2.3). Modern sigma-delta DACs take a different approach, oversampling and using noise shaping to push quantization noise out of the audible band, resulting in high fidelity with simpler analog filtering. Although no real DAC is perfectly “ideal,” they can achieve excellent results when the sampling rate is sufficiently high and quantization is fine enough.

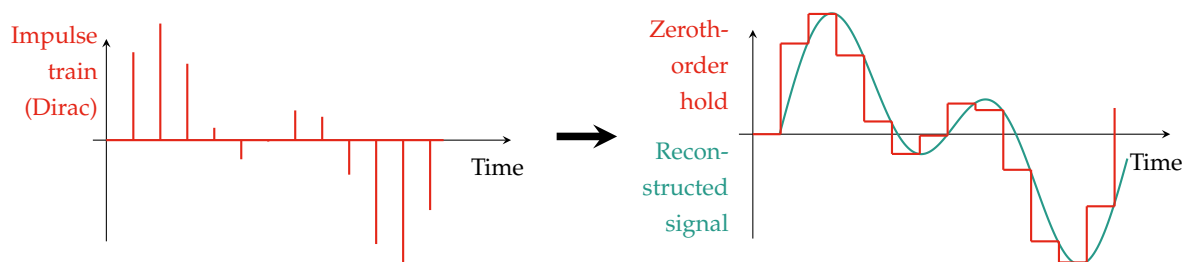


Figure 2.3: Reconstruction principle (DAC)

2.3 Fundamentals of Audio Resolution

This section delves into sampling and quantization concepts. For a more detailed explanation of these topics, refer to Chapter I of Roads's book [Roa23].

2.3.1 Quantization

When sampled, the amplitude of the analog signal must be represented digitally with a binary value, which implies a finite number of possible values. The number of possible discrete levels (or quantization levels) is determined by the bit depth (or quantization depth) of the digital audio system that corresponds to the number of bits used to store each sample. For instance, a 16-bit signal has $2^{16} = 65\,536$ possible levels, while a 24-bit signal has $2^{24} = 16\,777\,216$ levels.

Since not all values can be represented, each sampled amplitude value is rounded to the nearest discrete level. Quantization error is the difference between the actual sampled value and the quantized value. This error introduces noise into the signal, referred to as quantization noise. Higher bit depths reduce quantization noise, leading to a higher Signal-to-Noise Ratio (SNR), improving accuracy and dynamic range of the signal. For each additional bit in the bit depth, the SNR improves by approximately 6 dB.

Common bit depths in digital audio include 16-bit (standard for CDs) and 24-bit (common in professional audio recording). A trade-off has to be found as higher bit depths result in larger file sizes and more complex processing.

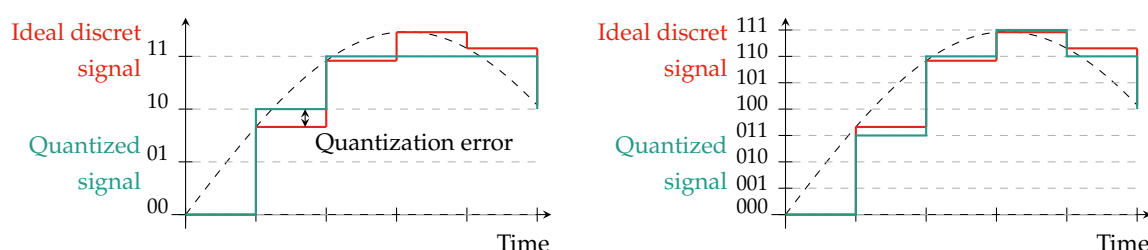


Figure 2.4: Quantization error difference for 2 and 3 bit depth signal.

Fig. 2.4 shows the difference between the ideal discrete signal if the set of level was infinite, and a set of level with a bit width of 2 and 3 bit (pedagogical values). The 3 bit depth signal shows less quantization error and thus, a better SNR.

The quantization error introduces a random component to the signal as the last bit in the data is rounded to 0 or 1 to match the nearest value of the data set. This randomization is heard as a noise called the noise floor. In digital audio, this noise sets a lower bound on the theoretical SNR. However, the actual noise floor in a real system may also include other sources, such as analog circuit noise or interference. Thus, quantization noise alone does not necessarily define the overall noise floor.

On the other hand, distortion occurs if the signal's amplitude exceeds the system's maximum level, known as 0 dBFS (decibels relative to full scale). The difference between the maximum level and the effective noise floor determines the system's dynamic range. For example, a 16-bit system provides a theoretical quantization-based dynamic range of approximately 96 dB, whereas 24-bit audio offers around 144 dB. In practice, actual dynamic range is often limited by factors beyond quantization, including analog hardware noise and interference. Nevertheless, higher bit depths allow for lower quantization noise and the capability to represent both very soft and very loud sounds with minimal distortion.

2.3.2 Sampling Frequency

The sampling frequency, or sample rate, is the number of times per second that an analog signal is sampled and converted into a digital form. Fig. 2.5 shows the same sampled signal with different sampling frequencies. One of the most important specificity of audio program is its absolute dependency on this slow clock rate. The sample rate must be respected, a single missed sample can alter significantly the audio quality.

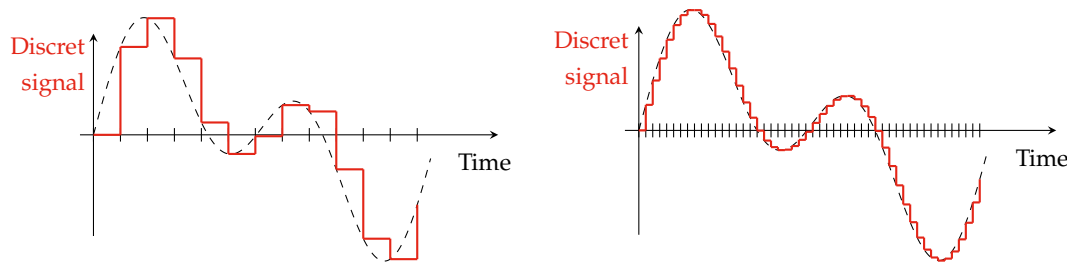


Figure 2.5: *Effect of sampling frequency. The signal on the right has a sampling frequency 4 times higher.*

The sample rate directly impacts the frequency range that can be accurately represented by the digital signal. It should be high enough to represent frequencies up to 20 kHz in order to reproduce sounds within the audible range of human hearing (typically between 20 Hz and 20 kHz). To ensure that, we use the Nyquist theorem. The Nyquist theorem states that to accurately represent a continuous signal and avoid aliasing, it must be sampled at a rate that is at least equal to or higher than the Nyquist rate, which is twice the highest frequency present in the signal [Sha49]. Not to be confused with the Nyquist frequency, that determine the maximum frequency that a system can accurately capture and recreate, corresponding to half the audio sampling frequency. So a sample rate of 40 kHz or higher ensures that all frequencies in the audible range can be correctly captured and generated.

If a continuous analog signal is sampled at a rate that is insufficient to accurately capture its frequency content, aliasing appear. When aliasing occurs, higher frequency components of the signal are misrepresented as lower frequencies in the digitized signal, leading to audible distortions degrading the quality of the audio (See figure 2.6).

To prevent aliasing, the signal must be sampled above the Nyquist rate and must be low-pass filtered to eliminate frequencies above half the sampling rate that could be aliased. This filter, called an anti-aliasing filter, is crucial for practical ADC systems dealing with high-frequency analog signals. Analog anti-aliasing filters can be very complex to design at low frequencies. In such cases, oversampling is often used to simplify the implementation of these filters.

Standard audio sample rates are 44.1 kHz or 48 kHz. The frequencies are slightly higher than the previously found 40kHz as we place the Nyquist frequency beyond our range of hearing. This allows for lighter filters to reduce aliasing, with minimal impact on audibility. The specific value of 44.1 kHz, 48 kHz is due to a combination of historical standards, technical requirements and

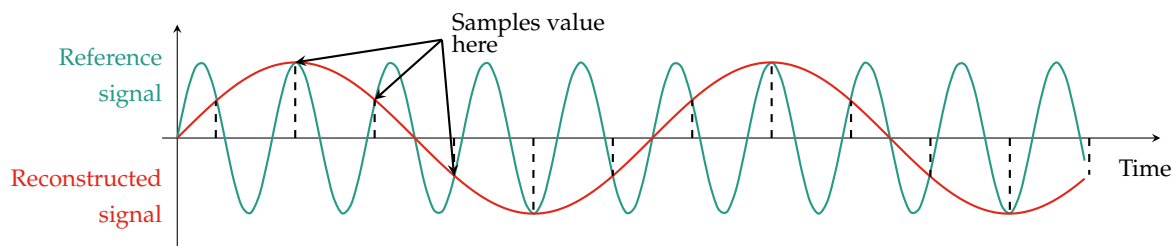


Figure 2.6: Effect of aliasing. Aliased reconstructed signal (red) due to under sampling the reference signal (green).

compatibility considerations. The first one is standard for audio CD, the second one is commonly used in video. Higher sample rates, such as 88.2 kHz, 96 kHz, and even 192 kHz [Stu97], are also used in professional audio recording and mastering environments.

The higher sampling frequencies, the better audio accuracy so there is no downside for the sound quality of using the higher sampling frequencies possible. However, it increases the needs in terms of processing power and memory and the difference is rarely noticeable to the human ear, which is why most actual systems do not go over few dozen of kHz.

To summarize, when a signal is sampled, the precision of the signal can be adjusted with the bit depth (vertical precision) and the sampling frequency (horizontal precision) as shown Fig. 2.7. Both of these steps can induce a loss of information, and the sampling frequency and quantization need to be carefully chosen.

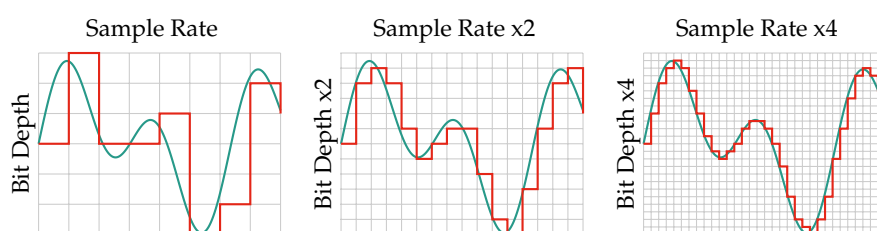


Figure 2.7: Difference of digital signal resolution (time and amplitude).

2.4 Real-Time Audio Constraints

2.4.1 Buffering

Buffering involves temporarily storing data in a region of memory to handle multiple data at once instead of processing each one separately. In Operating System (OS), buffering ensures a continuous stream of audio data, preventing audio glitches or interruptions caused by variations in processing time or data arrival rates.

Sample buffering reduces CPU load for several reasons. First, on a CPU the time needed to call a function to process a sample can be significant, especially due to the overhead of preparing the memory stack. Calling a function once to process multiple samples is thus much more efficient. Additionally, processing blocks of samples allows for a more efficient use of the cache memory, as many samples will likely read and write the same data in the cache. Accessing external memory is also more efficient because memory operations typically use bursts of data, making the reading or writing of a few elements suboptimal. Moreover, many optimizations can be applied at compile time if the compiler works with chunks of data, leading to better OS scheduling, branch prediction, and other enhancements. Buffering also helps smooth variations in CPU load during

the processing. For instance, when a controller is changed, it can induce spikes in computing load. Buffering allows these spikes to be absorbed by the sampling rate, maintaining real-time application performance. On a conventional computer with an OS, interruptions prevent the program from guaranteeing the computation of samples at the sample rate, as the system is not fully dedicated to audio processing. Buffering the samples smooths these interruptions and maintains the correct output sample rate.

However, because larger buffers require more memory, a trade-off is necessary to keep buffering optimal. At a certain point, increasing the buffer size further has little to no effect on performance and only consumes more memory. Additionally, buffering can increase latency as the arrival of samples must be awaited in order to buffer them. When using audio software, a balance must be struck between using a large enough buffer to ensure uninterrupted audio output and maintaining low enough latency to provide acceptable feedback.

On a standalone system such as a microcontroller or FPGA, buffering is less critical since the processor is fully dedicated to the audio processing tasks. Interruptions are entirely controlled by the user and can be managed effectively to avoid sample starving, even without buffering. However, buffering can still reduce CPU load by optimizing memory access and computations, making it useful in embedded systems while allowing for smaller buffer sizes.

The size of the buffer depends on many parameters: operating system, sound card, connection, memory, etc. Juillerat et al. [JAS07] were able to use a 16-sample buffer with *Linux* architectures and most sound cards when using Advanced Linux Sound Architecture (ALSA) drivers, though depending on the platform, they were occasionally limited to 512 samples. Wang [Wan18] demonstrated that Digital Audio Workstations (DAW) using frame based processing on operating system typically requires a buffer of 64 - 2048 samples. *Linux Xenomai* allows buffer sizes as small as 16 samples [VZT22]. As discussed before, with microcontrollers, buffers can afford to be much smaller thanks to the dedicated processor. For example, Michon et al. [MOLF19] used buffers as low as 8 samples using a *Teensy* microcontroller. In general, buffer sizes ranging from 32 to 1024 samples are typical.

2.4.2 Latency

For audio systems, latency generally refers to the delay taken for the signal to pass through a system or process (i.e., the delay between the time signal is input and the time it is output). Low latency is desirable in real-time audio applications, such as live performances, where any delay between input and output can be noticeable and cause disruptions. High latency can be acceptable in non-real-time applications, such as audio recording and post-production, where precise synchronization between different audio tracks is more important than immediate feedback.

The music and audio technology community has been focusing on audio latency for a very long time. Wang's PhD thesis [Wan18] provides a good review of the different potential sources of delay in digital audio systems. ADC/DAC, operating systems, and audio networking are usually identified as the main contributors to audio latency. A detailed study of ADC/DAC-induced latency is carried out in Chapter 4.3. For operating systems and audio networking, latency can be introduced by various factors, including processing time, buffering, and data transmission delays.

2.4.3 Throughput

Throughput refers to the rate at which data is processed, measured by how many samples can be handled per unit time. In audio systems, throughput determines whether processing can keep pace with the required sample rate on average. It should not be confused with latency, which, as explained earlier, refers to the delay from input to output and must remain bounded to ensure acceptable responsiveness.

If, on average, each sample can be processed in less time than one sampling period, the system can sustain real-time performance (see Fig.2.8). However, if the average processing time per sample exceeds the sampling period (Fig.2.9), the system fails to keep up with the sample rate, making real-time operation impossible.

Processing that exceed a single sample period may still be acceptable, provided buffering or other strategies allow the system to catch up and keep the maximum latency within an acceptable limit (Fig. 2.10). Although buffering increases latency, it can also improve throughput by enabling more efficient memory access or parallelization. In this way, higher throughput can be achieved to ensure real-time operation.

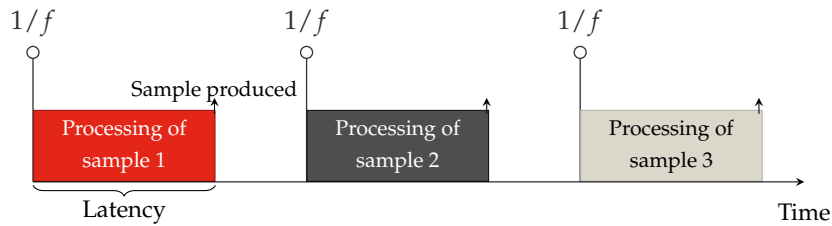


Figure 2.8: Sample processing: Latency < 1 Period; Throughput = 1 Sample/Period (real-time).

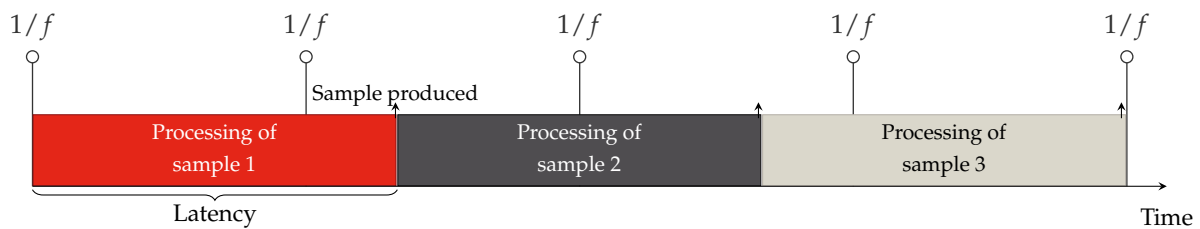


Figure 2.9: Sample processing: Latency > 1 Period; Throughput < 1 Sample/Period (not real-time).

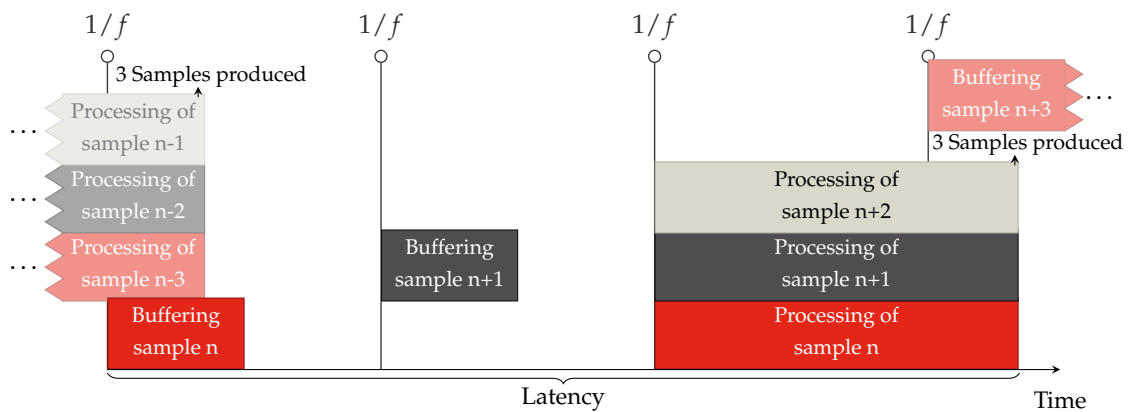


Figure 2.10: Sample processing parallelized: Latency > 1 Period; Throughput = 1 Sample/Period (real-time).

2.4.4 Jitter

Hardware Clock Jitter

Jitter is the small, often undesirable, variation or deviation in the timing of a signal's arrival or transmission from its expected position in time. In the context of audio DSP, jitter typically refers to fluctuations in the timing of sample delivery. It occurs when there are slight local deviations in

the clock signal, leading to small variations in the time intervals between samples (see Fig. 2.11). This can result in the phase distortion of the original sound leading in a high frequency clicking noise added to the signal. With modern powerful hardware systems and improvements in clock integrity, jitter is rarely a hardware issue with a single device.

Jitter should not be confused with clock drift. Clock drift occurs when a clock does not run at the exact same rate as a reference clock, leading to a gradual desynchronization of independent devices that are supposed to be synchronized. This issue can be mitigated by using a master clock to synchronize all devices. However, poor interconnections and substandard cable quality can complicate this process and exacerbate synchronization problems.

The threshold of audibility of jitter appears to be in the range of 10 ns to 30 ns [BG98]. On a standalone embedded system, clock jitter is usually very low. It is challenging to find typical values because jitter is highly dependent on many external factors (such as power supply, temperature, vibration, crosstalk, etc.). For instance, a value around 100 ps is given for the jitter of the Phase-Locked Loop (PLL) on the Teensy 3 [pjr] (often used in embedded real time audio processing). The design that we implemented on our FPGA has an output jitter ranging from 200 ps to 300 ps. Most audio codecs do not provide any jitter specifications, likely indicating that with modern systems, clock jitter is not a significant concern anymore.

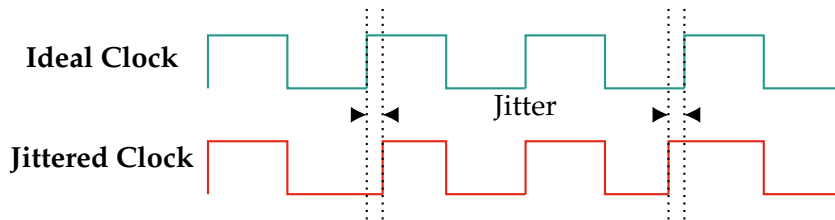


Figure 2.11: Clock Jitter (order of magnitude: few ps to few ns).

Software Network Jitter

Most of the time in literature, the term “jitter” is applied to software latency inconsistency, describing the phenomenon where latency can randomly change over time. Operating systems, software environments, and communication channels can introduce a significant amount of jitter [JAS07], leading to many samples of latency uncertainty. When this latency directly affects samples (e.g., Ethernet packets arriving with jitter or drifting sample clocks in the case of two distinct systems [MSKW24]), a buffer must be used to mitigate the jitter and guarantee a consistent sample rate output.

Fig. 2.12 shows an example of jitter in a communication channel. Each packet handles 2 samples. If jitter occurs (e.g., Packet 3 arrives too late), a sample (S_6) is not transmitted to the ADC in time, resulting in noticeable noise. The only solution is to buffer packets to allow for jitter compensation (as explained in Section 2.4).

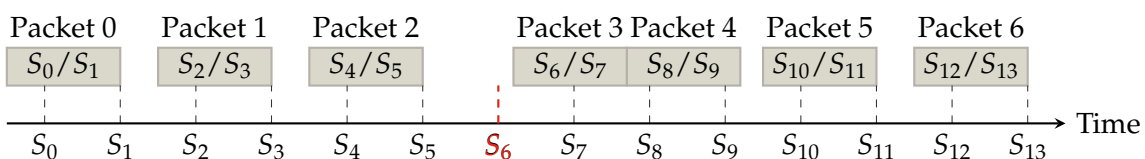


Figure 2.12: Software Jitter (order of magnitude: μ s to ms).

Software Control Jitter

Software jitter can also stem between a controller and the processed sample, giving different latency between a change of a controller and the effect on the sample. Since the controllers are processed at a different rate than the samples (see Section 2.6), the latency inconsistency is translated into a change in the reactivity of the controllers. Hence, jitter can be critical for digital musical instruments.

McPherson [MJM⁺16] measured the latency inconsistency for different platforms commonly used for digital instruments (embedded microcontrollers, single-board computers, and wireless devices) from a digital input stimulus to output sound using different communication protocols. He found that, in the best-case scenario, at least 1.5 ms of latency inconsistency would appear from a digital stimulus. However, a latency and jitter-optimized system such as *Bela* (based on the *BeagleBone Black* single-board computer and running on a *Xenomai Linux* kernel) can achieve jitter on the order of a single sample. As pointed out by McPherson's work, a maximum jitter of 1 ms seems to be an acceptable limit.

2.5 The Range of Latency: From Real-Time to Ultra-Low Latency

There is no strict threshold in the literature to define the difference between audio “real-time” latency, low latency, and ultra-low latency. This section proposes to formalize the thresholds defining these terms.

2.5.1 Real-Time Processing

In 1965, Martin James in his book *Programming real-time computer systems* [Mar65], gives a definition of real-time programming:

Definition 1. “A real-time computer system may be defined as one which controls an environment by receiving data, processing them and returning the results sufficiently quickly to affect the functioning of the environment at that time”.

Real-time programs must guarantee response within specified time constraints and even if real-time responses are often understood to be in the order of milliseconds, real-time does not inherently define a specific latency limit. In audio processing, real-time refers to ensuring that samples are processed in sync with the sample rate. However, as explained in Section 2.4, this primarily depends on buffer size and does not necessarily guarantee any latency constraints. An audio system operates in real-time as long as $t_{proc} < t_{buff}$, where t_{proc} is the processing time and t_{buff} is the delay between two successive buffers.

2.5.2 Low Latency

In audio applications, “real-time” and “low latency” are often used interchangeably. However, a system is often considered to have low enough latency when the delay does not alter the user experience, so in other words, when it is imperceptible. Nelson Posse Lago and Fabio Kon, in their *Quest for Low Latency* [LK04] state that “In interactive systems, adequate latency and jitter characteristics are determined by the interaction with the user: high latency or jitter may impair the user's performance or, at least, offer a frustrating and tiring experience”.

For example, a guitar player requires a low latency system so that there is no perceptible delay between when they play a note and when they hear it. On the other hand, if a computer sends pre-recorded music to a speaker over a wireless channel, low latency is not necessary, as the listener can tolerate a delay between when the song starts and when it is played without affecting the

experience. Still, in this example, the computer system is considered real-time as it should meet the specified timing constraints ensuring that the song is transmitted without missing any samples. These timing constraints, though, are not critical for the user experience.

The threshold for imperceptible latency is highly dependent on the application and remains a subject of debate. Lester et al. [LB07] demonstrated that the impact of latency on live sound monitoring for musicians is highly dependent on the type of instrument, and that even a few milliseconds of latency can be discernible by performers. Lago et al. [LK04] show that synchronization in haptics, rhythm, or external events requires very different latencies. Rubine and McAvinney [RM90] demonstrated that altering the duration of notes by as little as 2 ms can be noticeable, and performers attempting to tap at a specified rate show standard deviations in the time intervals between taps ranging from 1.5 ms to 4 ms. Friberg et al. [FS95] proved that an asynchrony in a steady pulse is detectable at 6 ms. Therefore, the noticeable limit can vary depending on the type of experience.

Asynchronies of up to around 50 ms can be subconsciously adjusted during ordinary music performance. For example, in a concert hall where the distance between instruments can generate dozens of milliseconds of latency due to sound propagation, these asynchronies are not perceived as such by performers or the audience [Ras79]. For digital audio instruments, gesture and haptic feedback must also be taken into account in the perception of simultaneity. Dahl and Bresin [DB01] demonstrated that an audio feedback latency up to 55 ms can be compensated for by a musician adapting their gestures, while Kaaresoja et al. [KBL14] concluded that audio feedback to haptic gestures can tolerate up to 70 ms of latency. Wessel and Wright [WW02] defined the higher acceptable computer's audible reaction to gesture at 10 ms. As pointed out by McPherson [MJM⁺16], this threshold is still one of the most commonly used by the audio community today. A good summary of perception latency thresholds for different applications is described in Wang's thesis [Wan18].

It is therefore more accurate to speak about acceptable latency rather than imperceptible latency. However, depending on the application, what constitutes acceptable latency can vary by an order of magnitude. In the literature, "low-latency" often refers to systems yielding around 10 ms [JSA08, BCA⁺19]. But for some algorithms, such as dereverberation [HAY⁺12] or singing voice detection [LWB15], hundreds of milliseconds of "latency-by-design" can be induced. These papers discuss these as low-latency compared to the existing state of the art and the fact that there is no direct feedback needed.

This shows that the definition of low latency needs to be specified for a specific range of applications. This thesis focuses on latency for interactive audio systems, where there is a feedback need for the output signal. Considering the scenarios discussed earlier, the following definition for a low-latency system can be proposed.

Definition 2. *A low-latency interactive audio system is a system with latency low enough to be acceptable and non-inconvenient in every feedback-induced scenario, with a threshold of under 10 milliseconds.*

For audio systems that do not guarantee such a stringent constraint but still allow for acceptable results (10 ms to 100 ms), the term audio "real-time" seems more suitable.

2.5.3 Ultra-Low Latency

For ultra low latency, there is to date an important effort make for the reach of a sub-millisecond latency system [MZ15, BCS⁺15]. As discussed before, in some cases a latency of 4 ms or even 2 ms can be detected. While most of the time this can be subconsciously adjusted, achieving sub-millisecond latency brings the advantage of being strictly imperceptible in any case. Moreover, as Lago and Kon pointed out [LK04], a latency lower than 10 ms, even if not directly perceptible, can provide several advantages for the overall system. For example, considering the fact that

speakers are generally placed a few meters away from the user, this distance adds dozens of milliseconds of latency due to sound propagation, leaving less than a millisecond for processing to maintain the low-latency constraint. Similarly, since latencies accumulate in a setup, providing a sub-millisecond latency allows for other systems to be daisy-chained with room for additional latency.

Because of the extensive difficulty of achieving under-millisecond latency in a complete system, and the fact that a 1 ms system leaves enough room for other system components latency, it seems reasonable to choose this value. The definition for ultra-low latency can then be stated.

Definition 3. *An Ultra low-latency interactive audio system is a system with latency low enough to be imperceptible, with a threshold of under 1 millisecond.*

These definitions are summarized in Table 2.1. While somewhat subjective and based on the current state of latency system efforts, they are appropriate for effectively classifying real-time audio systems.

Designation	Range	Usage
Real-Time	10 ms to 100 ms	Perceptible, case-dependent usability
Low Latency	1 ms to 10 ms	Non-inconvenient for feedback
Ultra-Low Latency	<1 ms	Imperceptible, chainable

Table 2.1: *Proposed latency designations for interactive audio system.*

2.6 Control Mechanisms

This section explains the control mechanisms employed in audio DSP algorithms, essential for real-time sound manipulation. It highlights that control processing is distinct from sample processing, necessitating different management strategies.

2.6.1 Control Rate

In audio DSP programs, it is common to distinguish two different rates: the sample rate (or audio rate), detailed in Section 2.3, and the control rate (Fig. 2.13).

Control rate (typically between 100 Hz and 1000 Hz) refers to the rate at which control signals or parameters (such as volume, panning, filter cutoff frequencies, etc.) are updated. Unlike sample rate, control rate only affects the processing delay of parameter changes, but not the signal integrity. A higher control rate allows for smoother and more precise adjustments to these parameters over time, resulting in finer control over audio effects and processing.

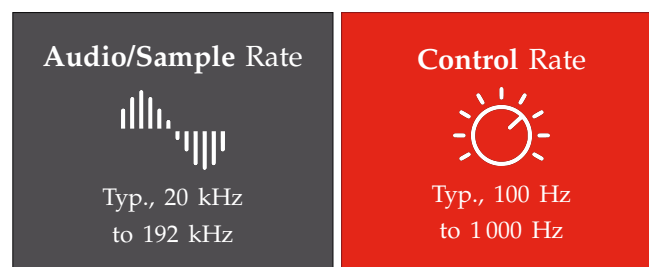


Figure 2.13: *Control rate and sample rate.*

Historically, the control rate is directly affected by buffering strategies. Instead of updating controls for every sample processed, which would consume significant CPU resources, controls are typically updated once per buffer. This approach optimizes performance by reducing the frequency of control updates to match the buffer size. For example, with a buffer size of 128 samples at a sampling rate of 48 kHz, the control rate would be $\frac{48\,000}{128} = 375$ Hz. This rate is generally sufficient to capture changes in parameters over time without the need for high-frequency updates, as changes in control parameters tend to occur more slowly than changes in the audio signal and are often human-controlled.

Algorithm 1 Audio Processing with Control Rate Processing Parameters

```

BUFFER_SIZE ← 64
while true do
    process_controller()                                ▷ Updated at control rate
    for i ← 0 to BUFFER_SIZE − 1 do
        process_sample(buffer[i])                      ▷ Updated at sample rate
    end for
end while
  
```

An example of how the control rate works is illustrated in Algorithm 1. Controllers are processed only once per buffer. The larger the buffer size, the longer the interval between controller processing.

Today, the control rate is slightly less critical as CPUs have become powerful enough to handle more computations at the sample rate. However, users are also exploring new capabilities that were impossible years ago. Therefore, it remains useful to conserve CPU resources wherever possible.

2.6.2 Controllers

To maintain control of the DSP over time, the user can use controllers that modify the processing values of the program (See “Controller” in Fig. 2.1). These controllers can be classic components of audio user interfaces, such as sliders, knobs, or buttons, which are digitally translated into integer or float values. These basic controls allow the user to adjust parameters like volume, pitch, and filter cutoff frequencies in real-time. However, controllers can also be more complex objects, such as signals or tables. Signals can modulate parameters continuously, allowing for dynamic changes that can create effects like vibrato or tremolo. Tables can store sequences of values or presets, enabling the user to switch between different configurations or automation patterns efficiently. These controllers are updated at the control rate and are usually mapped onto generic interfaces via protocols such as Musical Instrument Digital Interface (MIDI), Open Sound Control (OSC), etc.

It is important to note that there is a naming conflict here: “controllers” can either refer to user-accessible interface components (knobs, sliders, etc.) or to signals that control the DSP as explained before. For example, if a knob controls a sinewave frequency that modulates another signal, the sinewave signal itself is also referred to as a controller.

In contrast to controllers, sometimes data needs to be output from the DSP without being an audio signal. These outputs can include parameter values that change depending on the samples, such as VU meters. These outputs follow the same rules as controllers, being updated at the control rate rather than being constrained to the sample rate.

2.7 Memory Management

This section focuses on memory management, particularly as it applies to DSP compilation and implementation for embedded architectures. The concepts explained here may not be relevant for general-purpose operating systems with CPU or Graphics Processing Unit (GPU) implementations.

2.7.1 Compilation Time, Initialization Time

On audio programs, some computations are done only once at initialization time. They correspond to constant computations, which cannot be done statically because they depend on values known at execution time, such as the sample rate. This can represent a significant part of the computation. For example, filter coefficients often need to be calculated based on the sample rate. The same thing applies for delay lengths, usually specified in time (e.g., milliseconds) and thus sample rate-dependent. For all the computations that depend on values known at the compilation time, they can be processed during compilation, just like any other program.

An audio DSP program can also require the initialization of a large chunk of memory to store large arrays (i.e., waveform tables, delays, etc.). Although the initialization value of these variables may be known at compilation time (and therefore may not require computation at initialization), it still necessitates processing power during initialization to instantiate these large tables.

2.7.2 Memory Size and Accesses

Static arrays are heavily used in audio DSP programs, either for waveform tables or for delay lines (e.g., echos, reverbs, etc.), translating into large memory footprints (often requiring dozens of MiB of memory). For example, a delay of 1 second at a sample rate of 48kHz and a bit depth of 24b, the memory footprint is: $48\,000 \times 1 \times 24 = 1\,152\,000\text{b}$ or 1.1Mib. A large reverb or a long delay can, therefore, very quickly necessitate subsequent memory.

These large memory regions can be very rarely accessed. A waveform table or a delay might only require one memory access per sample. This number can increase drastically for a reverb. On the other hand, in the case of a Finite Impulse Response (FIR) filter, all the filter coefficients need to be read at each sample, placing a high demand on the memory without caching.

2.8 Audio DSL: The FAUST Example

The field of audio processing has witnessed significant advancements over the past few decades, driven by the growing demand for high-performances, real-time audio applications. Central to these advancements is the development of audio DSLs [Fow10], which are specialized programming languages designed to efficiently address the unique requirements of audio signal processing. Developed since the beginning of computer music [Ver82], these languages offer tailored abstractions and optimizations leveraging domain-specific optimizations that facilitate the development, implementation, and deployment of audio processing algorithms. Audio DSLs provide simplified and intuitive syntax adapted to audio processing tasks, using a dataflow programming model (see Section 2.1.2) in many cases, improving readability and maintainability with less code needed [JCFS13]. They present a viable alternative to C/C++ to design and implement efficient DSP code as native applications or plugins.

2.8.1 The FAUST Language

FAUST (Functional AUdio STreams) is a functional programming language developed at Grame-CNCM (Centre National de Création Musicale) since 2002, that targets high-performance signal

processing applications, libraries and audio plug-ins for a variety of platforms and standards. The usage and principles of the language are discussed in Chapter 3.1.4.

For this thesis, we choose the FAUST audio DSL for several reasons. FAUST is widely employed in music technology, and its programming model, which combines a functional programming approach with a block diagram algebra, makes it intuitive and readable for audio algorithms. Moreover, its open-source nature allows for modifications to the source code, enabling adaptation to the FPGA implementations studied in this thesis. But perhaps most importantly, FAUST is developed in Lyon, and we have had the opportunity to work directly with Yann Orlarey and Stéphane Letz, the creators of the language. This close collaboration between the Emeraude Team and Grame-CNCM has greatly facilitated our understanding of how the compiler works and how to modify it. While this thesis focuses on FAUST, the concepts and methods explored can be applied to other audio transcompilers as well.

Program Example

While it is advisable to visit the official FAUST website [Grab] to explore the programming language and learn how to write audio DSP programs with it, a brief example is presented here to illustrate the basics of FAUST's DSP structure. The following example (Listing 2.1) does not aim to showcase all the functionalities of the language but rather to provide key elements that are important for its implementation on FPGA. It can be tested on the online FAUST Integrated Development Environment (IDE) [Grac].

Lang: FAUST

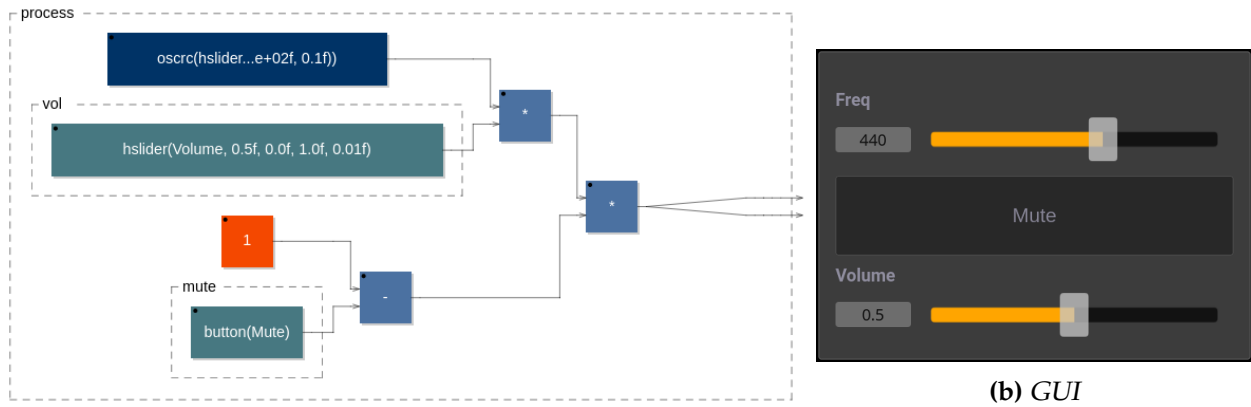
```
1 import("stdfaust.lib");
2
3 // sliders
4 oscFreq = hslider("Freq [knob:1]",440,50,700,0.1);
5 volume = hslider("Volume [knob:2]",0.5,0,1,0.01);
6 // buttons
7 mute = button("Mute [switch:3]");
8
9 process = os.oscrs(oscFreq) * volume * (1-mute) <: _,_;
```

Listing 2.1: Basic FAUST example. Program: basic.dsp [The].

File imports allow definitions from other source files to be included. The `stdfaust.lib` library (line 1) gives access to all the standard FAUST libraries. Next, three controllers, two sliders and a button, are instantiated (line 4 to 7). The `hslider` function defines a horizontal slider with the following parameters: label, initial value, minimum value, maximum value, and step. The button only needs a label. It is important to note that the label contains metadata (e.g., `[knob:1]`), which binds the first knob of the interface to this slider (see Chapter 6.1.6 for more details). The keyword `process` is equivalent to `main` in C/C++. This line defines the core of the DSP, specifying how the output should behave depending on the inputs. Here, the `os.oscrs` function generates a sinewave at the `oscFreq` frequency, which has been defined as a slider in the previous step. The sinewave is multiplied by the volume slider and the mute button. The split composition operator (`A <: B`) is used to distribute the outputs of `A` to the inputs of `B`. Here, the single-channel signal is copied to two channels, represented with `"_"`.

To summarize, this program generates a sinewave on two channels (left and right), with the frequency and volume controllable from sliders and muting available with a button. Fig. 2.14 shows the diagram automatically generated by the FAUST IDE where each function and connection can be clearly analyzed, along with the associated Graphical User Interface (GUI).

The work presented in this thesis, particularly the *Syfala* toolchain introduced in Chapter 4.5.3,



(a) Diagram of the DSP

Figure 2.14: Auto-generated interfaces by the compiler.

now enables the `basic.dsp` program presented in Section 2.8.1 to be directly compiled to an FPGA. With a simple command line (see Listing 2.2), this DSL program can be fully implemented on a SoC (in approximately 15 minutes), including optimized dispatching of CPU/FPGA computations and a customized audio interface. This process is fully automated and does not require the user to have any hardware design knowledge

Lang: CMD

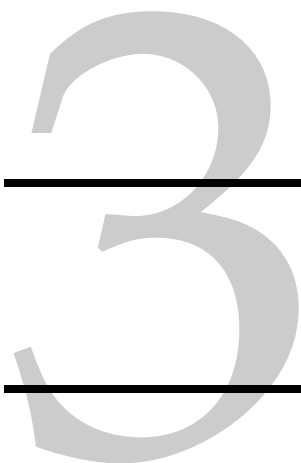
```
1 syfala ./basic.dsp
```

Listing 2.2: *Syfala* command-line for fully compiling the `basic.dsp` example for FPGA/CPU co-design implementation, producing a flash-ready output.

2.9 Conclusion

This chapter provided an overview of the fundamental concepts essential for real-time audio DSP, setting the stage for more advanced discussions on FPGA-based audio compilation. We established key principles of DSP, covering discrete signal representation, bit depth, and sampling frequency, which define audio quality. We categorized latency levels from real-time to ultra-low latency, clarifying often imprecise terminology. Primary real-time challenges like latency, buffering, throughput, and memory footprint were highlighted, along with distinct management of control and sample computation in audio programs. Finally, we introduced audio DSLs, focusing on FAUST, which streamlines the implementation and optimization of DSP algorithms through higher-level abstractions.

With this foundation, we are well-prepared to explore how these principles can be applied and optimized within FPGA architectures in the following chapters. Before that, a deep analysis of the state of the art is conducted in the next chapter. This analysis covers current techniques, solutions, and challenges in real-time audio processing, as well as the present state of real-time audio on FPGA platforms.



CHAPTER 3

State of The Art

Contents

3.1	Audio Domain Specific Languages	24
3.1.1	Classical Programming Languages for Audio	24
3.1.2	Audio Synthesis and Composition Languages	24
3.1.3	Graphical Programming Languages	25
3.1.4	Transcompiler Languages	25
3.2	Platform for Real-Time Audio Processing	26
3.2.1	CPU and Digital Signal Processor Chip	26
3.2.2	Embedded Systems	27
3.2.3	GPU	28
3.3	Audio Processing on FPGA	29
3.3.1	Audio Hardware Design	29
3.3.2	FPGA in Audio Industry	30
3.3.3	FPGA Programming and Domain-Specific Compilation	31
3.4	Real-Time Audio Processing Challenges	32
3.4.1	Audio Latency	32
3.4.2	Multichannel	34
3.5	Synthesis and Insights	35

Now that the basics of audio processing have been outlined, this chapter presents a literature survey of audio compilation, with a focus on FPGA design and audio implementation.

- Section 3.1 conducts a literature review of programming languages used for audio programming, analyzing general-purpose programming languages, audio-specific programming languages with a non-exhaustive explanation of major types.
- Section 3.2 reviews the various programmable platforms used for audio processing.
- Section 3.3 focuses on FPGA programming, presenting their advantages for audio processing, existing applications, and the complexities involved in using and compiling audio algorithms on these platforms.
- Section 3.4 surveys the current research and challenges in audio programming, analyzing trends in terms of performance and goals.

3.1 Audio Domain Specific Languages

Historically, C/C++ has been the dominant language for audio programming, however, audio DSP programs have a specific structure that can potentially be better expressed in DSLs (see Chapter 2.8). They provide a higher-level abstraction, simplifying the design and optimization of audio algorithms. A non-exhaustive list is presented next.

3.1.1 Classical Programming Languages for Audio

C/C++. C++ has long been the foundational language for audio development, offering powerful low-level control over audio processing. It allows developers to write highly optimized code, which is crucial for real-time audio applications where latency and efficiency are essential. A key resource on the use of C in audio is the book *Audio Programming* by Richard Boulanger and Victor Lazzarini [BL10], which details the application of C in various audio processing contexts. Framework like *JUCE* extend C++'s capabilities, providing a more accessible API for audio plugin development and cross-platform compatibility.

Rust. *Rust* is a systems programming language that has gained significant attention for its safety guarantees, performance, and memory management, making it an attractive option for real-time and embedded systems, including audio programming. Although it's not as traditionally entrenched in the audio DSP field as C++, Rust is starting to gain traction in audio-related applications, especially where safety and concurrency are critical.

3.1.2 Audio Synthesis and Composition Languages

Music I - V. The *Music* series [MM64], developed by Max Mathews in the 1950s and 1960s, is historically significant as it laid the groundwork for subsequent audio DSLs. *Music I* was the first computer program for sound synthesis, and its successors introduced increasingly sophisticated methods for digital audio processing. These languages introduced fundamental concepts such as unit generators, which are still in use in modern DSLs.

Csound. Evolved from the *Music* series, *Csound* [LYH⁺16, Bou00] is a versatile and powerful audio synthesis language. It offers a comprehensive set of tools for sound design, algorithmic composition, and signal processing. *Csound*'s flexibility comes from its ability to allow users to write complex signal processing algorithms using an *orchestra-score paradigm*. *Csound* has been extensively used in academia, experimental music, and sound design due to its open-source nature and extensive community support.

SuperCollider. *SuperCollider* [McC02] is a real-time audio synthesis and algorithmic composition language. It features a set of tools for live coding and sound design. SuperCollider is particularly noted for its server-client architecture, where the *SuperCollider* language (*sclang*) controls a separate audio synthesis engine (*scsynth*).

ChuckK. Well-suited for live coding performances, *ChuckK* [WC03] is a strongly-timed, concurrent, and on-the-fly audio programming language designed for real-time synthesis, composition, and performance. One of its unique features is its precise control over time and concurrency.

3.1.3 Graphical Programming Languages

Max/MSP. Developed by *Cycling '74*, *Max/MSP* [Man16] is a proprietary visual programming language designed for real-time audio and multimedia processing. *Max/MSP* allows users to create complex audio processing chains by connecting graphical objects that represent various audio functions. It is widely used in live performance, sound installation art, and experimental music due to its flexibility and real-time capabilities.

PureData. Created by Miller Puckette, the same developer behind *Max*, *Pure Data* (Pd) [P⁺96] is an open-source visual programming language. *Pd* shares many similarities with *Max/MSP* but emphasizes accessibility and extensibility through its open-source model. *Pure Data* is widely used in academic research, interactive installations, and DIY audio projects, often in contexts where cost and customization are crucial. *libpd* [BKL⁺11] is an embedded version of *PureData* that generates audio libraries compatible with *Objective-C*, *Python*, *Android*, and *iOS*.

Reaktor. *Reaktor* by *Native Instruments* is a proprietary visual programming environment for building custom virtual instruments, effects, and sound design tools. It combines modular synthesis with a flexible graphical interface, allowing users to create both simple and complex audio processing structures.

3.1.4 Transcompiler Languages

A transcompiler (or *transpiler*) is a type of compiler that takes source code written in one programming language and converts it into source code in another language that is at a similar level of abstraction. This differs from a traditional compiler, which typically translates high-level code into lower-level code or machine code. For audio, most transcompilers generate C++ code from high-level abstraction languages.

CMajor. *CMajor*, based on *Soul*, is a domain-specific language designed to generate C++ code for audio applications. It allows developers to write audio processing code in a more abstract, high-level language. This approach combines the ease of use of a DSL with the performance benefits of C++, making it an attractive option for developers who need to balance productivity with performance.

FAUST FAUST [Grad, OFL04] is a functional programming language for sound synthesis and audio processing with a strong focus on the design of synthesizers, musical instruments, audio effects, etc. The strength of FAUST lie in his compiler that can generate optimized code for a variety of languages (C++, C, Java, JavaScript, asm.js, LLVM, WebAssembly [LOF18], Web audio plugins [RLO⁺20], etc.) from a given FAUST program. It also provides libraries that implement hundreds of algorithms for sound synthesis and processing.

FAUST's efficient compilation makes it a high-level alternative to C/C++ for developing high performance signal processing applications, libraries, and audio plug-ins. This language is highly adapted for embedded audio processing as it provides native programming support for most embedded platforms [MOL⁺20a].

3.2 Platform for Real-Time Audio Processing

The implementation of a real-time audio system can use either a software-programmable processor, in which functional blocks are developed in software: CPUs, Digital Signal Processor chips,¹ GPUs, microcontrollers. Or use a hardware processor, in which functional blocks are implemented with logic blocks: FPGAs, ASICs.

3.2.1 CPU and Digital Signal Processor Chip

Real-time audio processing has been extensively studied and optimized for software-based systems such as CPUs over the past decades [JAS07, PPC⁺07, WB15]. Significant improvements in the computational power of modern CPUs now enable most real-time audio processing to be performed on personal computers. Dedicated hardware can now be convincingly replaced by DAWs, and the use of hardware circuits has become specific to certain applications. Recently, CPUs have been highly studied for neural network implementation [MOT⁺21, SSTS20], where embedded platforms are not easily suitable.

However, there is still an important need for dedicated hardware solutions for better parallelization capabilities, low power consumption, or integration reasons. A Digital Signal Processor chip is a specialized microprocessor designed for efficient real-time digital signal processing, particularly in applications such as audio, image processing, and telecommunications. Unlike general-purpose CPUs, Digital Signal Processors feature optimizations such as specialized instruction sets for fast Multiply-Accumulate (MAC) operations (which are fundamental to filtering, convolution, and Fourier transforms), hardware loops that eliminate instruction cycle delays, and SIMD (Single Instruction, Multiple Data) capabilities for parallel computation. They also employ tightly coupled memory architectures to minimize access latency and avoid bottlenecks. These enhancements enable Digital Signal Processors to perform intensive mathematical computations with high efficiency, making them well-suited for tasks requiring real-time performance and low power consumption. Although Digital Signal Processors are often referred to as hardware implementations [Mol09, ZHA⁺13] due to these architectural optimizations, algorithm development on a DSP is still software-based rather than relying on custom circuit design, as is the case with FPGAs or ASICs.

Since the 2000s, using a Digital Signal Processor chips has often been considered a favorable solution for achieving an application-specific hardware component with better computing efficiency than CPUs [PSK⁺03, CT98], and implementation optimization for musical applications has been studied during this period [SSK01, SSK02]. An early exploration in 2001 [MSAA01] implemented a full suite of professional audio effects on a Digital Signal Processor. They demonstrated that real-time audio processing implementation is very efficient due to the high performances and flexibility of the chip. They used a standalone processor that handled only digital computations, requiring external chips (such as audio codecs) to build a complete audio system. More recently, [BKPE09] used a SoC with an embedded Digital Signal Processor (eDSP). The audio effect SoC integrates a Digital Signal Processor core and many peripherals (PLLs, ADC, audio codec, host

¹Digital Signal Processing and Digital Signal Processor share the same acronym. To avoid confusion, in this thesis, a Digital Signal Processor is always referred to by its full name (without using the acronym), while Digital Signal Processing is referred to as DSP algorithms or simply DSP.

interface, memory, etc.), allowing them to implement a full audio system on a single chip using hardware/software co-design.

Optimizing implementation is still being studied today [BAC17, SAR⁺21]. However, the recent progress in the accessibility of designing hardware circuits has attracted new research into “true” hardware implementation on FPGA.

3.2.2 Embedded Systems

The previous section focused on DSP implementations on computers running General Purpose Operating Systems (GPOSs) (*Windows, MacOS, Linux, etc.*), or specialized Digital Signal Processor. This section discusses embedded CPUs, typically microcontrollers and single-board computers.

Embedded systems are often more practical than traditional computers or laptops when the device must be integrated into a specific context, whether for artistic purposes or operational requirements. Embedded CPU real-time audio processing has therefore been extensively studied for platforms such as embedded musical instruments. Many programming platforms with custom boards and software frameworks have emerged, including the OWL stage effects pedal embedding an ARM Cortex [WLK14], SPINE based on a CUI32Stem [HW14], the Eurorack platform based on FAUST embedding a Daisy Seed board [DL22], the Satellite CCRMA platform using a Beagle-Board/Raspberry Pi with a connected microcontroller [BJ17], and the Faselunare Microcosmos music development board based on a Teensy board [Fas] to name a few.

Single Board Computer

Open source *Linux*-based platforms like the *Raspberry Pi* and the *BeagleBone* are frequently used for audio applications due to their embedded GPOSs, which offer flexibility in using popular audio development software and drivers. However, these GPOSs are not optimized for low-latency applications and can introduce audio latencies on the order of tens of milliseconds. For instance, this implementation [DBV19] of deep neural networks (DNNs) on *Raspberry Pi 3 Model B+* yield latencies around 16 ms. Despite this, these platforms can still be real-time compatible if the timing constraints are not stringent, as demonstrated by the *JackTrip* implementation on *Raspberry Pi* [CO19].

However, the lack of proper audio input/output interfaces and DACs for sensors on these platforms often limits their use in embedded audio applications, requiring external microcontrollers and audio sound cards, which can make the system more expensive and complex. Currently, two relevant platforms address this issue: *Bela* [Bel] and *ELK* [Elk]. In 2015, Andrew McPherson published a first paper [MZ15] on a complete environment (hardware board and software support) designed to achieve submillisecond latency for audio DSP implemented on an embedded CPU. This project involved a custom expansion board for the *BeagleBone Black* single-board computer and used *Xenomai* real-time kernel extensions to achieve latencies under 80 μ s. Subsequently, this work evolved into the *Bela* board [McP17], which is now a commercially available ultra-low-latency real-time audio development platform. The *Bela* board integrates all necessary audio codecs, ADC/DAC converters, a simplified IDE for programming, and numerous sensors for creating interfaces for musical interactions. *Elk Audio OS* [TF21] is a real-time operating system for embedded audio. Based on *Linux Xenomai*, *Elk Audio OS* can achieve latencies below 1 ms, enabling platforms like *Raspberry Pi* to achieve ultra-low latency for audio processing. *Elk* has also developed an advanced *Pi* audio daughterboard to facilitate seamless control and interface with audio systems [Elk], although this platform appears to be discontinued. Using a specific real-time operating system like *Xenomai* can indeed provide better timing constraints but may make the system less flexible and more complex to set up. In contrast, microcontroller platforms such as *Teensy* and *Arduino* can offer hard real-time constraints with ease of use, but have more limited

processing power.

Microcontroller

The open-source *Teensy* board is highly popular in the research and DIY audio community [SUL18, Rus23, MLN22] due to its facilitation of real-time audio DSP implementations. It features an *ARM Cortex-M4* microcontroller and is often paired with *PJRC's* audio shield [pjra] equipped with an *SGTL5000* audio codec, providing 16-bit 44.1kHz stereo audio inputs and outputs. Recent work [MOLF19] has demonstrated that the *Teensy* board is suitable for low-latency audio applications, achieving latency of around 1 ms with an 8-sample buffer. *Cortex-M* CPUs are frequently used for real-time audio processing, with *STM32* microcontrollers being a common choice for such applications [MPSD20], especially in the industry. The *ESP32* platform is also frequently used [PW23, MOL⁺20b] for embedded audio, particularly because of its embedded Wi-Fi and Bluetooth connectivity capabilities. Although the well-known *Arduino* microcontroller has been used for real-time audio implementations [BQ13], its 8-bit 16MHz microprocessor lags significantly behind the previously mentioned platforms in terms of computational power and is not suitable for professional applications.

3.2.3 GPU

Audio signal processing on GPUs has emerged relatively recently, with significant contributions beginning in the 2010s. Savioja et al. [SVS11] demonstrated substantial performance improvements over CPU implementations: 300 times for additive synthesis, 8 times for Fast Fourier Transform (FFT), and 130 times for FIR implementation. With a latency of a hundred sample buffer, this application has shown potential for low-latency capabilities. Trebien et al. [TO09] implemented linear digital filters for synthesizing realistic sounds of colliding objects in real time. Their approach aimed to replace CPU-based techniques that typically use pre-recorded sounds, requiring substantial memory resources. Other studies [LCZ⁺12, GLMC17] also highlight the relevance of GPUs for audio signal processing applications, leveraging their high potential for parallelism (filtering with numerous coefficients, signals, and channels).

A few years ago, Belloch et al. studied extensively the use of GPUs for audio processing. Beginning with massive convolution [BGMV11], they demonstrated that GPUs were a promising alternative to CPUs for highly parallel processing and multichannel audio. In their work on Infinite Impulse Response (IIR) filtering [BBS⁺14], they achieved running 1256 concurrent IIR filters of order 256 with latency under a millisecond. They also explored sound spatialization techniques by implementing a WFS algorithm for 96 loudspeakers [BGQ⁺17]. During the same period, other research teams showed interest in implementing multichannel real-time audio DSP, such as Active Noise Control (ANC) algorithms [LFdG14], and sound field renderings with 157 loudspeakers [TIO13]. Hamilton et al. used GPUs for room acoustics modeling [HW13] and 3D acoustic simulations [HBW14], while Bilbao et al. applied GPUs for physical modeling [BW13], leading to GPU-accelerated 3D simulations of the acoustic space around a Timpani drum [BW12]. However, it is important to note that while these applications are impressive, not all of them achieve real-time performance.

The most recent GPU research work can be attributed to T. Skare, who explored GPU-accelerated modal synthesis [SA19b, SA19a], leading to the development of a highly efficient drum kit synthesis plug-in [Ska22].

3.3 Audio Processing on FPGA

3.3.1 Audio Hardware Design

As discussed earlier, while Digital Signal Processor chips implementations are sometimes referred to as hardware implementations, the term “hardware implementation” more accurately denotes FPGA or ASIC implementations of algorithms as hardware circuits. Hardware implementation is particularly compelling for audio DSP due to its potential for low latency and parallelization.

Numerous audio projects have leveraged FPGA technology, such as FFT visualizers [ZWZ14], music synthesizers [ZZC], digital drum kits [JSP20], and audio effect generators [CPG⁺15]. Pfeifle et al. [PB16] presented an advanced implementation of real-time finite-difference method physical modeling of musical instruments on FPGA. More recently, an FPGA-based accelerator for sound field rendering was implemented in [TI19]. Although the FPGA implementation doubled the performances compared to a PC, the implementation was still not suited for real-time applications. An interesting application in blind sailing implemented on the *Xilinx Zynq Z-7020* SoC was proposed by Singhani and Morrow [SM20]. They leverage hardware/software co-design to accelerate Head-Related Transfer Function (HRTF) calculations in the FPGA fabric while handling file storage and non-critical computations on the CPU, enabling low-latency real-time applications.

These papers collectively highlight the intricate nature of FPGA design, emphasizing the necessity for research publications that focus on the implementation of specific applications. Efficient implementations require detailed analysis and optimization of each part of the algorithm [DASP21], and optimizing each functions [LVVN20]. Design methodologies have been developed to streamline implementation and reduce development time, but they are application-specific. For instance, Erdem et al. [MWBM07] presented a methodology for designing finite difference (FD) schemes on FPGA with real-time performance considerations.

FPGA System on Chip

While FPGAs are sometimes used standalone, most audio processing systems require control, flexibility, and interface compatibility that is better served by software-based CPU implementations. Modern FPGAs often integrate a full CPU architecture, allowing for both hardware and software implementations. Referred to as FPGA SoC, these devices also embed drivers and a variety of peripherals, facilitating the development of complete systems on a single chip [Sni23]. The advent of SoCs has greatly facilitated the integration of audio processing by providing complete platforms capable of handling both control and processing tasks. In 2007, Pfaff et al. [PMS⁺07] used an FPGA embedded within an SoC to implement digital audio effects, diverging from the prevailing trend of employing software implementations for audio effects at this time. Their hardware/software co-design enabled real-time processing on dedicated hardware with software handling parameterization. They demonstrated that this dual implementation approach offers improved performances in terms of latency and throughput, while retaining the flexibility of a software-based system. Taking the example of the FFT visualizer [ZWZ14] mentioned earlier and adopting a hardware/software co-architecture, Jingbo Zhang et al. improved their application [JGS15]. Wegener et al. further emphasized the benefits of co-design by using an embedded *Linux* on the processor, enabling widely used programming environments like *PureData* to directly control the DSP implemented on the FPGA fabric [WSN22].

By proposing modular block systems for circuit implementation, SoCs allow for incremental integration, enabling the reusing of control components and upgrading processing blocks [CFS22]. This approach has made accessible the concept of creating reusable and scalable implementations for various audio processing algorithms [VJY19]. Templates or frameworks can be developed for specific audio needs in both hardware and software components, necessitating only minor adjustments to the architecture to implement different audio applications, thus enabling multiple

applications from the same design model. The accessibility of evaluation boards, which provide all the necessary peripherals and connectivity for using FPGAs and SoCs, has greatly facilitated hardware design and encouraged the audio community to explore this technology [DK20].

Custom Architectures

Most of the existing research involves manually designing hardware/software implementations for specific audio processes. While a few custom architectures for a more generic usage have emerged, none have been actively continued. Verstraelen proposed an innovative dedicated architecture of a programmable parallel machine synthesized on FPGA in 2014, which could be programmed from data-flow graphs of audio applications [VKS14], but this project is no longer active.

In 2023, Bert et al. [BDP⁺23] proposed the design of an Application-Specific Instruction Set Processor (ASIP) implemented using Transport Triggered Architecture (TTA). This approach combines software and FPGA-based hardware to reduce latency in the context of Networked Music Performance (NMP). Their preliminary work demonstrated sub-millisecond analog-to-analog latency (0.859 ms) in an FPGA local loopback test, indicating that this architecture can be adapted for ultra-low-latency applications. However, this work is not yet mature enough to be compared with a complete hardware/software system.

Comparison of CPU, GPU, and FPGA

In 2011, Theodoropoulos compared the implementation of two immersive-audio technologies (Beamforming and Wave Field Synthesis) on GPU and FPGA against a standard CPU implementation [TKG11]. They concluded that, at the time, mid-range GPUs offered similar performance to mid-range CPUs, but high-end GPUs and FPGAs could provide better performance for highly parallel audio processing systems. No recent comparisons between these platforms were found.

3.3.2 FPGA in Audio Industry

As achieving ultra-low latency in audio processing has predominantly relied on FPGAs or ASICs, some industrial audio devices have been relying on them for a couple of years now. Previously, creating such systems required advanced skills in microelectronics and posed challenges for audio DSP engineers and programmers. However, the increased accessibility of hardware design techniques has made them more widely used now.

In the industry, there is a growing adoption of FPGAs for real-time audio signal processing tasks. For example, *Allen & Heath* uses their “XCVI 96kHz FPGA engine” in their digital mixers [Heab] to achieve sub-millisecond latency, ensure phase coherency, support variable bit depths (up to 96-bit), enable routing capabilities through parallel processing, and operate at a high sampling rate of 96kHz [Heaa]. *Seeburg* also utilizes an FPGA as a DSP in their speakers, although specific details about its application are not fully disclosed [See]. *Antelope* products [Ant] incorporate a proprietary “Synergy Core” [Aud] platform with Digital Signal Processor and FPGA chips, leveraging FPGAs for their computational power to simulate non-linear analog circuits. In the case of the *UDO* Keyboard [UDO] and the *Novation Summit* Keyboard [Nov], FPGAs are used to implement digital oscillators operating at a very high audio sampling rate (approximately 24MHz), aiming to approximate analog behavior effectively. Finally, in their *X1* and *X2* matrix array audio modules, consisting of around a thousand speakers, *Holoplot* [Hol] uses a *Zynq 7045* SoC FPGA to compute a large number of IIR and FIR filters in real-time, in order to implement powerful WFS algorithms [Sta24].

This is a non-exhaustive list of examples of significant product embedding FPGAs. Several other products, such as *Tascam* Digital Mixers, *Lynx* PCI Express Audio Cards, *Waldorf Kyra* Synthesiz-

ers, *Schlappi Three Body* eurorack module, RME Converters, and probably many others without communication on it, use FPGAs. While reverse engineering these products is challenging, it appears that FPGAs are predominantly used in these contexts for their computational power and parallel capabilities, particularly to facilitate signal mixing.

futur3soundz offers development boards embedding FPGAs (such as *Xilinx Spartan-6* or *Xilinx Artix-7*) with audio DACs for DIY polyphonic synthesizers [fut]. Unlike previous consumer-grade products, this platform targets the development and maker communities. As far as our knowledge extends, it seems to be the only commercially available FPGA-based platform specifically designed for audio development.

Hardware boards implementing the *Dante* protocol [Dana], widely used in concert halls, stages, and other venues for transmitting audio over Ethernet on local networks [PCW⁺18], typically rely on FPGAs. For example, the *DANTE-MY16-AUD2* 16-Channel Dante Network I/O Card uses a *Xilinx Spartan 6* FPGA. In this specific application, the FPGA's power is harnessed to interface with multiple audio codec chips in parallel and manage a large number of audio channels efficiently [Danb].

3.3.3 FPGA Programming and Domain-Specific Compilation

The traditional approach for programming FPGAs involves using a Hardware Description Language (HDL) like *VHDL* or *Verilog*. Advancements in hardware design approaches have enabled designers to effectively decrease the time required for hardware design and simplify the programming of FPGAs using ().

Domain Specific HLS and Design Space Exploration

HLS is an automated process that translates algorithmic descriptions written in languages like C or C++ into hardware descriptions for implementation on FPGAs or ASICs (See Section 4.1.4 for more details.) HLS tools raise the abstraction level by automating tasks such as pipeline scheduling and resource allocation. However, designers must still design an effective architecture in their program to achieve high-performance implementations, a task that demands hardware expertise and thorough design space exploration [SW20].

To address this challenge, frameworks can restrict the target to a specific domain, such as Convolutional Neural Network (CNN) [VB16, CHZ⁺19], IoT [ZCDC15, ZRZ⁺17], or image processing. For the latter domain, many frameworks facilitate the use of HLS tools in conjunction with DSLs, such as HIPAcc [RSH⁺14], Halide [PBY⁺17], and Polymage [CVPB16]. These tools allow programmers to benefit from the high-level abstraction of software programming languages while avoiding the extensive code optimization typically required by HLS. However, there is no equivalent specifically designed for audio.

One of the advantages of HLS over traditional RTL design flows is the ability to easily generate multiple microarchitectures. This enables trade-offs between resource usage and performance by adjusting various synthesis options, thereby facilitating the exploration of the design space. Domain specific HLS helps the design space exploration and facilitates the creation of templates for automatic hardware generation. Consequently, it enables the development of parametric designs. For instance, Bournias et al. [BCL22, Bou23] leveraged HLS to design a multiscale parametric optical flow implementation and conducted design space exploration to optimize performance, resulting in a reusable and efficient algorithm.

Gianluca et al. [DSPS16] proposed a DSL based on *Scala* that can help in code optimization for HLS, subsequently generating hardware designs and reducing the effort for designers. *Spatial*, a high-level programming language and compiler developed at Stanford [KFP⁺18, ZRV⁺19], allows

for writing hardware-optimized applications by leveraging *Chisel* HDL. It provides hardware-specific abstractions for memory, control, and design implementation, simplifying the accelerator design process. *Spatial* can target various FPGAs (*Xilinx* and *Altera*) and does not rely on vendor-specific HLS tools. A relatively exhaustive list of HLS tools can be found in a 2015 survey [NSP⁺16]. An example of an application-specific HLS for signal processing is *GAUT* [Cou19], which allow for the rapid prototyping of DSP applications [TCHM09]. Unfortunately, it seems to be discontinued as the latest information is from a few years ago and their website is no longer available.

HLS for Hardware/Software Co-design on SoC

With the advent of SoCs integrated into FPGAs and the development of HLS tools, some works have proposed compiling high-level languages for both FPGA and CPU. Ordaz et al. [OK17] leveraged HLS tools to optimize software implementations with hardware functions without the need to write any HDL code. Pauget et al. [PPP18] attempted to compile the imperative language *Whiley* directly for FPGA. *FLOWER* [APM⁺21] is a toolchain for image processing that allows programmers to write their applications in a high-level library, automatically performing code optimizations for HLS. More recently, Chung et al. [CA22] proposed a complete compilation flow for generating CNN inference accelerators on FPGA without requiring hardware design expertise. However, similar tools for audio DSP do not exist.

High-Level Audio Compilation

The work by Vannoy et al. [VDD⁺19, Van20] represents one of the most detailed and advanced audio signal processing projects on FPGA using HLS tools. They use *MathWorks HDL Coder* and an *Intel Cyclone V* FPGA. Their complete design is available on GitHub, and they provide additional utilities such as a *Linux* sound card driver or a web client for control interface.

However, aside from our work, there appears to be no existing research proposing to specialize or adapt HLS tools for the field of real-time audio DSP. There is no audio-specific DSL that directly compiles high-level programming languages into FPGA implementations, nor is there a compilation flow that enables straightforward integration of FPGA with audio-specific languages.

3.4 Real-Time Audio Processing Challenges

FPGAs have emerged as a powerful platform for audio processing by offering significant advantages in terms of performances for latency, parallelism, and multichannel capabilities, enabling new levels of efficiency and functionality in both professional and consumer audio devices.

3.4.1 Audio Latency

Latency characteristics and requirements are discussed in Chapter 2.5. This section examines the current latency achieved in audio implementations on embedded systems.

Latency on Software-Based Audio Systems

Latency in general-purpose operating systems can vary greatly depending on the operating system, drivers, and communication systems [MSKW24]. In 2010, Wang et al. [WSR10] tested audio latency across different general-purpose OSs using onboard sound cards in personal computers. They found significant latency variations, ranging from 3 ms on *Linux* with ALSA drivers to 80 ms on *Windows XP*. Topliss et al. [TZM⁺14] evaluated latency performance on a *BeagleBone* with the default *Linux* distribution (*Angstrom*) and various configurations. They achieved latencies under

3 ms with the *AUDIO Cape BeagleBone* interface, but observed latencies exceeding 12 ms with a USB audio interface. Wang's PhD thesis [Wan18] provides a comprehensive review of potential sources of delay in digital audio systems. On general-purpose operating systems, latency primarily stems from OS buffering. A recent study by Vignati et al. [VZT22] compared the *Xenomai* real-time *Linux* kernel with the *PREEMPT RT* patch, another approach to real-time *Linux*. Results demonstrated that *Xenomai* achieves superior latency performance, providing approximately 1 ms for audio round-trip latency.

It is important to note that latency induced by buffering depends on the platform but may also depend on the application's computational demands. While smaller buffers generally reduce latency, some systems may require larger buffers to prevent underruns when computation time is high or the operating system introduces significant overhead. However, on platforms with fast scheduling and low overhead (such as *Bela* [Bel]), even demanding algorithms can often run reliably with small buffers. Consequently, buffer size decisions must consider both performance requirements and the characteristics of the target environment.

Audio Latency on FPGA

One of the primary advantages of using FPGAs for audio processing is their inherent capability to achieve low latency. When implementing audio DSPs on hardware, the main source of latency arises from the ADC/DAC (group delay), as there is no operating system involved in processing the sample stream. Thus, an FPGA can achieve a latency as low as one sample (the synchronous nature of the serial communication with the audio codec induces this one sample latency). Buffering can be implemented to handle multiple samples and efficiently manage heavy algorithms by processing many samples at once. However, the latency introduced by buffering remains significantly lower than the latency induced by operating systems.

There have been very few efforts aimed at implementing low-latency audio systems on FPGAs. Verstraelen et al. [VKS14] achieved a latency of only a few samples on their custom architecture. However, this latency does not include the group delay introduced by the audio codec. In their implementation of real-time spatial 3D audio synthesis for a blind sailing application, Singhanil et al. [SM20] achieved 1 ms latency for sound synthesis. This latency was mainly due to buffering the Inter-IC Sound (I²S) samples to 20 audio samples.

LowLAG [ES20] is a sound effect implementation on a *Zynq*-based programmable SoC platform. They employ a hardware/software co-design methodology to implement audio effects with lower latency compared to a full CPU implementation (the same effect running on *MATLAB* and *Simulink* on a PC). Using single-sample processing, they report a latency of 1 μ s, which corresponds to the hardware audio core processing time and is highly dependent on the specific audio effect implemented. This reported latency does not include the latency introduced by the audio interface components (such as the I²S transceiver, audio codec, AXI bus, etc.), and therefore, cannot be directly compared to complete system latencies, which, even with low sample processing latencies, can add thousands of microseconds of delay due to the audio sampling rate. Nevertheless, they have developed a low-latency implementation using reusable audio processing blocks and demonstrated that automatically generated processing blocks (via *HDL Coder* in their case) can achieve efficient latency. They further refined their approach in a subsequent study [ES21], achieving improved latency by implementing pipelining through *MATLAB* and optimizing the design. However, similar to their earlier work, their method for measuring latency heavily relies on the specific audio effect and does not account for the total system latency.

Vannoy et al. [VDD⁺19, Van20], using model-based design, achieved a latency of 180 μ s analog-to-analog for the full system, including the audio codec and communication to the hardware audio process. Among the works mentioned, this represents the smallest reported latency for FPGA processing in a complete system.

3.4.2 Multichannel

Audio Spatialization

In the fields of active acoustic control and audio spatialization, using a large number of speakers is usually necessary to obtain optimal results. Some sound spatialization techniques such as WFS [BdV93, Ahr12] can imply the use of thousands of speakers in specialized concert halls. The usual approach in handling real-time audio signal processing systems with such requirements typically involves employing a CPU alongside a multichannel audio interface, requiring a large quantity of resources, leading to a very high cost. For real-time applications, even modern CPUs struggle to compute thousands of channels in time, but GPUs seem to be a better fit for these applications.

Multichannel on GPU

Belloch et. al made significant advancements in multichannel audio processing on GPUs. They achieved real-time massive convolution on 16 channel [BGMV11] paving the way for real-time 3D spatial sound on GPUs. They also implemented a distributed version of a WFS algorithm using multiple *Jetson Nano* GPU-based SoCs [BBL⁺21]. While scalable, this approach highlights the complexity of managing multichannel systems, requiring in this case a new SoC-based board for each pair of speaker, and therefore a robust synchronization protocol. Furthermore, their implementation of a WFS algorithm using 96 speakers on a single GPU [BGQ⁺17] demonstrates that a centralized approach is feasible, but this application does not meet real-time requirements. This poses a significant barrier for spatial audio, where real-time applications are prevalent. Latency remains a major technical constraint. This example also underscores the challenges of interfacing a large number of channels with an embedded system, as they employed multiple audio interfaces (four *MOTU 24I/O* audio cards), resulting in increased costs and resource utilization.

In their real-time implementation of ANC on GPU, Lorente et al. [LFdG14] achieved processing capabilities exceeding 600 channels in the best case. However, testing under real-world conditions involved only a few speakers due to the limited availability of massive multichannel interfaces (they used only a single 24-channel audio card).

Multichannel Interfaces

The availability of audio interfaces offering more than 32 audio outputs is limited and often involves high costs. Furthermore, existing external audio interfaces introduce significant latency due to buffering, adding a challenge for real-time applications. Reussner et al. [RSR⁺13] presented a massive multichannel system comprising 640 loudspeakers connected via Ethernet audio interfaces in parallel, forming an extensive audio network managed by a PC.

Some specialized concert halls around the world are equipped to accommodate systems with thousands of speakers. For instance, the theater at TU Berlin in Germany [TB] hosts a system with 2700 speakers capable of synthesizing up to 42 sources. This setup is controlled by 15 individual *Linux* nodes, managing 832 audio channels via MADI fiber and MADI/ADAT interfaces. A massive WFS system with 558 individually addressable loudspeakers has been constructed at the Experimental Media and Performing Arts Center at Rensselaer Polytechnic Institute (USA) [Goe19]. The system consists of 18 modules, each containing 31 speakers. Each module embeds two 16-channel DANTE amplifiers, allowing for Ethernet communication for audio transmission. The processing algorithms are executed by several high-end CPUs (Apple MacPros in one of the application). In France, IRCAM's espace de projection [NCW12] is equipped with a sound spatialization system designed for artistic and scientific research. It features a belt of 264 speakers evenly distributed around the stage and audience for Wave Field Synthesis (WFS) diffusion, along with a dome of 75 speakers for three-dimensional Ambisonics diffusion. The 264-channel system is

managed using 6 MADI interfaces, each capable of handling 64 channels, and is operated by powerful CPUs. These custom-made systems involve extremely high costs and complex installations, requiring multiple PCs with several audio interfaces.

There are commercial WFS or equivalent systems available involving many speakers, such as *L-ISA* [LA], *Sonic Wave I* from *Sonic Emotion* [Son], and *TTa's Stagetracker FX* [Tot], offering up to 32, 64, or more outputs with a latency typically around 5 ms. These systems generally use *Dante* or MADI communication protocols, although their high costs often make them prohibitive. *Holoplot* [Hol] offers 96-speaker matrix array audio modules that integrate an FPGA, and equips the Las Vegas MSG Sphere with 1 578 modules, resulting in a 152 000-speaker system.

In their implementation of active acoustic control using a 30-microphone sphere, Epain et al. [EF07] employed *LMA COMPARS* controllers [MRW95], specialized multichannel, multiprocessor active noise control devices.

FPGA Parallelization Capabilities for Multichannel

FPGAs are well-suited for multichannel audio processing due to their architecture, which facilitates the independent processing of multiple audio channels without the performance bottlenecks common in sequential processing on traditional CPUs. The high degree of parallelization of FPGAs and their abundance of General-Purpose Input/Output (GPIO) pins enables the use of a large number of digital audio channels, making them especially interesting for sound spatialization and acoustic control techniques. Existing examples have demonstrated handling up to 64 speakers [TKG11]. The use of an FPGA can also reduce system expenses by allowing all audio channels to be managed by a single embedded system and by providing enough resources to compute audio samples at a high sampling rate.

There are relatively few multichannel implementations on FPGA documented in the literature, likely attributable to the challenges posed by the cost and complexity of multichannel interfaces as mentioned earlier. Some practical implementations of multichannel ANC on FPGA resort to using expensive DAC/ADC converters. In their implementation [SGHL20], Shi et al. used a €12 000 *NI 9039 Compact RIO* controller, embedding a *Xilinx Kintex-7* FPGA and *NI-9220 ADC/NI-9264 DAC* 16-bit modules (few thousand euros each), to implement a 24-channel Least Mean Squares (LMS) algorithm. Many papers present new FPGA ANC implementations with a high number of channels, showcasing the potential of FPGA technology in this research field. However, the complexity and expenses associated with the interfaces often lead researchers to synthesize their implementations with numerous channels but only test on a reduced number of channels [KKTS21].

3.5 Synthesis and Insights

FPGAs are powerful platforms for real-time audio processing, providing unmatched latency, the ability to manage numerous audio inputs/outputs, and significant computational power. If GPUs implementations generally struggle to meet real-time constraints and CPUs can do so mostly when the embedded operating system is optimized for low-latency operations, FPGAs, by their nature, allow for minimal buffering and parallel implementation, enabling latencies on the order of a single sample. However, their programming complexity limits their usage. While real-time audio processing on CPUs, GPUs, and embedded systems is well-documented, FPGA implementations remain rare.

On the other hand, DSLs have made it easier to implement real-time audio DSP algorithms, as demonstrated by their widespread adoption in the audio community. The current state of the FPGA audio industry reflects a growing interest in these platforms. Advances in FPGA design

methods suggest that simpler programming approaches, compared to traditional complex methods, could be used.

Using HLS could simplify this programming, but very few studies have applied it to real-time audio processing. To date, aside from our work, there is no high-level compilation flow for real time audio processing that incorporates hardware/software co-design and provides a complete workflow from high-level specifications to FPGA and CPU implementation.

4

CHAPTER 4

Hardware Design of Audio DSP

Contents

4.1	Basics of Hardware Design	38
4.1.1	SoC and FPGA	38
4.1.2	IP and Block-Based Architecture	40
4.1.3	Hardware Description Language	40
4.1.4	High Level Synthesis	41
4.1.5	Audio Compilation Flow	42
4.2	Implementation Guidelines	44
4.2.1	FPGA-Centric Design	44
4.2.2	Hardware/Software Partitioning	47
4.2.3	One-Sample and Multi-Sample Strategies	49
4.2.4	Memory Accesses	50
4.3	Interfacing with the Analog Domain: Audio Codecs	53
4.3.1	Audio Serial Interface	53
4.3.2	GPIO Usage of I ² S Protocols	56
4.3.3	Audio Codecs Architecture	57
4.3.4	Ultra-Low Latency Audio Codecs	60
4.4	Hardware and Software Audio Host Control	62
4.4.1	Simple Control with Baremetal	62
4.4.2	Embedded <i>Linux</i> and Interfacing With External World	66
4.5	FPGA Audio Compilation Scheme	66
4.5.1	From High Level DSP to Hardware and Software Components	67
4.5.2	Hardware Integration and User Interface	68
4.5.3	Our Audio DSP Compiler: Syfala	69
4.5.4	Additional Compiler Features	71
4.6	Conclusion	73

The previous chapter highlighted that the few works proposing real-time audio DSP implementations on FPGAs are mostly application-specific and not easily generalizable. This chapter builds on the fundamental principles discussed in Chapter 2 to provide a methodology for effectively implementing audio DSP on FPGA. Based on our research, it details each component of an FPGA audio implementation, from the hardware design of the DSP to the software control, including analog conversion and user interfaces. It also offers rules and guidelines for their implementation.

The guidelines provided in this chapter are designed to be hardware-independent, relying on principles applicable to any FPGA SoC board. They focus on the SoC CPU/FPGA architecture, examine the most common memory types, explore most used audio interfaces for external converters, and discuss the two primary methods of controlling DSP. By doing so, this chapter aims to equip readers with the knowledge to develop an optimized audio compiler for FPGA. The final section synthesizes all these principles and propose a step-by-step audio compilation scheme. The last part of this section presents a practical example of our audio compilation flow, demonstrating the compilation of FAUST audio DSP down to hardware/software co-design implemented on *Xilinx* SoC.

The chapter is organized as follows:

- Section 4.1 explains the basics of FPGA architecture and programming, detailing existing tools and approaches to hardware design.
- Section 4.2 proposes implementation guidelines based on the audio specificities explained in Chapter 2. These guidelines target hardware/software co-design enabled by SoC architectures.
- Section 4.3 focuses on the analog interface, exploring different DAC/ADC architectures and audio codec principles. It includes a study of existing converters to identify the best options for audio processing, targeting high quality and low latency.
- Section 4.4 explains the different approaches for controlling a DSP implemented on an FPGA, either using a lightweight bare-metal approach or a *Linux*-based embedded OS for greater flexibility.
- Section 4.5 presents our proposed compilation flow for an FPGA audio compiler. It details the workflow from high-level DSP algorithms down to FPGA bitstream and CPU implementation. Finally, it provides a practical example based on the FAUST programming language, showcasing the toolchain we developed for this work.

4.1 Basics of Hardware Design

FPGAs are a specific type of embedded platform that differs significantly from general-purpose processors in various aspects. Although they, like other programmable systems, are used to run user-defined applications, their architecture, programming approach, operational characteristics, and capabilities are fundamentally distinct from those of CPUs and GPUs. This section provides a detailed examination of these specificities.

4.1.1 SoC and FPGA

FPGA/CPU Comparison

A Field-Programmable Gate Array (FPGA) is an integrated circuit that can be configured by the user after manufacturing to perform specific logic functions. Details about FPGA architecture are provided in Chapter 5.1

There is a fundamental difference between a CPU and an FPGA in their approach to processing tasks. A CPU is a programmable software device that executes instructions from software applications sequentially. It is designed for versatility and efficiency in running a wide range of tasks by following a pre-defined set of instructions stored in memory. In contrast, the reconfigurable hardware of FPGAs allow for designing parallel and dataflow architectures by physically mapping logic functions. They can execute parallel processing tasks with high efficiency and low latency, making them ideal for applications requiring real-time performance and customization.

While CPUs are designed for general-purpose computing and are not optimized for specialized tasks, FPGAs offer the capability to create hardware-accelerated solutions tailored to specific applications, providing significant advantages in speed and performance for these specialized tasks. Although they are often referred to as “programmed” (including in this thesis), technically, FPGAs are not programmed but configured using HDL.

System on Chip

A wide range of FPGA-based boards, such as the *Digilent Zybo Z7* used in this thesis, uses a System on Chip (SoC) that integrates a CPU architecture on the same chip as the FPGA, along with various peripherals and additional I/O components.

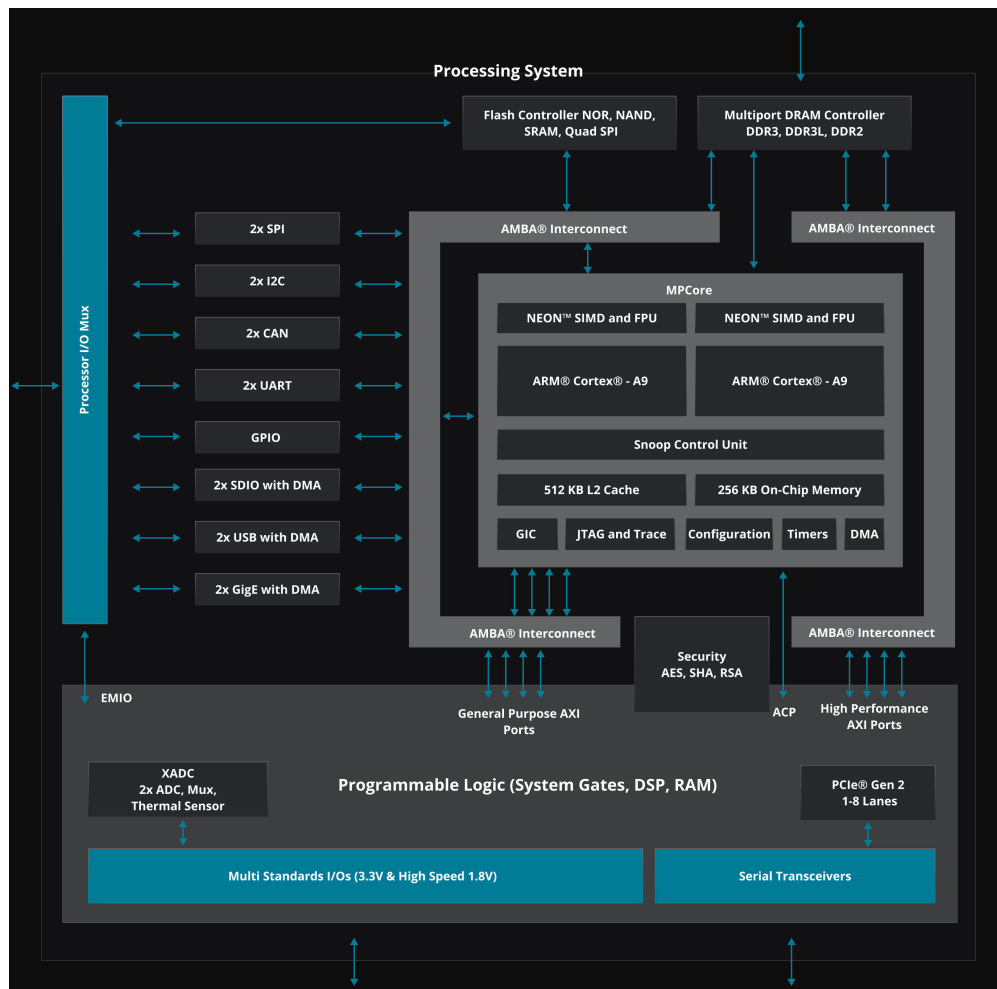


Figure 4.1: Zynq7000 SoC architecture from Xilinx’s website [AMD].

Fig. 4.1 illustrates the architecture of the Zynq-7000 SoC embedded on the Zybo FPGA development board. The dark gray block at the bottom represents the FPGA logic (Programmable Logic), which has direct external connections for general-purpose I/O, ADCs, and serial transmissions.

This FPGA logic communicates with the CPU, represented by the light gray box above, via an Advanced Microcontroller Bus Architecture (AMBA) bus. The CPU is a *dual-core Cortex-A9*, interfacing with the external world through various communication protocols such as SPI, I²C, and UART. Additionally, the CPU has access to external DDR3 Random-Access Memory (RAM) and a flash controller.

This combination of CPU and FPGA allows for a fine-grained partitioning of workloads, with the CPU focusing on sequential, control-oriented tasks, while the FPGA handles compute-intensive, deeply pipelined, massively parallel operations.

4.1.2 IP and Block-Based Architecture

An IP or Semiconductor Intellectual Property core (SIP core) refers to pre-designed and pre-verified hardware logic blocks or modules that can be integrated into an FPGA design. These IP cores serve specific functions or tasks and expedite the development process by providing tested, reusable components. They can be compared to functions in software implementations. However, because they encompass more than just a function (being actual units of logic or integrated circuit layout designs) the term *Intellectual Property* highlights their value as pre-verified, optimized components often developed and tested by experts.

For example, an IP core for Ethernet can handle network communications. Instead of designing this complex functionality from scratch, the Ethernet MAC IP core can be integrated and configured as needed. IPs can be seen as hardware blocks that can be connected together to form a complex SoC architecture. This block-based approach facilitates clear and easy development of hardware designs. *Xilinx* provides a tool called *Vivado IP Integrator*, which allows users to create complex system designs by instantiating and interconnecting IP and HDL modules in a block design.

The concept of IP is crucial to understand as it is largely used in hardware design. For instance, our audio DSP implemented on FPGA is encapsulated within an IP core, which comprises all the necessary computations for audio processing. Throughout this thesis, the term IP or hardware IP is frequently used to refer to the hardware circuit that realizes the desired audio algorithm. IP cores can be created either using high-level synthesis tools or by directly writing in hardware description languages.

4.1.3 Hardware Description Language

The traditional workflow for programming FPGAs involves using HDLs such as *VHDL* [IEE19] and *Verilog* [IEE06]. They allow for a description of the structure and behavior of an electronic circuit into a netlist (a description of the electronic components and connections of an electronic circuit), which can then be placed and routed to create an integrated circuit or configure an FPGA.

While HDLs are not discussed further here, it is important to note that they differ significantly from conventional programming languages. HDLs introduce explicit notations for expressing concurrency and concepts of time (such as timing and scheduling), which can make them more complex to use. Furthermore, *VHDL* and *Verilog* use three distinct types of structures (dataflow, behavioral, and structural), which contribute to a higher level of complexity in code comprehension compared to ordinary programming languages. Even though HDLs are often referred to as programming languages, it is more accurate to speak about specification languages or modeling languages.

Efforts have been made to raise the abstraction level in FPGA programming to reduce complexity. For example, *SystemC* [IEE12], and *Chisel* [Chi], which is based on the *Scala* programming language [Sca], aim to enhance productivity compared to *VHDL* or *Verilog* by embedding HDL in a

software language. However, users still need to write programs at a circuit level with timing constraints. Audio DSP are commonly implemented using high-level programming languages such as *Python* and *MATLAB*, or *C++*. Translating these software-based implementations into a hardware description language is possible thanks to high-level synthesis, but involves extensive and in-depth effort.

4.1.4 High Level Synthesis

() is a design approach that allows for the automatic translation of abstract behavioral specifications into FPGA hardware design that meet the desired performance and functionality requirements. It starts with a high-level description of how a digital system should behave. This description outlines what the system needs to do but does not specify deeply how it should be implemented at the hardware level. The HLS tool then takes this abstract description and automatically converts it into a lower-level hardware design, specifically at the Register-Transfer Level (RTL). The RTL design represents how data moves between registers and how various operations are performed at the hardware level, providing a more detailed blueprint for how the system is physically implemented on a chip.

Compared to traditional HDL, HLS introduces a layer of abstraction that manages timing, scheduling, resource sharing, and memory banking. These tools are more productive than HDLs but still require manual hardware optimization by the user through directives and pragmas. Comparisons between HDL and HLS implementations have been conducted. For instance, Akkad et al. [ALRM⁺] implemented FFT algorithms and found that while HLS implementations may be slightly less optimized at a low level, they can achieve nearly the same results as HDL designs while offering greater flexibility. Tisserand et al. [Arn] demonstrated that HLS is viable for arithmetic and algorithmic exploration, providing comparable results to HDL implementations with code optimization and hardware implementation experience.

HLS tools, or High Level Tools (HLT), can be differentiated into two main categories based on their input sets: those based on programming languages (*C*, *C++*, *SystemC*, etc.) and those based on model-based design (*Simulink*, *LabVIEW*, etc.).

Model-Based Design

In the field of digital signal processing, a common approach to high-level synthesis is through the use of Model-Based Design (MBD) such as *Simulink*. MBD enables FPGA programming by connecting blocks (*MATLAB* functions, *Simulink* models, *Stateflow* diagrams, etc.) to create intricate algorithms. Then, vendor-specific product such as *Xilinx System Generator* [Xilg] or *Intel DSP Builder*, or non-vendor-specific tools like *MathWorks HDL Coder* [Mat], are used to compile the model to FPGA bistreams.

Vendor-dependent HLS tools are optimized for specific hardware platforms and often produce more efficient implementations than *HDL Coder*. However, they require repeating the entire workflow if the target hardware changes to a different vendor's platform, which can be time-consuming and inefficient. A recent in-depth comparison between vendor-specific and vendor-agnostic HLS tools was conducted at Lund University [PC19]. They concluded that while universal tools like *HDL Coder* facilitate FPGA programming, vendor-dependent tools offer better optimization in terms of resource usage and performance.

The model-based design approach is very convenient and widely used, as many engineers in the audio field are already familiar with *Simulink* or *LabVIEW*. A guide on how to implement real-time audio DSP using the *Xilinx System Generator Simulink* library was published in 2021 [DML⁺21]. On the other hand, it has a much less flexible counterpart than a traditional programming language because it obliges the programmer to work with pre-existing components.

Software Programming Languages

The workflow based on programming language input is commonly referred to simply as HLS, although as we have seen, HLS and HLT also encompass MBD. Programming language HLS is now offered by all FPGA vendors as a mean to enhance productivity for hardware designers. It involves compiling a software programming language (typically a restricted syntax of C, C++, or *Python*) down to hardware. Examples of such tools include *Xilinx's Vitis HLS*, *Mentor Graphics' Catapult-C*, and *Intel/Altera's HLS compiler*. The term "compilation" here is somewhat simplistic because HLS transforms a sequential program into a functionally equivalent circuit containing many parallel computations, enabling significantly greater computational power.

Limits of High Level Synthesis

Traditional programming languages such as C++ operate on control flow semantics, as opposed to the data flow semantics used in HDLs. Translating from a high-level control flow semantic sequential language to a hardware description is a complex task that requires providing a substantial amount of information to the compiler. To achieve the best implementation, each computation of the algorithm needs to be directed. Generally, the compiler can automatically handle many implementation optimizations (e.g., loop pipelining, unrolling, memory banking, buffering), but this automatic inference is often not perfect, and the entire program needs to be analyzed by hand and requires explicit user instructions.

To achieve this, the use of libraries and language directives allows the user to provide specifications and information about the hardware constraints to the compiler. This provides control over implementation and allows for optimization of the design architecture, making it significantly more adaptable and robust compared to MBD, although it is less straightforward.

This preprocessing step adds complexity to using a software programming language, as a programmer must pre-optimize the program before giving it to HLS. The high-level software programming abstractions still require the user to apply hardware optimization techniques (not software) and thus have a good understanding of hardware design. This becomes a significant obstacle for non-specialized engineers when using these tools. This is one of the primary challenge with HLS: each program can have numerous different implementations, some very inefficient and some highly optimized. Without the programmer's intervention through directives, the compiler is limited in its ability to infer specific optimizations, as it is designed to be sufficiently generic to implement any program without predefined design space constraints.

This leads to a significant effort in implementing specific designs, as there are no truly generic solutions for automating the implementation process. A key postulate of this thesis is that, by restricting the application of HLS to a specific domain such as audio, it is possible to reduce the design space exploration and achieve a degree of automation in certain aspects.

4.1.5 Audio Compilation Flow

The Problem of Existing Techniques

This thesis advocates for a compilation flow for FPGA-based audio systems, extending beyond HLS. The term *compilation flow* seeks to emphasize the capability to program an FPGA using software-based languages (whether it is domain-specific languages or general-purpose languages like C++), without requiring in-depth hardware design knowledge. None of the previously cited approaches allow for this.

While HLS aims to simplify the design process compared to HDL, it is still not an efficient way to provide a solution to facilitate audio hardware design. Both approaches (Model-Based and programming language-based) have their own limitations. Model-Based design does not offer

enough control to fully address the complexities of audio DSPs. This is primarily because audio DSP treatments, at least those in the time domain, are not standardized enough to take advantage of an IP-based design flow, such as the one proposed in *MathWorks HDL Coder*, for instance. On the other hand, HLS with programming languages provides too much design possibilities and requires the programmer to infer hardware constraints that are not trivial.

Compilation Approach

An audio compilation flow from high-level specifications to hardware design requires the generation of an audio IP that includes the hardware circuit corresponding to the implemented audio DSP. As discussed in Section 4.1.2, this IP can be generated using either HDL or HLS, requiring a decision to be made between these two approaches for the compiler.

Hardware Description. Direct hardware generation with HDL has the advantage of controlling exactly the generated code. Whereas HLS tools often infer many decisions during the synthesis process (such as pipelining, memory partitioning, etc.) that the programmer does not always have control over. Hence, it may seem as a wiser choice to compile directly hardware description given that some language, such as FAUST for instance, are functional and hence exhibit all the possible parallelism in their original program specifications [TDO06]. However, from our experiment, it requires a huge amount of work to generate hardware that is compatible with the resource available on the FPGA for complex functions. First, in many cases, the audio program needs to be partially sequentialized to avoid excessive FPGA resource usage, which is challenging to implement and requires synchronization mechanisms. Second, the implementation of vendor specific bus protocols, such as Advanced eXtensible Interface (AXI) bus for *AMD/Xilinx* platform, is a huge amount of work while it comes for free when using vendors HLS tools. Finally, it appears, even if audio programs can be expressed in a domain specific language, writing a hardware compiler for this DSL would be almost as much work as writing a HLS tool.

High Level Synthesis with Programming Languages. Given that the problem with HLS is providing too much control, it can be reduced by using an abstraction overlay. By focusing exclusively on audio programs, it is possible to minimize the amount of exploring space required and simplify the process of hardware compilation. An audio DSP can have clearly defined characteristics such as sampling frequency, number of input/outputs, memory size, throughput, etc. These specifications allow for setting constraints and providing guidelines to the HLS tool without having to know the precise content of the DSP algorithm itself. Thus, HLS can optimize the design space exploration time, resulting in efficient and optimized IP hardware synthesis. The HLS approach is more adaptable and straightforward to generate hardware IP, even though it requires reliance on existing vendor tools, which are often proprietary.

Streamlining HLS for Audio Applications

Using these tools means that the compiler must “translate” the source audio program, which contains no hardware implementation information, into an HLS-optimized input file. This necessitates a careful analysis of the fundamental concepts of audio DSP discussed in Chapter 2, and the application of these concepts to modify the code structure for the HLS.

When implementing algorithms on FPGAs, trade-offs between throughput, latency, power consumption, and resource usage must be evaluated to meet the requirements of the target application while maximizing system efficiency. The balance between these aspects can be finely tuned using HLS directives, which is discussed in Chapter 5. However, the first step is to effectively write the high-level algorithm to ensure its efficient implementation.

Finally, an audio compilation flow not only focuses on audio IP generation but also involves considerations such as coordinating the overall system including the CPU, managing data transfer between the FPGA and external memory, and interfacing with external audio sources and outputs. The following sections discuss specific techniques, methodologies, and best practices for the compilation of audio DSP specifically apply on FPGA.

4.2 Implementation Guidelines

The computational power of an FPGA is limited by the amount of resources it provides. Unlike a CPU that performs operations sequentially, and thus can theoretically execute an algorithm of any complexity as long as the execution delay allows it, an FPGA can be limited by the number of resources. Therefore, it is essential to optimize these resources to enable the execution of complex algorithms. Although there are many potential optimizations that may be made on the synthesis tool (as discussed in Chapter 5), the very first step for saving resources is to reduce the amount of computations executed on the FPGA.

4.2.1 FPGA-Centric Design

Hardware Acceleration: The Common Approach

In many applications, FPGAs are used alongside a CPU to improve performance by offloading specific computational tasks to dedicated hardware implementations (see Fig. 4.2). This approach, known as hardware acceleration, leverages the parallelism and computational power of FPGA architectures to accelerate functions that may be limited by a traditional CPU implementation. The traditional workflow starts by identifying functions that can be accelerated on the FPGA from a CPU-based algorithm. These functions are generally highly parallelizable and would be a throughput or latency bottleneck on a CPU (such as signal processing, encryption, compression, image processing, etc.).

Traditional CPU Execution:



CPU Execution With FPGA Acceleration:

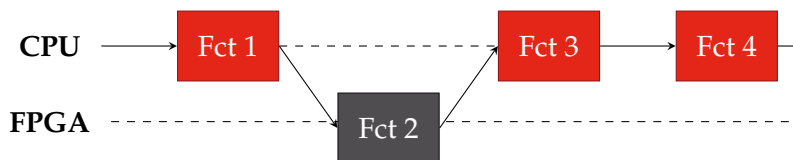


Figure 4.2: Example of an FPGA accelerated function (Fct 2) on a CPU execution.

Then, after evaluating if these functions can be accelerated and meet the desired performances by determining how much parallelism is needed and how resources should be used, they are implemented on the FPGA as an accelerator. This results in a CPU-centric architecture in which the FPGA serves as a peripheral function where variables are passed as arguments and returned to the CPU after being processed. The CPU initiates the program and utilizes the FPGA as needed. The FPGA has its own memory, but most of the interfacing is handled by the CPU. In this scenario, the processor is often referred to as the *Host CPU*.

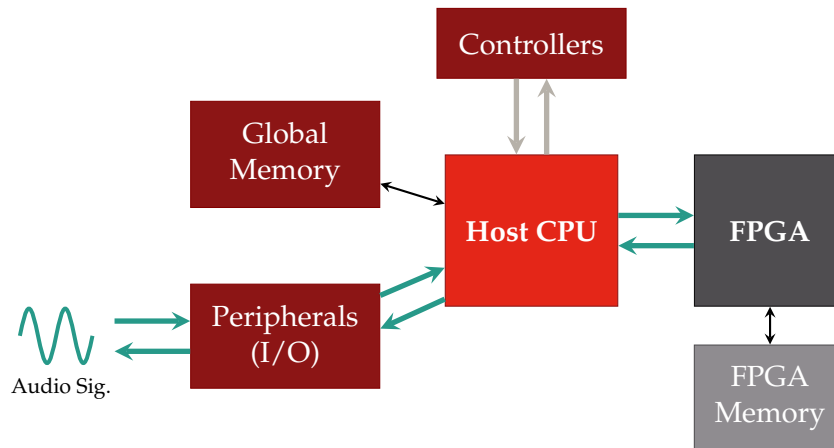


Figure 4.3: CPU-centric architecture. The green arrows represent the audio path, the gray arrows are the control path.

This architecture is illustrated Fig. 4.3. The main characteristic of this approach is that the CPU manages all interfaces with other peripherals. Input and output signals (such as audio signal) are handled by the CPU and then passed to the FPGA. In practice, the processor does not handle each data transfer directly: an AMBA bus is accessible from both the peripherals and the FPGA, facilitating communication. However, the bus is still managed by software.

Our approach to implementing hardware/software partitioning is quite different, especially in the workflow, as it starts with an IP block (hardware function) on FPGA, and offload computation on the CPU with a software “control” function.

Software Control: Our FPGA-Centeric Approach

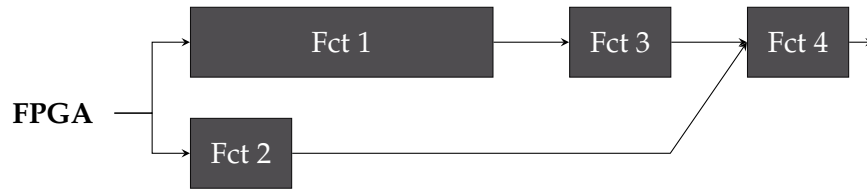
Instead of using the FPGA as a peripheral acceleration function for the CPU, our approach is using the hardware as the main platform for the implementation and the CPU as a peripheral.

Chapter 2.1 highlighted that an audio algorithm can be viewed as a functional entity that receives an infinite stream as input, processes it, and produces an infinite output stream, potentially with some controller and memory. This dataflow processing is highly suited to hardware functional architecture, with no buffering needed, no interrupt and the same processing for all the samples. Therefore, designing the architecture around the CPU may not be ideal, as most (if not all) of the computations are carried out by the hardware.

Our workflow consists of fully implementing the algorithm on the FPGA without relying on any peripheral acceleration functions. Subsequently, the program is analyzed to determine if any computations could be offloaded to the CPU. Fig. 4.4 illustrates this principle. It can be seen that while function processing may take longer when executed on the CPU compared to the FPGA, this does not matter as long as the process is not impacted by the waiting time. However, meticulous attention is required to ensure that *Fct 2* completes before *Fct 4* begins. This is often a cumbersome process, necessitating manual verification of each function’s schedule by the programmer. But in the context of audio programs, due to the asynchronous nature of control rate and sample rate, control functions can be offloaded to the CPU without concerns about scheduling conflicts. The nature of the offloadable functions is discussed in Section 4.2.2.

In contrast to the previous common approach, when computations were offloaded from the CPU when they presented a performance bottleneck and could be parallelized, our current approach involves identifying functions that are not required to be implemented on the FPGA and can be done on the CPU without degrading the performance. Instead of increasing CPU performance

Full FPGA Implementation:



FPGA Implementation with CPU control:

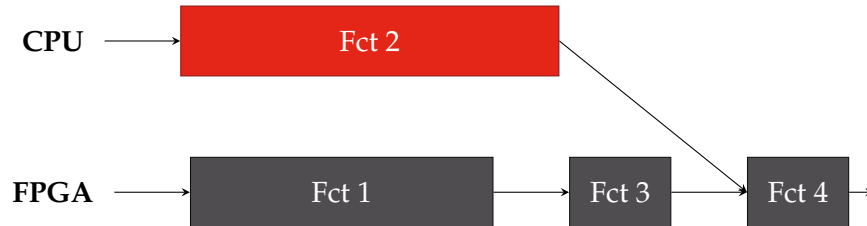


Figure 4.4: Example of FPGA offloading function (Fct 2) on a CPU.

through FPGA use, this reduces FPGA resource usage by leveraging the CPU.

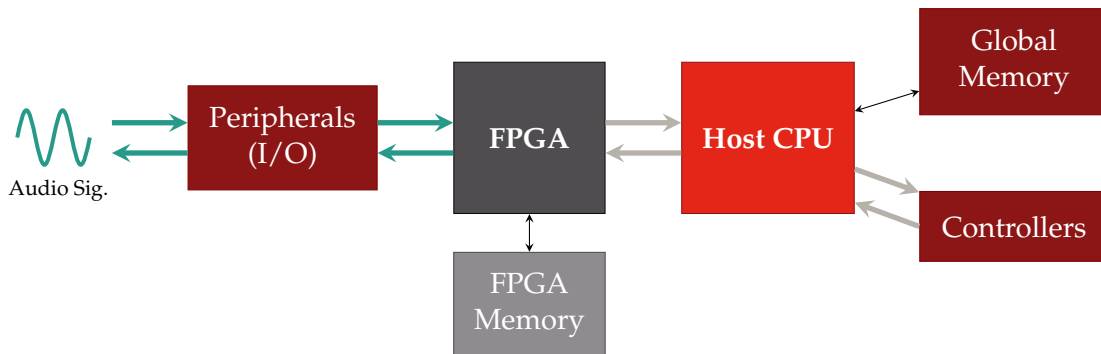


Figure 4.5: FPGA-centric architecture. The green arrows represent the audio path, the gray arrows is the control path.

As a result, the interface with external signals is handled by the FPGA and the hardware function is directly interfaced with the audio signal, the CPU only handles controllers and offloaded computations. Fig. 4.5 illustrates the FPGA-centric approach. In contrast to Fig. 4.3, where audio signal is going through the CPU, the FPGA-centric approach handles audio processing directly, without using a CPU-FPGA interface. This architecture allows for a full control over audio signal, potentially no interface buffers (as the hardware function directly handles the samples), a low latency, and better resource management for a continuous signal such as audio.

▼ Implementation Guideline 4.2.1 — FPGA/CPU Computations

The FPGA should handle the audio interface and perform all critical computations. The CPU should be used to offload non-critical operations.

4.2.2 Hardware/Software Partitioning

The usual approach of FPGA/CPU partitioning, which offloads specific functions of sample processing, necessitates meticulous scheduling analysis to ensure that all critical timing requirements are met and synchronization between the two systems is maintained. This makes it design-specific and unsuitable as a generic guideline, which is the goal of this chapter. A more generic and less constraining splitting method is possible: the control rate/sample rate partitioning. Leveraging the asynchronicity between these two rates allows for distinct implementations of their respective dependent computations without scheduling concerns. Since most audio DSP can be divided into control and sample computations, this guideline is broadly applicable leading to a more general methodology.

Differentiate Control and Samples Computations

Chapter 2.6.1 discussed the distinction between sample rate and control rate. The control rate can be significantly lower than the sample rate, making computations that depend on a control not as tightly time-constrained. This demonstrated that controls are typically updated every 10 to 100 samples. Therefore, all control computations can be handled by the CPU, provided it can process these tasks in a period of less than 100 samples. As long as the CPU respects this time constraint, it ensures that updated controls are ready for the FPGA when needed, therefore ensuring schedule constraints.

Instead of requiring very precise synchronization at the computation/function level, as in Fig. 4.2-4.4, this approach requires synchronization every 10 to 100 samples, which is a much less stringent constraint.

▼ Implementation Guideline 4.2.2 — Sample Rate/Control Rate Dispatching

Sample rate computations should be implemented on the FPGA. Control rate computations should be implemented on the CPU.

It is important to understand that this implies that the CPU can read all the controllers of an audio program, but it also means that any computations that rely on these controllers can be computed on the CPU, as explained in Chapter 2.6.2. To clarify this, let's take an example of a resonant filter implemented using a biquad.

A C function can be written to compute a single sample of a biquad filter as follows:

Lang: C

```

1 float computeBiquad(int frequency){
2     th = 2*PI*frequency/SAMPLE_RATE;
3     c = cosf(th);
4     s = sinf(th);
5
6     int impulse_previous=impulse_actual;
7     float temp0_previous = temp0_actual;
8     float temp1_previous = temp1_actual;
9     temp0_actual = ((s * temp1_previous) + (c * temp0_previous));
10    temp1_actual = (((float)(1 - impulse_previous) + (c * temp1_previous)) - (s *
temp0_previous));
11    float sample = temp1_actual;
12    impulse_actual=1;
13    return sample;
14 }

```

Listing 4.1: Biquad example. Program: biquad.c [The]

This program implements a resonant filter that takes a frequency as a parameter and computes one output sample of the filter, using two global variables as delay of one sample. This example is taken from a biquad implementation in FAUST; it is the `nlf2` function programmed by Julius O. Smith in `filter.lib`, defined in the library as “Second order normalized digital waveguide resonator.” Biquads and their various implementations are detailed in Julius’ book *Introduction to Digital Filters* in Chapter 9, “Implementation Structures for Recursive Digital Filters” (p. 199). A full understanding of the code is not required for what follows. The optimization presented here is intended to be applicable to any program, so a detailed understanding of the internal functions is unnecessary. Only the general structure needs to be grasped. Specifically, it should be noted that this function is divided into two sections. The initial section (lines 2 to 4) involves the computation of the cosine and sine of `th`, which itself depends on the frequency variable and other constants. The second section (lines 6 to 13) involves two steps:

- Duplicate variables in order to store them from one sample to the next one and create a delay.
- Multiply the variable by the previously computed sine and cosine values.

When considering the implementation of this program on an FPGA, it is important to take into account that the initial phase is highly demanding in terms of resources due to the complexity required for implementing the sine and cosine functions. However, it can be observed that this part does not depend on the value of a sample, but only on constants (`PI` and `SAMPLE_RATE`) and the frequency variable. This variable frequency is managed by a controller, and therefore depends on the control rate. Thus, there is no need to recalculate this value for each sample, and it can be computed on the CPU. This allows for computing the values of `c` and `s` on the CPU, as they only depend on `th` and do not need to be computed more frequently.

For the second part, all variables depend on variables from the previous sample (`temp0_previous` and `temp1_previous`) and must therefore be computed at sample rate. This part cannot be implemented on the CPU and must be executed entirely on the FPGA in order to ensure the FPGA’s computational capabilities are fully used for sample computations.

Therefore, it is possible to reduce the computations on the FPGA without impacting the computation of samples or requiring precise scheduling synchronization. By performing all sample rate computations on the FPGA and all control rate computations on the CPU, as depicted in Fig. 4.6, it ensures timely computation of the samples while reducing the computational load on the FPGA from control rate processing. Hence, it is essential to be able to differentiate and separate the computations of the sample rate from the control rate.

▼ Implementation Guideline 4.2.3 — *Identify Control Rate Computations*

Any computations that do not rely on the value of a previous or current sample can be executed at the control rate and thus, executed on the CPU.

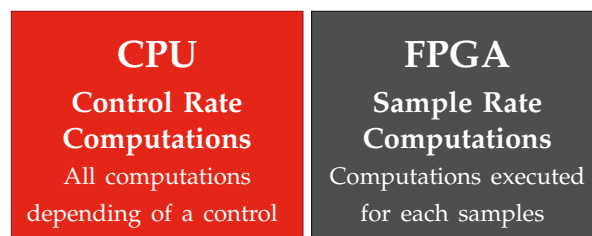


Figure 4.6: *Computations dispatching guideline.*

DSP Initialization

Constants and table initialization, including waveform tables and delays, do not involve sample computations or timing-constrained operations. As a result, these tasks can be performed on the CPU before initiating the DSP. The same principle applies to computations performed only once during initialization. Provided these computations can be done without the DSP being started, they can be processed by the CPU while the FPGA waits, then copied to the FPGA. This implies implementing a mechanism that allows the CPU to give a start signal to the FPGA. Before receiving this signal, the FPGA should not process any samples or read from the shared memory.

The specificity of these values is that they are read throughout the audio process and, therefore, should be stored in memory, as opposed to control values, for example, which are read by the FPGA but usually do not need to be stored. Thus, while control values are sent directly from the CPU to the FPGA, it is inefficient to send initialization values to the FPGA for it to store and later read them. Instead, the CPU can store these values in a shared memory and provide the address to the FPGA, which will read them as needed. Shared memory can be safely used in this context since the CPU initializes values once before the DSP starts, and the FPGA reads them only after receiving the start signal from the CPU, preventing read or write conflicts.

▼ Implementation Guideline 4.2.4 — *DSP Initiation*

Constant initialization and one-time computations can be performed on the CPU before the FPGA audio DSP begins operation and stored on a shared memory (Typ. DDR). Once these tasks are completed, the CPU issues a command to the FPGA to start processing.

4.2.3 One-Sample and Multi-Sample Strategies

As discussed in Chapter 2.6.1, the audio sample rate is typically 48 kHz, with 768 kHz being a common upper limit. Hence, it is much slower than the FPGA clock rate which is typically between 100 and 400 MHz. For instance, if the FPGA clock rate is 125 MHz and the sample rate is 48 kHz, there is a ratio of approximately 2 604 between them. This means that the FPGA has 2 604 clock cycles between the arrival of two successive input samples.

Each elementary function of the FPGA (adder, multiplier, memory access, etc.) has a defined cycle latency. This latency depends on the implementation of the function itself and other parameters

such as pipelining, which is discussed in detail in Chapter 5. Typically, the latency of functions spans only a few cycles, so 2 600 cycles provide ample room for implementing relatively complex designs.

This allows for the one-sample strategy: the IP has enough time to fully process a sample before the next sampling period. Consequently, it processes the samples individually and can output a sample at each tick of the audio clock. The one-sample strategy eliminates the need for buffering, thereby ensuring best latency by guaranteeing a single sample latency. This approach is ideal for applications requiring ultra-low latency.

However, if the processing latency exceed the available FPGA cycles per sample period, the design will be unable to meet the required sampling rate, resulting in missed samples. In such cases, and if optimization techniques like pipelining and parallelization discussed in Chapter 5.2 are insufficient, a multi-sample strategy becomes necessary.

A multi-sample strategy involves buffering multiple samples for simultaneous processing. Although this approach introduces higher latency, it may be the only viable solution for implementing very complex designs. Several implementations for multi-sample processing exist, each of which involves buffering and consequently results in increased latency. Detailed descriptions of these methods can be found in Chapter 5.4.5.

4.2.4 Memory Accesses

Memory Types

FPGAs embed various types of memory to handle different tasks and optimize performance. The following paragraphs provide an overview of the different types of memory typically found on FPGAs, with a focus on access delays, as this will present an important bottleneck for real-time audio applications.

Block RAM (BRAM). Block Random-Access Memory (BRAM) is dedicated memory built into the FPGA fabric. It is suitable for storing moderately large amounts of data that need to have very fast access, often used for buffers, caches, and large memory arrays. BRAM typically comes in fixed-size blocks, with common sizes including 18 Kbits or 36 Kbits, although the exact size can vary between FPGA families. The port widths that access the blocks can be configured, allowing designers to choose between different aspect ratios (wide shallow memory or narrow deep memory, defining the number of word stocked and their bitwidth). Most BRAMs have dual-port capability, enabling simultaneous read/write operations, and parity/check bit options for error checking.

Distributed RAM (LUT). Distributed RAM uses the FPGA's look-up tables (LUTs) to implement small memory elements. These memory elements are typically smaller than those implemented in BRAM, often ranging from 16 to 64 bits per LUT, and they also provide very fast access for small memories. Distributed RAM is specifically tailored for applications such as small data buffers, shift registers, and small lookup tables. It offers high flexibility in terms of size and placement, allowing designers to finely adjust the memory configuration according to their needs. However, because it utilizes logic resources within the FPGA fabric, the use of distributed RAM can impose constraints on the size of the overall design that can be implemented.

UltraRAM (URAM). URAM consists of high-density memory blocks available on certain newer FPGA families, such as *Xilinx UltraScale+*. Each UltraRAM block offers a capacity of 288 Kb and can be configured with various aspect ratios. URAM is particularly beneficial for applications that require very large memory capacities with fast accesses, such as data processing and extensive

buffering. Compared to Block RAM (BRAM), URAM features larger block sizes and generally provides greater capacity.

External Memory. While not strictly “on-chip,” many FPGAs include dedicated hardware for interfacing with external memory, such as DDR, Static Random-Access Memory (SRAM), and Flash memory interfaces. This allows for addressing very large external memory capacities (in gigabytes), significantly exceeding the limitations of on-chip memory. Although the latency varies based on the memory type and interface, it is generally slower than that of on-chip memory. Using external memory often results in higher power consumption and necessitates careful timing design, but it is almost a requirement for applications needing large datasets and bulk storage.

Shift Register (FIFO). Shift registers in FPGAs are specialized structures used for implementing sequential delay lines or FIFOs. They consist of a chain of flip-flops or dedicated shift register primitives. Typically designed for serial input and serial output accesses, shift registers can facilitate very high-speed operation. However, they have a fixed structure, making them less flexible than general-purpose memory.

Comparing Latency: Which Option is Faster?

The relative speed of BRAM versus distributed RAM depends on the implementation. BRAM is optimized for storing large amounts of data, whereas distributed RAM stores small chunks of data across the logic fabric. For larger memory blocks, BRAM is generally faster than distributed RAM, as the latter would require extensive logic and interconnects to link multiple LUTs to form a larger RAM. In contrast, for small memory requirements, distributed RAM is typically faster because it can be mapped directly to a single LUT, offering very low latency due to proximity to logic elements. Additionally, BRAM offers dual-port capabilities, allowing simultaneous read and write operations, a feature not available with distributed RAM. Therefore, in scenarios requiring concurrent read and write access, BRAM may be the preferable choice.

On *Xilinx* platforms, both URAM and BRAM are natively one clock latency read and one clock cycle write, so the latencies of the operations are the same. Shift registers provide sequential access and are not relevant for comparison with random access memories. Finally, external memory is generally the slowest memory, with access times measured in tens or even hundreds of clock cycles for DDR.

Audio Objects Storage

Delay Lines. Some audio programs use delay lines that can require megabytes of memory. As discussed in Chapter 2.7.2, a 1-second delay at a 48 kHz sample rate represents 1 MiB of memory. Although this is highly dependent on the FPGA, most FPGAs typically offer around 10 MiB of Block RAM (with the best *Xilinx Ultrascale+* FPGAs offering around 30 MiB). For reasonably sized delay lines, it may not always be feasible to fit them into the FPGA’s Block RAM.

While URAM could be a solution, this type of memory is not available on many FPGAs currently, particularly the ones we are using.

Therefore, it is logical to implement long delay lines on external memory such as DDR. External memory, which is typically in the gigabyte range, can store dozens of minutes, if not hours, of delay lines. The slow access time of external DDR memory is a potential important performance bottleneck, limiting the number of accessed objects per sample. However, delay lines are usually accessed only twice per sample computation (one read and one write access), making the number of external memory accesses limited enough to be included within the sample processing time.

Static Arrays. Static arrays such as large tables can be used, for example, to implement waveform tables or filter coefficients. These tables can contain thousands of values, leading to implementations of several MiB. Unlike delay lines, many values from the table can be accessed at each sample. For instance, in the case of filter coefficients, the entire table needs to be accessed at the sample rate. In such cases, external memory might be too slow, even with efficient burst mechanisms.

BRAM should be prioritized for these arrays. If the BRAM size is insufficient, URAM and then Distributed RAM (LUT) can be used. Distributed RAM is used as a last resort to conserve resources and allow for the implementation of larger designs. If external memory has to be used, multiple memory accesses to the same array element (to the same sample, for instance) should be cached on the FPGA. This is also true on a General-Purpose Processor (GPP) where several accesses to the same sample should be carried out through a scalar temporary variable that can be cached in a register by the compiler.

Other Objects. Finally, all other objects used in current sample computations, primarily independent variables that act as buffers and are read and written once for each sample, should prioritize the use of BRAM to avoid starvation due to high memory latency.

The following rule should be followed to ensure the optimized implementation of the audio program memory (illustrated in Fig. 4.7).

▼ Implementation Guideline 4.2.5 — Memory

Small and cached objects should be stored on Block Rams.

Delay lines and waveform tables should be stored in external DDR.

Large arrays should be stored in Block RAMs as much as possible. If external memory must be used, caching mechanisms should be implemented if feasible.

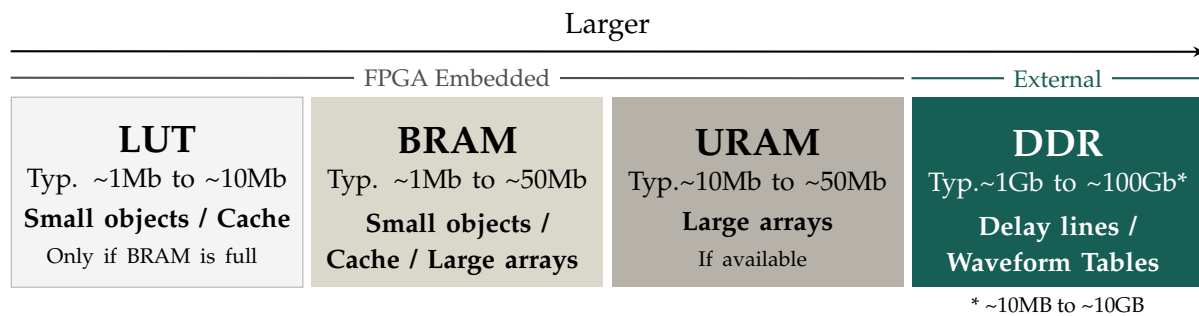


Figure 4.7: Memory dispatching guideline. Cache corresponds to frequently accessed/used data in the current sample computation object, which should be transferred to BRAM for utilization.

This is an optimized strategy. However, we have not implemented it yet in our compiler and are currently using a simpler implementation. Our current implementation, detailed in Algorithm 2, employs a straightforward greedy approach that checks if the total memory required exceeds the FPGA block RAM limit; if it does, the entire memory is mapped to DDR, otherwise, it is mapped to FPGA. This approach is considerably less efficient as BRAM can saturate quickly with non-time-critical objects that could be implemented on external DDR, limiting the size of implemented algorithms. However, it is a straightforward strategy to implement and automate within the compiler. Moreover, our experiments show that delay lines and waveforms were not critically large

in the majority of audio DSPs we tested. Nonetheless, there is still room for optimization in this aspect.

Algorithm 2 Current memory mapping

Require: Total memory required M , FPGA block RAM limit B

Ensure: Mapping of memory to FPGA or DDR

if $M \leq B$ **then**

 Map entire memory to FPGA

else

 Map entire memory to DDR

end if

4.3 Interfacing with the Analog Domain: Audio Codecs

When working with digital circuits such as FPGAs, interfacing with the analog world presents a significant challenge. While SoCs can integrate ADCs and DACs, they are rarely suitable for audio applications. For example, the *Zynq-7000* SoC includes a 12-bit, 1MS/s ADC (and no DAC), which lacks the resolution required for audio signals that typically demand at least 16 bits. Audio-specific converters, which incorporate audio filters and possess characteristics tailored to the low-frequency, high-resolution requirements of audio signals, are often essential. However, using separate converters necessitates careful management of hardware interfaces and software protocols to ensure smooth signal conversion and to avoid performance bottlenecks. The principles of analog-to-digital and digital-to-analog conversion are detailed in Chapter 2.2, while this chapter focuses on their implementation and hardware integration.

4.3.1 Audio Serial Interface

An audio serial interface (ASI) allows for the transfer of non-buffered audio data between processors and audio converters. The I²S standard, introduced in 1986 by *Philips Semiconductor*, is widely used today for communication between processors and audio codecs. Other ASI protocols, such as S/PDIF and AC97, are also common for digital audio communication. However, I²S and its variant remains the predominant protocol for Inter-IC audio communication.

Standard I²S

I²S is a serial bus interface specially designed for communicating digital audio data between integrated circuits (ICs). The I²S protocol sends Pulse-Code Modulation (PCM) two's complement audio data from a controller to a target.

The bus consists of at least three lines (see Fig. 4.8): Bit clock (BCLK) or continuous serial clock (SCK). Word Select (WS) or left-right clock (LRCLK) or frame sync (FS) Data line (SDATA) or serial data (SD) (also called SDIN, SDOUT, DACDAT, ADCDAT and more.)

The bit clock synchronizes the data by generating a pulse for each bit transmitted on the data lines. The frequency of the bit clock is defined by the following formula:

$$f_{BCLK} = f_s \times n_{Bit} \times n_{Ch} \quad (4.1)$$

With f_s the sample rate, n_{Bit} the number of bits per channel (sample width or bit depth), rounded to the next power of 2 value, and n_{Ch} the number of channels (2 for standard I²S).

For example, a 48 kHz sampling rate with a bit width of 24 bit I²S signal has a clock frequency of $f_{BCLK} = 48\,000 \times 32 \times 2 = 3.072\text{ MHz}$

I²S allows two channels to be sent on the same data line, Word Select specify which of the stereo channel (left or right) is transmitted. When the Word Select signal is low (0), the left channel data is transmitted; when it is high (1), the right channel data is sent. As each channel is transmitted sequentially, the WS clock generates a 50% duty-cycle signal at the same frequency as the sample rate.

Data is signed, encoded as two's complement with the Most Significant Bit (MSB) first (Big-endian). This allows the number of bits per frame to be arbitrary, with no negotiation required between transmitter and receiver.

The Word Select clock is typically synchronized to the falling edge of the Bit Clock. The first bit of Data is transmitted one Bit Clock period after the Word Select clock changes. This ensures that the previous word is completely received and can be properly stored before the next word arrives in the subsequent clock period. The receiver latch the Data on the rising edge, allowing for a half-period clock delay between transmitter write and receiver read, which facilitate the timing requirements.

Typically, the data line is unidirectional. Communication with DAC/ADC audio codecs requires two data lines: one for the input and one for the output.

The protocol can also include a Master Clock (not part of the I²S standard), typically at 256 times the LRCLK (4 or 8 times the bit clock, depending on the bit width), which is usually generated by the processor. This clock ensures the synchronization of the internal operations of the analog-to-digital and digital-to-analog audio converters.

Taking the previous example, a 48 kHz sampling rate signal has a Master Clock frequency of $f_{mclk} = 48\,000 \times 256 = 12.288\text{ MHz}$

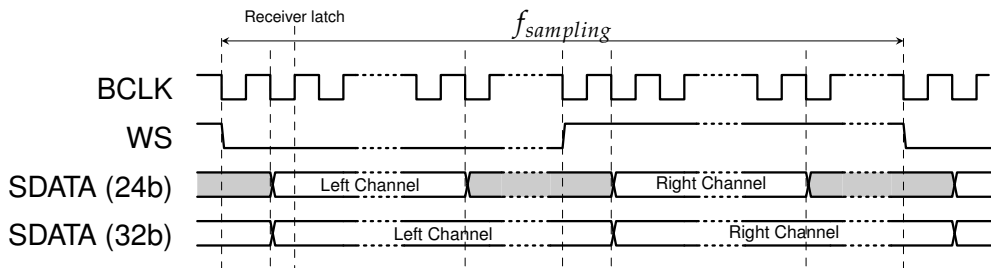


Figure 4.8: I²S (standard) timing diagram.

It is worth noting that, although I²S was primarily designed for inter-integrated circuit communication, the protocol is sometimes used for external connections to audio devices. However, because the I²S bus lacks any error detection mechanisms and was not designed for use with cables (most integrated circuits do not have the appropriate impedance for coaxial cables), this can lead to significant decoding errors.

Left and Right Justified

Left and right justified streams can also be encountered, though they are less common than standard I²S.

Left-justified streams transmit data immediately upon the transition of the Word Select clock, with no clock period delay. The data begins right at the start of the Word Select signal (see Fig. 4.9).

Right-justified streams transmit the data such that the last bit is synchronized to be sent just before the transition of the Word Select clock. If necessary, zero bits are added before the word, resulting

in a delay between the Word Select transition and the transmission of the first bit, with the length of this delay depending on the bit width (see Fig. 4.10).

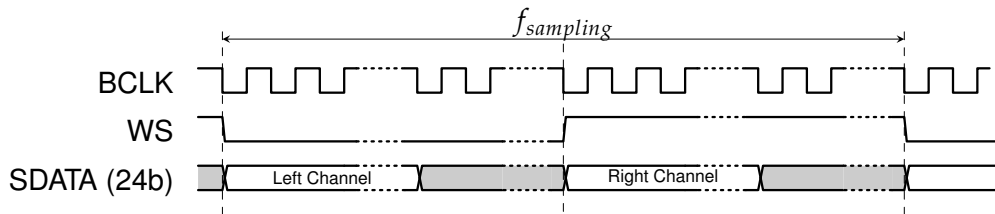


Figure 4.9: Left Justified timing diagram.

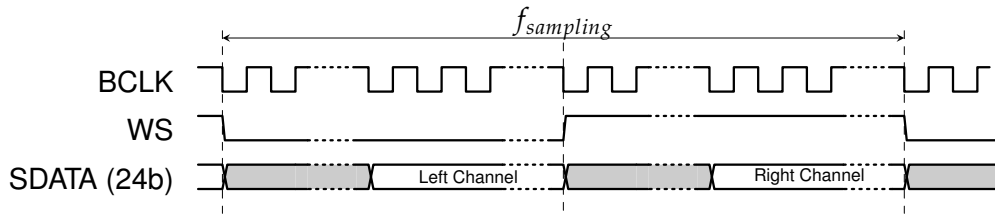


Figure 4.10: Right Justified timing diagram.

TDM

In a Time Division Multiplexing (TDM) transmission, multiple audio channels are transmitted over a single serial data line in a time-multiplexed manner. This approach enables the transmission of more than two channels using the same number of lines than standard I²S. The data for each channel is sent in succession, synchronized with the word and bit clocks.

TDM I²S follows the same principles as the standard I²S protocol, with key differences primarily in the Word Select clock. Since TDM transmits more than two channels, the Word Select clock cannot encode the channel number as in standard I²S. Instead, a Word Select pulse is generated for one bit clock cycle to indicate the start of a new transmission. Immediately following the falling edge of this pulse, the first bit of the first channel is transmitted, followed by the remaining bits of that channel. After all bits of the first channel are sent, the first bit of the second channel is transmitted, and this continues sequentially until the last channel is complete (see Fig. 4.11).

Consequently, it is essential for both the transmitter and receiver to be configured with the same number of channels and the number of bits per channel. The Word Select pulse serves to synchronize the beginning of each channel's transmission. Typical configurations include TDM8, which transmits eight time-multiplexed channels, and TDM16, which transmits sixteen channels.

Transmitting additional channels increases the number of bits while maintaining the same sampling rate, which results in a higher Bit Clock frequency. The period of the Word Select clock must remain equal to the sampling period to ensure that a sample from each channel is transmitted at the designated sampling frequency.

Equation (4.1) remains applicable. However, the number of channels is now greater. For instance, in a TDM16 configuration at a 48 kHz sampling rate with a bit width of 24 bits (rounded to 32 bits), the required clock frequency can be calculated accordingly: $48\,000 \times 32 \times 16 = 24.576$ MHz.

While Left/Right justified and TDM configurations are considered outside the I²S standard, they are often referred to as Left/Right justified I²S and TDM I²S, including in this thesis. This terminology facilitates a better comprehension, as TDM is a generic term that can refer to various protocols.

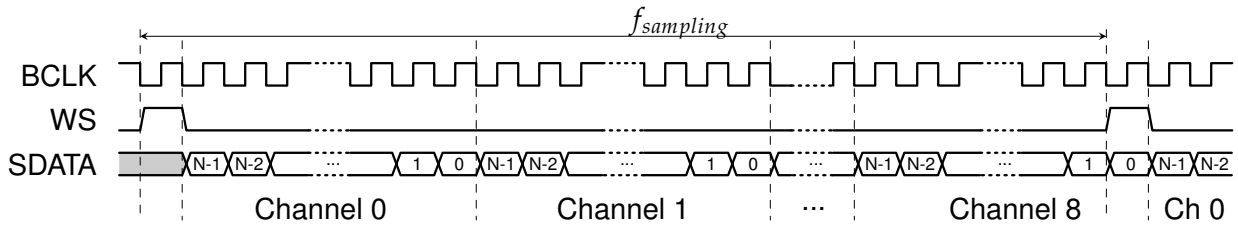


Figure 4.11: 8-Channel TDM (TDM8) timing diagram.

4.3.2 GPIO Usage of I²S Protocols

Having an audio system with many (i.e., more than 32) audio inputs and outputs at a low cost is one of the challenges that FPGA can help solve. Increasing the number of audio channels can be achieved either by using multiple separate I²S interfaces (standard or right/left justified) or by using TDM within the I²S protocol (or by combining both methods).

To manage and test these different protocols, we developed a custom I²S transceiver IP that can handle both standard I²S and TDM (we did not require right or left justified modes). This IP is further detailed in Chapter 6.2.1. The resource usage and allocated GPIOs are presented in Table 4.1 for a standard I²S IPs and for a TDM8 implementation (TDM16 has not been tested yet).

Number of channels		4	8	16	32	256
Standard I ² S	LUT	222	274	386	606	3 601
	Register	300	496	892	1 697	12 697
	GPIO pin	4	6	10	18	130
TDM I ² S	LUT	159	208	271	358	2 878
	Register	267	501	697	1 227	9 546
	GPIO pin	3	3	4	6	34

Table 4.1: Size and pins usage for unidirectional (output only) standard I²S and TDM8 implementation for various numbers of I/O channels. Numbers obtained from Syfala I²S component on AMD/Xilinx Zybo-Z20 board.

TDM8 is obviously more efficient in terms of GPIO usage, as up to 8 channels can be grouped on the same bus. Each additional 8 channels requires only one more wire (Data In), since the clocks Word Select and Bit Clock can be shared between buses. Note that if the communication is bidirectional, two additional wire (Data In and Data Out) are needed for every 8 channels, bringing the total number of required GPIO pins for 256 channels to 66. For standard I²S, each new pair of 2 channels requires this one wire. Moreover, TDM is much more economical because the 8 channels share the same logic, whereas each pair of standard I²S channels likely duplicates the logic (although this is not entirely accurate, and is discussed in more detail in Chapter 6.2.1).

TDM configuration should be preferred for a large number of I/O channels. However, using TDM requires a higher clock speed, better signal integrity protection on the wires, and a more complex configuration of the audio codec (each audio codec must be configured to read the correct slot). Standard I²S is straightforward and allows for easy scalability of the system: a new audio codec simply requires a new GPIO, and its configuration does not depend on the number of audio codecs already in use. For up to 16 channels, the resource savings with TDM are relatively small, and the GPIO usage is easily manageable, making the effort to use TDM in these cases potentially not worth it.

▼ Implementation Guideline 4.3.1 — I²S/TDM Protocol

For a high number of channels, TDM can be used to save resources and GPIO. However, standard I²S allows for easy scalability and is manageable for up to a few dozen channels.

4.3.3 Audio Codecs Architecture

An audio codec is a chip that performs the conversion between analog and digital audio signals. It encodes analog audio as digital signals and decodes digital back into analog by embedding both ADC and DAC.

Architecture and Specificities

Audio codecs are capable of operating at low-frequency sample rates (typically below a few hundred MHz) and high resolution (typically 24 bits). In contrast, standard ADC/DACs typically operate at lower resolutions (under 16 bits) but support higher frequencies (above several MHz). This distinction results in significant differences in system complexity.

As discussed in Chapter 2.3.2, analog anti-aliasing filters are a crucial component of ADCs and can be complex to design. Audio codecs operate at relatively low frequencies and must incorporate these complex filters. They also contain complex low-frequency reconstruction filters for the DAC. The low-pass filter necessary for signal reconstruction must perform two functions: remove all frequencies above the Nyquist rate and compensate for the zeroth-order hold's effect (see Chapter 2.2.2), which results in a *sinc* response function. This requirement leads to a transfer function that necessitates a costly filter to be executed effectively. More detailed information about this reconstruction process can be found in Chapter 3 of Smith's book [Smi03].

Moreover, audio codecs usually embed one or more Digital Signal Processors. These allow for pre-processing and conditioning of the signal. Most of them are programmable, enabling the codec to operate in standalone mode by processing the signal internally without using the digital interface. Depending on the codec, they can provide many useful pre-processing functions such as filters, limiters, volume controls, and mixing. However, embedded programmable DSPs have a limited number of instructions and do not allow for flexible signal processing.

It is possible to use an audio codec without using the embedded DSP. This is especially useful if the processing is entirely done on another chip and the raw signal needs to be digitized. Avoiding the use of the embedded DSP can significantly reduce latency, although this is generally not the default configuration. Advanced audio chips, including those embedding a DSP are configurable, typically using an I²C communication interface. This configuration handles numerous parameters, including ADC/DAC sampling frequency, the audio signal path inside the codec, amplifier volume, and more.

Most audio codecs support linear PCM representation and protocols such as I²S, right/left justified, or TDM, as detailed in Section 4.3.1. They also offer sampling frequencies aligned with audio specifications, such as 44.1 kHz, 48 kHz, 96 kHz, 768 kHz among others. These features make codecs complex and specific chips, distinguishing them as a particular type of ADC/DAC. The diagram of the SSM2604 audio codec, taken from the datasheet [Anag] (p.1), is presented in Fig. 4.12. The SSM2604 is a very simple, configurable audio codec. It's a simplified version of the SSM2603 [Anaf] that we used, presented here for its simplicity in the schematic. The diagram shows the main components included in the chip: input amplifier, the DSP that handles the signal processing, ADC/DAC converters, a bypass, and the digital and control interface.

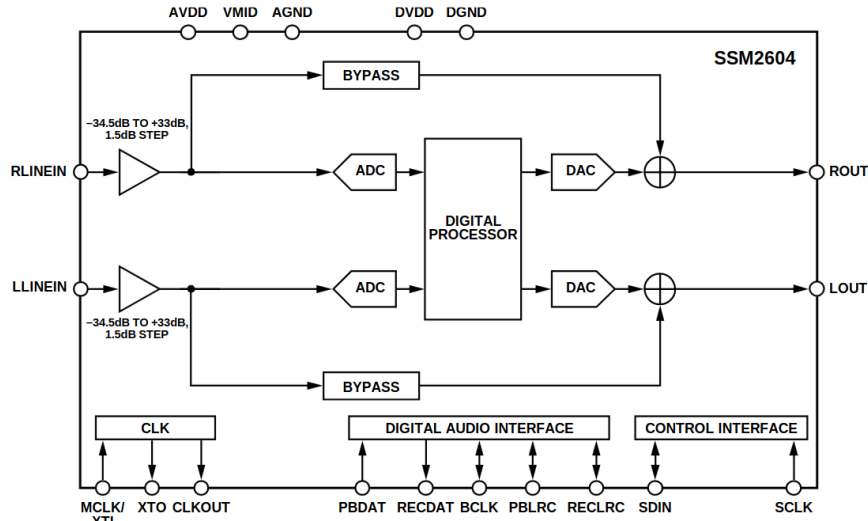


Figure 4.12: SSM2604 audio codec diagram taken from the datasheet [Anag] (p.1).

ADC Types

Analog-to-Digital and Digital-to-Analog conversion principles are detailed in Chapter 2.2. This section focuses on the different types of ADCs/DACs and their implementation.

There are several ways to implement an ADC. Various architectures coexist, depending on the required performance. Fig. 4.13 shows the approximate distribution of the four main types of ADCs based on their respective needs. A more accurate version of this diagram can be found here [FF15].

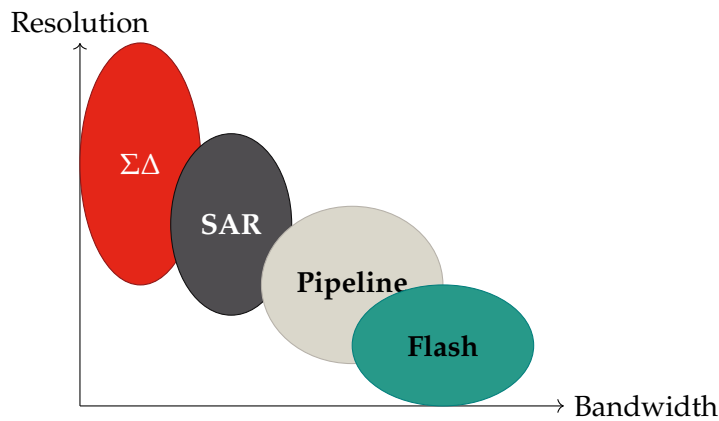


Figure 4.13: Resolution and bandwidth characteristics of different ADC architectures.

Successive Approximation Register (SAR) determines a digital output by iteratively comparing an analog input to an internal variable signal, refining the approximation until reaching the input voltage's binary representation.

Flash ADC uses a bank of comparators to directly convert an analog input into a digital output using a one-hot encoding scheme, achieving very high-speed conversion but limited resolution due to its exponential complexity with increasing bit depth.

Pipeline ADC is a combination of the two precedent type, it splits the analog input into multiple stages, processing each stage simultaneously to achieve high-speed and high-resolution conversion.

Sigma-delta ADC use high-frequency sigma-delta modulation (convert analog signals into high-frequency, low-resolution digital signals) and a low-pass filter to convert the data stream from that high sampling rate with low bit depth to a lower rate with higher bit depth. A more detailed description of the main ADC technologies is available on the Analog Device website [Anaa]. An exhaustive and thorough discussion of ADC and DAC technologies can be found in the *Data Conversion Handbook* [Kes05].

As shown in Fig. 4.13, the sigma-delta ADC is one of the most suitable for audio applications, providing high resolution (16 to 32 bits) and low frequencies (<1 MHz). Most audio ADCs today use sigma-delta conversion. This architecture is further discussed in the next section.

DAC Types

The application range of ADCs is less straightforward and it's difficult to find a survey that classify each architecture depending on the needs (resolution/frequency). However, DAC architectures can be divided in two main families: Resistor based DAC and oversampling based DAC.

Resistor based DAC architectures, such as binary weighted DAC, R/2R DAC, string DAC, are, as their names suggest, using a network of resistors to generate corresponding voltage levels based on the input digital code. They are commonly used due to their simple architecture, low power consumption, and are fast (short settling time). DAC architectures are extensively studied in the literature, with their inherent advantages and disadvantages thoroughly analyzed [BK05, BJ10]. However, resistor based DAC can suffer from bad accuracy as they rely on the precision of the embedded resistors as well as on the voltage reference stability.

Compare to resistor based DAC, sigma-delta DAC have a lower Differential nonlinearity (DNL) and guarantee to be monotonic (output either increases or remains constant for increasing digital inputs, in other words, the derivative of the transfer function never changes sign) [LMDS18]. Differential nonlinearity is the primary specification used to determine DAC accuracy. It measures the deviation between actual and ideal step sizes in digital-to-analog conversion, indicating how uniformly the analog output changes with each increment of the digital input. This makes them particularly well-suited for audio applications, where precise and consistent output is critical for sound quality. Sigma-delta DACs also offer high resolution and low distortion, single voltage power supply and lower power dissipation at a relatively low cost, compared to resistor-based DAC.

Sigma-Delta Modulation

Sigma-delta (or delta-sigma) modulation is employed in sigma-delta ADCs and DACs. This method involves oversampling, which encodes signals into low bit-depth digital signals at a very high sample frequency. Therefore, audio converters based on the sigma-delta architecture require an internal master clock that operates at a much faster rate than the target sample rate. Sigma-delta modulation uses a negative feedback loop during quantization to lower bit depth. This loop continuously corrects quantization errors and shifts quantization noise to higher frequencies, beyond the original signal's bandwidth. It is highly adapted for high resolutions low output bandwidth (less than a few hundred kHz) application, making it the most used architecture, for ADC as well as DAC in audio codecs [BK05].

Sigma-delta ADCs (Fig. 4.14) use a decimator digital filter following the sigma-delta modulation. This process converts the high-frequency, low-bit-resolution signal outputted from the modulation into a low-frequency, high-resolution digital signal. Conversely, DACs (Fig. 4.15) use an interpolator to convert the low-frequency, high-bit-resolution digital input signal into a high-frequency, high-bit-resolution signal, which is then fed into the modulation.

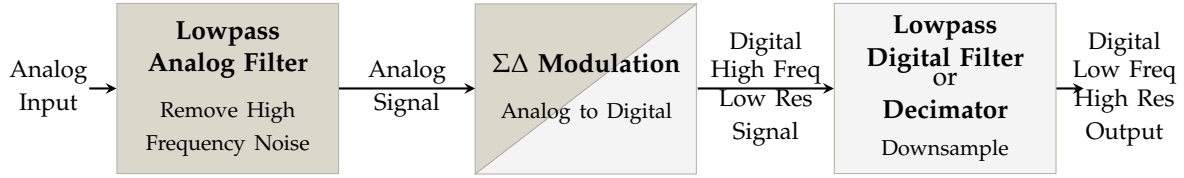


Figure 4.14: Sigma-delta ADC Architecture.

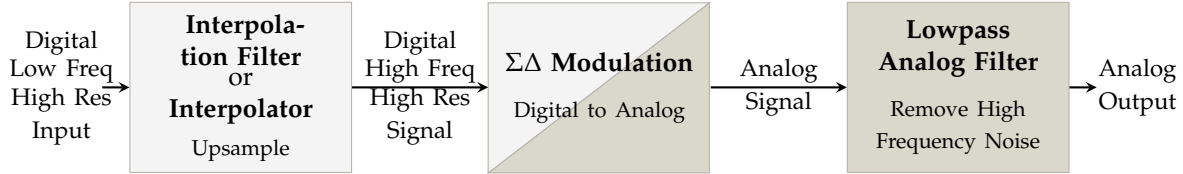


Figure 4.15: Sigma-delta DAC Architecture.

These decimator and interpolator filters are the main cause of delay for sigma-delta converters [Wan18].

Sigma-Delta Latency

The topic of sigma-delta converter latency has been extensively covered in Wang’s PhD thesis [Wan18]. As noted in the thesis, no research has provided accurate latency measurements of hardware sigma-delta audio codecs. Wang demonstrated that the majority of latency in a sigma-delta architecture is caused by the group delay of digital decimation and interpolation filters [Wan11]. They characterized the latency of four audio codecs. However, they used dated chips, two of which are now obsolete or not recommended for new designs. Furthermore, they did not test audio codecs specifically designed for low latency.

The principle of sigma-delta modulation is not detailed further in this thesis. However, more information can be found here [ST⁺05]. In the next section, we analyze the latency of several relevant codecs to determine if the current latency is sufficient for low and ultra-low latency audio applications or if a change of architecture (SAR, Pipeline, Flash) could lead to significant improvements.

4.3.4 Ultra-Low Latency Audio Codecs

Table 4.2 briefly presents the audio codecs adapted to our needs used in this thesis, summarizing the capabilities of currently available audio chips.

The minimum latency detailed here corresponds to the measured time between the analog input at the ADC and the analog output from the DAC, with an I²S bypass between them. It represents the minimum latency expected in a full system for the conversion part, excluding computation time. Detailed experimental latency measures and testing methods are covered in Chapter 6.2.3

The table shows that low-latency-specific audio codecs are significantly below the ultra-low latency specification. The *ADAU1787*, for example, represents less than 1% of the 1 ms threshold defined for ultra-low latency. Thus, codecs with sigma-delta architecture can be considered sufficient for ultra-low latency applications.

Faster Performance Analysis: Comparing Alternatives

If audio codec latency is dependent on the sampling rate and our goal is to minimize it, the use of higher sampling rate (non-audio specialized) ADCs and DACs should be considered.

Audio codec	Specificity	Sample Rate (kHz)	Res. (bit)	Min Latency (μ s)
SSM2603	Cost effective, easy to use	8 to 96	24	161
ADAU1777	Low latency, cost-effective	8 to 192	24	69
ADAU1787	Very low latency	8 to 768	24	8.4

Table 4.2: Overview of audio codecs capabilities with minimum measured latency analog to analog (detailed in Chapter 6.2.3)

The fastest audio codec we found (ADAU1787 [Anac]) operates at a frequency of 768 kHz.

High-resolution “general-purpose” ADCs (24-bit or more) with sampling rates exceeding 1 MHz are rare. These ADCs typically employ either SAR or sigma-delta architectures and do not exceed 4 MS/s. Notable examples include the AD4030-24 [Anab] from *Analog Devices* (2 MS/s max), the LTC2380-24 [Tecb] from *Linear Technologies* (2 MS/s max), and the ADS1675 [Texpa] from *Texas Instruments*, which can achieve up to 4 MS/s.

No DAC offering more than 20-bit resolution at a sampling rate exceeding 1 MS/s were found.

In summary, no significant improvements in latency seems possible with non-audio DACs, as they offer characteristics close from audio DACs. ADCs exhibited potential for higher speeds: the exact latency for the cited ADCs is not provided, but their higher sampling rates and the use of SAR architecture suggest they are likely faster than audio codecs, although their sampling frequency is only 2 to 5 times more than audio codec’s ADC. Moreover, it is important to note that higher sampling frequencies reduce the processing time available for each sample. Sampling at 2 MS/s is very fast for audio applications and may require significant computational power depending on the implemented algorithms. We did not test these audio codecs, as the 768 kHz sampling rate achieved by our fastest audio codec already imposes substantial processing time constraints. Nevertheless, it is worth considering that latency improvements may be feasible in this area.

In a more technical approach for low latency optimization, Wang proposed an adjustment of the sigma-delta architecture by demonstrating that various decimation architectures are feasible, indicating potential for optimizing the main contributor to latency in these architectures [Wan18].

Even though the choice is limited as most, if not all, audio codecs use sigma-delta ADCs and DACs, this section has demonstrated that their technology is suitable for ultra-low latency audio applications. However, the fact that sigma-delta architecture induces more latency than other architectures (particularly for DACs) suggests that there is potential for improvement in this specification.

▼ Implementation Guideline 4.3.2 — ADC/DAC Architecture

Sigma-delta ADC and DAC architectures are highly suitable for ultra-low latency audio applications, offering high resolution, low distortion, lower power dissipation, and a relatively low-cost solution.

FPGA gives the possibility of implementing converters directly on the chip. FPGAs are well-suited for implementing sigma-delta DACs and ADCs, especially if the FPGA provides differential GPIO, as they can operate at very high speeds. A promising work involving our team has made on this subject [MSL⁺23].

4.4 Hardware and Software Audio Host Control

Controlling an audio DSP is one of the key elements of audio programming. Chapter 2.6.1 demonstrated that control signals can be differentiated from audio signals, as they do not require the same processing rate. Section 4.2.2 shows that controllers and every computation depending on them can be processed on the CPU at a much slower rate than the sampling rate. This offloaded processing not only allows the FPGA to save resources but also facilitates interfacing with control interfaces. Typically, an audio program is controlled using a physical interface containing controllers such as knobs, potentiometers, buttons, etc., or using a software interface running on a computer. In both scenarios, a CPU is better suited for handling communication with the interface.

Controller Synchronization Mechanism

An inherent problem when working with two different rates is that synchronization issues may arise. In Section 4.2.2, we discussed how the slow refresh rate used by the controller avoids the need for deep scheduling analysis for FPGA implementation, as controller updates do not need to be synchronized with a specific FPGA function. This is only partially true. While controllers can indeed be updated at any point during FPGA processing, there is a strict constraint that all controllers must be updated during the same sample processing.

This is crucial because control parameters are transmitted sequentially, and some may have coherence conditions (e.g., they are all derived from the same value). If dependent controllers are applied to different samples, it could lead to an inconsistent state of the control parameters, resulting in poor sound quality.

To prevent this, care must be taken to ensure proper synchronization. In such cases, the FPGA must wait to receive all relevant parameters before processing them. This synchronization mechanism has been implemented in the FAUST compiler by Yann Orlarey.

Bare-Metal and Embedded Operating System

The control can be handled by the CPU either through a bare-metal implementation or via an OS. A bare-metal system executes instructions directly on hardware without operating system. These systems are programmed for specific applications, offering faster and more efficient implementations compared to OS-based systems. Embedded platforms (such as microcontrollers) often use bare-metal implementations, which allows for the easy access to peripherals and meet time-critical latency requirements. However, they offer less flexibility than systems with operating systems. Moreover, many services (such as peripheral management, communication protocols, and task scheduling) must be re-implemented, sometimes with considerable complexity.

Embedded operating systems provide a wide range of services and applications by being based on General Purpose Operating System (GPOS) such as *Linux*. This compatibility allows them to support many software components that are very complex to implement on a bare-metal system, such as TCP/IP stacks, HTTP protocols, OSC or MIDI interfaces, and WAV or MP3 audio codec libraries.

4.4.1 Simple Control with Baremetal

A bare-metal system does not benefit from most software components, therefore, controlling a DSP running on a bare-metal system typically involves simple communication protocols, often serial, which are available and implementable on the embedded system.

For hardware controllers, electronic components such as potentiometers and sliders can be directly connected to the FPGA's GPIO and processed by the hardware implementation of the DSP, provided the FPGA has an embedded ADC. While FPGAs generally have at least a simple ADC, this solution is neither scalable nor adaptable and quickly uses up all available GPIO pins as the number of controllers increases. The conventional solution is to use a separate ADC with serial communication protocols such as SPI or I²C. ADCs usually allow for the conversion of multiple analog signals into a serial bus, facilitating scalability and simplifying connections. On the other hand, communication with a computer can be easily achieved using UART serial communication.

In both cases, using SPI/I²C or UART communication requires specialized transceivers for encoding and decoding the signal to the appropriate protocol. Although these can be designed in hardware on the FPGA, CPUs usually have embedded peripherals that handle these protocols (see Fig. 4.1 p.39). Thus, handling control communication requires less effort than implementing drivers on the FPGA, making the use of the CPU for control computation even more consistent.

Hardware Interface

For my Ph.D., a simple controller board has been designed. It uses the MCP3008, a 10-bit ADC with a sampling rate of up to 200 kSps (at a 5V power supply). The MCP3008 can convert up to 8 analog signals via an SPI bus. This setup allows for the conversion of any analog signal. For control applications, this signal can be the output of a potentiometer (knob/slider) or a switch, for example. Fig. 4.16 shows the PCB of the board, illustrating the placement of 8 components. Each component can be a potentiometer, slider, or switch. The design of the board is generic enough so that a wide range of combinations of controllers (i.e., buttons, rotary potentiometers, faders, etc.) can be explored and soldered on it. Different possible combinations are shown in Fig. 4.17.

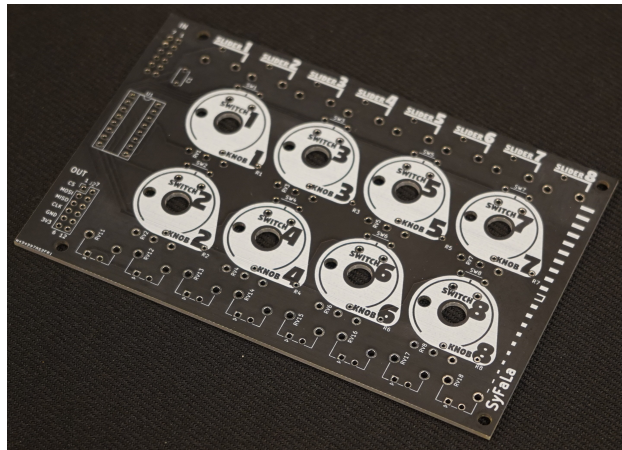


Figure 4.16: Custom controller board PCB.

The objective of this board is to be as simple as possible, both in terms of fabrication, requiring only the hand-solderable MCP3008 chip and the desired controllers, and in terms of communication, using the SPI protocol, which is easily integrable into embedded systems. The choice of using SPI was made to leave the I²C bus available for audio codec configuration. Although both could coexist on one bus with different addresses, using two separate buses allows for different GPIO placements and different communication rates.

This board was presented at the SMC 2022 conference [PMR⁺22].

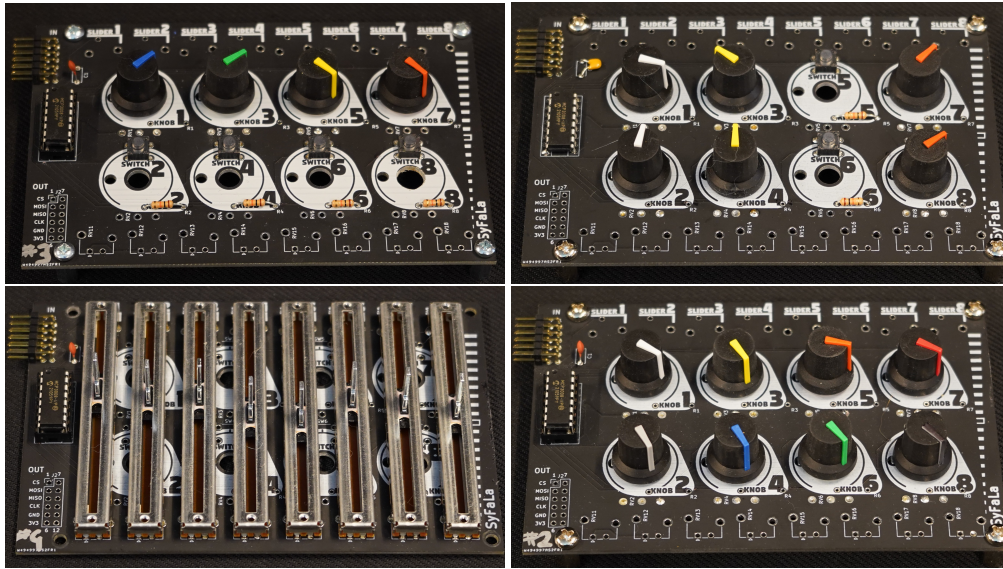


Figure 4.17: Custom controller board with different combinations (4 knobs/4 switches; 6 knobs/2 switches; 8 sliders; 8 knobs).

Software Interface

A CPU can easily instantiate a UART serial connection (USB communication) and send and receive messages from a computer with a USB connection. By setting up a remote serial console with terminal emulators such as *PuTTY* or *Minicom* on the host computer, the user can transmit data to the embedded CPU. A Command Line Interface (CLI) or a Terminal User Interface (TUI) can facilitate this communication. A Graphical User Interface (GUI) can also be developed to handle the UART communication in the background and bring transparent and easy control to the user. On the embedded CPU side, the UART protocol is generally straightforward to implement, as libraries facilitate the setup. Interpreting messages from the computer and eventually formatting messages back to the computer must be implemented. These messages sent back can include non-audio output data, such as VU meters, as discussed in Chapter 2.6.2.

The main consideration for UART implementation is the baud rate, which is the rate at which data is transmitted and received in a serial communication system, typically measured in bits per second (bps). The baud rate defines the speed of data transfer and needs to be set to the same value on both sides to ensure synchronization between the communicating devices. It must be chosen carefully according to the control rate needs. A common baud rate is 115 200 bps, which is typically sufficient to send a reasonable number of controller values (~10) at the control rate. A baud rate of 115 200 bps allows for 11 520 bytes per second of data (per byte: 8 data bit, 1 parity and 1 stop bit). With a hypothetical control rate of 100 Hz, this translates to 115 bytes per control cycle, enabling approximately 28 controllers with 32-bit values. However, if the control rate is 1 000 Hz, this number drops to 2-3 controllers.

If a large number of controllers needs to be communicated, or if the control rate is too fast, the baud rate can be increased, sometimes up to 921 600 bps. If this higher baud rate is still insufficient, the control rate may need to be decreased.

▼ Implementation Guideline 4.4.1 — *Implementation: Bare-metal*

Bare-metal implementation of DSP control on the CPU is more suitable for low-latency and simple applications.

It can easily handle tens of controllers with reasonable control rate.

Implementation

This section describes our proposed implementation of bare-metal CPU control.

In a bare-metal system, the CPU exclusively handles the control tasks: reading the values from controllers (knobs, sliders, switches, GUI), processing the computations based on these values, and sending the updated values to the FPGA. In this scenario, the implementation can follow a best-effort strategy (see Fig. 4.18): the CPU continuously loops and sends new controller values to the FPGA as quickly as possible. Since the computation of the control values is blocking, processing a large number of controllers simply takes longer, thereby automatically decreasing the control rate. The programmer just needs to ensure that the control rate remains within acceptable limits for their specific application needs. This can be a critical step in some applications, such as digital instruments, where latency is more restrictive, as discussed in Chapter 2.5.2. Additionally, if software control is used, it is crucial to ensure that the baud rate is sufficient, as discussed in the previous section. With this implementation running on a 667 MHz dual-core *Cortex-A9* on a *Zybo Z20*, using our controller board (8 controllers) and a 48 kHz sampling rate, the latency between a controller change and the output signal consequence (including the DAC-induced delay) was measured at 50 μ s, making it suitable for digital instruments.

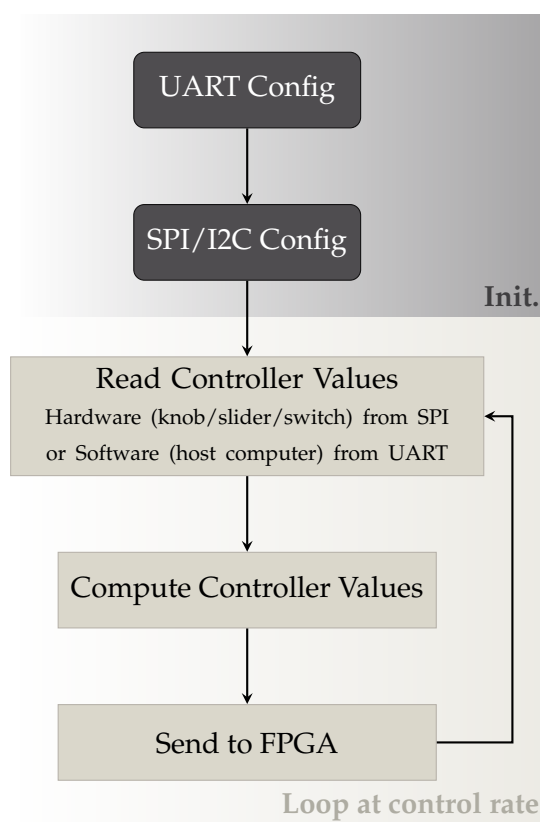


Figure 4.18: CPU bare-metal control implementation.

The interface between the CPU and the FPGA is then managed with an AMBA bus, which allows for communication between the FPGA fabric and the embedded CPU on a SoC, using protocols such as AXI, as detailed in Chapter 5.5.1.

4.4.2 Embedded *Linux* and Interfacing With External World

The control interface described in Section 4.4.1 is sufficient for prototyping an FPGA-based audio platform, but as soon as it is intended to be used by many users, the question of interfacing the system with the external world, Ethernet in particular, quickly comes as essential.

Modern integrated CPUs on SoCs are powerful enough to run an operating system. However, integrating an OS necessitates a long and challenging process to compile and configure the embedded operating system for the specific target platform. The communication between the embedded OS with the FPGA, as with a bare-metal system, uses an AMBA bus at the hardware level. However, the OS adds a level of abstraction, necessitating the use of dedicated drivers (e.g., a kernel driver on *Linux*) from the CPU side. Providing these drivers and handling concurrent access to memory shared by the operating system and the FPGA is generally a complex task.

Today, OS integration is typically facilitated by FPGA vendor tools that generate drivers to ease the interfacing of the IP synthesized on the FPGA with the OS present on integrated SoCs. However, this process remains complex and is highly dependent on the quality of the tools provided by the vendors. For instance, *Xilinx/AMD* offers *Petalinux*, a *Linux* distribution dedicated to the *Zynq* SoC family. *Petalinux* is built using the *Yocto* open-source toolchain, which is widely used but quite intricate.

For this project, *Xilinx's Petalinux* could not be effectively configured and built on desktop computers using recent OS versions, leading to the decision to build a distribution from scratch. Using a custom *Linux* distribution has many advantages: precise control of the distribution's content, facilitated reuse of existing software components, etc. The complete process of our *Linux* implementation is precisely detailed in [CPF⁺23] and is not further discussed here. While this thesis laid the groundwork for this project and led to the first successful attempts with *Petalinux*, all the OS-based work currently used in the toolchain, including the custom OS and kernel configuration, was made possible by Pierre Cochard (team engineer) and Antoine Fraboulet (associate professor at INSA Lyon). That being said, *Petalinux* is quite difficult to use on the *Zybo*, with limited documentation, and the HLS drivers were poorly documented. Achieving a first working version was a significant contribution of this thesis.

▼ Implementation Guideline 4.4.2 — *Implementation: OS*

OS-based implementation of DSP control on the CPU is more flexible, allowing for various control protocols and approaches such as OSC (Open Sound Control) through Ethernet or Wi-Fi, MIDI, web interfaces running on an HTTPD server, etc.

Embedding an OS also enables greater programming flexibility for the FPGA. Many FPGA bitstreams can be uploaded to the operating system's file system and stored on the board. This allows for dynamically changing the FPGA program from the CPU.

4.5 FPGA Audio Compilation Scheme

The previous sections explored the foundational components and configuration processes necessary for deploying audio DSP on FPGA. This section synthesizes all the previously discussed concepts to detail how a compilation flow from high-level audio DSP to FPGA hardware design should operate.

4.5.1 From High Level DSP to Hardware and Software Components

An audio DSP to FPGA compilation flow is not limited to the single compilation of signal processing algorithm to FPGA hardware. Fig. 4.19 presents the global view of the building blocks of an “audio DSP to FPGA compiler.” A single source (named `audio.dsp` in Fig. 4.19) generates both hardware (i.e., audio IP) and software (i.e., control program) according to the dispatching discussed in section 4.2.2. The hardware part is the section of the program that is mapped on the FPGA and the software part is the section of the program that is compiled to the associated CPU.

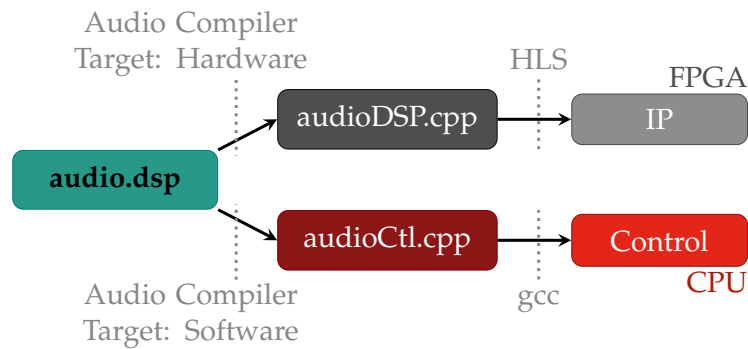


Figure 4.19: Audio to FPGA compiler global view.

The audio DSP can be written using an audio DSL or directly in C++. If a DSL is used, most existing audio compilation flows have already implemented the hardware/software split when compiling for CPUs. The splitting results in two different files, ideally written in C++ for ease of management throughout the compilation flow, although other languages can be used.

Control Part

For the software part, the file needs to be compiled to be executed on the CPU. If the implementation is bare-metal, the file needs to be cross-compiled on a computer and then executed directly on the embedded CPU. In the case of an embedded OS, the file can be cross-compiled on a computer or compiled directly from the embedded OS.

The `audioCtl.cpp` in Fig. 4.19 file should include everything needed for the FPGA communication and controllers computation as illustrated in Fig. 4.20. This involves setting up communication protocols (SPI/I²C/UART, etc.) as discussed in Section 4.4. If the implementation is OS-based, this also includes calling and configuring kernel drivers. The software file also handles the external memory initialization and constant initialization according to Section 4.2.2 guidelines.

For the FPGA communication, HLS tools typically provide APIs and libraries to manage the AMBA bus. With *Vitis* HLS, the creation of an AXI4-Lite interface on the hardware IP automatically generates a set of C driver files. These C driver files offer a set of APIs that can be integrated into any software running on a CPU and used to communicate with the device via the AXI4-Lite slave interface. Driver files are created for bare-metal and *Linux* modes. In other words, the hardware file declares the AXI Interconnect, while the software file instantiates and initialize it. If an embedded user interface is used (see Section 4.5.4), it is also declared here.

Hardware Computations

The hardware part will be implemented as an audio IP. It can be written directly with HDLs or, like in our case, use HLS. For HLS, an FPGA audio compiler should then be able to infer automatic directive to the HLS tools to create audio IP with adapted interface from the C++ `audioDSP.cpp` file. This involves creating an audio stream interface, preparing the communication with the CPU for

control computations by declaring an AXI interconnection, and handle BRAM initialization, as illustrated in Fig. 4.20.

For convenience, the audio stream can be managed by a parallel interface within the IP and then converted to I²S by a separate I²S transceiver IP. This approach simplifies the compilation process and makes the `audioDSP.cpp` file clearer. If other generic audio functions need to be implemented to process the audio stream (such as bypass, mute, permanent filters, etc.), they are also declared in this file. Note that this file should also infer the appropriate directives to optimize the hardware implementation, which are discussed in detail in Chapter 5.

4.5.2 Hardware Integration and User Interface

Design Integration

At this stage, the implementation consists of a software control application running on a CPU and a hardware IP processing audio samples, both prepared to communicate with each other. The design is still missing many components: AMBA interconnections, clocks, I/O port mapping, memory mapping, interfaces, etc. Namely, a hardware design that integrates the CPU, the hardware IP, and all necessary peripherals.

There are several ways of integrating a hardware IP and a CPU into a hardware design. With *Xilinx* tools, the easiest approach is probably using a block design. A block design includes all the necessary components for the architecture. Each component takes the form of a dedicated IP, whether generated with HLS, provided by *Xilinx* tools, or handwritten in HDL. This allows for the easy integration and configuration of clock systems, AXI interconnections, GPIO interfaces, hardware logic, and custom IPs (such as an I²S transceiver). The block design then defines the connections and also includes port mapping to define which signals go to physical GPIOs, as well as other parameters such as memory mapping and timing requirements.

An example of block design used for this project is showed in Appendix E.2. Block design can be developed using a graphical interface with *Xilinx Vivado* software, which generates *Tcl* files. These *Tcl* files can also easily be generated by an audio compiler to automate the design creation process. The *Tcl* scripts are then processed with the *Xilinx Software Command-line Tool (XSCT)*, making the entire workflow scriptable, which is essential for a compiler. The file should contain all the previously detailed components, as illustrate in file `BlockDesign.tcl` in Fig. 4.20.

`audioCtl.cpp` (for baremetal controllers) and `audioDSP.cpp` files depicted in Fig. 4.20 are presented in the Appendix F for a simple example of a gain DSP. They are used in our *Syfal*a compiler, which is detailed in next section.

Configuration Automation

The compiler should handle various audio parameters to ensure compatibility with a wide range of user-written audio DSPs. These parameters include the number of input/output channels, sampling rate, bit width, number of controls, and compatibility with audio codecs (including audio serial protocol types and speeds, as well as clock requirements). In the case of *Xilinx*, many hardware functions are provided through configurable IP blocks. For example, the *Clocking Wizard* allows for the precise configuration of clock generators, and similar tools are available for AXI interconnections. These IP blocks can be configured using *Tcl* files, allowing the compiler to adapt them to the constraints of the audio DSP.

Some components may need to be manually written (using HLS or directly in HDL) to achieve maximum control over parameters and the lowest possible latency. For example, we had to hand-write the I²S transceiver instead of using the provided IP to gain control over sample rate, bit width, channel number and TDM capabilities. This is described in Chapter 6.2.1.

audioCtl.cpp	audioDSP.cpp	BlockDesign.tcl
- Peripheral Init. (UART, SPI, I2C, etc.) - FPGA Communication Init. - External Memory Init. - Controllers Init. - Audio IP Init. (Start signal, shared memory pointer, etc.) - User Interface Init. - Kernel drivers Init. (Linux Only) - Control rate computations	- Audio parallel interface (Communicate with an I2S transceiver) - CPU communication interface - External memory interface - BRAM Init. - Audio generic functions (Bypass, mute, etc.) - Sample rate computations	- I2S transceiver - AMBA interconnect (AXI) - Clock management - External memory interconnect - Port mapping - Memory mapping - Hardware logic - Reset management

Figure 4.20: Software (audioCtl.cpp), hardware (audioDSP.cpp) and block design (BlockDesign.tcl) file content.

Once everything is scripted, the compiler can automatically generate the adapted architecture design.

Graphical User Interface

If a host computer is used to provide a user interface in the case of a bare-metal implementation, as discussed in Section 4.4.1, the audio compiler should automatically generate an adapted interface based on the DSP program.

Controllers information can be extracted from the source program to create user commands and graphical elements for a GUI. A simple approach for automatically generating a user interface is to create a CLI or a TUI based on the extracted controller information. The compiler generates an API containing details about each controller (such as number, name, type, resolution, min/max values, etc.) and creates a terminal-based script that communicates with the FPGA through UART. A more advanced method involves generating a graphical interface with elements like knobs and sliders based on the controller information. This can be facilitated if the source DSP is processed through a DSL compiler, such as FAUST, which provides a GTK-based GUI directly adapted from the FAUST DSP source program (see Fig. 4.21).

4.5.3 Our Audio DSP Compiler: Syfala

All guidelines and concepts provided before are intended to offer a generic approach to what an audio compiler should be. This section demonstrates how we applied them to build our audio compiler: *Syfala* [PMR⁺22, PMR⁺23b, Emeb]. *Syfala* is one of the main contributions of this thesis, it represents more than four years of work heavily impacted by this thesis. But it is also the result of the collective efforts of the entire *Emeraude team*, especially Professor Tanguy Risset, Dr. Romain Michon, and Pierre Cochard, the team's engineer, along with Yann Orlarey and Stéphane Letz, the creators of the FAUST language.



Figure 4.21: Example of an automatically generated GUI with FAUST.

The Syfala Compiler

Syfala is a FAUST or C++ to FPGA compiler. The compilation flow associates a traditional compiler (the FAUST compiler) with a high-level synthesis tool (*Vitis HLS*) which acts as a middle-end by optimizing the resulting implementation for FPGA targets. Our flow uses the standard *Xilinx Vivado* synthesis tool as a back-end. Apart from proprietary *Xilinx* tools, the platform is open-source and available on GitHub [Emeb]. An example of the complete toolchain is provided in the last chapter, Chapter 6.

Compiler Complete Overview

The complete implementation of the *Syfala* FPGA audio compiler is illustrated in Fig. 4.22. From a single audio DSP, three distinct files are generated. The core DSP computation `audioDSP.cpp` is mapped on the FPGA, the control part `audioCtl` is implemented on the computing system and the user interface UI is executed on the host processor or carried out in hardware. Standard interfaces allow the user to use the same compilation commands for all audio programs.

In the case of *Syfala*, the audio compiler used is the FAUST compiler. This approach could be applied to any audio programming language, but using FAUST significantly benefited us. Its open-source nature allowed us to modify the source code, collaborate with the language's creator, and leverage its intrinsic construction, which facilitates tuning different elements to fit an FPGA configuration. All these points are detailed in Chapter 6.1.

The FAUST compiler is tuned to automatize the generation of the three files (`BlockDesign.tcl`, `audioCtl.cpp` and `audioDSP.cpp`) in line with the functionalities detailed in Fig. 4.20 p.69. Everything is scripted through makefile and TCL scripts, making the entire toolchain automated, as detailed in the following sections. This part of the work was largely led by Pierre Cochard.

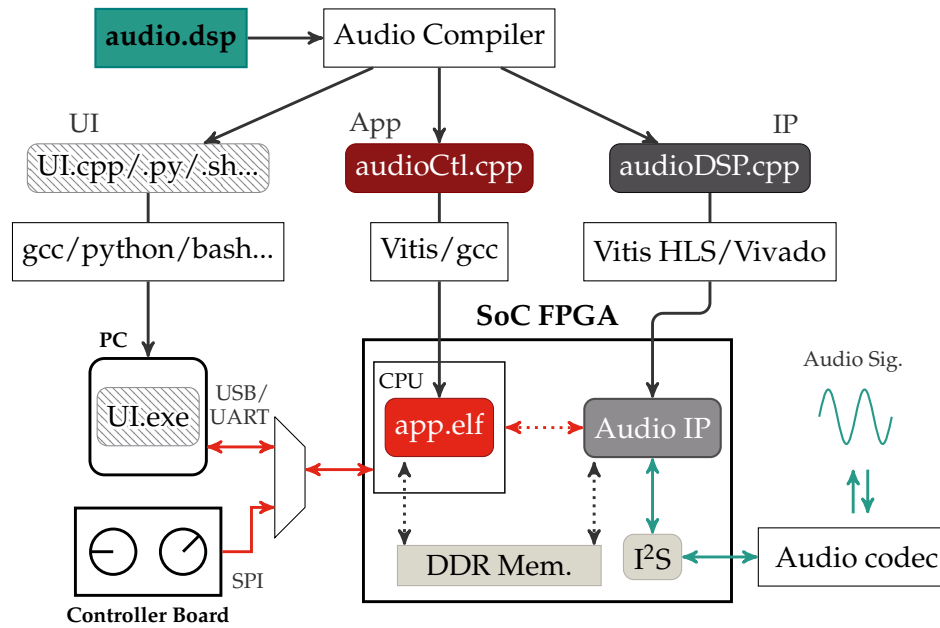


Figure 4.22: *Syfala* FAUST to FPGA compiler complete architecture. The audio path is shown in green, and the control path is in red. AMBA communication is indicated with dotted arrows. Rounded boxes represent elements generated during the compilation flow.

4.5.4 Additional Compiler Features

Terminal User Interface

In a bare-metal environment, control over the implemented program is very lightweight. Controllers directly manage the DSP, but it can be useful to control non-DSP parameters, such as sample rate, sample width, and other settings, via the embedded CPU. Note that these parameters can be modified in the source file, but this requires the file to be recompiled.

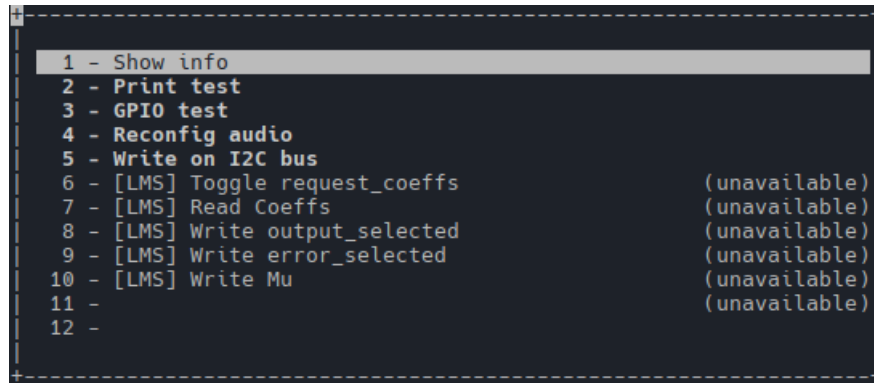
This is different from the TUI previously mentioned, which is implemented on the host computer to control the DSP. The bare-metal control interface is coded directly on the CPU and handles non-DSP-related parameters. It can be entirely managed on the SoC, for example, by using keyboard or simple buttons connected to the CPU. The interface can be displayed on an embedded screen or on a host computer through UART. In the latter case, care should be taken to avoid conflicts if the host computer already uses the UART connection to handle the controller TUI.

An example of the TUI developed for this project is presented in Fig. 4.23. This interface provides access to various implementation details, such as audio configuration, IP interface, control interface, etc., allowing users to verify that the running program is aligned with their specifications. The TUI also supports debug tests and manual I²C bus writes, which is useful for configuring connected audio codecs. Furthermore, custom menus have been added to allow for manual adjustments of specific application parameters (Marked as “unavailable” in this example).

Compilation Report

The compilation flow is a long and complex process that involves multiple software tools and scripts. Providing clear reports is essential for maintaining a good overview of the process and improving productivity. A concise synthesis of all the compilation step reports can help avoid the need to analyze each step’s logs if a problem occurs. Additionally, it should include a summary of all important parameters used during the process to ensure they align with user specifications.

The various types of FPGA resources that can be used, along with the design latency, are crucial



```

1 - Show info
2 - Print test
3 - GPIO test
4 - Reconfig audio
5 - Write on I2C bus
6 - [LMS] Toggle request_coeffs (unavailable)
7 - [LMS] Read Coeffs (unavailable)
8 - [LMS] Write output_selected (unavailable)
9 - [LMS] Write error_selected (unavailable)
10 - [LMS] Write Mu (unavailable)
11 - (unavailable)
12 -

```

Figure 4.23: Example of our TUI running on the SoC. It is fully controllable using the user buttons embedded on the development boards.

metrics that must be carefully analyzed (detailed in Chapter 5). These details should be easily accessible for diagnosing issues and optimizing the design.

A clear and concise report interface has been developed for this project (see Appendix E.1). The interface first details the user parameters (DSP, target FPGA, audio parameters). Reports from the HLS tool are then scraped to present only the critical information: resource usage and timing constraints. If any element exceeds 100%, the design will not fit on the FPGA, rendering the compilation unsuccessful. The same reporting process is applied to the implementation step, which includes the full hardware architecture. The CPU application is analyzed, and its main parameters are printed, such as protocol parameters (I²C speed, UART baud rate, debug verbose level, etc.). To facilitate debugging, the compiler report also scrapes the GPIO mapping file. This provides a clear presentation of the signals assigned to each GPIO, allowing the user to verify that the configuration matches their requirements.

Automating Compilation With Scripts

To make a compiler user-friendly, tasks should be automated as much as possible and controlled through simple commands. As discussed in Section 4.5.2, most *Xilinx* software can be automated using TCL scripts, which streamlines the compilation process. Moreover, other elements that do not rely on *Xilinx* software, such as GUI generation or compilation reports, can also be easily scripted. These scripts need to be executed wisely based on the commands given by the user and commands should be carefully designed to provide the user with sufficient control for efficient compilation management.

For example, compiling high-level files to hardware design using HLS generates a report on estimated resource usage and timing schedules. This synthesis is relatively fast (a few minutes) compared to synthesizing the entire design, which typically takes 10 to 30 minutes for a simple audio design. It is beneficial to perform only the HLS step initially to verify if the resource usage and timing schedules are compatible with the desired implementation. This iterative approach at the HLS level reduces compilation time, avoiding a full implementation that will not fit on the FPGA if the audio IP is already too large.

It is therefore essential to give the user control over this step, providing an option at compilation to either synthesize the entire design or just perform the HLS synthesis. *Syfala* uses command-line interfaces, offering greater portability and efficiency compared to a GUI. The command-line options not only allow users to selectively build specific parts of the project, saving time, but also provide control over various DSP parameters, such as sampling rate, bit width, and buffer sizes. Additionally, they enable users to choose different designs for the implementation, such

as audio interfaces (IP, TMD, integrated sigma-delta), Ethernet interface, multichannel interfaces, and more.

Some examples of *Syfala* command-line usage are provided below. The Command-Line Interface reference is provided in the Appendix G and is also available in the repository [Emea].

Lang: CMD

```
1 syfala examples/faust/virtualAnalog.dsp
```

Listing 4.2: *Syfala* command-line for fully compiling a FAUST program (virtualAnalog.dsp) into an FPGA bitstream and a CPU executable.

Lang: CMD

```
1 syfala examples/faust/virtualAnalog.dsp --arch --ip
```

Listing 4.3: *Syfala* command-line for compiling a FAUST program (virtualAnalog.dsp) and performing only the HLS synthesis step.

Lang: CMD

```
1 syfala examples/cpp/templates/gain.cpp
```

Listing 4.4: *Syfala* command-line for compiling a C++ program (gain.cpp) into an FPGA bitstream and a CPU executable.

Lang: CMD

```
1 syfala examples/cpp/templates/gain.cpp --csim tests/csim/csim_cpp_template.cpp --
  csim-inputs tests/stimuli --csim-iter 64
```

Listing 4.5: *Syfala* command-line for performing a C simulation (csim) on a C++ program (gain.cpp) with 64 iterations, using csim_cpp_template.cpp as the simulation test file, and fetching input samples from the tests/stimuli directory.

Lang: CMD

```
1 syfala examples/faust/virtualAnalog.dsp --b Z20 --sample-rate 24000 --sample-width
  24
```

Listing 4.6: *Syfala* command-line for compiling a FAUST program (virtualAnalog.dsp) into an FPGA bitstream and CPU executable, specifying the target board (Zybo Z20), sample rate (24 kHz), and sample width (24-bit).

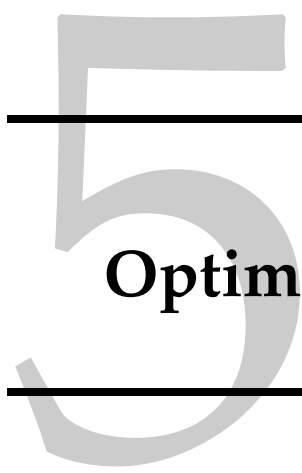
4.6 Conclusion

This chapter analyzed the hardware design aspects crucial for implementing audio DSP on FPGA platforms. We started by covering the fundamentals of FPGA design, including SoC architectures, IP block-based designs, and the use of HDLs and HLS to streamline the process. Next, we presented implementation guidelines as a practical framework for developing audio IP, based on DSP characteristics from Chapter 2. We also discussed interfacing with the analog domain, focusing on audio codecs and the protocols necessary for digital-to-analog integration. Our exploration of low latency codecs highlighted their alignment with ultra-low latency requirements. Lastly, we explored user control in audio systems, comparing simple baremetal controls with more complex embedded *Linux* systems. Baremetal offers a lightweight, easy-to-implement solution, while

OS-based systems provide greater flexibility and compatibility at the cost of added complexity.

All of these guidelines were then consolidated in a proposed FPGA audio compilation scheme, illustrating the journey from high-level DSP design to the integration of hardware and software components. This discussion culminated in one of the primary contributions of this thesis, and of the Emeraude Team: the *Syfala* compiler. *Syfala* is an audio compiler designed for programming FPGAs using FAUST and C++. This achievement underscores the significant effort required to develop a user-friendly and effective audio compiler, while also providing the insights and tools that made it possible.

In summary, this chapter has provided a robust framework for translating DSP algorithms into efficient, real-time hardware implementations. While these guidelines are sufficiently generic to be applied across various approaches, the next chapter demonstrates that an additional optimization step is necessary when using HLS.



Optimizing Audio on FPGA with C++ High Level Synthesis

Contents

5.1	Introduction and Principles	77
5.1.1	Resources and Latency: The Key Metrics in FPGA Design	77
5.1.2	AMD/Xilinx HLS Tools Principle	79
5.2	Optimizing Hardware Design With Sequential Language	85
5.2.1	Guiding the Implementation with Directives and Pragmas	85
5.2.2	Unrolling loops: The Sequential/Parallel Trade-Off	86
5.2.3	Pipelining Loops to Minimize Latency and Maximize Throughput	88
5.3	Introduction to Pragmas	89
5.3.1	Trivial Program Example	89
5.3.2	Pragma Unroll: Sequential/Parallel Control	90
5.3.3	Interpreting Latency Reports	91
5.3.4	Pragma Array_partition and Bind_storage: Control Over Memory Layout	93
5.3.5	Pragma Pipeline: Optimizing the Design Throughput	95
5.3.6	Pragma Interface: Configuring the IP Interface	96
5.4	Methodology for Hardware Designing Audio DSP in C++	100
5.4.1	Default Implementation	100
5.4.2	Optimizing Pipeline	102
5.4.3	Nested Loops: A Key to Major Optimizations	104
5.4.4	Unrolling: Enhancing Parallelism When Pipelining is Insufficient	109
5.4.5	Multi-Sample Design Guidelines: Exploiting FPGA Parallelism	111
5.4.6	Multi-Sample Case Study	117
5.5	Interfacing DSP IP With CPU and Memory	121
5.5.1	IP Interface	122
5.5.2	Memory Access Optimization	124
5.6	Final Evaluation and Insights on HLS Optimization for Audio DSPs	126
5.6.1	Further Recommended Guidelines	127
5.6.2	Concluding Thoughts on Automating HLS for Audio Applications	130

HLS tools significantly raise the level of abstraction compared to HDLs and allow the use of software languages to program FPGAs. Since these software languages are built to target instruction-based processors like CPUs and GPUs, they require a layer of hardware specification to guide the design. While most HLS tools can infer some hardware specifications based on the source program, designs in HLS often still require manual hardware optimization.

Although the previous chapter provides guidelines for implementing audio DSP programs using HLS as a backend, it does not specify the coding practices necessary for preparing the input file for the HLS. Despite the software programming abstractions, programmers must employ non-trivial hardware optimization techniques to achieve efficient implementations.

Many optimizations must be carefully analyzed, including loop pipelining, unrolling, memory banking, and buffering. While most of these optimizations can be managed by the tool, they are not specifically tailored for audio programs and may be unsuitable, necessitating explicit user directives. In addition to directives, HLS sometimes necessitates adapting the entire program's structure. For instance, HLS tools often optimize loop pipelining (i.e., reusing the same hardware for successive computations with minimal cycles between them) which is especially effective when the loop body contains multiple parallelizable instructions. However, since many audio programs (such as an FIR filter) are inherently sequential, optimizing them may require restructuring the source code to assist HLS tools in enhancing performance.

Focusing on audio programs, this chapter provides a methodology for writing code effectively to enable HLS to optimize implementation as much as possible. It offers information on how to guide HLS and provide the appropriate explicit directives. Although the FIR filter is used as an example throughout this chapter, the emphasis is not on optimizing this particular example but on providing generic guidelines applicable to any audio program. This approach may not yield the absolute best implementation but results in an optimized solution applicable to many audio programs.

The chapter is organized as the following:

- Section 5.1 explains the fundamentals of FPGA implementation metrics and provides insight into how the *AMD/Xilinx* synthesis tools work.
- Section 5.2 presents the principles of hardware design and optimization, and explains how to “guide” these principles using sequential programming languages.
- Section 5.3 details the major pragmas of *Vitis* HLS and explains how to use them.
- Section 5.4 proposes a methodology for rearranging C++ code and using pragmas to achieve efficient hardware optimization of audio programs. It provides several straightforward rules to enhance the implementation and includes a case study with an FIR filter for multi-sample approaches.
- Section 5.5 focuses on the IP interface for external word communication and memory optimizations.

It is also complemented with summarized design guidelines to provide the reader with essential insights for optimizing the design of audio DSP on FPGAs.

Throughout this chapter, all reports, resource usage percentages, implementation results, and similar metrics are based on the implementation on a *Zybo Z10 (Zynq 7010 SoC)*. Although these results are generally applicable, resource usage may differ when synthesizing the program for other platforms.

5.1 Introduction and Principles

This section introduces the FPGA and fundamental concepts necessary to understand hardware optimizations. An FPGA is primarily limited by the resources it provides. Understanding the different types of resources and their usage is crucial for properly implementing a design. This is followed by an analysis of the functionality of the software tools we use, equipping the reader with the essential knowledge to comprehend the other sections of this chapter.

5.1.1 Resources and Latency: The Key Metrics in FPGA Design

FPGA Architecture

An FPGA chip is made up of a finite number of logic blocks, a hierarchy of reconfigurable interconnects, and I/O blocks to allow the circuit to access the outside world. The main resources are presented briefly here, but we do not delve into the details, as this thesis does not require an in-depth understanding of FPGA hardware.

- Configurable Logic Blocks (CLBs) is the primary elements inside an FPGA, which contain two basic components: look-up tables (LUTs) and Flip-Flops (FFs). These basic logic unit are the building blocks of the FPGA and are used to implement combinational and sequential logic. Configurable Logic Block (CLB) are sometimes referred to as slices or logic cells.
- Digital Signal Processing blocks (DSP Block or DSP slices)¹, are designed for efficient arithmetic operations such as multiplication and addition.
- Block Random-Access Memorys (BRAMs) provides on-chip memory, which can be configured for various storage needs, such as data buffers and FIFOs.
- Configurable Interconnects are programmable routing resources that connect the CLBs, DSP Blocks, and memory blocks, allowing for the creation of complex logic circuits.
- Input/Output Blocks (IOBs) interface the FPGA with external devices, supporting various I/O standards for communication.
- Clock Management Blocks, including Phase-Locked Loops (PLLs) manage and distribute clock signals throughout the FPGA, ensuring synchronized operations across different parts of the chip.
- Some FPGAs may also feature embedded hard IP cores, such as PCIe controllers or Ethernet MACs, which provide optimized implementations of commonly used functions.

A look-up table is a programmable element used to implement combinational logic in digital circuits. It is typically made of 1 bit SRAM memories and multiplexers that allow it to implement any N to 1 logic function. Flip-Flops are binary storage elements used to synchronize logic and save logical states. FFs are essential for creating registers, counters, and state machines within digital circuits, as they allow the storage and retrieval of binary information in synchronization with a clock pulse. They introduce the notion of sequential logic inside an FPGA that would only be able to do combinational, time-independent logic without it. More information about LUT architecture can be found in this paper [Alt] from *Altera*. A more detailed study of FPGA architecture is available in [KTR08], and a recent analysis of their architectural evolution can be found in [BB21].

¹DSP blocks represent a third meaning of the acronym DSP. To avoid confusion, this is referred to as DSP blocks throughout this thesis, except in resource usage tables and figures, where the context should be sufficient for clarity.

Essential Resource Monitoring in Hardware Design

When implementing a hardware design, its complexity is constrained by the available resources, as exceeding 100% of the available resources for the target FPGA makes the design impossible to implement. Resource usage is particularly important to monitor as optimization can often reduce the consumption. Tracking resource usage can guide the optimization work. For instance, saturation of LUTs requires different optimization strategies compared to the saturation of BRAM. Implementation reports typically focus on LUT and FF usage rather than CLB utilization, as the number of LUTs and FFs per CLB can vary depending on the FPGA.

Depending on the resource type, their monitoring is more or less critical.

- LUTs are the fundamental logic blocks; the more complex the design, the more LUTs it requires. This metric is crucial to monitor, as LUTs are the most likely to reach saturation first.
- Flip-Flop (FF) usage is generally proportional to LUT usage, as a CLB contains both in proportion. This is also an important resource to monitor, given its dependence on design complexity.
- DSP Blocks are used for arithmetic operations, which represent a significant part of an audio DSP. Moreover, they can be used instead of LUTs for certain functions and vice versa (LUTs instead of DSP Blocks, though with relatively lower efficiency), making them as important a resource as LUTs to verify.
- BRAM represents the on-chip memory. As shown in Chapter 4.2.2, many audio objects are stored in BRAM, which can fill up quickly. Therefore, monitoring BRAM usage is critical.
- Configurable interconnects are a critical metric, but they are handled by the software during implementation and routing phase, the user does not have direct control over them for the purposes of this thesis.
- I/O blocks only becomes a concern if, for example, more ports are instantiated than what the chip can provide. However, this is not typically a design problem that can be optimized.
- Clock management blocks are generally used for the overall design architecture, such as generating clocks with specific frequencies for IP cores. These blocks should not be affected by the audio algorithm implemented, as all programs use the same clock interface.

To summarize, four resource types can limit audio DSP implementation: LUTs, flip-flops (FFs), DSP Blocks, and BRAM. This chapter aims to provide key strategies for programmers to optimize these resources effectively, enabling the design of complex algorithms.

IP Latency

In this thesis, latency refers to the time required to obtain the output sample of the audio IP after a sample is entered. This time can be expressed in microseconds or in clock cycles. The clock cycle period is determined by the master clock of the IP, and each IP can use a different clock if necessary. For example, with an input clock of 125 MHz, the clock cycle period is $\frac{1}{125\text{MHz}} = 8\text{ ns}$. If samples are processed at 48 kHz, this corresponds to a sampling period of 20.83 μs or 2 603 clock cycles at 125 MHz. If the IP uses a single-sample strategy, meaning no buffers are used and samples are processed one by one as they arrive, the IP has a maximum of 2 603 cycles to process each sample. Even with a multi-sample strategy, the IP must maintain coherence with the sample rate and ensure that the execution time does not exceed 2 603 cycles per sample. For instance, if 16 samples are processed at a time, the maximum latency would be $16 \times 2\,603 = 41\,648$ cycles. As discussed in Chapter 2.6.1, the sample rate is a critical constraint: if a sample is missed, the signal

is distorted. Therefore, the latency of the IP is a very important metric to control, as exceeding the maximum threshold invalidates the design.

5.1.2 AMD/Xilinx HLS Tools Principle

This section provides a quick review of the software tools used for this thesis in order to give the reader basis to understand next sections. It shows the key elements of graphical tools from the *AMD Xilinx Vitis* HLS software.

The *Vitis* HLS tool [Xila] allows users to synthesize a C/C++ function into RTL to design complex FPGA algorithms. The software facilitates debugging with two verification tools: C Simulation Tool, which allows for testing and validating the functionality and behavior of the C code with a C testbench. And C/RTL Co-Simulation which validates the RTL produced by HLS synthesis by reusing the C/C++ test bench used in C simulation and verifying that the RTL is functionally identical to the C source code. Moreover, it generates detailed reports about the simulation, synthesis, implementation, and co-simulation processes. These reports provide insights into the high-level synthesis process, including estimated FPGA resource usage, estimated latency and operating frequency, interface specifics, and more.

This section examines how the reports are interpreted in the context of our experiments. All the following reports are based on the program `fir_multisample.cpp` [The] provided in Appendix F p.198. Understanding the program is not necessary at this stage, readers who wish to comprehend it can refer to Section 5.4. The report files used are listed in the legend of each screenshot.

The program is a basic FIR filter. It generates samples for a sawtooth waveform in the `generate_input_samples` function. The `linear_buffer` function is used to store previous samples for the FIR filter, and these samples are processed through the FIR filter (which is a multiply and accumulate function) in `compute_fir`, with the coefficients of the FIR filter being in an external file (`coeffs_115.hpp`). The processed samples are then written to the output with the `write_output_samples` function. This simple program also uses a buffer of `BLOCK_NSAMPLES` to process several samples per IP run.

Resource Usage Main Reports

After synthesizing the program, *Vitis* generates several reports in separate files. The main synthesis summary report for the top-level function is automatically opened after a synthesis, and is named `csynth.rpt` by default. For our development, the most important part of these reports is the resource usage, which shows an estimation of the utilization of main resources (detailed in Section 5.1.1). This allows us to see if any resource exceeds 100% of its availability and to modify the source code accordingly.

Modules & Loops	Issue Type	Latency(cycles)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
▼ syfala		2659	-	2660	-	no	3	10	2758	3238	0
▼ computemydsp		2658	-	2658	-	no	3	10	2544	2840	0
▶ computemydsp_Pipeline_1		18	-	18	-	no	0	0	7	49	0
▶ computemydsp_Pipeline_generate_input_samples	II Violation	210	-	210	-	no	0	0	117	207	0
▶ computemydsp_Pipeline_linear_buffer		117	-	117	-	no	0	0	19	77	0
▼ computemydsp_Pipeline_compute_fir_coeffs	II Violation	2302	-	2302	-	no	1	5	1677	1147	0
▶ compute_fir_coeffs	II Violation	2300	20	20	115	yes	-	-	-	-	-
▶ computemydsp_Pipeline_write_output_samples		22	-	22	-	no	0	0	235	673	0

Figure 5.1: Example of synthesis summary report form *Vitis* HLS: resource estimates. File: `csynth.rpt`

An excerpt of a synthesis report is given in Fig. 5.1. The first column of the table lists the function names. `syfala` is the top-level function, equivalent to the main function in a C program, and it contains every other function. `syfala` includes the function `computemydsp`, which itself includes

generate_input_samples, linear_buffer, compute_fir_coeffs, and write_output_samples. To be precise, these are not C functions but loops and modules that HLS uses to handle the implementations. They use a naming convention of the following type: `computemydsp_Pipeline_<loop_name>` indicating that this module contains pipelined loops. These pipelined loops can be expanded from each module (to simplify the view, only the loop of `compute_fir_coeffs` is expanded in the figure). All the modules are sequential, but they all contain loops that are pipelined. Details about this are given in Section 5.3. It is important to understand that this module subdivision is not specifically equivalent to the functions in the C program. These modules are derived by *Vitis* from the C code, primarily based on loops or other optimizable algorithmic components.

The next column addresses issues that prevented HLS from correctly implementing and optimizing modules to meet the requirements.

Then, the latency in FPGA cycles is provided, which indicates the time needed for a function to process data. All the modules in `computemydsp` are sequential (non-pipelined). Thus, all latencies of sub-functions add up to give the latency of the top-level function, which is the cycle latency of the IP, i.e., the number of FPGA cycles needed between the start and the end of the IP.

The latency of a single iteration of a loop is specified in the *Iteration Latency* column, and the number of times this loop is iterated is specified in the *Trip Count* column. These fields are thus only available for lines corresponding to loops. The *interval* (or initiation interval) is different from latency: latency is the time for one data item to be processed, whereas the interval determines the rate at which data can be processed, corresponding to the throughput. In this example, since none of the modules are pipelined, the interval is equal to the latency. However, the loops within the modules are pipelined, as highlighted by the `compute_fir_coeffs` loop, which indicates a latency of 2300 cycles but an interval of 20 cycles. More details about pipelining and how this part of the report is calculated are explained in Section 5.3.3.

Finally, the report provides the usage of the main resources detailed in the previous section: BRAM, DSP Block, FF, and LUT. By giving the target FPGA architecture to the tools, it has access to the maximum number of available resources and can display them as a percentage of use. Other columns can be shown but are not relevant at the moment.

Module-Specific Reports

For each module, the tool generates a dedicated report with more detailed information, such as the exact utilization and distribution of each resource. The convention for these reports is `<Module_name>_csynth.rpt`. For example, Fig. 5.2 shows the resource distribution for the `syfala` function in `syfala_csynth.rpt`.

This top-level function only calls the `computemydsp` function and defines the interface. The report displays the resource distribution based on the instances called within the function (Fig. 5.3), in this case: the interface (AXI) and the call to `computemydsp`. Other sections are folded as they are not relevant here.

It is possible to go into the nested `computemydsp` function report by opening `computemydsp_csynth.rpt` (Fig. 5.4). The first table presents the resource distribution, with the total showing the resource usage for the `computemydsp` function alone this time. The distribution by instance clearly illustrates how the submodules utilize these resources, with the modules corresponding to the loops in the functions called by `computemydsp`, (already shown in Fig. 5.1).

To delve further, the file `computemydsp_Pipeline_compute_fir_coeffs.rpt` can be analyzed. This module is the core of the DSP, handling the FIR filter with the multiply-and-accumulate function. This time, the instances called are not user-defined modules but adder and multiplier implementations provided by HLS, as illustrated in Fig. 5.5. There are various types of adders and multipliers

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	24	-
FIFO	-	-	-	-	-
Instance	3	10	2752	3200	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	14	-
Register	-	-	6	-	-
Total	3	10	2758	3238	0
Available	432	360	141120	70560	0
Utilization (%)	~0	2	1	4	0

Figure 5.2: Utilization estimates for the syfala function. File: syfala_csynth.rpt.

Detail

Instance

Instance	Module	BRAM_18K	DSP	FF	LUT	URAM
grp_computemydsp_fu_80	computemydsp	3	10	2544	2840	0
control_s_axi_U	control_s_axi	0	0	208	360	0
Total		2	3	10	2752	3200

DSP

Memory

FIFO

Expression

Multiplexer

Register

Figure 5.3: Instance detail for the syfala function. File: syfala_csynth.rpt

Summary

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	2	-
FIFO	-	-	-	-	-
Instance	1	10	2435	2515	-
Memory	2	-	64	8	0
Multiplexer	-	-	-	315	-
Register	-	-	45	-	-
Total	3	10	2544	2840	0
Available	432	360	141120	70560	0
Utilization (%)	~0	2	1	4	0

Detail

Instance

Instance	Module	BRAM_18K	DSP	FF	LUT	URAM
grp_computemydsp_Pipeline_1_fu_30	computemydsp_Pipeline_1	0	0	7	49	0
grp_computemydsp_Pipeline_compute_fir_coeffs_fu_50	computemydsp_Pipeline_compute_fir_coeffs	1	5	1677	1147	0
grp_computemydsp_Pipeline_generate_input_samples_fu_36	computemydsp_Pipeline_generate_input_samples	0	0	117	207	0
grp_computemydsp_Pipeline_linear_buffer_fu_44	computemydsp_Pipeline_linear_buffer	0	0	19	77	0
grp_computemydsp_Pipeline_write_output_samples_fu_59	computemydsp_Pipeline_write_output_samples	0	0	235	673	0
fadd_32ns_32ns_32_6_full_dsp_1_U21	fadd_32ns_32ns_32_6_full_dsp_1	0	2	237	222	0
fmul_32ns_32ns_32_4_max_dsp_1_U22	fmul_32ns_32ns_32_4_max_dsp_1	0	3	143	140	0
Total		7	1	10	2435	2515

Figure 5.4: Utilization estimates and instance detail for the computemydsp function. File: computemydsp_csynth.rpt.

that HLS can provide, which can be tuned with directives. This report details the cost of each operation, allowing for precise tuning to balance resource usage.

It is also interesting to observe the usage of other resources, such as memory, as shown in Fig. 5.6. This figure illustrates the BRAM configuration that *Vitis* HLS uses to store the 115 coefficient words. Similarly, the exact implementation and usage of other elements such as expressions, LUTs, and registers are detailed in Fig. 5.7. These details provide many insights to guide kernel optimization.

Summary

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	222	-
FIFO	-	-	-	-	-
Instance	-	5	380	362	-
Memory	1	-	0	0	-
Multiplexer	-	-	-	563	-
Register	-	-	1297	-	-
Total	1	5	1677	1147	0
Available	432	360	141120	70560	0
Utilization (%)	~0	1	1	1	0

Detail

Instance

Instance	Module	BRAM_18K	DSP	FF	LUT	URAM
fadd_32ns_32ns_32_6_full_dsp_1_U10	fadd_32ns_32ns_32_6_full_dsp_1	0	2	237	222	0
fmul_32ns_32ns_32_4_max_dsp_1_U12	fmul_32ns_32ns_32_4_max_dsp_1	0	3	143	140	0
Total	2	0	5	380	362	0

Figure 5.5: Utilization estimates and instance detail for the module inside the compute_fir function. File: computemydsp_Pipeline_compute_fir_coeffs.rpt.

Memory									
Memory	Module	BRAM_18K	FF	LUT	URAM	Words	Bits	Banks	W*Bits*Banks
coeffs_U	computemydsp_Pipeline_compute_fir_coeffs_coeffs_ROM_AUTO_1R	1	0	0	0	115	32	1	3680
Total		1	0	0	0	115	32	1	3680

Figure 5.6: Memory detail for the module inside the compute_fir function. File: computemydsp_Pipeline_compute_fir_coeffs.rpt.

Expression									
Variable Name	Operation	DSP	FF	LUT	Bitwidth P0	Bitwidth P1			
add_in54_10_fu_615_p2	+	0	0	14	7	3			
add_in54_11_fu_625_p2	+	0	0	14	7	2			
add_in54_12_fu_635_p2	+	0	0	14	7	2			
add_in54_13_fu_483_p2	+	0	0	14	7	1			
add_in54_1_fu_525_p2	+	0	0	14	7	4			
add_in54_2_fu_535_p2	+	0	0	14	7	4			
add_in54_3_fu_545_p2	+	0	0	14	7	4			
add_in54_4_fu_555_p2	+	0	0	14	7	4			
add_in54_5_fu_565_p2	+	0	0	14	7	4			
add_in54_6_fu_575_p2	+	0	0	14	7	4			
add_in54_7_fu_585_p2	+	0	0	14	7	3			
add_in54_8_fu_595_p2	+	0	0	14	7	3			
add_in54_9_fu_605_p2	+	0	0	14	7	3			
add_in54_509_p2	+	0	0	15	8	4			
empty_23_fu_498_p2	+	0	0	15	8	4			
icmp_in51_fu_477_p2	icmp	0	0	10	7	5			
Total		16	0	222	114	54			

Multiplexer				
Name	LUT	Input Size	Bits	Total Bits
ap_N5_fsm	106	21	1	21
ap_done_int	9	2	1	2
ap_sig_allocacmp_i_1	9	2	7	14
fTemp_address0	54	10	4	40
fTemp_address1	54	10	4	40
grp_fu_383_p0	49	9	32	288
grp_fu_383_p1	37	7	32	224
grp_fu_387_p0	49	9	32	288
grp_fu_387_p1	37	7	32	224
grp_fu_391_p0	26	5	32	160
grp_fu_395_p0	26	5	32	160
i_fu_90	9	2	7	14
samples_address0	49	9	8	72
samples_address1	49	9	8	72
Total	563	107	232	1619

Register				
Name	FF	LUT	Bits	Const Bits
add_in54_13_reg_755	7	0	7	0
ap_CS_fsm	20	0	20	0
ap_done_reg	1	0	1	0
coeffs_load_reg_780	32	0	32	0
fTemp_load_10_reg_886	32	0	32	0
fTemp_load_11_reg_891	32	0	32	0
fTemp_load_12_reg_916	32	0	32	0
fTemp_load_13_reg_921	32	0	32	0
fTemp_load_14_reg_946	32	0	32	0
fTemp_load_15_reg_951	32	0	32	0
fTemp_load_1_reg_791	32	0	32	0
fTemp_load_2_reg_806	32	0	32	0
fTemp_load_3_reg_811	32	0	32	0
fTemp_load_4_reg_826	32	0	32	0
fTemp_load_5_reg_831	32	0	32	0
fTemp_load_6_reg_846	32	0	32	0

Figure 5.7: Expression, multiplexer and register detail for the module inside the compute_fir function. File: computemydsp_Pipeline_compute_fir_coeffs.rpt.

Accurate Reports

The synthesis process is relatively fast (a few minutes) compared to the classic FPGA workflow. This allows these reports to be generated and interpreted without requiring the entire process to run. However, synthesis only provides an estimation that can sometimes be quite far from the actual implementation result. These results are only estimates because the tool cannot account for the final optimizations or routing in the synthesized or implemented design.

To obtain a more accurate implementation size of the RTL design, *Vitis* HLS can run the full implementation with placement and routing options. This yields a very precise report of the timing and resource usage for the IP. However, this process is significantly longer. Moreover, the report may still be slightly inaccurate, as the true implementation depends on the final global design.

This difference in report accuracy is summarized in Table 5.1 which shows the results yielded

by the estimation and the post-implementation reports. It also includes the *Vivado* estimation and the final actual implementation. The HLS estimation is very pessimistic, showing more than double the actual usage for LUT estimation. This behavior appears to be general for the majority of programs we tested, although there are occasional exceptions where the trend is reversed.

Resource	HLS Syn.	HLS Impl.	Vivado estimation	Final design
DSP	10	10	10	10
FF	3 521	2 923	2 933	2 923
LUT	4 119	1 975	2 143	1 978
BRAM	3	5	2	2

Table 5.1: Difference in Vitis HLS (synthesis and implementation) and Vivado resource usage estimation.

All these reports are in .rpt format, which can be opened as plain text, facilitating the extraction of data for the custom compilation report of our audio compiler (discussed in Chapter 4.5.4). The reports in this section were presented with screenshots to familiarize the user with the actual graphical interface. However, for the sake of consistency, all the reports in this thesis are exported and presented as tables.

Function Call Graph

In addition to the reports, *Vitis* HLS provides a function call graph that allows a simple visual representation of the program hierarchy, as shown in Fig. 5.8. The Function Call Graph displays functions as nodes and the call relationships between them as edges, showing the hierarchy and organization of the code. The diagram offers information about latency and throughput, helping identify the critical path in the design and potential bottlenecks.

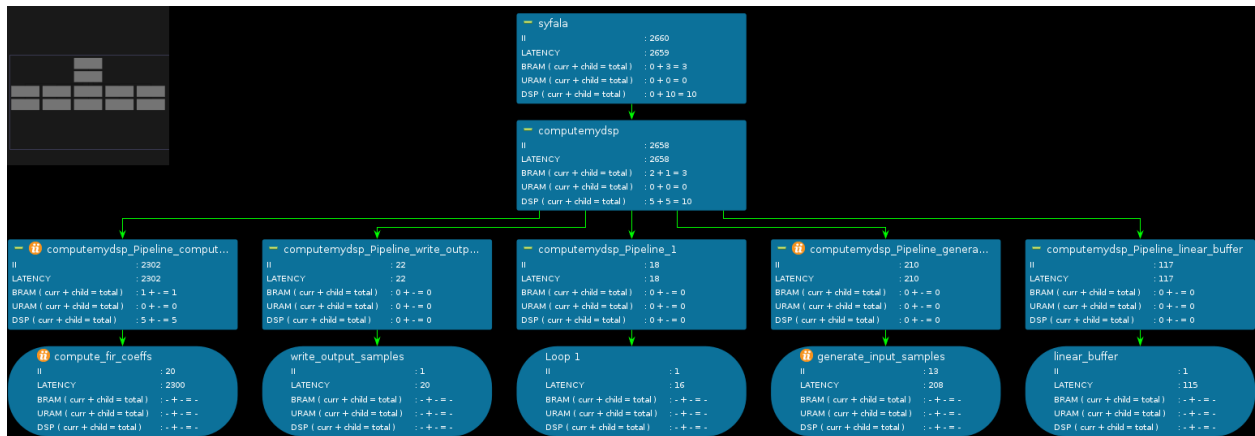


Figure 5.8: Function Call Graph.

Schedule Viewer

To analyze latency, scheduling, and dependencies precisely, the schedule viewer can be used. It displays a timeline with each operation, such as arithmetic or logical operations, represented as a bar indicating when it starts and ends in clock cycles. This visualization shows the sequence and parallelism of operations. By highlighting the links to previous and subsequent operations, the schedule viewer helps identify data dependencies and loop dependencies that are preventing parallelism. It also provides a means to quickly find timing violations.

With the *Vitis* interface, one aspect that must be carefully considered is the operation/control step indication on the top horizontal axis, which shows clock cycles. This does not represent the total execution time if it refers to a call function/module. For example, Fig. 5.9 shows the schedule view of the `computemydsp` function. The four functions we previously discussed can be easily identified (`generate_input_samples`, `linear_buffer`, `compute_fir_coeffs`, and `write_output_samples`). However, it might be tempting to think that each of these functions has a 1-cycle execution time, which is not the case. This only applies to the calling process. By opening the schedule viewer for the `generate_input_samples` module (Fig. 5.10), it can be seen that the module has a latency of 13 cycles. Therefore, programmers must ensure that they examine the schedule for the lowest-level module if they want to interpret the horizontal axis as the execution cycle count. To our knowledge, the schedule viewer does not allow for determining the execution time of higher-level functions like `computemydsp`.



Figure 5.9: Schedule Viewer of `computemydsp`.



Figure 5.10: Schedule Viewer of `generate_input_samples`.

The pipelined (reused) loops are represented only once on the scheduler (with a single bar). To determine the number of times they are executed, the *trip count* column of Fig. 5.1 p.79 can be used.

Vitis also provides a dataflow viewer for dataflow implementations, but this is not detailed here.

Now that the tools have been described, the next section covers how to optimize the C program for hardware implementation, leveraging these tools to analyze optimization results and iteratively refine the program for improvement.

Usability and Consistency Issues in *Vitis* HLS

Before concluding this section on the use of the *Vitis* HLS tool, it is important to address the usability of the tool itself, as it has significantly impacted both this thesis and the writing of this manuscript.

While HLS tools are complex and alternative options from major manufacturers are limited, *Vitis* HLS is a highly inconsistent and sometimes poorly documented software. The highly unpredictable behavior of this tool makes it challenging to automate optimizations. Although most default behaviors produce good results for basic programs, our experience has shown that for more complex programs, there are almost always nonsensical optimizations that require user intervention to be corrected. Furthermore, the default behavior frequently changes from one version to another, making it difficult to provide generic coding guidelines, as we attempt to do in this thesis. One of the most frustrating examples is the threshold for automatic loop pipelining.

In version 2022.2, a threshold is introduced, as indicated in the documentation [Xilb]:

“The config_compile -pipeline_loops command enables loops to be pipelined automatically based on the iteration count. All loops with an iteration count below the specified limit are automatically pipelined. The default is 64.”

However, in the next version, the behavior is inverted while keeping the same threshold.

“The config_compile -pipeline_loops command enables loops to be pipelined automatically based on the iteration count. All loops with a trip count greater than the specified limit are automatically pipelined. The default limit is 64.”

Although these are only default behaviors and can be overridden with pragmas, they add a layer of confusion when attempting to establish generic optimization guidelines.

At the start of this thesis the tools used were in the 2019 version. The migration to version 2020 required numerous changes to the scripts and demanded a significant amount of work. We attempted to keep up with updates until version 2022.2, at which point the migration to each new, incompatible version every six months became too labor-intensive. Newer versions appear to correct some nonsensical behaviors bugs we encountered and could facilitate HLS use. However, version 2022.2 was the last version we could use without frequent crashes on our *Linux* system, leading to the decision to remain with this version.

This entire thesis uses version 2022.2, and if the reader wishes to achieve the same results as those presented here, synthesis should be performed with version 2022.2. However, it is strongly advised that readers who wish to further this work update to the latest version, as it appears to address many of the bugs present in earlier versions. A solution to these issues may be to bypass the HLS tools by generating HDL languages directly from the FAUST or C++ with our compiler. This approach has been studied and is discussed in the future work Chapter 7.

5.2 Optimizing Hardware Design With Sequential Language

Synthesizing a hardware design from a C++ program involves providing numerous guidelines to the compiler to direct the implementation process. A single sequential program can be represented in multiple hardware configurations, each varying in resource utilization and latency. The HLS tool can infer some design rules based on the C program, but a high-level control flow language like C/C++ does not provide sufficient information for the tool to automatically determine the most optimized implementation. The programmer must provide additional information to the HLS. In *Vitis*, this is done through pragmas and directives embedded directly within the source code.

5.2.1 Guiding the Implementation with Directives and Pragmas

The pragma directive is a method specified by the C standard for providing additional information to the compiler. It is a compiler-specific directive, meaning that different compiler vendors may use it for their own purposes. In *Vitis* HLS, pragmas can be used to optimize the design, reduce

latency, improve throughput performance, and decrease area and device resource usage in the resulting RTL code.

Vitis HLS offers optimization commands in the form of directives and pragmas. Directives can be specified as TCL commands and scripted separately from the source code, enabling different optimizations without modifying the code itself, whereas pragmas are applied directly within the source code. For clarity and because our audio DSP programs typically have only one file, only pragmas are used in this thesis.

Another important aspect of *Vitis* HLS is the use of “Configuration Commands”, which allow users to configure the *Vitis* HLS tool to control the results of synthesis and simulation. Although they do not directly optimize the code, configuration commands influence the behavior of certain pragmas and the default behavior of the HLS tool (including inferred hardware optimizations). Therefore, their use is also valuable for hardware optimization.

5.2.2 Unrolling loops: The Sequential/Parallel Trade-Off

To understand how pragmas work, it is important to understand why they are needed. Consider a simple example of implementing an FIR filter with N coefficients. An FIR filter algorithm applies a series of coefficients to a sequence of input data points by performing a multiply-and-accumulate operation: each input sample is multiplied by a corresponding filter coefficient, and the results are summed to produce the filter’s output. More details about the FIR filter are provided in Section 5.4.1. The C++ code for this algorithm is straightforward: it involves a loop of N iterations performing the multiply-and-accumulate operation, as depicted in Listing 5.1.

Lang: C

```
1 for (int n = 0; n < N; ++n) {
2     result += samples[n] * coeffs[n];
3 }
```

Listing 5.1: C program of an FIR filter

Designing a hardware circuit corresponding to this simple algorithm can take many different forms. For example, the loop can be implemented in a fully sequential manner (see Fig. 5.11): only one multiplier and one adder are instantiated, and they are used sequentially N times to handle the N iterations of the loop. In this case, a register is used to store the value from the previous iteration.

If the system can only handle one value at a time (i.e., a value needs to be in the register before the next sample and coefficients are processed by the multiplier), then the execution time is approximately:

$$T_{exec} = (T_{Mult} + T_{Add}) \times N$$

With T_{Mult} and T_{Add} the latency of the multiplier and the adder respectively.

On the other hand, one can consider a fully parallelizable implementation (see Fig. 5.12): each iteration of the loop has its own hardware to perform the loop function. This necessitates the instantiation of N multipliers. For summation, it is not feasible to compute the sum of the products fully in parallel. Instead, an adder tree can be used. The depth of the adder tree scales as $\log_2$ of the number of coefficients in the filter. Thus, the number of adders required is given by $\sum_{k=1}^{\log_2(n)} \frac{n}{2^k}$ (where n is the number of coefficients), or approximately $n - 1$.

Given the same hypothetical conditions as before, the execution time is:

$$T_{exec} = T_{Mult} + (\log_2(N) \times T_{Add})$$

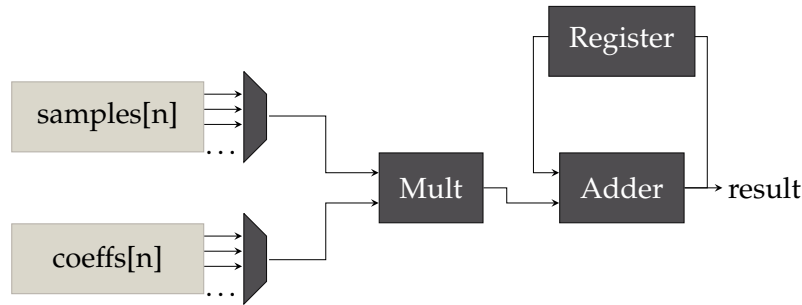


Figure 5.11: Simplified fully sequential implementation of an FIR filter.

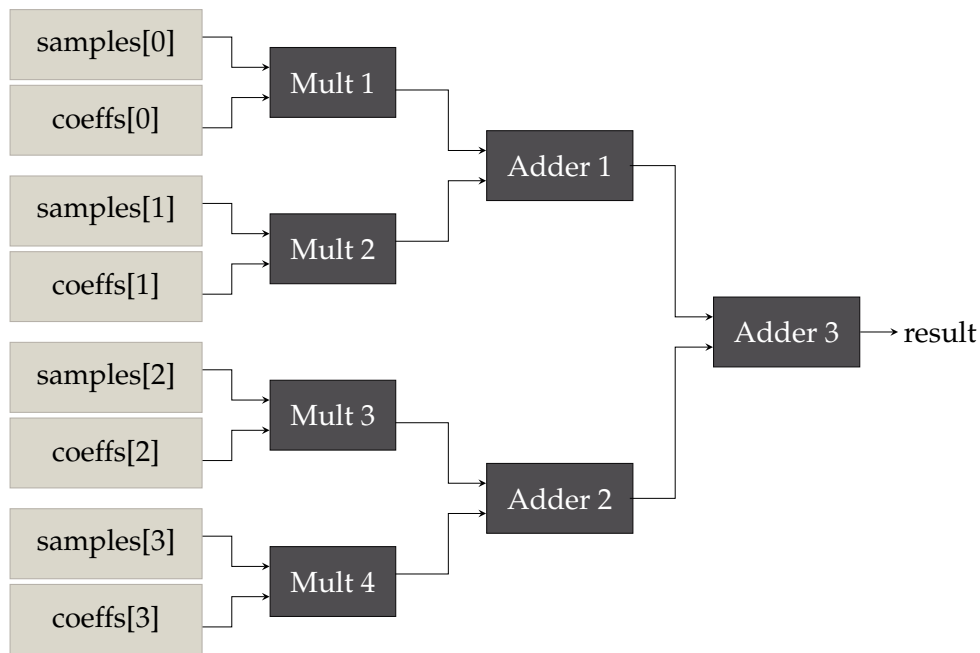


Figure 5.12: Simplified fully parallel implementation of an FIR filter.

For a loop to be parallelized, it must inherently allow parallelization. Even though techniques like buffering and variable copying can sometimes be employed to introduce concurrency, some algorithms are inherently sequential and offer no opportunities for parallel execution. This inherent sequential nature can be due to dependencies between iterations, where each iteration relies on the results of the previous one, making it impossible to process them concurrently. These two examples provide a simplified view of "opposite" implementations for clarity. They do not account for memory dependencies or include any pipeline optimizations.

In this simplified scenario, the first sequential implementation requires one multiplier and one adder, but results in significant latency. The second parallel implementation requires N multipliers and $N - 1$ adders, but it is much faster.

Based on the principle that intermediate implementations are also possible by adjusting the ratio of parallelization to sequentialization, this approach results in a range of implementations that balance resource usage and latency as illustrated in Fig. 5.13. Partially parallelized implementations can be the preferred solution offering some improvement of performance while not requiring as many resources as a full parallelized design.

The programmer must choose an implementation strategy with the objective of using less than 100% of the available resources and achieving a latency lower than the maximum permissible

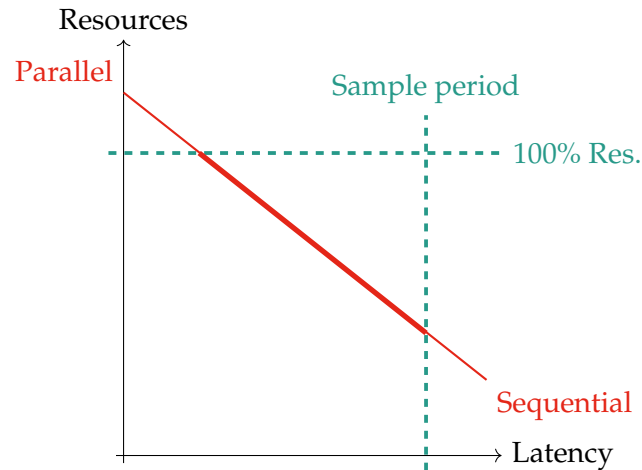


Figure 5.13: Theoretical trade-off between resource usage and latency in hardware implementations

latency (e.g., the sample period for a single-sample implementation). The method of adjusting this trade-off through *Vitis* and the `unroll` pragma is discussed in the next section.

5.2.3 Pipelining Loops to Minimize Latency and Maximize Throughput

The previous sequential example was explained using a non-pipelined approach. When a loop or a function contains several operations executed sequentially, pipelining allows subsequent iterations of the loop to overlap and run concurrently by enabling the operations to handle new data even if the global function has not yet completed processing. This enables concurrent execution of operations, allowing the function to process new inputs every N clock cycles, where N is the initiation interval (II) of the loop or function. An II of 1 means a new input is processed every clock cycle.

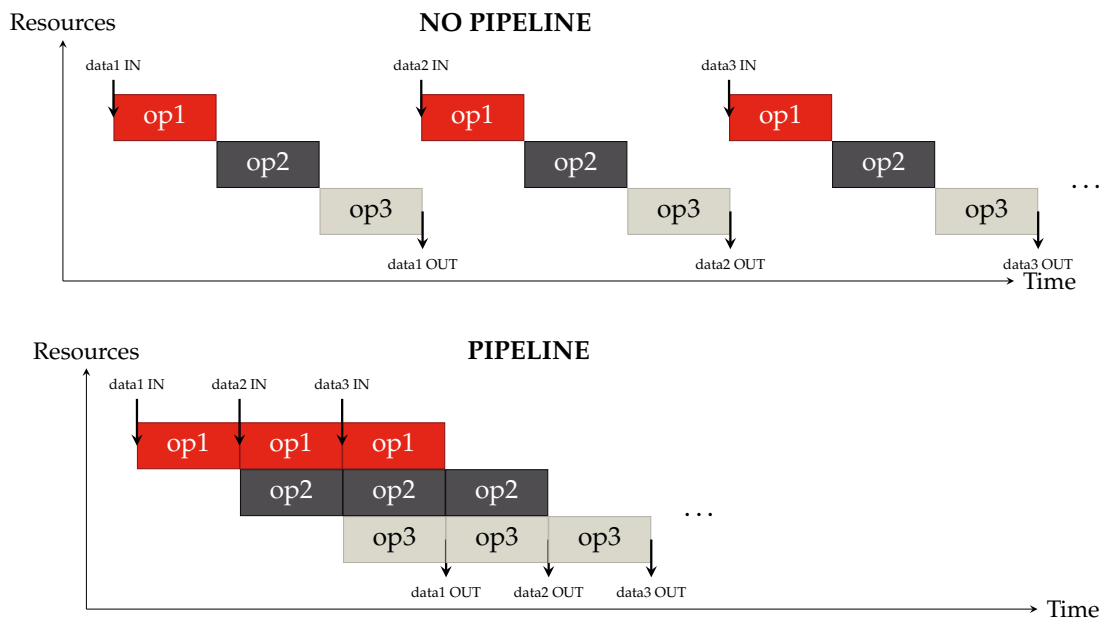


Figure 5.14: Pipeline principle.

In Fig. 5.14, the top example illustrates the default sequential operation (without pipelining). If each operation (op1, op2, and op3) takes 1 clock cycle to execute, the entire function takes 3 clock

cycles to complete. Consequently, each new data point can only be processed after 3 clock cycles, resulting in an initiation interval of 3.

In the second example, with pipelining enabled, the first operation can begin processing data as soon as the data is passed to op2, and op1 is free. The entire function still takes 3 clock cycles to execute, but each new data can be processed every clock cycle, reducing the initiation interval to 1. Although the latency remains at 3 clock cycles, the throughput improves significantly. For this example, processing 3 data points takes 9 clock cycles without pipelining, compared to 5 clock cycles with pipelining.

Pipelining reuse the same hardware for successive computation. The initiation interval is therefore a critical factor in optimizing the performance of hardware designs. *Vitis* HLS strives to generate the minimum II for a design. For some design and with appropriate techniques and optimizations, it is possible to achieve an II of 1, however, it requires the source program to be structured in loops whose body contains many instructions that can be run in parallel. By minimizing the II, the design can handle more data in a given period with minimal or no additional hardware cost, thereby maximizing the efficiency and performance of the hardware. An initiation interval that is sufficiently low can result in an almost fully parallelized implementation without the need for hardware duplication, making it more efficient than unrolling in many cases.

5.3 Introduction to Pragmas

By demonstrating how to use pragmas to achieve the desired synthesis behavior with a simple program, this section aims to provide the necessary insights to understand the complexity of hardware design constraints and the interactions among pragmas (such as memory dependencies, loop dependencies, latency bottlenecks, etc.). However, given the large number of pragmas addressing various use-cases and the fact that they may slightly change between different versions of *Vitis* HLS, this section does not aim to provide an exhaustive application of pragmas. For extensive details, the *Vitis* documentation should be consulted.

When synthesizing a program with *Vitis* HLS, the tool can use either the *Vivado* IP flow or the *Vitis* Kernel flow. This choice determines how the design is implemented, affecting the default settings and behavior of HLS. Throughout this thesis, only the *Vivado* IP flow is used (with all the default configurations it implies). For more detailed information, refer to the relevant documentation. [Xilf]

5.3.1 Trivial Program Example

To better visualize how to use pragmas, this section examines a very basic example and details how to configure the synthesis. It uses the program `basic_unroll.cpp` [The], which simply performs a loop of N iterations where it accumulates the values of an input array A . The result of the accumulation is then returned by the top-level function.

This program, while not representing any specific audio application, provides valuable insights into the behavior of various sequential audio examples, such as the FIR filter. The core of the program is shown in Listing 5.2.

```
1 for (x = 0; x < N; x++) {  
2     out_accum += A[x];  
3 }
```

Lang: C

Listing 5.2: Simple example of a loop.

5.3.2 Pragma Unroll: Sequential/Parallel Control

The trade-off depicted in Fig. 5.13 p.88 can be adjusted in the source program by using pragmas at the loop level. If the automatic pipeline is disabled using the HLS `pipeline off` pragma inside loops, a C loop is implemented as fully sequential: each operation within the loop is implemented once, and the loop is executed sequentially for N iterations.

After synthesizing `basic_unroll.cpp` with only the HLS `pipeline off` pragma, the schedule shown in Fig. 5.15 is generated. It can be seen that there is only one adder (line 13) and one multiplier (line 11). The trip count of this module in the synthesis summary (not shown here for clarity) indicates that this loop is executed 100 times, corresponding to the sequential approach described in the previous section.

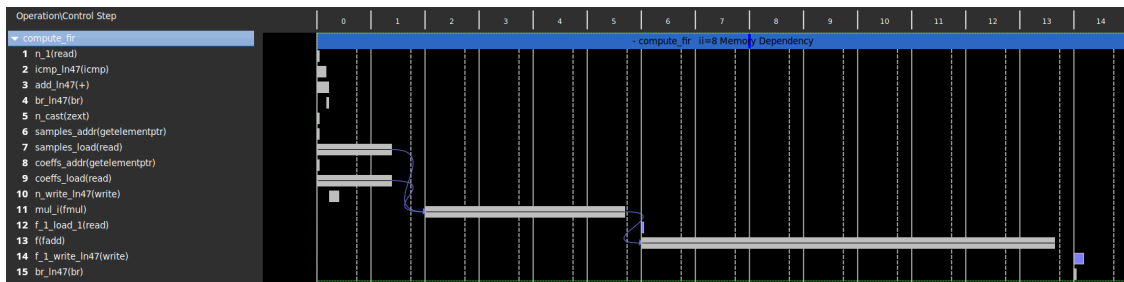


Figure 5.15: Schedule viewer for `basic_unroll.cpp` with no pragmas or configuration commands.

A fully parallel implementation can be achieved using the pragma `HLS unroll`. This pragma can take a factor as an option to partially unroll the loop, providing control over the degree of parallelization, as shown in the following listing:

```
Lang: C
1 #pragma HLS unroll factor=<F>
```

By specifying a factor F , the synthesis creates the logic for F iteration in parallel, and this parallel design is executed sequentially by the loop N/F times. If the factor is not specified, the loop is fully unrolled.

Fig. 5.16 shows the latency and resource usage for different unroll factors. Note: In this and many other figures, there are two y -axes: the left axis represents LUT and FF counts (associated with the gray and green curves), while the right axis represents latency (in clock cycles), corresponding to the red curve. Unrolling loops implies a trade-off by achieving higher performance but at the cost of using extra resources.² The graph represents values from 1 (no unroll) to 100. Between 51 and 100, all the values are the same because, above 50, the synthesis creates a single iteration “loop” with the logic for 100 parallel loop instances, which is a full unroll.

Simplifying the concept, unrolling a loop by a factor of 2 doubles the hardware cost while reducing the loop’s latency by half. Similarly, unrolling by a factor of 4 quadruples the resource usage and reduces latency by a factor of 4. This pattern continues as the unrolling factor increases. So the latency reduction follows a $1/X$ trend, becoming quickly marginal compared to the linear increase in hardware cost. Therefore, excessive loop unrolling provides diminishing benefits. This trend is clearly illustrated in the graph, where resource usage increases linearly, while latency reduction follows an inverse function.

²BRAM and DSP Block usage are not displayed in this graph and the following as they do not change significantly with the unroll factor.

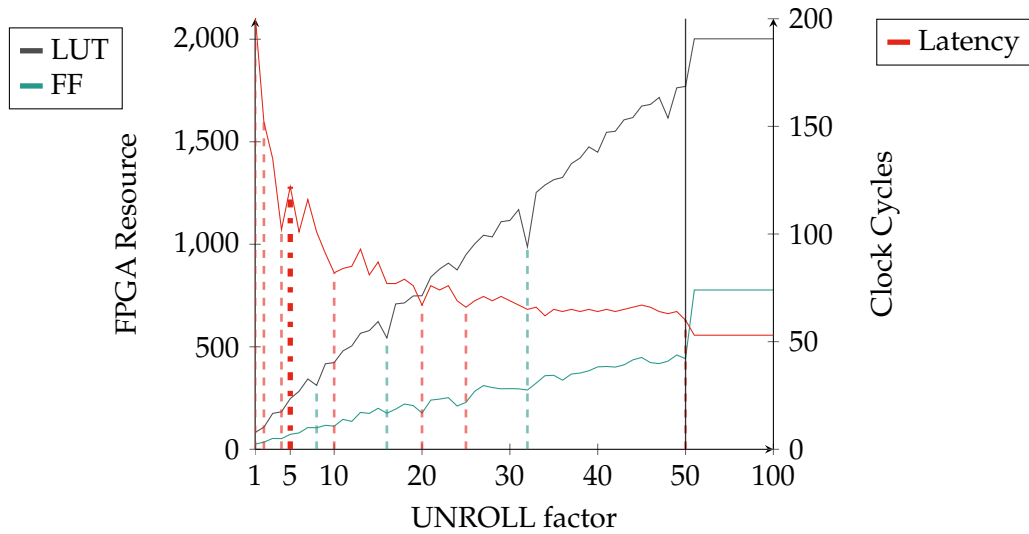


Figure 5.16: Resource usage and latency of `basic_unroll.cpp` (no pipeline) for varying unroll factors.

The plot is not as linear as expected, and it appears that some values are more optimized than others. This is a major concern for our application because they appear difficult to predict. A linear plot would typically allow control over the resource/latency trade-off and predict the synthesis: if resource usage is too high, one could simply lower the unroll factor. However, in this case, a reduction in the unroll factor can lead to increased resource usage depending on the values. This unpredictability makes automation of the optimization process by an audio compiler particularly challenging.

These optimized unroll values are difficult to predict because they arise from various factors. For example, resource usage tends to be better for values indicated by vertical lines, corresponding to powers of 2 (8, 16, and 32), as these values generally simplify the design. This can be verified in Fig. 5.17 showing the resource usage by expression for an unroll factor of 15 and 16. Although the unroll factor of 16 does use more expressions, it can employ OR operations instead of additions, which are less costly.

Other optimizations may appear depending on the unroll factor and the nature of the design (such as variable bit width, number of iterations, memory type, etc.). These optimizations are the reason why the plots are not perfectly straight, but they are challenging to explain without a detailed hardware analysis of the resulting design, which is beyond the focus of this thesis.

It is crucial to enable the Unsafe Math Optimization configuration command when using unrolling with floating-point arithmetic, as IEEE floating-point operations are neither commutative nor associative (see Appendix A). Changing the order of computation can lead to small variations in the results.

5.3.3 Interpreting Latency Reports

For latency, the “sweet spots” of the unroll factor are related to perfect partial unrolling. When N , the total number of loop iterations, is not perfectly divisible by the unroll factor F , the HLS tool implements an exit check in the loop to terminate it mid-execution during the last iteration. This partially executed loop makes the design less efficient and increase the latency.

In such cases, the last partially unrolled loop is not included in the trip count. For example, Table 5.2 shows the latency and trip count report generated by *Vitis* for an unroll factor of 40. It indicates that the trip count is 2. The design implements two loops with the logic for 40 parallel

Variable Name	Operation	DSP	FF	LUT	Bitwidth P0	Bitwidth P1
add ln35 10 fu 531 p2	+	0	0	14	9	9
add ln35 11 fu 567 p2	+	0	0	32	32	32
add ln35 12 fu 545 p2	+	0	0	14	9	9
add ln35 13 fu 558 p2	+	0	0	13	10	10
add ln35 1 fu 377 p2	+	0	0	14	9	9
add ln35 2 fu 412 p2	+	0	0	13	10	10
add ln35 3 fu 443 p2	+	0	0	32	32	32
add ln35 4 fu 294 p2	+	0	0	14	9	9
add ln35 5 fu 347 p2	+	0	0	13	10	10
add ln35 6 fu 383 p2	+	0	0	14	9	9
add ln35 7 fu 424 p2	+	0	0	13	10	10
add ln35 8 fu 434 p2	+	0	0	12	11	11
add ln35 fu 403 p2	+	0	0	39	32	32
add ln840 10 fu 483 p2	+	0	0	14	7	4
add ln840 11 fu 493 p2	+	0	0	14	7	4
add ln840 12 fu 511 p2	+	0	0	14	7	4
add ln840 13 fu 521 p2	+	0	0	14	7	4
add ln840 14 fu 473 p2	+	0	0	14	7	4
add ln840 1 fu 274 p2	+	0	0	14	7	2
add ln840 2 fu 256 p2	+	0	0	14	7	2
add ln840 3 fu 300 p2	+	0	0	14	7	3
add ln840 4 fu 284 p2	+	0	0	14	7	3
add ln840 5 fu 324 p2	+	0	0	14	7	3
add ln840 6 fu 310 p2	+	0	0	14	7	3
add ln840 7 fu 363 p2	+	0	0	14	7	4
add ln840 8 fu 334 p2	+	0	0	14	7	4
add ln840 9 fu 457 p2	+	0	0	14	7	4
add ln840 fu 241 p2	+	0	0	14	7	1
ap return	+	0	0	32	32	32
out_accum 5 fu 575 p2	+	0	0	32	32	32
icmp ln1027 fu 467 p2	icmp	0	0	10	7	6
Total		31	0	521	368	311

Variable Name	Operation	DSP	FF	LUT	Bitwidth P0	Bitwidth P1
add ln35 10 fu 553 p2	+	0	0	13	10	10
add ln35 11 fu 559 p2	+	0	0	14	9	9
add ln35 12 fu 573 p2	+	0	0	14	9	9
add ln35 13 fu 588 p2	+	0	0	13	10	10
add ln35 14 fu 598 p2	+	0	0	12	11	11
add ln35 1 fu 298 p2	+	0	0	14	9	9
add ln35 2 fu 311 p2	+	0	0	13	10	10
add ln35 4 fu 403 p2	+	0	0	14	9	9
add ln35 5 fu 432 p2	+	0	0	39	32	32
add ln35 6 fu 465 p2	+	0	0	14	9	9
add ln35 7 fu 506 p2	+	0	0	13	10	10
add ln35 8 fu 607 p2	+	0	0	32	32	32
add ln35 9 fu 512 p2	+	0	0	14	9	9
add ln35 fu 324 p2	+	0	0	32	32	32
add ln840 fu 365 p2	+	0	0	14	7	5
ap return	+	0	0	32	32	32
out_accum 5 fu 615 p2	+	0	0	32	32	32
icmp ln1027 fu 349 p2	icmp	0	0	10	7	6
or ln840 10 fu 455 p2	or	0	0	7	7	4
or ln840 11 fu 483 p2	or	0	0	7	7	4
or ln840 12 fu 493 p2	or	0	0	7	7	4
or ln840 13 fu 530 p2	or	0	0	7	7	4
or ln840 14 fu 540 p2	or	0	0	7	7	4
or ln840 1 fu 284 p2	or	0	0	7	7	2
or ln840 2 fu 270 p2	or	0	0	7	7	2
or ln840 3 fu 339 p2	or	0	0	7	7	3
or ln840 4 fu 355 p2	or	0	0	7	7	3
or ln840 5 fu 383 p2	or	0	0	7	7	3
or ln840 6 fu 393 p2	or	0	0	7	7	3
or ln840 7 fu 409 p2	or	0	0	7	7	4
or ln840 8 fu 419 p2	or	0	0	7	7	4
or ln840 9 fu 445 p2	or	0	0	7	7	4
or ln840 fu 255 p2	or	0	0	7	7	1
Total		33	0	444	384	325

Figure 5.17: Resources usage per expressions for an unroll factor of 15 (left) and 16 (right).

loop instances, leaving 20 instances to execute to reach the total number of iterations of 100. The latency of one loop is reported as 26 clock cycles, resulting in 52 clock cycles for the two loops executed sequentially. The Iteration Latency represents the time required for a data point to be processed by the function, so the 64 clock cycles correspond to the latency of the two sequentially executed loops (52) plus the time required to execute the remaining 20 loop instances. Executing the first 20 parallel loop instances within the loop takes 12 clock cycles (shown in the scheduler, not included here for clarity). Adding this to the 52 clock cycles gives a total of 64 clock cycles, which aligns with the *Vitis* report. This demonstrates that when using a non-multiple unroll factor, the programmer should be cautious with the report, avoiding the assumption that latency is simply the trip count times the iteration latency.

Function	Latency	Iteration Latency	Trip Count
syfala	65	-	66
LOOP1	64	26	2

Table 5.2: Latency and trip count report of an unroll factor of 40 for the program basic_unroll.cpp. Taken from the Performance and Resource Estimates table in csynth.rpt report.

Now that the mechanics of partial unrolling are understood, it can be acknowledged that designs not perfectly divisible by the unroll factor are likely to result in suboptimal implementations. Since sequential loops inherently implement parallelized circuits, using them partially for the last iteration is inefficient.

However, if Fig. 5.16 shows small improvements for unroll values that are divisors of the iteration count, highlighted with vertical red bars (1, 2, 4, 10, 20, 25, 50), this is not the case for a factor of 5. Conversely, some non-perfect factors also demonstrate optimization. The reason of these two behaviors lies in the memory access bottlenecks presented in next section, which prevents the loop from being implemented entirely in parallel.

5.3.4 Pragma Array_partition and Bind_storage: Control Over Memory Layout

The schedule viewer for an unroll factor of 5 generates the scheduling of operations shown in Fig. 5.18. It indicates that not all operations are fully parallelized, although some are concurrent. The bottleneck arises from the reads of variable *A*, labeled as *A_load_<X>* in the scheduler. These reads are paired two by two: *A_load_1* (line 12) and *A_load_2* (line 16), followed by the next read *A_load_3* (line 22) and *A_load_4* (line 26), with each subsequent read occurring one clock cycle later.

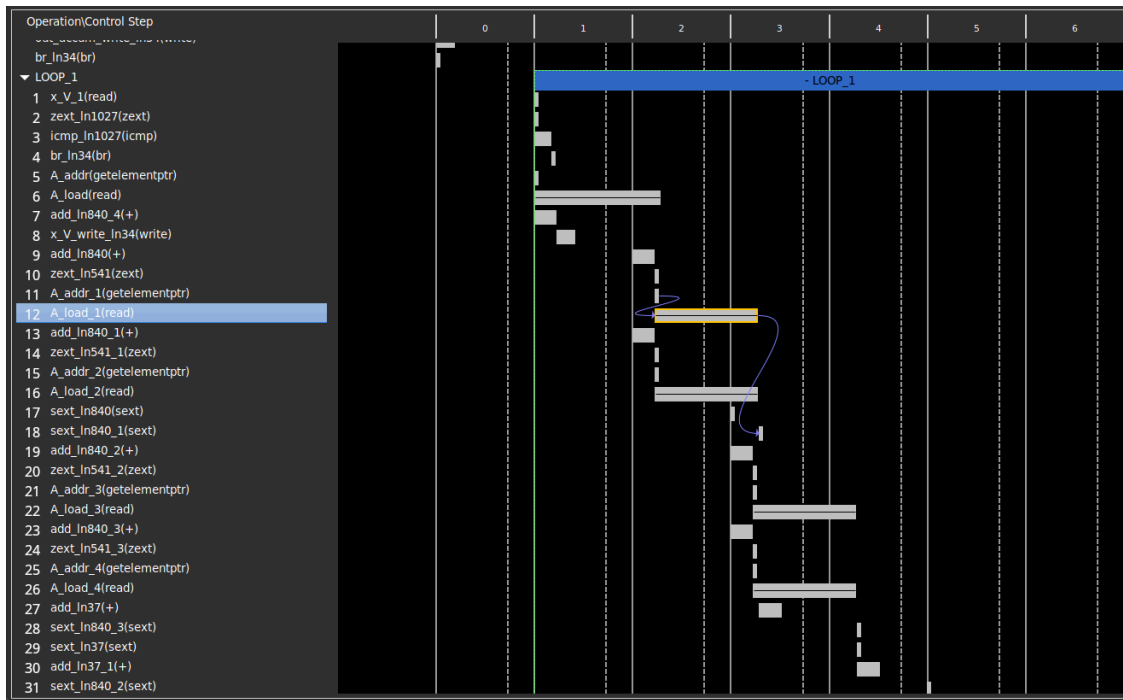


Figure 5.18: Schedule viewer for the program `basic_unroll.cpp` with no pipeline and an unroll factor of 5.

What happens is that, by default, *Vitis* HLS uses a dual-port RAM for interface variables (i.e., top-level function arguments), which allows up to two reads per cycle. Thus, with a dual-port RAM, an odd unroll factor that requires an odd number of reads in the loop results in a sequential read inside the loop. For example, with an unroll factor of 4, there is 25 loops with 2×2 reads, whereas a factor of 5 results in 20 loops with $2 \times 2 + 1$ reads. Given that a read has a latency of 2 cycles, this results in $2 \times 2 \times 25 = 100$ cycles for the factor of 4, versus $(2 \times 2 + 2) \times 20 = 120$ cycles for the factor of 5. This illustrates why an unroll factor of 5, even though it is a perfect divisor of 100, does not yield specifically optimized results.

To resolve this, the variable *A* can be fully partitioned into individual elements, allowing access to all elements in one cycle. By adding the following pragma to the main function, *Vitis* completely partitions *A*:

```
1 #pragma HLS array_partition variable=A complete dim=0
```

Lang: C

The resulting synthesis, shown in Fig. 5.19, illustrates the impact of this partitioning: the latency is significantly improved, but at the cost of substantial resource usage.

By using multiple small memories instead of a single large memory, the partitioning significantly increases the number of read (and write) ports for storage, allowing all 100 reads to be completed

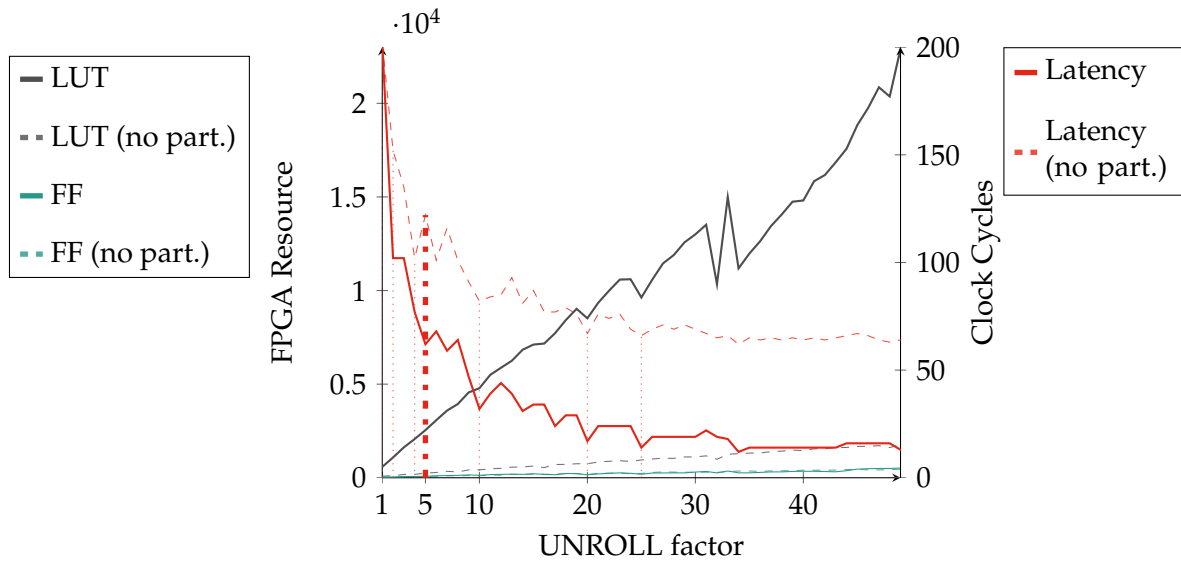


Figure 5.19: Resource usage and latency of `basic_unroll.cpp` (no pipeline) with a complete partitioning of `A` for varying unroll factors. The scale for resource usage (left) changes significantly compared to the previous plots.

in one cycle. The scheduler (Fig. 5.20) demonstrates that all reads are now fully concurrent. Even successive additions are performed within the same cycles, resulting in a latency on the order of 10 cycles, or even less, for all 100 iterations.

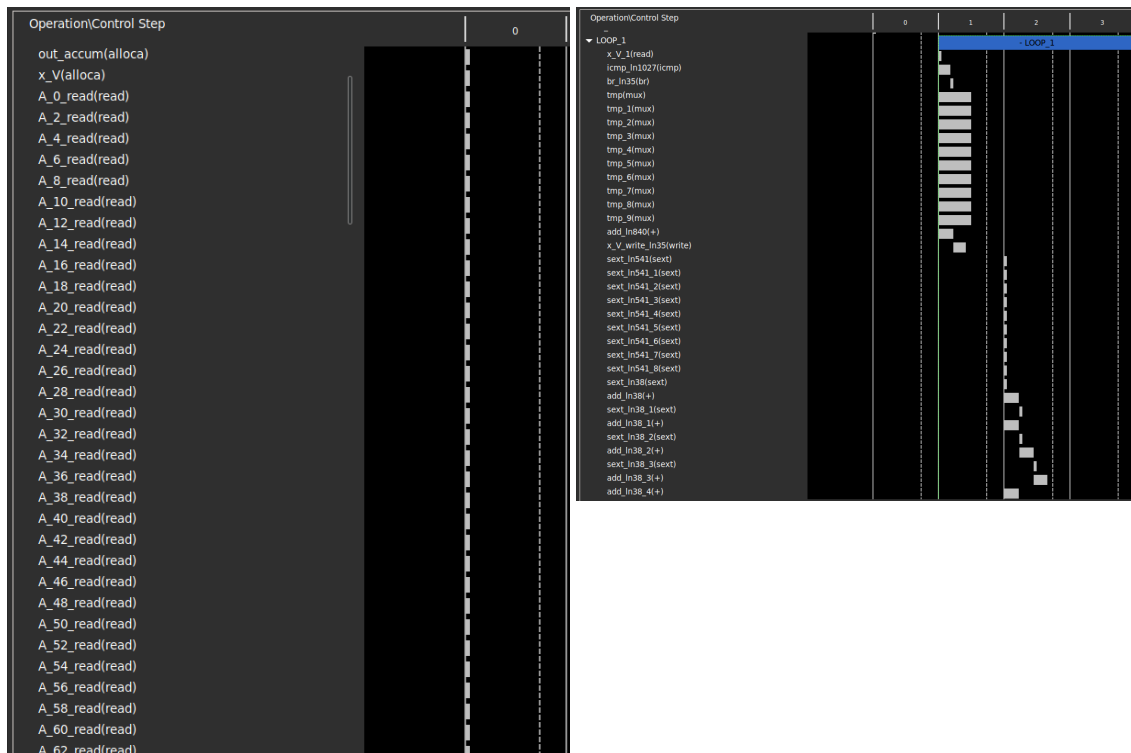


Figure 5.20: Schedule viewer for the program `basic_unroll.cpp` with a complete partitioning of `A` and a full unroll. Left: beginning of the scheduler, including all the reads. Right: end of the function with the computations.

Performing a full partition is indeed costly in terms of resources and may be too aggressive for a viable design. However, it is possible to specify a partition factor in the pragma to control

the degree of partitioning. This approach can potentially improve both latency and throughput but still requires more memory instances or registers. Such pragmas are particularly useful for sequential algorithms with memory dependencies, like an FIR filter.

Nevertheless, this time the latency on Fig. 5.19 shows effective optimization for all divisors of the iteration count, including the factor of 5.

To conclude on the important pragma for memory mapping, the HLS `bind_storage` pragma allows specific memory types to be assigned to variables. If this pragma is not specified, *Vitis* HLS determines the appropriate memory type automatically. Arrays in the C code can be targeted to any memory resource, such as FIFO, RAM, and ROM, which can be implemented using different resources such as BRAM, LUTRAM, or URAM. The programmer can thus tailor the design to use the desired memory type, as discussed in Chapter 4.2.4.

5.3.5 Pragma Pipeline: Optimizing the Design Throughput

By default, C function loops in *Vitis* HLS are inherently rolled or pipelined to enhance performance. Thus, pipelining a loop corresponds to simply not applying any pragma. Depending on the *Vitis* HLS version, this can only be applied to loops with an iteration count above or below a threshold, which is set to 64 by default. As discussed at the end of Section 5.1.2, this default behavior is highly dependent on the tool version and is not even properly documented for our version, 2022.2.

In our case, and without any pragmas, a non-nested loop is always pipelined, regardless of the iteration count. This unusual behavior has been reported, and *Vitis* support has acknowledged it, stating they will attempt to patch this bug.

With nested loops, the following behavior applies:

- If the trip count of inner loop exceeds `syn.compile.pipeline_loops` threshold: pipeline the innermost loop .
- Else, if trip count of inner loop is less than or equal to `syn.compile.pipeline_loops` threshold: pipeline the parent loop.

Moreover, with nested loops, pipelining an outer loop results in the automatic unrolling of any inner loops.

Synthesizing the initial `basic_unroll.cpp` program without using the HLS `pipeline off`, and varying the unroll factor results in the plot in Fig. 5.21 illustrating the resource usage and latency. The resource usage remains similar to the implementation without pipelining (showed with dashed plot), but the latency shows a significant improvement. After an unroll factor of 50, the loops are fully unrolled, meaning no hardware is reused. As a result, pipelining is not possible, and there is no difference in performance compared to the previous results.

The latency plot exhibits a sawtooth pattern, which is attributed to the memory bottleneck previously discussed in Section 5.3.4. Specifically, when using odd unroll factors, the loop requires an odd number of reads. Consequently, some reads are executed sequentially rather than concurrently, causing occasional spikes in latency. This variation is more pronounced with lower unroll factors due to the larger number of loop iterations they generate, impacting latency more significantly compared to higher unroll factors, where the effect is less noticeable due to fewer loop iterations.

Despite this, pipelining substantially enhances performance by enabling different stages of the loop to execute concurrently. Although resource usage is comparable, pipelining allows for the processing of new data at a faster rate, approaching or even achieving a new data processing cycle every clock cycle at barely no cost.

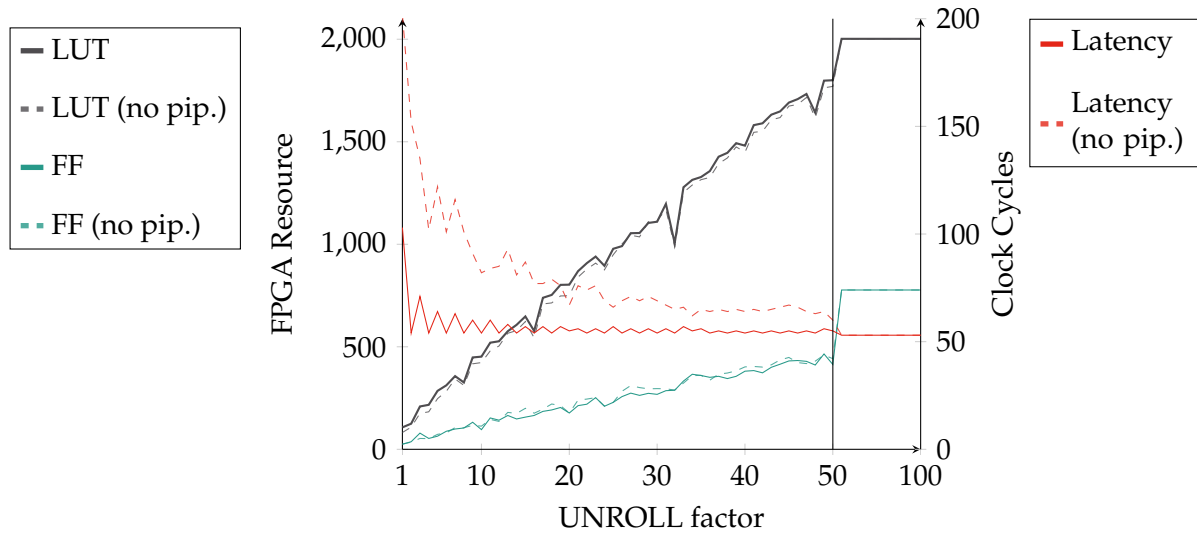


Figure 5.21: Resource usage and latency of `basic_unroll.cpp` pipelined, for varying unroll factors.

The pipeline pragma also provides options for fine-tuning it when it is enabled. It can, for example, specify the pipeline style (stall, flushable, or free-running), though this aspect won't be discussed further here. More importantly, the pragma allows for the specification of the initiation interval (II) using the `II` option, as shown in the following listing:

Lang: C

```
1 #pragma HLS pipeline II=<II>
```

By default, when pipelining is applied, *Vitis* HLS minimize the II, according to the specified clock period constraint. The focus is primarily on meeting the timing requirements rather than strictly minimizing the II unless the `II` option is explicitly specified. Specifying an `II=1` with the pipeline pragma does not guarantee a pipeline with an initiation interval of one clock cycle every time, achieving that consistently would be ideal but is not always feasible. However, by setting a very low II, *Vitis* HLS becomes much more aggressive in trying to meet this constraint, which can occasionally lead to timing violations if the design can't meet the strict requirements. If *Vitis* HLS is unable to create a design with the specified II, it issues a warning and creates a design with the lowest possible II. The factors preventing the tool from reaching the desired II are detailed in the report, and it becomes the programmer's responsibility to analyze the design and the warning messages to determine the necessary steps to achieve the required initiation interval. In this example, forcing the II to 1 does not alter the outcome, as the tool already generates the lowest possible II for such a simple architecture.

To conclude on pipelining, it's crucial to understand that *Vitis* HLS can only apply pipelining optimizations to loops. If a loop is manually unrolled in the program, pipelining optimization becomes impossible.

5.3.6 Pragma Interface: Configuring the IP Interface

When synthesizing the C++ code into an RTL design, it's essential to configure the port interfaces (inputs and outputs of the IP) and the execution model.

Defining Interfaces: Port-Level and Axi Interface Protocols

The top-level function arguments of the C/C++ code are synthesized into RTL I/O ports and are automatically implemented using port-level protocols or AXI interface protocols. As discussed in the introduction of Section 5.3, the *Vivado* IP flow is chosen over the *Vitis* kernel flow for building the IP. One of the main reasons for this choice is that the *Vitis* kernel flow only supports AXI4 interfaces, whereas the *Vivado* IP flow also supports port-level protocol interfaces. This flexibility is particularly advantageous for handling stream signals (such as audio signals) directly, bypassing the need for AXI interfaces.

The *Vivado* IP flow supports different interface paradigms, with each paradigm supporting various interface protocols to communicate with external systems:

- **Stream Paradigm:** Data is streamed into the kernel from another streaming source or streamed out of the kernel. This paradigm supports FIFO and AXI4-Stream interface types.
- **Register Paradigm:** Data is accessed by the kernel through register interfaces via register reads and writes. This paradigm supports port-level handshakes interfaces [Xile], AXI4-Lite, and no protocol.
- **Memory Paradigm:** Data is accessed by the kernel through memory interfaces. This paradigm supports port-level memory interfaces [Xile], AXI4 Memory Mapped (*m_axi*), and BRAM interfaces.

Details about AXI implementations are provided in Section 5.5.1

The interface generated by default (when no pragma is specified) in *Vitis* is derived from the data type and direction of the arguments of the top-level function and its return type. This synthesis process is detailed in the *Vitis* documentation [Xild]. A table derived from this documentation synthesizes the default and supported paradigms and interfaces (see Table 5.3).

C-Argument Type	Supported Paradigms	Default Interface Protocol
Scalar variable (pass by value)	<i>Register</i>	None
Array	<i>Memory, Stream</i>	Memory interface
Pointer	Memory, Stream, <i>Register</i>	Input:none / Output:handshake
Reference	<i>Register</i>	Input:none / Output:handshake
hls::stream	<i>Stream</i>	Fifo

Table 5.3: Default paradigms and interfaces inferred by *Vitis* HLS, as referenced in the *Vitis* documentation [Xild]. Default paradigms are indicated in *italics*.

The generated interface is more clearly visible when viewed using the *Vivado* block design environment. The IP synthesized from `basic_unroll.cpp` without any pragmas (see Listing 5.3) in the *Vivado* block design environment produces the IP block illustrated in Fig. 5.22.

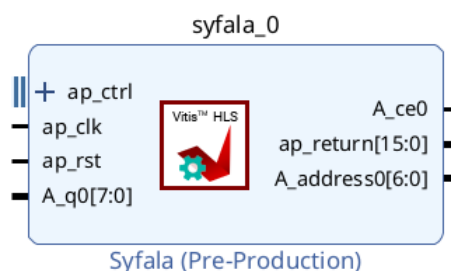


Figure 5.22: IP synthesized from `basic_unroll.cpp` with no pragmas.

Lang: C

```

1 ap_int<16> syfala (ap_int<8> A[N])
2 {
3     ap_int<16> out_accum = 0;
4     ap_int<16> x;
5 LOOP_1:
6     for (x = 0; x < N; x++) {
7         out_accum += A[x];
8     }
9     return out_accum;
10 }

```

Listing 5.3: basic_unroll.cpp program with top-level function arguments

The input ports are displayed on the left side of the IP, while the outputs are on the right. In the source C++ program, the table A is represented as an array and is thus implemented here with a memory interface. This automatically infers ports such as A_ce0 (chip-enable), A_q0 (input data: read), and A_address0 (address). The return argument is implemented using a handshake protocol.

Instead of using a statically defined table for A, it can be beneficial to use a pointer. Similarly, rather than passing the return value directly, it can be declared as a pointer to potentially accommodate multiple return values. This modification results in the program shown in Listing 5.4 and generates the IP illustrated in Fig.5.23.

Lang: C

```

1 void syfala (int* A, int* output)
2 {
3     int out_accum = 0;
4     int x;
5
6     for (x = 0; x < N; x++) {
7         out_accum += A[x];
8     }
9     *output=out_accum;
10 }

```

Listing 5.4: Modification of the basic_unroll.cpp interface for pointers.



Figure 5.23: IP synthesized from basic_unroll.cpp with pointers (no pragmas).

By default, as shown in Table 5.3, input pointers arguments are implemented as simple wire ports without associated handshaking or control signals. Output pointers are by default implemented with an associated output valid signal to indicate when the output data is valid. In Fig.5.23, A (an input pointer) is implemented without a control protocol (ap_none), meaning there are no additional control signals associated with it. The output pointer is implemented with a wire handshake control with the output_r_ap_vld signal to indicate when the output data is valid. When the port has no I/O protocol (either by default or by design), the input data must be held stable until it is read, and it can be challenging to determine when to read the output data.

The remaining `ap_ctrl` interface is related to the execution mode of the IP.

Defining Execution Mode: Block-Level Control Protocols

The execution mode of the HLS design determines how and when the IP begins execution, how it handles new inputs, and how it signals completion of operations. There are several execution modes:

- Sequential: requires the current transaction to complete before starting a new one. This mode is similar to the one-sample approach detailed in Chapter 4.2.3.
- Overlapping: allows the next transaction to begin before the current one has completed, enabling concurrent execution.
- Auto-Restarting: automatically restarts the execution once the current operation completes, often useful for continuous data streams.

The execution mode is determined by the block-level control protocol assigned to the HLS design. For the *Vivado* IP flow, the default execution mode is sequential (`ap_ctrl_hs`). This means that the IP requires one iteration to complete before it can start a new one, ensuring that each transaction is fully processed before moving on to the next.

Controlling the Interface With Pragma

To summarize the previous paragraphs, in the default *Vivado* IP flow, the tool generates two main types of interface ports on the RTL design to manage the flow of both data and control: block-level control protocol and port-level interface protocols. Additionally, every IP design includes a third set of ports necessary for the operation of the IP: Clock and Reset Ports (`ap_clk` and `ap_rst`). These ports are essential for the synchronization and initialization of the IP.

The Interface pragma can be used to modify the block-level control protocol of an IP. This is done by specifying the port name "return" and selecting one of the supported protocol. For instance, considering the program used to illustrate the Figure 5.23, the following pragma can be added at the beginning of the top-level function.

Lang: C

```
1 #pragma HLS interface mode=ap_ctrl_none port=return
```

The `return` keyword is a specific identifier that indicates the pragma should be applied to the control protocol. This removes the control protocol (`ap_ctrl`), leaving only the input and output ports along with the clock and reset signals.

Interface pragma can also be used to configure each port interface to use other port-level protocols or an AXI interface protocol. For example, an implementation is provided using the following pragmas:

Lang: C

```
1 #pragma HLS interface ap_hs port=A
2 #pragma HLS interface s_axilite port=output
```

It configures the output variable to use an AXI-Lite bus and apply a handshake protocol to the A port. This change is depicted in Fig. 5.24.

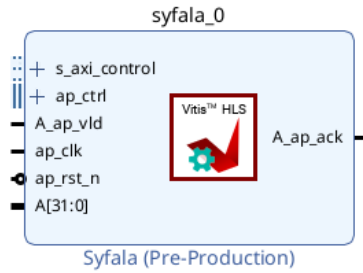


Figure 5.24: IP synthesized from `basic_unroll.cpp` with AXI-Lite interface and handshake protocol on A.

Other pragmas can indirectly alter the interface by modifying the nature of the arguments, thereby changing the default inferred protocol. For example, a user can generate an IP from the example studied in Section 5.3.4 with the following pragma.

Lang: C

```
1 #pragma HLS array_partition variable=A dim=0 complete
```

It creates an IP with 100 distinct inputs, each 8 bits wide in parallel, corresponding to the 100 values of the table A.

5.4 Methodology for Hardware Designing Audio DSP in C++

This section proposes a systematic methodology for optimizing hardware implementations of audio programs by strategically reorganizing C++ code and applying pragmas. By following a set of clear and practical guidelines, this approach enables significant improvements in hardware performance and resource utilization for audio DSPs implementations.

The methodology is designed to streamline the process of converting high-level C++ code into efficient hardware descriptions, making it easier to achieve optimal results in FPGA implementations. However, it does not provide the optimal implementation for a specific case. Achieving such optimization would require a detailed analysis of the specific code. The goal of this section is to offer guidelines that are sufficiently generic to be applicable to most audio programs without requiring knowledge of the exact content.

5.4.1 Default Implementation

A Typical Example: The FIR Filter

The FIR filter is a good representation of typical audio programs because, while it involves straightforward operations (multiplication and addition), it can become computationally intensive when the filter order is high. A Finite Impulse Response (FIR) filter operates by convolving the input signal with a vector of constant coefficients, resulting in a finite response. Unlike Infinite Impulse Response (IIR) filters, FIR filters do not have internal feedback.

An FIR filter of order N applied on a signal $(x[n])_{n \geq 0}$, produces a signal $(y[n])_{n \geq N-1}$ such that:

$$y[n] = \sum_{i=0}^{N-1} C_i \cdot x[n-i]$$

Where $(C_i)_{0 \leq i \leq N-1}$ are the coefficients of the filter. Although the detailed design process is beyond the scope here, FIR filters can implement a variety of responses, such as passband or stopband filtering over specific frequency ranges. In many design approaches, increasing the number of coefficients can allow for sharper transitions or narrower bands.

FIR filters typically require storing recent input samples for each new output calculation. In software (e.g., on a CPU), this is often done with a circular buffer, allowing the same data structure to be reused efficiently without physically shifting the entire array. In hardware-based designs (e.g., FPGAs), a shift register is more common: input samples are shifted at each new y coefficient computed. Indeed, as the computation of $y[n]$ need samples $x[k]$ for $n - (N - 1) \leq k \leq n$, the computation of $y[n + 1]$ uses the same coefficients plus $x[n + 1]$ and dropping $x[n - (N - 1)]$ (i.e. $x[k]$ for $n - (N - 2) \leq k \leq n + 1$). There are many ways to implement shift registers, here we use shifting because it is more intuitive. However, the discussion on the different types of buffers is addressed in Section 5.5.2.

The core of the C++ code we are starting from is shown in Listing 5.5 (NCOEFFS is the number of coefficients, which are stored in the coeffs array). This code is encapsulated within a top-level function that declares all I/O audio ports and is synthesized into the IP by Vitis HLS. However, the core computation of the filter occurs in the compute function. It is executed after a read function, which retrieves the next input sample, and before a write function, which outputs the result (here, fTemp).

Lang: C

```

1 void compute ([...])
2 {
3     samples[0] = input_sample;
4     for (int n = 0; n < NCOEFFS; n++) { //Filter
5         fTemp += samples[n] * coeffs[n]; //fTemp contains the result y[n]
6     }
7     for (int j = NCOEFFS-1; j > 0; j--) { //Linear buffer
8         samples[j] = samples[j-1];
9     }
10 }

```

Listing 5.5: Basic FIR filter implementation.

Hardware Implementation

By default, Vitis HLS pipelines non nested loop (behavior for 2022.2 version). The linear buffer must wait for the filter computation to finish before shifting the values because of the dependency on samples, so the two loops are sequential. The first loop executes NCOEFFS times in a pipelined process, followed by the sample loop, which also executes NCOEFFS times pipelined. Although not the most efficient approach, this behavior is somewhat constrained by the inherent memory dependencies of the algorithm.

Since the loops are pipelined, the latency can still be low if the initiation interval is low enough. The samples loop (linear buffer) has no memory dependencies per iterations, allowing an II of 1, so executing the NCOEFFS iterations takes approximately NCOEFFS cycles. However, the ftemp loop has a read/write dependency on ftemp, which induces an II of several clock cycles (8 in our tests).³ This means that the design needs roughly 9 times NCOEFFS cycles to execute, or 1 065 cycles with NCOEFFS equal to 115 in our example. Given a maximum latency of 2 603 cycles in a one sample approach at 48 kHz, this allows for the implementation of an FIR filter with fewer than

³These values are taken from the synthesis results.

300 coefficients, an outcome that is really far from what we could expect and quite inadequate for many real-world applications.

In next section we explore the different C++ code transformations that we propose to improve the result of *Vitis* HLS synthesis.

5.4.2 Optimizing Pipeline

Section 5.2 discussed two major principles of hardware design: parallelization and pipelining. Fig. 5.21 p.96 shows that, regardless of the unrolling factor, pipelining tends to be more efficient than non-pipelining. For each loop in the program, an effective way to optimize the design is thus to minimize the initiation interval. However, controlling the II without delving deep into schedule analysis can be complex.

This section details essential guidelines that have proven highly effective for implementing our audio program, while avoiding an overly detailed analysis of the program itself. Each of these guidelines aims to optimize latency. By default, *Vitis* produces a design that balances resource usage and latency. However, since audio programs are generally sequential, latency is often more critical than resource usage. Moreover, latency tends to be a strict constraint, whereas resource usage can be managed by opting for a more powerful FPGA, within the technological limits available.

Explicit Pipelines: More Effective Than Inferred

HLS automatically pipeline all loops, as long as they are not nested (nested loops are discussed in Section 5.4.3).

Automatic pipelining can be verified in the logs by looking for messages of the type:

Lang: HLS

```
1 INFO: [XFORM 203-510] Pipelining loop 'compute_fir' (syfala_ip/syfala_ip.cpp:50)
in function 'computemydsp' automatically.
```

However, many of our tests have shown that even when pipelining is inferred, explicitly declaring a pragma pipeline can push *Vitis* to further reduce latency, albeit at the cost of increased resource usage.

▼ Design Guideline 5.4.1 — Explicit Pipelining

Explicitly declaring a pragma pipeline in loops where pipelining is inferred can help decrease latency. Depending on the design, the effect on latency can be beneficial or negligible.

Related pragmas:

- #pragma HLS Pipeline (see Section 5.3.5)

While this approach does not work for every design and is dependent on the specific program, it has never shown negative effects. In the worst case, it does not change the implementation at all, so specifying explicit pipelining in any loop that should be pipelined is a good coding practice.

Forcing a Low II

Explicitly declaring a pipeline reduces latency, but it may not minimize the initiation interval as much as possible. Adding the option II=1 to the pipeline directive force the tool to minimize the

initiation interval even further. This can force some costly partitioning to avoid memory dependencies.

While this approach can be quite expensive in resources, potentially saving only a few hundred cycles, it may allow the design to fit within the constraints of a given FPGA. However, it often introduces timing violations. Conversely to the previous rule, forcing a low II can have negative effects, as it can consume a significant amount of resources to save only a few cycles. This approach should be used with caution.

▼ Design Guideline 5.4.2 — Lower Initiation Interval

Declaring a pragma `pipeline II=1` in pipelined loops can significantly reduce latency. Depending on the design, the effect on latency can be beneficial or negligible while very costly in resources.

Related pragmas:

- `#pragma HLS Pipeline II=1` (see Section 5.3.5)

Analyze Loops Dependencies

The two previous pipelines, if declared only in certain loops of the program, can create new dependencies and increase the iteration latency of other loops. This is demonstrated in the one-sample case study in Appendix B, where one of the two loops has an II of 1 without a pragma, and the other has a high II. Attempting to minimize the II of the second loop with an explicit pragma causes the first loop to increase its II.

Removing Memory Dependencies

If unrolling a loop does not sufficiently reduce the latency, it is likely due to memory dependencies that prevent some parallelization, such as the read/write dependency in the FIR example with the samples array. Partitioning the relevant arrays using the pragma `array_partition` can significantly reduce latency, though it comes at the expense of highly increased resource usage.

Most of the time, the pragma `pipeline II=1` automatically partitions the relevant arrays. However, sometimes the tools require it to be explicitly declared. Even if the tool already performs partitioning, using the explicit pragma can marginally improve the design, either by using slightly fewer resources for the same latency or achieving slightly lower latency for the same resource usage. This pragma can be combined with the pipeline pragma (with or without specifying an II) to explore the optimal implementation.

▼ Design Guideline 5.4.3 — Arrays Partitioning

Explicit array partitioning with a pragma `array_partition` can enhance the design, even if inferred partitioning is already in place. However, partitioning arrays that are not essential can incur significant resource costs without improving latency.

Related pragmas:

- `#pragma HLS Array_partition variable=<name> complete/factor=<int>` (see Section 5.3.4)
- `#pragma HLS Unroll` (see Section 5.3.2)

To identify which array causes a bottleneck and should be partitioned, the violation indicators provided by the scheduler can help determine which operations and variables are limiting the parallelization.

5.4.3 Nested Loops: A Key to Major Optimizations

As seen in Section 5.4.1, the FIR filter due to the loop dependency of the MAC iterations, necessitates sequential execution and imply high II for the pipelined loop. This inherent sequentiality limits opportunities for parallelization and if a program only contains an FIR filter with a one-sample approach, pipeline optimization is very restricted, even if other optimizations are possible and discussed in Section 5.4.4.

In many cases, these sequential loops are nested within another loop, which is common when the program adopts a multisample approach or even in a one-sample approach if there are multiple inputs/outputs. An example of such nested loops is shown in Listing 5.6 with an FIR filter example from the full program `fir_multiinstances.cpp` [The].

Lang: C

```

1 void compute([...])
2 {
3     samples[0] = input_sample;
4
5     for (int p = 0; p < PARALLEL_EXEC; p++) {
6         for (int n = 0; n < NCOEFFS; n++) {
7             fTemp += samples[n] * coeffs[n] * p ;
8         }
9         output_sample[p]=fTemp;
10    }
11
12    for (int j = NCOEFFS-1; j > 0; j--) {
13        samples[j] = samples[j-1];
14    }
15 }

```

Listing 5.6: Nested loops of an FIR filter with parallel executions. This is a pedagogical example; the p factor does not have a real representation.

The FIR filter is now computed across `PARALLEL_EXEC` iterations. This `PARALLEL_EXEC` can represent the number of buffered samples, the number of channels, or other metric implying to execute the same FIR filter several times. In this example, there is only one input but multiple outputs, though other configurations are also possible. The crucial point is that the inner loop of the FIR filter is nested within the outer loop that iterates `PARALLEL_EXEC` times. Typically, `PARALLEL_EXEC` ranges from dozens to hundreds, while `NCOEFFS` varies from hundreds to several thousands. The $*p$ does not represent any specific value here, it is used solely to ensure different filter architectures for each parallel execution.

There are two optimization possibilities: pipelining the inner loop or pipelining the outer loop, which infers unrolling the inner loop. By default, *Vitis* HLS chooses one or the other depending on the `syn.compile.pipeline_loops` threshold (see Section 5.3.5). For all the following examples, a value of 16 is chosen for `PARALLEL_EXEC`, creating 16 distinct instances of the filter.

Pipelining Inner Loop: Inferring Sequential Outer Loop

By pipelining the inner loops in this case, the outer loop becomes perfectly sequential, and the latency of the FIR execution is repeated `PARALLEL_EXEC` times, leading to excessive latency as shown

in Table 5.4. The architecture of the inner loops remains the same as described in Section 5.4.1, which explains the latency of approximately $9 \text{ cycles} \times 16 \times 115$.

Nb Coeffs	Nb Instances	Latency (cyc.)	BRAM	DSP	FF	LUT
115	16	15 307	3(2%)	5 (6%)	1 099 (3%)	2 017 (11%)

Table 5.4: Resource usage for `fir_multiinstances.cpp` with pipelined inner loop.

Pipelining Outer Loop: Unrolling the Sequential Loop

With nested loops, pipelining an outer loop results in the automatic unrolling of any inner loops. The inner loop iterating over n is sequential, while the outer loop, iterating over p , is designed to be parallel. In order to pipeline the outer loop, thus unroll the inner loop efficiently, the inner loop must be parallel. Since the FIR computation is fundamentally sequential due to the nature of its convolution operations, achieving full parallelization of the inner loops is challenging. Aggressive partitioning of arrays might help, but it often results in an impractical use of resources.

It is possible to try to pipeline the outer loop using the pipeline pragma, effectively parallelizing all unrolled inner loops and leading to a very low latency. However, this lead to an excessive consumption of resources, as shown in Table 5.5, particularly because of the intensive partitioning required for the `fTemp` variable. For mid-complexity algorithms, such as a simple FIR filter with a few thousand coefficients, this aggressive partitioning can make *Vitis* HLS take a considerable amount of time to find a suitable implementation, sometimes leading to crashes. It is possible to explicitly specify a higher II for the outer loop to relax parallelism while controlling resource usage. However, this method is not suitable for complex algorithms and still takes a huge amount of time for *Vitis* to process.

Nb Coeffs	Nb Instances	Latency (cyc.)	BRAM	DSP	FF	LUT
115	16	284	2 (1%)	93 (116%)	20 609(58%)	18 213 (103%)

Table 5.5: Resource usage for `fir_multiinstances.cpp` with pipelined outer loop.

Loop Interchange: Pipelining the Outer Loop and Unrolling the Parallel Loop

The goal of loop interchange is to move the sequential loop to the outer level, enhancing the effectiveness of pipelining for the sequential loop and improving unrolling efficiency for the concurrent loop. When this optimization is applied, results are remarkably better than those achieved with other optimizations.

Separating each loop requires adapting memory dependencies. In this case, this means changing the `fTemp` variable into an array `fTemp[p]` of size `PARALLEL_EXEC`. This transformation is commonly used in loop vectorization. The second transformation involves interchanging the loops on n and p . This is possible because the only dependency is the accumulation of `fTemp[p]` (which is an internal dependency within the n loop). Consequently, the loops can be interchanged. The resulting code is shown in Listing 5.7.

This manipulation also involved loop distribution. If the loop over `output_samples` was not separated from `fTemp` first, it would not be possible to perform this interchanging as it does not iterate over `NCOEFS`.

Lang: C

```

1 void compute([...])
2 {
3     samples[0] = input_sample;
4
5     for (int n = 0; n < NCOEFFS; n++) {
6         for (int p = 0; p < PARALLEL_EXEC; p++) {
7             fTemp[p] += samples[n] * coeffs[n] * p ;
8         }
9     }
10    for (int p = 0; p < PARALLEL_EXEC; p++) {
11        output_samples[p]=fTemp[p];
12    }
13    for (int j = NCOEFFS-1; j > 0; j--) {
14        samples[j] = samples[j-1];
15    }
16 }

```

Listing 5.7: *Interchanged nested loops of an FIR filter.*

Here is a synthesis of the methodology:

- The code had two nested loops: a sequential loop with many iterations inside a parallel loop with few iterations.
- Pipelining the inner loop forces the outer (parallel) loop to become sequential, which is highly inefficient.
- Pipelining and minimizing the II on the outer loop is possible but prohibitively costly, as it requires fully unrolling the long loop.
- By inverting the loops, we can now pipeline the sequential loop as the outer loop. This way, the inner loop, which has a low iteration count, is unrolled, and this is not costly in terms of resources. Even if the II isn't optimal, avoiding the sequentialization of the upper loop makes this approach very efficient.

▼ Design Guideline 5.4.4 — Loop Interchange

For nested loops, the outer loop should be sequential while the inner loop should be parallel. In the case of multi-instances, such as multichannel or multisample processing, this often involves interchanging loops

At this stage, it is crucial to ensure that the functional behavior of the program remains unchanged. It is very easy to introduce errors when manipulating loops, which can completely alter the functionality. *Vitis CSim* should be used in this case to create test benches and ensure consistent results.

This approach leverages the potential for parallel execution within the inner loop, where the iteration count is usually smaller and more manageable, while keeping the outer loop sequential to maintain the structure and flow of the algorithm. The results for 115 coefficients are provided in Table 5.6.

This is a highly efficient transformation and one of the most significant contributions of this thesis. By interchanging the loops, the results are substantially improved compared to all previous approaches.

Nb Coeffs	Nb Instances	Latency (cyc.)	BRAM	DSP	FF	LUT
115	16	1 900	3 (2%)	7 (8%)	2 545 (7%)	2 872 (16%)

Table 5.6: Resource usage for `fir_multiinstances.cpp` with interchanged loops and no pragmas (inferred pipelined outer loop).

The Alternative Approach: Flattening Loops

Flattening the nested loops results in a single loop with $NCOEFFS \times PARALLEL_EXEC$ iterations. To flatten the loop, the pragma `loop_flatten` directive is applied to the inner loop, and the outer loop should not be pipelined (either by adjusting the `syn.compile.pipeline_loops` setting or by placing an explicit pragma `pipeline off` in the inner loop).

Here is where it becomes interesting: when flattening the loops with the “normal” configuration (outer loop iterating over `PARALLEL_EXEC` and inner loop iterating over `NCOEFFS`), the tool cannot minimize the initiation interval. Using the example from Listing 5.6, it results in an II of 9, which is then propagated through the $NCOEFFS \times PARALLEL_EXEC$ loop, totaling 1 840 iterations, leading to a latency of more than 16 000 cycles. In contrast, with the interchanged loops, the II is minimized to 1, resulting in a latency of around 1 840 cycles. The method of using pragmas is shown in Listing 5.8 (loops can be interchanged to compare the two implementations), and resource usage is detailed in Table 5.7. This result is quite unintuitive, as the flattened loop does not seem to be affected by the initial order of the two loops. However, it highlights once again that optimizing HLS is often very difficult to predict.

▼ Design Guideline 5.4.5 — Loop Flattening

Flattening nested loops provides a good balance between resource usage and latency, but it is less efficient if the initiation interval (II) cannot be minimized to 1.

Related pragmas:

- `#pragma HLS loop_flatten`

Lang: C

```

1 /* INVERTED */
2 coeff_loop: for (int n = 0; n < NCOEFFS; n++) {
3     par_loop: for (int p = 0; p < PARALLEL_EXEC; p++) {
4 #pragma HLS loop_flatten
5 #pragma HLS pipeline
6         fTemp[p] += samples[n] * coeffs[n];
7     }
8 }
```

Listing 5.8: Pragmas to force loop flattening.

Loop	Latency (cyc.)	BRAM	DSP	FF	LUT
Non Interchanged	16 732	3 (2%)	7 (8%)	1 882 (5%)	2 739 (15%)
Interchanged	2 020	3 (2%)	10 (12%)	2 176 (6%)	2 970 (16%)

Table 5.7: Resource usage for `fir_multiinstances.cpp` with flattened loops, for 115 coefficients and `PARALLEL_EXEC=16`.

Results

Once the loops are inverted, it is possible to apply the optimization rules (Rules 5.4.1 p.102, 5.4.2 p.103, and 5.4.3 p.103) to refine the implementation. The results of various implementations are shown in Fig. 5.25. BRAM and DSP Block usage are not shown here as these programs use very little BRAM (up to 3 maximum), and DSP Block usage consistently remains around 10.

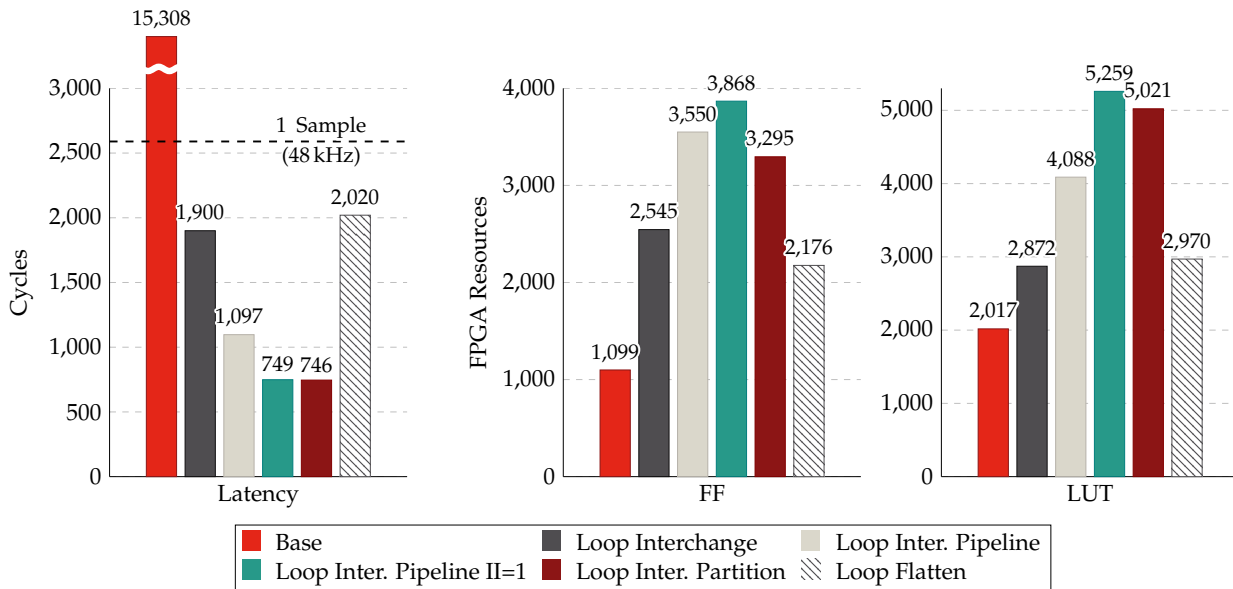


Figure 5.25: Latency, FF and LUT estimation usage for different nested loop implementations, for 115 coefficients and PARALLEL_EXEC=16.

Base: The basic program `fir_multiinstances.cpp` with no loop manipulations. Corresponds to Listing 5.6.

Loop Interchange: Loops are interchanged. Corresponds to Listing 5.7.

Loop Interchange with Pipeline: Same as Loop Interchange with an explicitly declared pipeline pragma in the outer loop.

Loop Interchange with Pipeline II=1: Same as Loop Interchange with Pipeline, but the explicit pipeline pragma specifies a II of 1.

Loop Interchange with Partitioning: Same as Loop Interchange with Pipeline II=1 with full partitioning of `fTemp`.

Loop Flattened: Same as Loop Interchange with flattened loops. Corresponds to Listing 5.8.

Note for *Loop Interchange with Partitioning*: as specified in Rule 5.4.3 p.103, array partitioning can be used alongside pipelining to yield different results. Here, it is used with pipeline II=1 to optimize latency.

Generic Methodology: Is It Applicable?

The proposed optimization approach is not universally applicable, hardware design is highly dependent on specific factors. Even small changes in memory dependencies can render the suggested methods ineffective. However, this generic optimization is particularly useful when dealing with nested loops where the outer loop can be parallelized and the inner loop is sequential. Such configurations frequently occur in audio processing scenarios. The approach becomes especially relevant when dealing with multisample or multichannel processing, where duplicating sequential audio programs often leads to this loop structure. While this method is particularly effective for FIR filters, which are fundamental in many audio algorithms, its principles can be broadly applied to similar scenarios, making it a valuable technique for optimizing hardware designs in audio processing applications.

5.4.4 Unrolling: Enhancing Parallelism When Pipelining is Insufficient

If pipelining is insufficient or not applicable, the only option to meet timing requirements may be to unroll the loops. This applies primarily in two scenarios: in a single-sample, single-channel strategy, where loops are typically not nested and optimizations are minimal, or in a multi-instance strategy, where a large number of iterations makes the pipeline insufficient.

One Sample Strict Constraint: The Sample Rate

In the one-sample strategy, no buffering is used. This means that a sample must be produced once every sample period to maintain the correct sample rate. Fig. 5.26 illustrates the one-sample approach to processing, with each sample being processed by three sequential functions: read, compute, and write. If the compute function is short enough, the IP can process a sample in less than one sample period, thereby ensuring proper sample rate processing. However, if the processing time exceeds one sample period (see Fig. 5.27), the throughput is insufficient, and the design becomes non-functional.

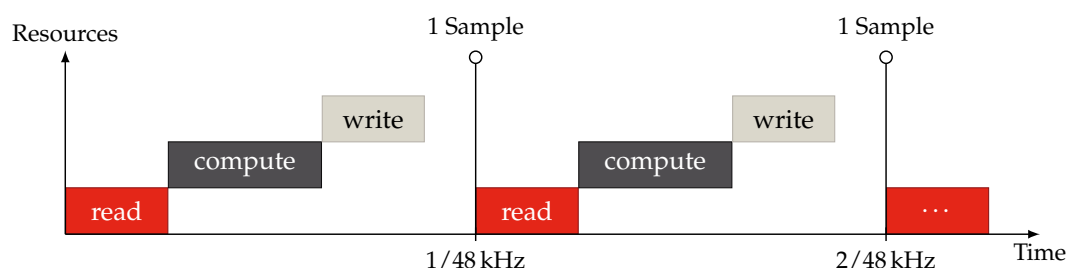


Figure 5.26: One-sample strategy real-time compatible: If the compute function is fast enough, a sample can be provided at the sampling rate.

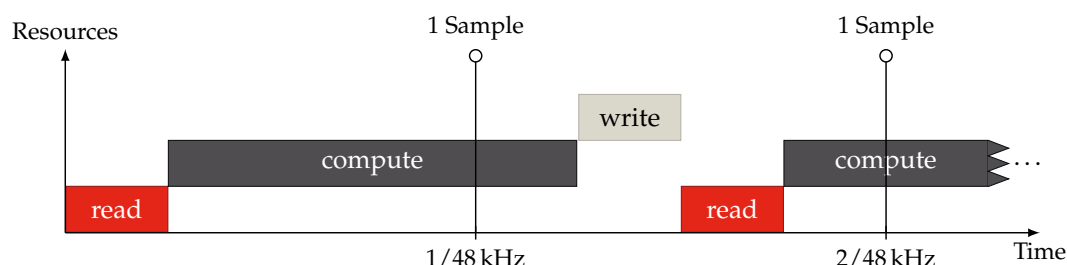


Figure 5.27: One-sample strategy not real-time compatible: If the compute function is too long or not parallel enough, a sample can't be provided at the sampling rate.

High Iteration Pipelining

Pipelining allows for very good latency performance while minimizing hardware costs. However, it cannot achieve an iteration latency of less than 1 clock cycle, even in the best-case scenario. This means that the maximum number of iterations is roughly limited by the number of available clock cycles. For a 48 kHz sampling rate, this implies that no loop can iterate more than around 2 600 cycles (in practice, even fewer, as the loop is not the only logic present). For example, an FIR filter would be limited to a maximum of 2 600 taps, which, although reasonable, is far below the potential of an FPGA.

Even though it is more likely to be a limitation in a single-sample approach, this also applies to multi-sample scenarios. In the multi-sample case, the number of iterations is equal to the number of clock cycles per period multiplied by the number of buffered samples, assuming each sample

is perfectly parallelized. Therefore, unrolling is the only solution to reduce the number of clock cycles used per loop by duplicating the hardware and executing multiple iterations concurrently.

Parallelize the Function

Using the pragma unroll directive, the compute function can be parallelized into sub-functions. By creating multiple independent operations rather than a single collection of operations, the unroll pragma transforms the original sequential loop into multiple sequential loops that handle parallel operations, effectively distributing the workload.

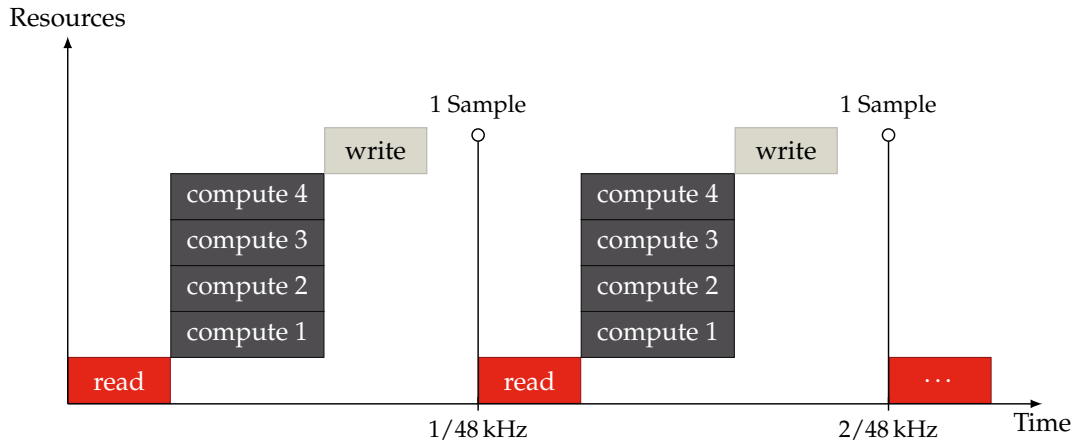


Figure 5.28: *Unrolling loops: By parallelizing the compute function, it can fit within the timing constraints at the cost of increased resource usage.*

When considering loop unrolling, several parameters should be taken into account:

Dependency check. If each iteration of a loop depends on the previous one, optimization opportunities are limited. However, if the iterations are independent, pipelining can achieve a very low initiation interval, and if not, unrolling can probably be effective.

Pipelining considerations. If a loop is pipelined (meaning the same hardware is reused sequentially) with an initiation interval of 1 clock cycle, the benefits of unrolling the loop is minimal, as the loop is already nearly parallelized. Unrolling is most effective when applied to loops with an Π greater than 1. The higher the Π , the greater the potential benefit of unrolling.

Resource and latency trade-off. As discussed in Section 5.3.2, the latency reduction achieved through unrolling follows a $1/X$ trend, while the resource cost increases linearly by a factor of X . Unrolling a loop with a too high factor becomes inefficient, as the additional resource cost outweighs the latency benefits.

Resource cost evaluation. Before unrolling, the programmer should carefully evaluate the resource cost of the function. If the function is lightweight, requiring only a few hundred resources, unrolling can multiply these low resource requirements and potentially save clock cycles at a relatively low cost, even if the Π is already low. Conversely, unrolling may not be worthwhile for high- Π loops that are resource-intensive, as the resource costs may outweigh the performance gains.

▼ Design Guideline 5.4.6 — Unroll to Parallelize

Loop unrolling provides control over latency in non-nested loops. It is most effective for loops with a high initiation interval and no dependencies and is advisable for low resources-cost loops but should be avoided for resource-intensive ones. Beyond a certain unrolling factor, the latency gains become minimal while the resource costs increase significantly.

Related pragmas:

- `#pragma HLS unroll factor=<N>` (see Section 5.3.2)

All the previous optimizations applicable to the one-sample approach, particularly the effect of different unroll factors, are studied in a case study provided in Appendix B. This real-world example highlights the inconsistencies in HLS optimizations and the necessity, despite the generic nature of the given guidelines, to thoroughly analyze the synthesis report to achieve an optimal implementation. While next section provides a case study for the multi-sample approach, interested readers can refer to the one-sample approach, which presents different challenges and solutions.

5.4.5 Multi-Sample Design Guidelines: Exploiting FPGA Parallelism

Leveraging the parallelization power of FPGAs can be particularly effective when working with buffers of samples. Buffering can increase throughput and can help maintain the sample rate when the computation for a single sample cannot be completed within one sample period. By buffering, multiple samples can be processed simultaneously, which significantly optimizes pipelining and unrolling. This creates nested loops, the benefits of which are detailed in Section 5.4.3.

However, the specific case of multisample processing introduces unique considerations. First, handling multiple samples requires waiting for these samples, necessitating a buffering mechanism. This contrasts with multichannel processing, where all samples can typically be processed immediately. Second, processing multiple samples at once can greatly optimize memory accesses by enabling vectorized operations.

Simple Buffering for Sequential IP Execution

This example uses the `fir_multisample.cpp` [The] program provided in Appendix F p.198, which is essentially the same as the `fir_onesample.cpp` [The] example (also provided in Appendix F p.197) but with an added multisample loop, as done in Section 5.4.3.

The simplest approach to implement buffering with the current IP is to accumulate N samples in memory before processing them all at once. Fig. 5.29 illustrates this concept. The hatched blocks represent the processing tasks of previous or subsequent IP executions, while the solid blocks depict the processing tasks of the current IP execution. In this example, the program waits until it has read 4 samples, after which it processes them and writes all of them in parallel.

The key feature of this approach is that the execution mode of the IP is sequential, meaning that two consecutive calls do not overlap. The signals `IP_start` and `IP_done` demonstrate that no new `IP_start` is initiated before the previous `IP_done` is completed. The only concurrent process is the buffering of incoming samples (represented in red), which occurs while the IP is processing the previous batch of samples. This buffering can be efficiently managed by an external FIFO without altering the IP's execution mode.

This sequential execution simplifies implementation since no additional control mechanisms are required. For this reason, we have primarily focused our work on this approach. However, the

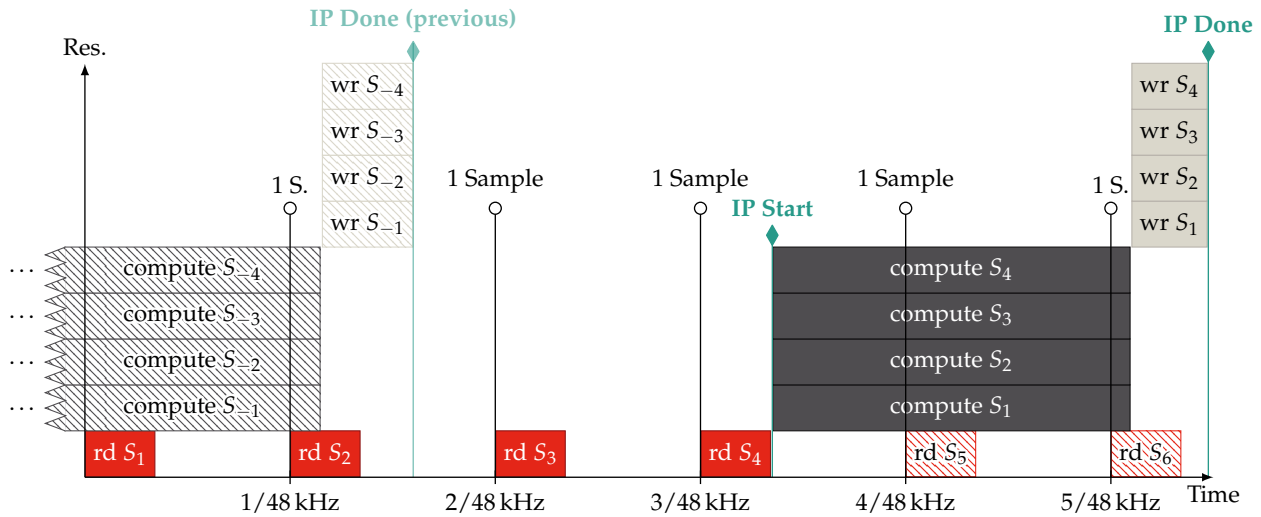


Figure 5.29: Multi-sample: simple buffering implementation.

concurrent execution has also been studied and is presented in the corresponding section p.117.

Buffering Size Limit

Buffering allows for handling IP designs with an execution delay longer than the sample rate. As the algorithm's complexity increases and processing time grows, more samples need to be buffered. However, there is a limitation to this approach. Each buffered sample requires more computation, and while parallelization can optimize implementation, there is a threshold beyond which additional buffered samples no longer benefit from parallelization. Beyond this point, buffering more samples increases the IP latency, rendering further buffering ineffective.

The resource usage and latency (both for the entire system and per sample) of `fir_multisample.cpp` for different buffer sizes are shown in Fig. 5.30. To maintain linear results and avoid HLS optimization issues for certain unroll factors, `fTemp` is fully partitioned.

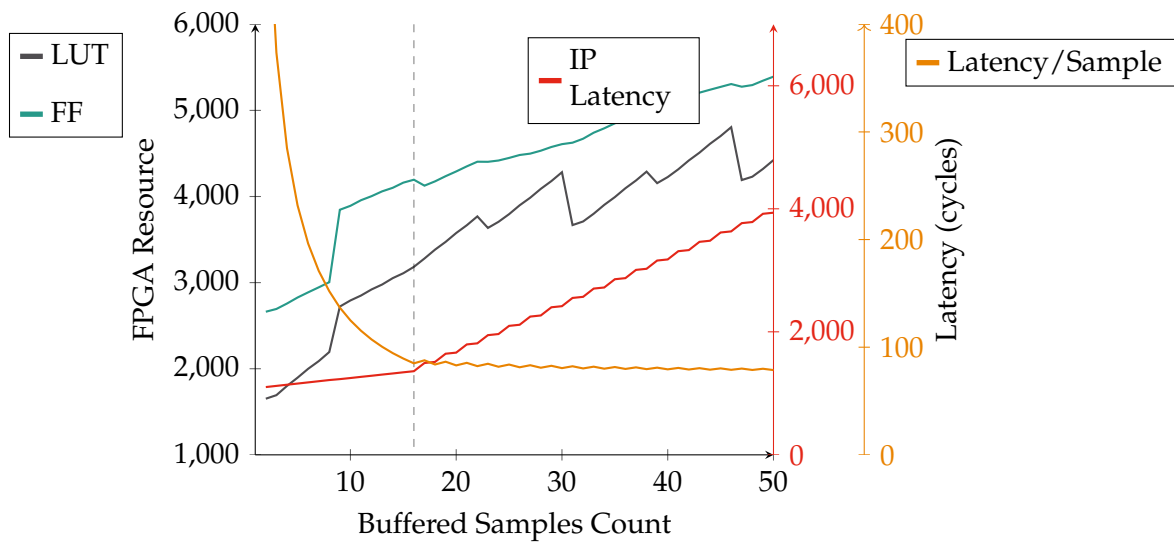


Figure 5.30: Latency and resource usage for `fir_multisample.cpp` for 115 coefficients with varying number of buffered samples and complete partitioning of `fTemp`.

In this example, the design efficiently parallelizes the processing of up to 16 samples. Beyond that

point, each additional sample does not significantly improve the average computation time (or latency per sample), reducing the effectiveness of buffering. This sequential latency arises because each additional sample slightly increases the initiation interval of the coefficient loop. Although this loop is parallelized, it is not fully optimized for aggressive parallelization, meaning that some computations still occur sequentially, resulting in a slight increase in II for each additional sample.

This behavior is inherent to the design and the type of operations involved. On this plot, the buffering threshold is clearly identified at 16 samples. For other applications, particularly those involving external DDR access, this number can be significantly higher.

For demonstration purposes, setting the II to 1 can force the inner function to be fully parallelized. This adjustment minimizes the increase in II based on the number of buffered samples but significantly impacts resource usage, which becomes quickly saturated. Fig. 5.31 illustrates the results obtained with a forced minimized II. While the IP latency slightly increases, it remains relatively constant, allowing for improved latency per sample as more samples are buffered. However, resource usage increases rapidly and becomes very high.

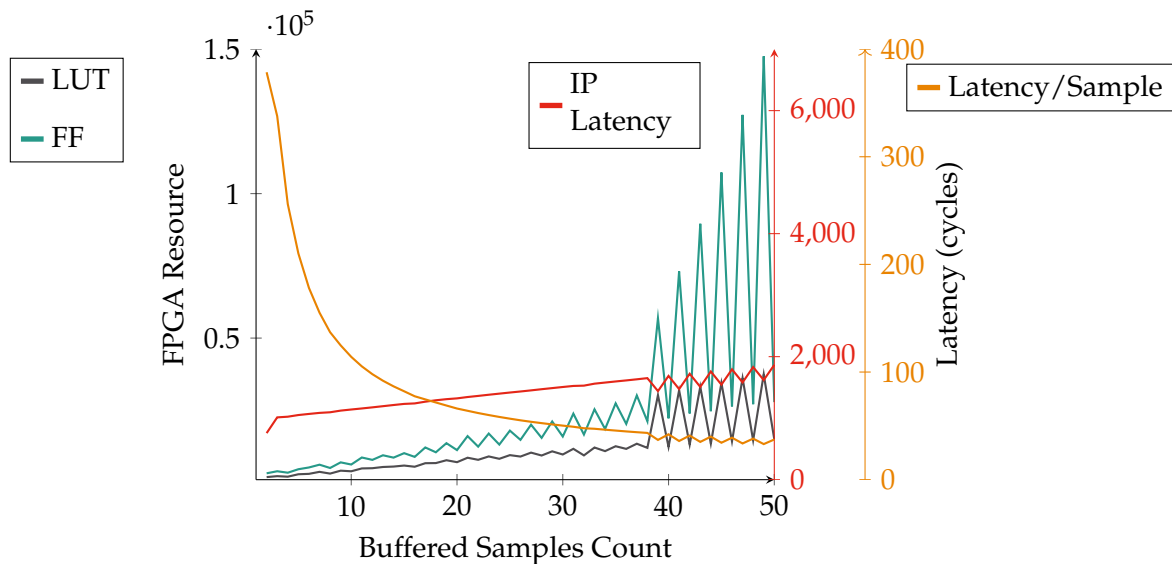


Figure 5.31: Latency and resource usage for `fir_multisample.cpp` for 115 coefficients with a varying number of buffered samples on a fully parallelized implementation forcing $II=1$.

▼ Design Guideline 5.4.7 — Buffering

Buffering can reduce the IP processing delay per sample, but the effectiveness of the buffer size is limited by the design.

Buffer Size Considerations with Input Signal

In the scenario used previously for the Fig. 5.31 and 5.30, samples are immediately available in a buffer since a function generates them. The FIFO does not have to wait for each sample to arrive, as it would if the samples were coming from an external input signal. In real-world applications, if samples are coming from an input signal, buffering more samples increases the waiting time before processing them. In Fig. 5.29, a buffer size of 4 is not optimal, as evidenced by the stall that occurs during the full sample period while reading sample 3. The IP processes the four samples

in 3 sample periods, resulting in a full period of inactivity. By reducing the buffer size to 3, there is no stall lasting more than one sample period (see Fig. 5.32).

To determine the optimal buffer size, the number of cycles per sample should be analyzed. The buffer size should be chosen to be just under the number of cycles required for one sample. If it is larger, the system does not provide a sample at each sampling period. If it is significantly less, the system produces all the samples in a short period and then experience stalls for N sample periods while waiting for the buffer to refill.

HLS does not account for the waiting time between buffer samples and therefore does not include this in its report. From its perspective, the buffers are always assumed to be full, and the IP latency reported reflects only the time from IP start to IP done. In Fig. 5.29, the waiting time between writing sample S_{-n} and the IP start is not captured by the HLS tool.

▼ Design Guideline 5.4.8 — Minimize Buffering Latency

If input samples are buffered, the chosen buffer size should be sized to use nearly all available clock cycles per sample. A shorter buffer size leads to stalling periods during sample buffering.

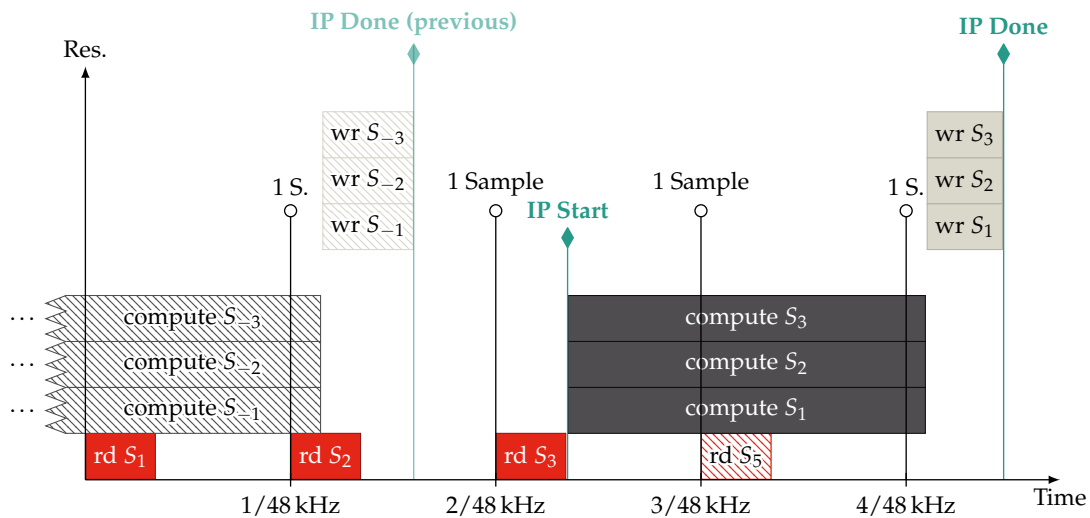


Figure 5.32: Multi-sample: Optimized buffer size.

Multisample Buffer Structure

The loop interchange technique discussed in Section 5.4.3 was demonstrated with a one-sample example (only one sample is added to the samples array for each IP iteration) and needed to separate the samples loop from the buffer loop. Applying this technique to a multisample program is more complex due to the dependencies between the FIR filter computation and the linear buffer. The multisample FIR filter implementation is illustrated in Listing 5.9.

Lang: C

```

1 for (int n = 0; n < BLOCK_NSAMPLES; n++) {
2     compute_fir_coeffs: for (int i = 0; i < NCOEFFS; i++) {
3         fTemp[n] += samples[i] * coeffs[i];
4     }
5     linear_buffer: for (int j = NCOEFFS-1; j > 0; --j) {
6         samples[j] = samples[j-1];
7     }
8 }

```

Listing 5.9: Basic implementation of an FIR filter with a multisample approach.

In this program, the `compute_fir_coeffs` and `linear_buffer` loops are sequential. The `linear_buffer` loop must wait until the `compute_fir_coeffs` loop has completed processing the current sample before it can shift the buffer. Fig. 5.33 illustrates the buffer process for the sequential approach. Breaking the outer loop, as shown in Listing 5.6, is not feasible in this scenario because the buffer needs to be shifted once after each complete execution of the `compute_fir_coeffs` loop.

A solution to separate the two loops is to increase the size of the sample array by `BLOCK_NSAMPLES` elements and perform a global shift of `BLOCK_NSAMPLES` samples at the end of the computation for `BLOCK_NSAMPLES` outputs, as illustrated in Fig. 5.34.

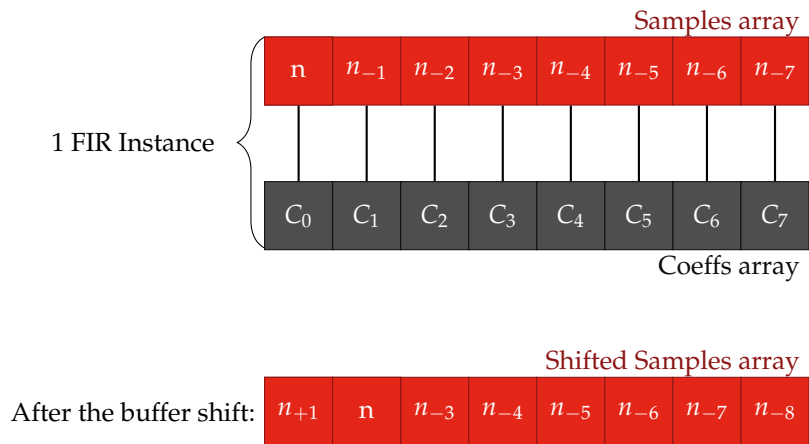


Figure 5.33: Classic linear buffer operation for one sample.

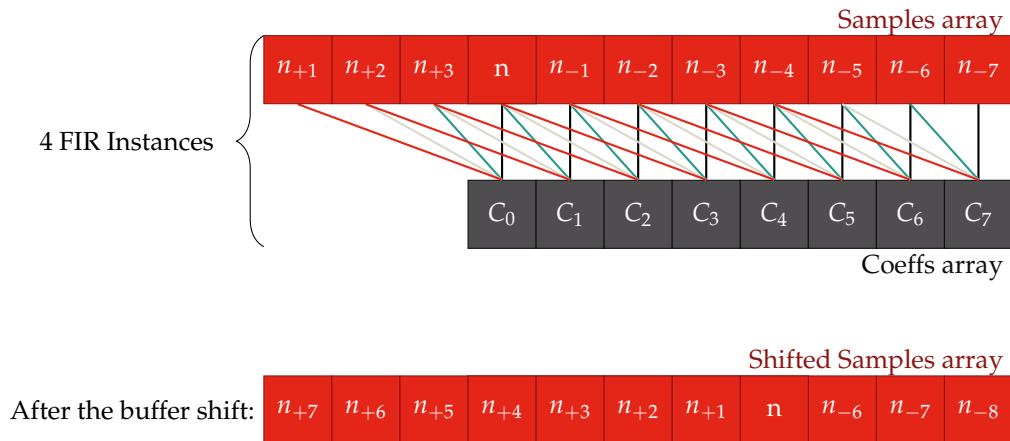


Figure 5.34: Global shift of the linear buffer for `BLOCK_NSAMPLES=4` samples.

The code for accumulating values in `fTemp` now requires adjustments for each sample. Each filter

computation reads a partial segment of the sample array corresponding to the state of the array for the current sample in a one-sample approach. The revised code is shown in Listing 5.10. By adapting the indices of the samples accessed in the FIR filter loop, the loop can be generalized to handle all buffered samples.

Lang: C

```

1  compute_fir_samples: for (int n = 0; n < BLOCK_NSAMPLES; n++) {
2      compute_fir_coeffs: for (int i = 0; i < NCOEFFS; i++) {
3          fTemp[n] += samples[i+BLOCK_NSAMPLES-1-n] * coeffs[i];
4      }
5  }
6  linear_buffer: for (int j = NCOEFFS+BLOCK_NSAMPLES-1; j >= BLOCK_NSAMPLES; --j) {
7      samples[j] = samples[j-BLOCK_NSAMPLES];
8  }

```

Listing 5.10: Global shifting of the buffer, the `linear_buffer` loop is executed once per buffer and the `compute_fir_samples` and `compute_fir_coeffs` loops are now interchangeable.

The buffer shifting is now executed only once per sample block, and the filter computation nested loops are now independent, allowing them to be interchanged. Performance improvement are presented in Section 5.4.6.

▼ Design Guideline 5.4.9 — Buffer Global Shifting

In multisample programs, the linear buffer should perform a global shift once per sample block, rather than shifting for each individual sample.

Dataflow

Instead of calling the functions inside the IP sequentially, concurrent execution can be applied, with the *Vitis* dataflow pragma. Simply put, dataflow is similar to pipelining, but it operates at the function level: a function does not need to fully process a sample before starting the processing of the next one (see Fig. 5.35).

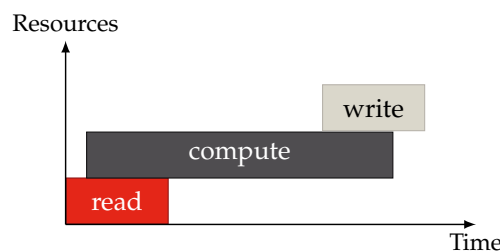


Figure 5.35: Example of dataflow pragma behavior.

Dataflow is particularly useful in multisample programs for beginning the processing function while read accesses are still ongoing, especially when the read or write functions involves accessing multiple values and when data is stored in external memory with significant access times. However, in our audio applications, the read and write functions are very quick because they handle one or several samples directly in onboard memory. There is minimal benefit to using dataflow pragma in an audio multisample setup on FPGA, as it would only save a few cycles from reading or writing a small number of values.

While dataflow can still be useful for pipelining two successive processing functions, such as the FIR filter and linear buffer functions in our example, this should be considered in context. First, the functions should be parallelizable, which is not the case in the FIR/buffer example. Additionally, often with audio programs, these functions can be combined into a single common function, reducing the need for separate pipelining.

▼ Design Guideline 5.4.10 — Dataflow

For audio DSP with no or few external memory accesses, dataflow is not useful.

IP Overlapping

With execution modes in HLS designs, it's theoretically possible to implement a form of dataflow at the IP level, where the IP can start processing a new sample without waiting for the current one to be fully processed. An example of a dataflow implementation is illustrated in Fig. 5.36.

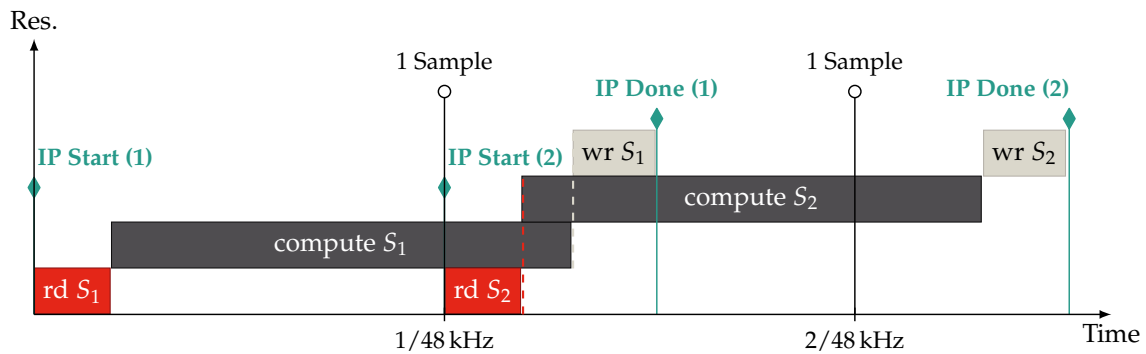


Figure 5.36: Example of overlapping IP execution mode.

In contrast to Fig. 5.29, where each new start of the IP is initiated only after the previous instantiation is complete, this dataflow approach allows a new instance of the IP to begin processing before the previous one finishes. This method requires more complex mechanisms for block-level control protocols to enable overlapping execution by managing and freeing resources from the first iteration in time for the execution of the second iteration.

This approach combines the low latency of a one-sample process where processing begins as soon as a sample is available, with the processing capabilities of a multisample system where a sample may take longer than a single sample period to process. The overlapping approach ensures a throughput of one sample per sample period, as it does not buffer samples.

However, achieving a fully functional IP overlapping implementation has proven challenging, and we were unable to obtain functional behavior. While this work is still ongoing, the usefulness of this approach should be evaluated. As noted in Section 5.4.5, a properly dimensioned buffer can closely replicate the behavior of IP overlapping. Moreover, even though IP overlapping yields results comparable to loop pipelining, the transformation explained in Section 5.4.3 (Loop Interchange) will not be possible with this approach. Therefore, we expect the results to be worse than those obtained with the multi-sample strategy discussed later.

5.4.6 Multi-Sample Case Study

This section proposes a workflow for optimizing multisample programs, leveraging the design guidelines discussed earlier. The program used in this demonstration are `fir_multisample_`

base.cpp, fir_multisampl_subloops.cpp, and fir_multisample.cpp [The] (see Appendix F p.198 for the last one).

Default Behavior

All the following examples assume the following command configuration:

- -umo (unsafe math optimization) is enabled (see Appendix A).
- syn.compile.pipeline_loops threshold is left at its default value of 64. This means that if the iteration count of the inner loop in a nested loop is less or equal to 64, automatic pipelining of the outer loop is applied. Conversely, if the iteration count is more than 64, the inner loop is automatically pipelined.
- If not specified otherwise, the block is 16 samples wide and the number of coefficients is 115.

The fir_multisample_base.cpp is synthesized with Vitis HLS without any pragmas. It contains the “normal” structure of a multisample program, using one outer loop to iterate through the samples. This loop essentially encapsulates all the functions from the fir_onesample.cpp example, as shown in Listing 5.11. The automatic pipelining process pipelines the inner loops of compute_fir and linear_buffer, as they iterate over NCOEFFS, which exceeds the syn.compile.pipeline_loops threshold (115 coefficients in the example).

Lang: C

```

1  for (int n = 0; n < BLOCK_NSAMPLES; n++) {
2      generate_input_samples();
3      linear_buffer();
4      compute_fir(&fTemp);
5      write_output_samples(audio_out_0, audio_out_1, &fTemp);
6  }
```

Listing 5.11: Excerpt of fir_multisample_base.cpp showing an outer loop calling the functions of the fir_onesample.cpp program for the number of buffered samples.

The resulting resource usage is provided in Table 5.8.

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
17 026	1 064	3 (2%)	7 (8%)	1 548 (4%)	2 499 (14%)

Table 5.8: Resource usage for fir_multisample_base.cpp with no pragmas, 115 coefficients and 16 buffered samples.

The reported latency corresponds to the latency for processing 16 samples. The second column provides the equivalent latency per sample, which corresponds to the average computation time, calculated as the total latency divided by the number of samples.

A first observation can be made: the latency is roughly equal to 16 times that of the one-sample program, while the resource usage remains the same. This indicates that the sample processing is perfectly sequentialized.

Loop Distribution

The next step is to apply loop distribution. This requires changing the fTemp variable into an array. Additionally, a global shifting mechanism for the linear buffer must be implemented (see Rule 5.4.9 p.116). Consequently, all the buffered samples need to be placed in the sample array,

increasing its size by BLOCK_NSAMPLES elements. This leads to the implementation already analyzed in Listing 5.10 p.116.

The updated code can be found in the file `fir_multisample_subloops.cpp` [The], and resulting resource usage is provided in Table 5.9.

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
15 335	958	3 (2%)	5 (6%)	1 625 (4%)	2 689 (15%)

Table 5.9: Resource usage for `fir_multisample_subloops.cpp` with no pragmas, 115 coefficients and 16 buffered samples.

The latency is slightly lower, while resource usage is somewhat higher. Not much has changed since no significant optimizations have been implemented yet.

Loop Interchange

Following the Rule 5.4.4 p.106, the loops in the `compute_fir` function are interchanged. The updated program is provided in the file `fir_multisample.cpp` [The] (see Appendix F p.198), and the results are presented in Table 5.10.

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
2 957	184	3 (2%)	10 (12%)	3 281 (9%)	4 069 (23%)

Table 5.10: Resource usage for `fir_multisample.cpp` with no pragmas, 115 coefficients and 16 buffered samples.

This time, while the new design is slightly more costly in terms of resources, the gain in latency is significant. With 16 samples, it only takes 185 cycles per sample to be processed.

Latency Lowering

Rule 5.4.1 p.102 advises adding an explicit pipeline directive to the outer loop of the FIR filter function. The results are provided in Table 5.11.

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
1 606	100	1 (0%)	10 (12%)	4 310 (12%)	6 383 (36%)

Table 5.11: Resource usage for `fir_multisample.cpp` with `pragma pipeline` in the outer loop of `compute_fir` function, 115 coefficients and 16 buffered samples.

This pragma can be changed to `pragma HLS pipeline II=1`, which forces the tool to minimize the II according to Rule 5.4.2 p.103. The results are provided in Table 5.12

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
1 258	79	1 (0%)	14 (17%)	5 592 (15%)	9 256 (52%)

Table 5.12: Resource usage for `fir_multisample.cpp` with `pragma pipeline II=1` in the outer loop of `compute_fir` function, 115 coefficients and 16 buffered samples.

Finally, explicitly partitioning fTemp to eliminate any memory dependency (Rule 5.4.3 p.103) further assists the tool in implementing a design with lower latency. In this case, the three different pipeline combinations (inferred, explicit, and explicit with II=1) previously studied yield different results, each improving upon the design without partitioning. The results are displayed in Table 5.13.

Pipeline	IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
Implicit	1 359	84	3 (2%)	10 (12%)	3 183 (9%)	4 195 (23%)
Explicit	1 587	99	1 (0%)	10 (12%)	4 200 (11%)	5 997 (34%)
Expl. & II=1	1 239	77	1 (0%)	14 (17%)	5 482 (15%)	8 897 (50%)

Table 5.13: Resource usage for fir_multisample.cpp with different pipeline implementations in the outer loop of the compute_fir function and a complete partitioning of fTemp, 115 coefficients and 16 buffered samples.

For nearly every case and resource type, the design improves with explicit partitioning, although the gains can be modest in some instances. Based on the specific needs and the desired balance between resource usage and latency, the programmer can select the preferred implementation.

Resource Savings With Loop Flattening

When prioritizing resource savings over latency reduction, Rule 5.4.5 p.107 can be followed to achieve a good balance between resource usage and latency. Adding the following pragma to the inner loop of compute_fir in order to flatten the nested loops of the FIR filter yields the results shown in Table 5.14.

Lang: C

```
1 #pragma HLS loop_flatten
2 #pragma HLS pipeline
```

IP Lat. (cyc.)	Sample Lat. (cyc.)	BRAM	DSP	FF	LUT
2 282	142	3 (2%)	5 (6%)	1 795 (5%)	2 707 (15%)

Table 5.14: Resource usage for fir_multisample.cpp with pragma loop_flatten in compute_fir, 115 coefficients and 16 buffered samples.

Results

The previous tables are summarized in the graph results shown in Fig. 5.37. These different configurations highlight various trade-offs between latency and resource usage based on the extent of optimizations applied.

These results show an impressive latency reduction by more than a factor of 14, with a resource cost increase by a factor of 3.5 for aggressive latency optimizations. The resource cost is debatable, as it highly depends on the number of coefficients, but the latency reduction applies to any FIR filter case. The interchanged loops example demonstrates that simply swapping loops, without any program analysis (no pragmas), results in a latency gain of almost a factor of 6. Lastly, the flattened loop example shows that a latency gain of more than a factor of 7 can be achieved with almost no increase in resource usage.

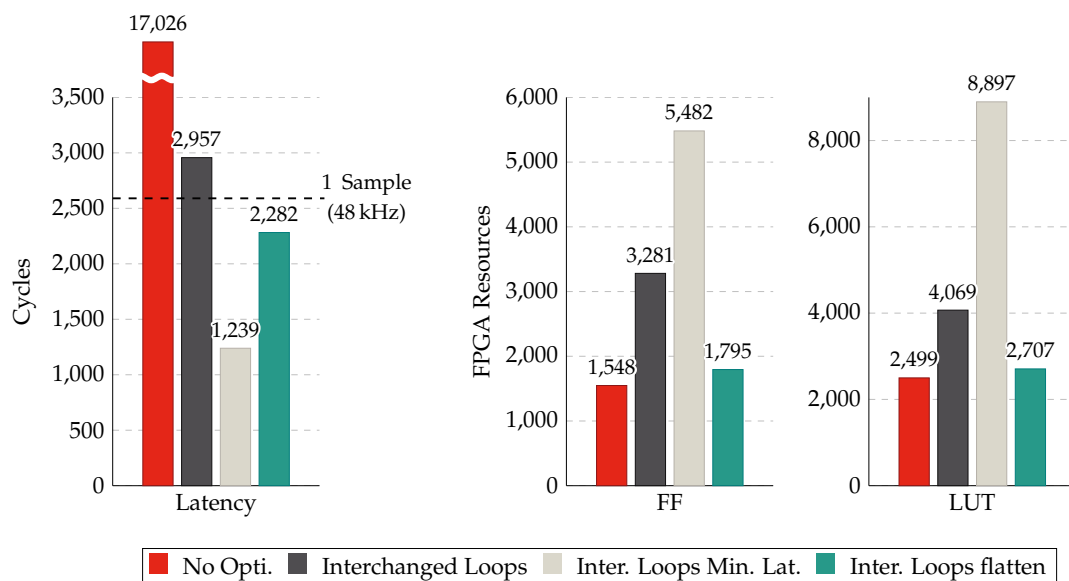


Figure 5.37: Resource usage and latency results for different implementations of an FIR filter in a multisample approach, 115 coefficients and 16 buffered samples.

No Opti: Represents the program without any code rewriting, corresponding to the file `fir_multisample_base.cpp`.

Interchanged Loops: This version includes loop interchange (and thus global shift buffer) but no pragmas, corresponding to the file `fir_multisample.cpp`.

Interchanged Loops Minimize Latency: This program applies loop interchange with pragma `pipeline II=1` and array partitioning of `fTemp` to optimize latency. While there may be further improvements possible, this setup achieves good results with minimal code changes.

Interchanged Loops flatten: Based on the Interchanged Loops, with loop flattening balancing the resource usage and latency efficiently.

Of course, in this example, the low number of coefficients makes the use of multisample debatable, as it could fit within a one-sample approach. This is primarily for pedagogical purposes and to enable quicker synthesis. This method is extended to thousands of coefficients in Chapter 6.4.1.

As for the results presented in Section 5.4.3, BRAM and DSP blocks are not included here, as these metrics are not highly relevant and are almost not utilized. In the case of FIR filters, BRAM is primarily occupied by the filter coefficients. Larger filters require more memory for storing coefficients, which can become problematic if BRAM is saturated, as external memory is typically too slow for accessing coefficients on a per-sample basis. Although LUT RAM can be used, it consumes a significant number of LUTs, which are often already heavily used in FIR implementations. However, experimental results discussed in Chapter 6.4.1 demonstrate that even for filters with tens of thousands of coefficients, BRAM is sufficient for mid-range FPGA boards used in these experiments. In this FIR filter case, DSP blocks are rarely used. However, experimental results, particularly in the one-sample scenario with specific implementations detailed in Appendix D, Table D.2, demonstrate that their usage can become intensive in certain implementations.

5.5 Interfacing DSP IP With CPU and Memory

To achieve an efficient architecture, the interface must be carefully configured. Proper hardware/software partitioning should be paired with a well-designed communication interface between the FPGA and the CPU. The audio signal path should be scalable to support multiple channels, and all control and communication signals must be correctly managed. On the memory side, to avoid

bottlenecks, access can be optimized at the code level by using efficient memory architectures and guiding the HLS tool to optimize memory accesses.

5.5.1 IP Interface

AXI Interface

On *Xilinx* FPGAs, data transfer between the CPU and a hardware IP is commonly performed using the AXI4 interface. AXI is an interface protocol defined by the AMBA4 specification. It is a widely-used bus protocol designed for high-performance, low-latency communication between IP blocks within an FPGA.

There are 3 different types of AXI4 interfaces:

- **AXI4 or Full AXI:** For high-performance memory-mapped requirements. The Master/Slave interface is composed of five channels (Read Address, Write Address, Read Data, Write Data, and Write Response) and can burst transactions of up to 256 data transfers.
- **AXI4-Lite:** A simplified, lightweight version of the AXI protocol for simple, low-throughput memory-mapped communication. This version is more straightforward to use but cannot burst like AXI. It is commonly used for control and configuration purposes, allowing the CPU to interact with peripheral registers.
- **AXI4-Stream:** A unidirectional protocol optimized for high-throughput, high-speed, low-latency streaming data transfers. It defines a single channel for the transmission of streaming data, based on the AXI4 write data channel. Unlike AXI4, AXI4-Stream interfaces can burst an unlimited amount of data.

More information about AXI can be found in [Xilc].

To implement the more suitable version for the CPU and FPGA communication, the requirements of the communication should be defined. The hardware/software partitioning discussed in Chapter 4.2.2 implies that the CPU handles only control computations. These computations are executed at a slow control rate, meaning that communication does not need to be constrained by low latency or high throughput. The interface only requires sufficient bandwidth to transfer all the control data at the control rate. The number of controllers and, thus, the number of data transfers required highly depends on the audio DSP. Most of them typically require a dozen, however in certain fields such as audio spatialization, algorithms may necessitate hundreds of them. The Axi4-Stream's high throughput is unnecessary for these needs. Embedding the complexity and overhead of Full-AXI appears unnecessary, although it might be required if the number of controllers is excessively high. The Axi4-Lite is the optimal choice for controller transfer, as it is specifically designed for this type of slow, short communication.

IP Interface

This section discusses how to configure the IP interface based on audio specific needs. As detailed in Section 5.3.6, the top-level function arguments define the IP interface. The next example uses the template interface of the IP employed in our *Syfala* compiler, as presented in Listing 5.12.

Lang: C

```

1 // -----
2 // HLS TOP-LEVEL FUNCTION
3 // -----
4 void syfala (
5     sy_ap_int audio_in[INPUTS],
6     sy_ap_int audio_out[OUTPUTS],

```

```

7   float arm_control_f[FAUST_REAL_CONTROLS],
8   int arm_control_i[FAUST_INT_CONTROLS],
9   float arm_control_p[FAUST_PASSIVES],
10  int arm_ok,
11  bool enable_i2s,
12  int* control_block,
13  #if SYFALA_DEBUG_AUDIO //-----
14      float arm_debug[FAUST_OUTPUTS],
15  #endif //-----
16  bool* i2s_rst,
17  float* mem_zone_f,
18  int* mem_zone_i,
19  bool bypass,
20  bool mute,
21  bool debug
22 ){
23 #pragma HLS INTERFACE s_axilite port=arm_control_f
24 #pragma HLS INTERFACE s_axilite port=arm_control_i
25 #pragma HLS INTERFACE s_axilite port=arm_control_p
26 #pragma HLS INTERFACE s_axilite port=arm_ok
27 #pragma HLS INTERFACE s_axilite port=enable_i2s
28 #pragma HLS INTERFACE s_axilite port=control_block
29 #if SYFALA_DEBUG_AUDIO //-----
30     #pragma HLS INTERFACE s_axilite port=arm_debug
31 #endif //-----
32 #pragma HLS INTERFACE m_axi port=mem_zone_f latency=30 bundle=ram
33 #pragma HLS INTERFACE m_axi port=mem_zone_i latency=30 bundle=ram
34
35 #pragma HLS array_partition variable=audio_in type=complete
36 #pragma HLS array_partition variable=audio_out type=complete
37 [...]

```

Listing 5.12: Excerpt of the IP interface.

First, `sy_ap_int audio_in[INPUTS]` and `sy_ap_int audio_out[OUTPUTS]` represent the audio inputs and outputs. These are custom `ap_int` types with a width adapted to the audio bitwidth defined by the user (or 24 bits by default), and they communicate directly with a *VHDL* I²S transceiver that bridges with external I²S communication. They are implemented as arrays of size `INPUTS` or `OUTPUTS` to facilitate the scalability of the design by changing the number of inputs and outputs simply by modifying the `INPUTS` and `OUTPUTS` macros, and allowing their use in loops. This array-based approach creates a single bus for inputs and one for outputs. For scalability and ease of integration with the I²S transceiver, separating each channel onto different wires simplifies the design. This avoids the need for a complex multichannel bus interface. With two `array_partition` directives applied, *Vitis* HLS splits the arrays into independent registers, allowing access to individual inputs and outputs (as already experienced in Section 5.3.6), while maintaining the scalability and scriptability of the arrays.

For the CPU/FPGA interface, `float arm_control_f`, `int arm_control_i`, and `float arm_control_p` are the three types of input controllers (respectively floating-point values, integer values, and pointers). As discussed before, these should fit within an AXI-Lite interface and are therefore declared as such with the `pragma HLS INTERFACE s_axilite port=arm_control_<TYPE>`.

`arm_ok`, `enable_i2s`, `arm_debug` and `control_block` are control signals sent from the CPU to manage the IP functionalities.

`bool bypass`, `bool mute`, and `bool debug` do not follow any specified protocol. They are simply control signals that provide easy control over the IP. `bypass` and `mute` are inputs controlled by switches to bypass or mute the IP. These signals are directly wired from the FPGA switches to the IP. `debug` is a single-wire output used to facilitate debugging by, for example, outputting a one or

a zero depending on the IP state. This signal can be easily read with an oscilloscope.

`float* mem_zone_f` and `int* mem_zone_i` are interfaces to external DDR memory. On SoC *Zynq* families, external DDR is accessible from the FPGA through the AMBA bus. By using `m_axi` instead of `axilite`, we ensure high-performance burst accesses to the DDR. The memory is divided into separate zones for integer and floating-point data.

5.5.2 Memory Access Optimization

Memory Performance

As with general-purpose CPUs, external DDR access can be a major performance bottleneck. Memory access latency must be carefully managed in *Vitis* HLS by providing precise access times. If the tool does not have accurate access times, the latency report may be incorrect and can lead to poor scheduling of the IP, resulting in an inefficient implementation. The performance of memory accesses is challenging to predict, and there are limited official performance metrics provided by *Xilinx* for external DDR access in HLS designs for the *Zybo* and *Genesys* boards.

To measure the memory access time, we first needed to measure the effective number of cycles taken to compute one sample by the IP. To achieve this, we connected the `ap_start` signal, which triggers the start of the computation, and the `ap_done` signal, which indicates the completion of the current sample's computation, to an oscilloscope through the board's GPIOs. This setup allowed us to precisely measure the actual IP latency in terms of the number of cycles. From our experiments, if no access to DDR memory is carried out in the code and so all the memory of the program can be stored on FPGA block RAMs, the evaluation reported by the HLS tool is very precise, if not perfect. But if reads or writes occur in DDR memory, the precision of the report relies on the latency given as a pragma to *Vitis* HLS (e.g., 30 cycles in Listing 5.12 line 32-33).

By adjusting the interface pragma latency to make the estimated latency reported by the tools match the actual measured latency, we were able to determine the latency of DDR access.

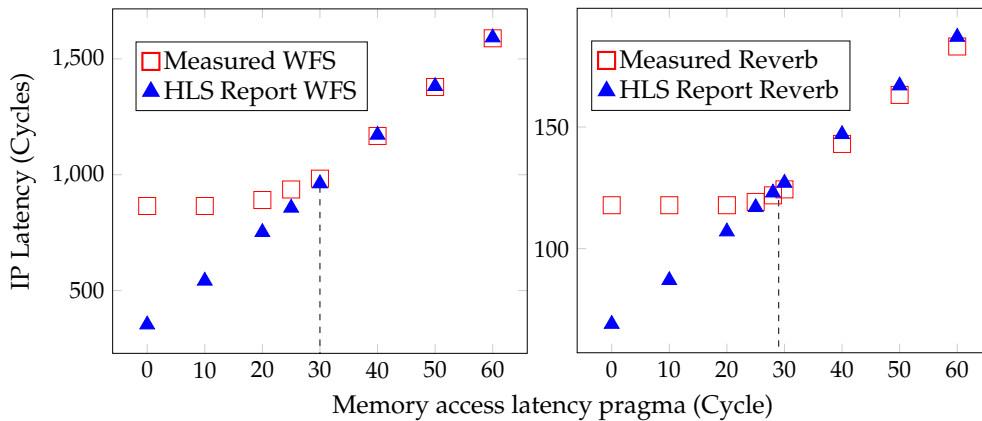


Figure 5.38: Estimated vs. measured latency of the IP for different values of the memory latency pragma for two different audio DSP algorithms (WFS and Reverb) on *Zybo* board.

Our measurements are presented in Fig. 5.38 for two audio DSP programs implementing a Wave Field Synthesis and an artificial reverberation algorithm that have been synthesized and executed for different values of the latency pragma. These measures show that, in these two cases, the optimal value for the latency pragma is 30 cycles. Below 30 cycles, the prediction is not realistic, and above 30 cycles, scheduling is not optimal. This optimal value could be very different for other applications, depending on their memory access patterns. However, for all the audio programs that we tested on both *Zybo* and *Genesys* FPGA boards, this 30 cycles value was approximately

the best value to use for a 125 MHz clock. Hence, we believe that the restricted nature of audio applications facilitates this memory performance prediction problem.

Buffer Type

There are various possible buffer implementations in hardware. In software, a circular buffer with incremental read and write indexes is often used, which is much more efficient than explicitly shifting all coefficients with a linear buffer. For FPGAs, however, using explicit shifting is often preferred due to its intuitive nature.

With shifting in linear buffering, the number of read and write operations is proportional to the length of the filter. This method is inefficient on a CPU because it requires many operations. On an FPGA, pipelining helps and each read and write operation generally adds one clock cycle, resulting in a latency roughly equal to the number of coefficients. An example of a shifting linear buffer implementation is shown in Listing 5.13.

Lang: C

```

1 void compute_fir(float* fTemp) {
2     compute_fir:for (int n = 0; n < NCOEFFS; ++n) {
3         *fTemp += samples[n] * coeffs[n];
4     }
5 }
6 void linear_buffer(){
7     linear_buffer:for (int j = NCOEFFS-1; j > 0; --j) {
8         samples[j] = samples[j-1];
9     }
10 }
```

Listing 5.13: Linear buffer example.

This approach can be optimized using a circular buffer (Listing 5.14). Instead of shifting all the values of the buffer one by one, a circular buffer shifts the reference pointer. This method significantly reduces the amount of data movement and can improve performance depending on the design.

Lang: C

```

1 void compute_fir(float* fTemp) {
2     compute_fir:for (int n = 0; n < NCOEFFS; ++n) {
3         *fTemp += samples[(idx+NCOEFFS-n)%(NCOEFFS)] * coeffs[n];
4     }
5     idx=(idx+1)%NCOEFFS; //Circular Buffer
6     //For NCOEFFS power of 2, use &(NCOEFFS-1) instead of %NCOEFFS
7 }
```

Listing 5.14: Circular buffer example

For an arbitrary number of iterations, such as in the FIR filter with 115 coefficients, there is a latency gain of approximately 115 clock cycles (see Table 5.15). This gain arises because there is no need to perform the full iteration of the buffer. However, this approach uses the modulo operation that can be costly and time-consuming. Although the latency added by the modulo operation does not significantly impact the overall latency (as the initiation interval for this example remains unchanged), resource usage increases due to the complexity of the modulo operation.

When the number of iterations is a power of 2, the modulo operation can be replaced by an *AND* operation, which is significantly more lightweight. Table 5.16 illustrates that, for 128 coefficients (a power of 2), the circular buffer using this approach not only reduces latency but also consumes

fewer resources. This is because the modulo operation is replaced with an *AND* operation, which is nearly free in terms of resource usage. The modulo operator is automatically replaced with a *AND* operator when the number of iterations is a power of 2, thus simplifying the implementation and reducing resource usage. While this gain might seem negligible, it is very valuable when dealing with thousands of coefficients. Generally, the circular buffer remains a good implementation practice, especially when the design involves multiple buffers or large data sets.

Buffer Type	Latency (cyc.)	BRAM	DSP	FF	LUT
Linear	1 065	3 (2%)	7 (8%)	1 270 (3%)	2 246 (12%)
Circular (mod)	960	2 (1%)	7 (8%)	1 653 (4%)	2 433 (13%)

Table 5.15: Resource usage for `fir_onesample.cpp` and `fir_onesample_circularbuffer.cpp` for 115 coefficients.

Buffer Type	Latency (cyc.)	BRAM	DSP	FF	LUT
Linear	1 182	3 (2%)	7 (8%)	1 271 (3%)	2 248 (12%)
Circular (&)	1 051	2 (1%)	7 (8%)	1 227 (3%)	2 195 (12%)

Table 5.16: Resource usage for `fir_onesample.cpp` and `fir_onesample_circularbuffer.cpp` for 128 coefficients.

▼ Design Guideline 5.5.1 — Circular Buffer

Using incremental indexes circular buffers instead of shifting linear buffers is efficient when the buffer is large, is a power of 2, or when there are multiple buffers.

However, this rule must be taken with caution. The `idx` value inside the array index complicates the parallel execution of the loop and tends to keep the *II* high for high unroll factors. For example, if we try to force an *II*=1 and unroll the loops with a factor of 16 (with 128 coefficients), the linear buffer can achieve an *II*=5 for `compute_fir` and *II*=2 for `linear_buffer`, whereas the lowest *II* is 8 for the circular buffer. This becomes worse with higher unroll factors, as the one-sample FIR filter results show in Section 6.4.1.

▼ Design Guideline 5.5.2 — Linear Shifting Buffer

With high unroll factors, when latency is highly critical, using a shifting linear buffer can be more effective.

5.6 Final Evaluation and Insights on HLS Optimization for Audio DSPs

This concluding chapter provides final, generic optimization strategies for high-level synthesis. We conclude by analyzing the code optimizations highlighted throughout this chapter, considering their integration feasibility into an “audio to FPGA” compiler.

5.6.1 Further Recommended Guidelines

This brief section presents a few key strategies for achieving better results when designing hardware for audio programs. These are general principles that we used and found to significantly improve our designs. This is by no means exhaustive, but it aims to summarize best practices that have proven helpful.

Code Rearranging

Since pipelining requires the use of loops, it is likely that the implemented program contains many of them. It is very efficient to merge loops whenever possible.

Lang: C <pre style="margin: 0;">1 //Non optimized version: 2 for(int k = 0; k < N_COEFS; k++){ 3 y_hat += C_hat[k]*x_n[k]; 4 } 5 for(int k = 0; k < N_COEFS; k++){ 6 x_n_norm += x_n[k]*x_n[k]; 7 }</pre>	Lang: C <pre style="margin: 0;">1 //Optimized version: 2 3 for(int k = 0; k < N_COEFS; k++){ 4 y_hat += C_hat[k]*x_n[k]; 5 x_n_norm += x_n[k]*x_n[k]; 6 } 7</pre>
--	---

Figure 5.39: Excerpt of a program showing two loops that can be merged.

Listing 5.39 shows an example of two distinct loops with identical boundaries that can be merged. Although it may seem trivial, this optimization nearly halves the latency while conserving resources in our experiments. The pragma `HLS loop_merge force` is intended to perform this optimization automatically, but we encountered inconsistent results with *Vitis* HLS 2022.2. As a result, manually merging loops proved to be more reliable.

▼ Design Guideline 5.6.1 — Merged Loops

Merging consecutive loops into a single loop can reduce overall latency, increases resources sharing, and improves logic optimization.

It is crucial to minimize the computations performed within the loop if they are not sequential. Any computation that can be done once outside the loop should be moved out to save the overhead of repeating it for every iteration of the loop. Listing 5.40 demonstrates an example of performing computations outside the loop.

Lang: C <pre style="margin: 0;">1 //Non optimized version: 2 for(int k = 0; k < N_COEFS; k++){ 3 C_hat[k] += x_n[k]*(mu*error)/(4 x_n_norm+0.001f); 5 }</pre>	Lang: C <pre style="margin: 0;">1 //Optimized version: 2 float inv_x_norm=(mu*error)/(x_n_norm 3 +0.001f); 4 for(int k = 0; k < N_COEFS; k++){ 5 C_hat[k] += x_n[k]*inv_x_norm; 6 }</pre>
--	---

Figure 5.40: Excerpt of a program (LMS) showing the offloading of computations from a loop.

Here, $(\mu * error) / (x_{n_norm} + 0.001f)$ is constant during all iterations of the loop and can be computed once before the loop, then stored and read within the loop. This approach saves multipliers, adders, and even a division from being executed in each iteration.

▼ Design Guideline 5.6.2 — *Computation in Loops*

All computations that are not dependent on the loop iteration should be performed before the loop and used as constants during the iteration.

If the loops are pipelined, moving the operations outside the loop may not have a significant impact, as the logic is already optimized by the pipeline and the latency minimized by the initiation interval.

Arithmetic Operations

Division falls within the category of computationally expensive operations and should be avoided whenever possible.

▼ Design Guideline 5.6.3 — *Costly Operators*

Costly operators such as division, modulo, square root, exponential and logarithmic functions, and trigonometric functions (sine, cosine, etc.) should be avoided.

Using fixed-point arithmetic is generally much more efficient than floating-point for DSP on FPGAs. However, generating efficient fixed-point code is far from trivial. Currently, FAUST cannot generate fixed-point DSPs, and writing correct and efficient fixed-point C++ code by hand can be tedious and error-prone. This is the main reason it has not been used in this thesis, but efforts should be focused on this aspect in the future. Using fixed-point arithmetic has been a challenge that the Emeraude Team has been working to resolve for years.

▼ Design Guideline 5.6.4 — *Fixed-Point Arithmetic*

Fixed-point arithmetic should allow for more efficient implementations than floating-point arithmetic, but writing fixed-point code is complex.

An operation (such as multiplication, addition, or division) can be bound to a specific resource for implementation. Using the `Bind_op` pragma, it is possible to specify that a particular operation is implemented using LUTs or DSP Blocks. This can help balance resource usage and lead to changes in latency.

▼ Design Guideline 5.6.5 — *Binding Different Resource*

If FPGA resources are saturated, it can be beneficial to bind an operation to a different resource to alleviate the load on the saturated resource.

Latency Optimization Priority

For strictly parallel loops, the loop with the highest latency should be optimized first. If a program contains an *if* statement with two distinct hardware paths, *Vitis* fully parallelizes them, resulting in the total latency of the IP being determined by the longest latency between these paths. During optimization, if the latency of one hardware path falls below that of the other, the tool may allocate less effort to further optimization, as the overall latency is no longer affected. This could potentially render all the optimizations discussed in this thesis ineffective.

▼ Design Guideline 5.6.6 — Longest Path Latency Priority

For parallel hardware, a circuit is only efficiently optimized if its latency is the highest among all the parallel circuits.

Low Sample Rate

If none of the previous optimizations are sufficient to meet the timing requirements, the final option is to lower the sampling rate. This provides more time to compute each sample, allowing more complex algorithms to be processed while still meeting the sampling rate constraint (as illustrated in Fig. 5.41). Since the sampling frequency does not affect the FPGA clock frequency, the computation function requires the same amount of time, but there is more time available between two samples.

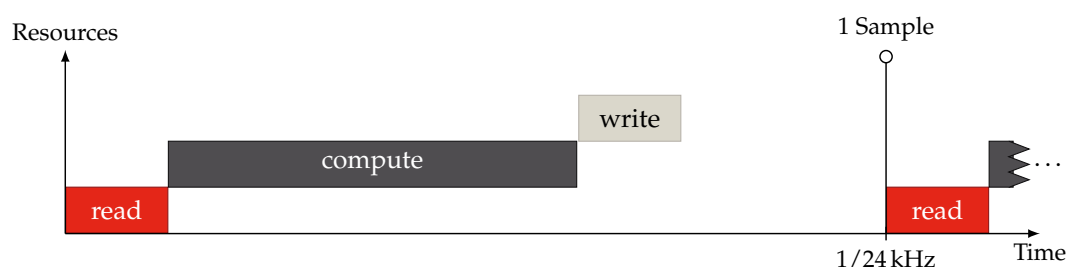


Figure 5.41: Effect of lower sample rate: the longer compute function can now fit within the timing constraints without requiring further optimization.

Lowering the sample rate increases latency and directly impact audio quality, so it should be used with caution. However, many audio programs do not require a 48 kHz sampling frequency, such as active noise-canceling systems, which can operate at lower frequencies. Reducing the sample rate should be considered as a last resort when other optimization techniques are insufficient.

▼ Design Guideline 5.6.7 — Lower Sample Rate

Reducing the sampling rate allows more time to compute but impacts the audio quality.

About the Performance Pragma

Instead of manually handling pipelining, unrolling, and partitioning, *Vitis* HLS provides a convenient pragma called performance. This pragma allows you to specify the number of clock cycles between the successive starts of a loop and lets the tool infer the unroll, pipeline, array-partition, and inline pragmas to achieve the desired performance. While it doesn't always suc-

ceed in meeting the requirements, it generally produces a highly optimized result. In most cases, for our example, it infers a pragma pipeline `II=1`, applies partitioning if needed, and unrolls the loop according to the number of iterations divided by the target interval.

I did not discuss it earlier as I found it challenging to fine-tune when trying to reach a specific interval. Specifying a number of clock cycles can be less intuitive than using an unroll factor that divides the number of iterations by an integer. Moreover, using pipelining and unrolling separately provides more control over loop optimization, as we have seen in this chapter.

However, the performance pragma is still an efficient and straightforward option worth trying. Most of the time, using pragma `HLS performance target_ti=NCOEFFS` in a nested loop results in the same latency as the lowest observed in our experiments. However, if this approach uses too many resources, the only solution is to increase the `target_ti` value without a specific rule to follow. This could reduce the latency but generally won't significantly reduce resource usage, making it less efficient. Finally, it relies on *Vitis* HLS masked optimizations, which are already very complex and inconsistent, making it difficult to recommend.

5.6.2 Concluding Thoughts on Automating HLS for Audio Applications

This chapter presents a thorough exploration of the complexities and inconsistencies inherent in HLS optimization. Achieving optimized implementations heavily depends on the behavior of HLS tools and often necessitates a deep analysis of the program, which complicates the idea of automating the process through a compiler.

We introduced the *Xilinx* HLS tool to familiarize readers with its use in this work, as understanding hardware design is essential for the thesis. Then, we established the basics of hardware optimizations, focusing on unrolling and pipelining. The third section detailed the use of pragma directives for guiding hardware implementation, with a focus on those most relevant to this thesis. While an expert user might rely on documentation, this aimed to provide explanations tailored to the needs of audio DSPs implementation, prioritizing clarity over exhaustiveness. In Section 5.4, we presented a generic approach to optimizing audio programs using pragmas, with our main contribution being the loop interchange technique. This simple manipulation resulted in performance improvements exceeding tenfold. Finally, we discussed the generation of efficient interfaces for hardware IP through adapted pragmas and provided some general best practices for designing audio IP.

The issues and considerations discussed in this chapter are common challenges in hardware design. Typically, design analysis identifies bottlenecks to apply the appropriate pragmas and assist HLS. However, the goal of this thesis is to automate this process within an audio compiler. The aim is not to achieve the absolute best implementation but to develop a solution that, in most cases, optimizes the design effectively. Therefore, we have not investigated deeply into design analysis, as the specific conclusions we would have drawn here would probably not apply universally. Instead, we sought to identify general behaviors in pragmas by testing them with audio applications use cases to apply them across different architectures.

This chapter has highlighted two key insights about HLS. First, the tool often defaults to resource-saving behaviors that result in high latencies, which are problematic for audio programs due to their inherent sequentiality and strict timing constraints. Then, effective optimizations can be achieved with minimal effort once the correct approaches are known. For example, a simple loop interchange can reduce resource usage by over tenfold, and a well-placed pragma pipeline can decrease latency without additional resource consumption. Clear patterns for optimization can be identified and applied with minimal additional knowledge, though they remain dependent on the overall algorithm and may sometimes be challenging to achieve.

6

CHAPTER 6

Real-World Applications and Outcomes

Contents

6.1	Compiling from a DSL: FAUST to FPGA	133
6.1.1	FAUST Audio Program	133
6.1.2	FAUST Hardware/Software Dispatching	134
6.1.3	FAUST Compiler Modifications	134
6.1.4	Compiling Process Overview	135
6.1.5	Hardware and Software Integration	137
6.1.6	Control Binding	138
6.1.7	Experimental Results and Contributions	139
6.2	Ultra Low Latency Audio Path	140
6.2.1	I ² S Transceiver	141
6.2.2	Audio Codecs Configuration	142
6.2.3	External Audio Codec Latency Measurements	143
6.2.4	Troubleshooting SSM2603: Audio Distortion Due to Faulty Initialization	146
6.3	Multichannel Interfaces	147
6.3.1	Frugal Board	148
6.3.2	Ultra Low Latency Board	150
6.3.3	Syfala Toolchain Compatibility	154
6.4	Experimental Results	154
6.4.1	FIR Filter	155
6.4.2	Frugal WFS System	158
6.4.3	Active Noise Control: LMS and FxLMS	159
6.4.4	Modal Synthesis	162
6.4.5	Comparative Insights: Evaluating Our Approach Against Existing Works	163

Chapters 4 and 5 present guidelines and methodologies for implementation optimizations. This chapter outlines the implementation of these approaches and presents the experimental results obtained from them.

The general concepts introduced in Chapter 4 can be relatively easily implemented in a compiler. The idea is that, starting from a high-level description of an audio DSP, the concepts discussed in this chapter can be automated. This is demonstrated in Section 6.1. Using FAUST, we modified the compiler to incorporate general changes in the generated C++ code, aligning it with the *Implementation Guidelines* provided in Chapter 4, but not the guideline of Chapter 5 (guidelines of Chapter 5 will be discussed in section 6.4 and could not be directly included in the FAUST compiler as explained below). These changes involve adjusting the program generation to support hardware-software co-design, including computation dispatching, memory allocation, and control handling. While the section specifically details how this was achieved with the FAUST compiler, it is presented in an explanatory and general manner to allow for potential application to other DSLs or general-purpose languages.

The method of interfacing with the analog world through external audio codecs, as outlined in Chapter 4, is discussed in Section 6.2. This section describes our custom I²S transceiver, which facilitates communication with these external converters. Since all audio codecs use the I²S standard, the information presented here is broadly generalizable, regardless of the DSP program or FPGA platform used. Our work presents the I²S transceiver as an independent IP core that can be not directly linked to the compiler, which is why it is discussed in a distinct section. For our specific requirements, we configure this IP directly from the compiler to match the audio program (e.g., number of inputs/outputs, sampling rate). However, the process is thoroughly documented and can be easily adapted to other compilers.

Hence, the results of the first two sections (6.1 and 6.2) constitute a functional FAUST-to-FPGA toolchain, enabling ultra-low-latency FPGA applications directly from FAUST DSP code. This toolchain is fully scripted, from the DSL to the FPGA bitstream, and does not require any prior knowledge of hardware design. However, to achieve resource-optimized designs, the HLS input code must be prepared following the guidelines presented in Chapter 5. Initially, we implemented these guidelines directly in C++ code without using FAUST, allowing us to fine-tune the design without modifying the FAUST compiler. Although our initial goal was to automate these code enhancements within the compiler, the task proved to be more complex than anticipated, and this functionality was not integrated into the FAUST compiler yet.

Implementing such optimizations would require significant work to enable the compiler to identify specific patterns and configurations within the code. In the case of the FAUST compiler, this challenge was even greater, as FAUST automatically unrolls all loops in the generated C++ code. Implementing loop interchanges, for example, was not feasible at the time. Modifying the compiler to integrate Chapter 5's *Design Guidelines* would have required extensive effort beyond the scope of this thesis. Instead, the goal of this thesis was to identify potential optimizations and explore how they could be manually applied without requiring a deep understanding of the DSP program. This approach lays the groundwork for future automation within the compiler. Section 6.4 demonstrates the experimental results obtained by manually applying Chapter 5's guidelines.

Finally, Section 6.3, which addresses the challenge of handling multichannel processing, primarily focuses on the Printed Circuit Board (PCB) design and fabrication process. Although this section presents PCBs designed for our *Xilinx* FPGA boards, it is intended to be didactic, offering useful information for building multichannel interfaces. While certain technical details (such as connectors or voltage levels) may vary depending on the FPGA board or audio codec used, the challenges and solutions discussed here are broadly applicable to any multichannel board.

Hence, as a summary, this chapter is organized as follows:

- Section 6.1 presents the FAUST to FPGA workflow of our *Syfala* compiler.
- Section 6.2 discusses the communication and configuration with audio codecs. Low-latency audio codecs are tested in real conditions, giving a real understanding of such audio codec capabilities.
- Section 6.3 details the two PCB boards developed during this thesis, enabling the interfacing of up to 512 channels to a single FPGA with low-latency audio codecs.
- Section 6.4 presents our experimental results and provides insights into the possibilities that this work and our toolchain can offer.

6.1 Compiling from a DSL: FAUST to FPGA

All the elements presented in this thesis constitute a complex system with various technical aspects. Our *Syfala* toolchain has automated and scripted these processes, providing programmers with a high level of abstraction for FPGA programming in the audio domain.

By starting with an audio domain-specific language like FAUST, it is possible to fully automate the process and develop one of the key contribution of this thesis: a FAUST-to-FPGA compilation flow. Building on the groundwork of Chapter 4, this section presents the entire workflow of *Syfala*, from a FAUST DSP program down to the FPGA bitstream and CPU executable, incorporating audio-specific optimizations.

This example relies on *Xilinx* software, is compiled for the *Zybo Z7* FPGA platform embedding a *Xilinx Zynq-7000* FPGA SoC, and the DSP is implemented using a one sample approach (see Chapter 4.2.3).

6.1.1 FAUST Audio Program

The FAUST program used in this example is a simple audio synthesizer that incorporates oscillators, low-frequency oscillation (LFO), noise, and panning. This example, called *VirtualAnalog*, serves as the "Hello World" of our tests: a simple, lightweight program that uses several techniques. It produces a signal that's easily recognizable by ear, facilitating a quick review of its functionality.

Lang: FAUST

```

1 import("stdfaust.lib");
2 // sliders
3 oscFreq = hslider("oscFreq [knob:1]",80,50,500,0.01);
4 lfoFreq = hslider("lfoFreq [knob:2]",1,0.01,8,0.01);
5 lfoRange = hslider("lfoRange [knob:3]",1000,10,5000,0.01) : si.smoo;
6 noiseGain = hslider("noiseGain [slider:7]",0,0,1,0.01) <: _*_;
7 masterVol = hslider("masterVol [slider:8]",0.8,0,1,0.01) <: _*_;
8 panning = hslider("pan [knob:4]",0.5,0,1,0.01) : si.smoo;
9 // buttons
10 activateNoise = button("activateNoise [switch:6]");
11 killSwitch = 1-button("killSwitch [switch:5]");
12
13 LFO = os.lf_triangle(lfoFreq)*0.5 + 0.5;
14
15 process = os.osrcrc(440)* 0.25 * killSwitch * os.sawtooth(oscFreq) + no.noise*noiseGain
    *activateNoise : fi.resonlp(LFO*lfoRange+50,5,1)*masterVol <: _*(1-panning),_*
    panning;
```

Listing 6.1: *VirtualAnalog* FAUST example

Line 1 imports the standard FAUST library, which includes useful DSP functions and modules.

Line 4 to 9 define the controls sliders and knobs.

Line 12 and 13 define two control switches.

Line 15 generates a triangle wave that oscillates between 0 and 1 at the lfoFreq.

Line 17 is the main process:

- `os.oscr(440)* 0.25 * killSwitch * os.sawtooth(oscFreq)` generates a sawtooth waveform at the frequency set by `oscFreq`, modulated by a 440 Hz sinusoidal oscillator, scaled by 0.25, and gated by `killSwitch`.
- This signal is added to `no.noise*noiseGain*activateNoise` that generates noise controlled by `noiseGain` and toggled by `activateNoise`.
- Then, the generated signal is input to `fi.resonlp(LF0*lfoRange+50,5,1)` that passes the combined signal through a low-pass resonant filter with a cutoff frequency modulated by the LFO, centered around `LF0*lfoRange+50` Hz.
- This is then multiplied with `masterVol` that scales the final signal.
- Finally, `_<: _*(1-panning),_*panning` splits the signal into stereo channels with panning controlled by `panning`.

Some technical choices made here may seem odd, but this is a pedagogical example that does not aim to be musically relevant. The program can be tested on the FAUST online IDE [Grac].

6.1.2 FAUST Hardware/Software Dispatching

The FAUST compiler generates both hardware (i.e., audio IP) and software (i.e., control program) .cpp files from the program on Listing 6.1. This automatic split is enabled by two specific features of FAUST.

FAUST Automatic Control Detection

FAUST natively differentiates computations that depend on a controller from those that depend on a sample. With the C++ backend, the compiler names these functions `controlmysdp` (for controller processing) and `computemydsp` (for sample processing). Specifically for this project, we modified the FAUST compiler to allow for a better separation between control rate and sample rate computations, as well as for a better control of the DSP memory layout. These modifications were identified through this thesis and are detailed in Section 6.1.3. However, they were implemented in the compiler by Stéphane Letz, one of the FAUST developers.

FAUST Architecture Files

FAUST uses a system of file wrapper to separate the audio DSP interface into a different file from the DSP algorithm itself. This external file, called the *architecture file*, defines how to connect a FAUST program to the external environment, specifically regarding the audio drivers and controller interfaces being used. The source DSP can be compiled with two distinct architecture files selected at compile time. Each architecture file produces a different compiled output file, containing the source FAUST DSP code translated to C++, along with the additional code defined in the respective architecture file.

6.1.3 FAUST Compiler Modifications

In order to include all the implementation guidelines expressed in Chapter 4, the compiler has been modified. These modifications introduce a new `-os2` compilation option to the FAUST com-

piler for the C and C++ backends. The modifications are as follows:

- A distinct `controlmysp` function is generated. It contains the code executed at the control rate using the updated values of all controllers (i.e., buttons, sliders, etc.). The `controlmysp` function is called on the CPU side.
- The `computemysp` function processes a single sample. It does not contain the loop generated in the standard version of the FAUST compiler, which would normally process a buffer of input/output samples. Hence, the name of the new option: “-os2” stands for one sample. The `computemysp` function is called on the FPGA side.
- Two additional `iControl` (for integer values) and `fControl` (for float values) arrays are used as parameters to share the control state between the `controlmysp` and `computemysp` functions. It is used to do the FPGA/CPU communication through the AMBA interface.
- DSP memory is divided between a section kept in the DSP class/structure, which is implemented on the FPGA BRAM, and a separate section typically allocated on the external DDR. The DDR usually contains long delay lines. The user can adjust the memory layout using the additional `-fpga-mem` command option to precisely control how memory is divided. This option defines the memory capacity threshold at which the external DDR is used, allowing the user to adjust it based on the amount of memory available in Block RAMs.
- Two new functions have been added to copy constants computed on the CPU side to the FPGA. This modification facilitates the hardware-software partitioning detailed in Chapter 4.2.2 and facilitates the initialization of constants on the CPU.

6.1.4 Compiling Process Overview

The compiling process is illustrated in Fig. 6.1. This approach calls twice the FAUST compiler and produces two separate files (`audioDSP.cpp` and `audioCtl.cpp` on Fig. 6.1), each based on the DSP and custom architecture file (respectively `hardware.cpp` and `software.cpp` on Fig. 6.1).

- The `software.cpp` architecture file, which is used for the CPU implementation, calls the `controlmysp` function but not `computemysp`. It writes the controller values into the `iControl` and `fControl` arrays. It is called with the `-os2 -a software.cpp` FAUST command line.
- Conversely, the `hardware.cpp` architecture file, which is used for hardware IP synthesis, calls the `computemysp` function but not `controlmysp`. It reads the controller values from the `iControl` and `fControl` arrays. It is called with the `-os2 -a hardware.cpp` FAUST command line.

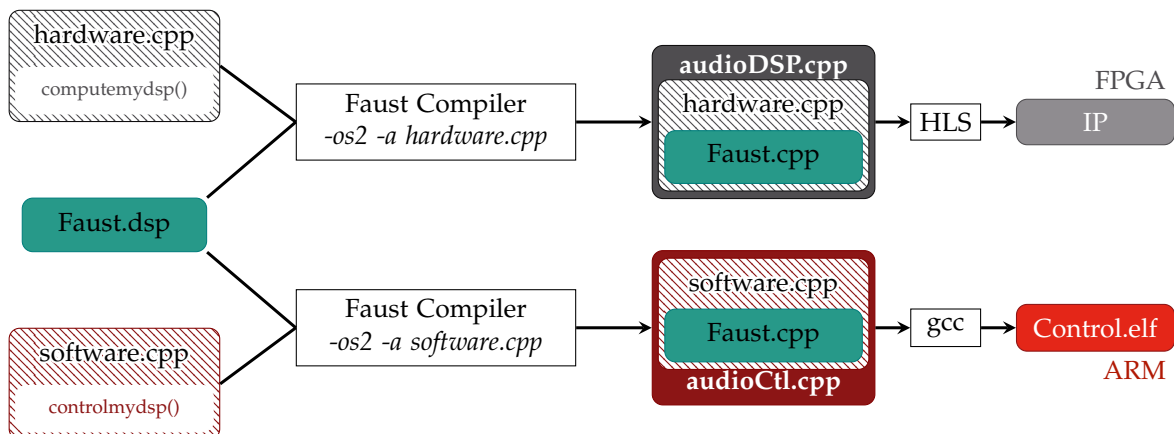


Figure 6.1: FAUST to FPGA compilation example.

These concepts are illustrated by the C++ code excerpt in Listing 6.2 generated by the FAUST compiler with the `-os2` option from the DSP program 6.1.

Lang: C

```

1 void controlmysp(dsp* dsp, int* RESTRICT iControl, float* RESTRICT fControl, int*
  RESTRICT iZone, float* RESTRICT fZone) {
2   fControl[0] = dsp->fConst1 * (float)(dsp->fHslider0);
3   fControl[1] = dsp->fConst2 * (float)(dsp->fHslider1);
4   fControl[2] = 4.656613e-10f * (float)(dsp->fButton0) * mydsp_faustpower2_f((float)(
    dsp->fHslider2));
5   fControl[3] = fmaxf(1.1920929e-07f, fabsf((float)(dsp->fHslider3)));
6   fControl[4] = dsp->fConst1 * fControl[3];
7   fControl[5] = 1.0f - dsp->fConst0 / fControl[3];
8   fControl[6] = 0.25f * (1.0f - (float)(dsp->fButton1));
9   fControl[7] = dsp->fConst2 * (float)(dsp->fHslider4);
10  fControl[8] = mydsp_faustpower2_f((float)(dsp->fHslider5));
11 }
12 [...]
13 void computemysp(dsp* dsp, FAUSTFLOAT* RESTRICT inputs, FAUSTFLOAT* RESTRICT
  outputs, int* RESTRICT iControl, float* RESTRICT fControl, int* RESTRICT iZone,
  float* RESTRICT fZone) {
14  dsp->iVec0[0] = 1;
15  dsp->fRec0[0] = fControl[0] + (dsp->fRec0[1] - floorf(fControl[0] + dsp->fRec0[1]));
16  dsp->fRec1[0] = fControl[1] + dsp->fConst3 * dsp->fRec1[1];
17  float fTemp0 = tanf(dsp->fConst4 * (0.5f * dsp->fRec1[0] * (2.0f * (1.0f - fabsf(2.0
    f * dsp->fRec0[0] + -1.0f)) + -1.0f + 1.0f) + 5e+01f));
18  float fTemp1 = 1.0f / fTemp0;
19  float fTemp2 = (fTemp1 + 0.2f) / fTemp0 + 1.0f;
20  dsp->iRec3[0] = 1103515245 * dsp->iRec3[1] + 12345;
21  float fTemp3 = fControl[4] + dsp->fRec4[1] + -1.0f;
22  int iTemp4 = fTemp3 < 0.0f;
23  float fTemp5 = fControl[4] + dsp->fRec4[1];
24  dsp->fRec4[0] = ((iTemp4) ? fTemp5 : fTemp3);
25  float fRec5 = ((iTemp4) ? fTemp5 : fControl[4] + dsp->fRec4[1] + fControl[5] *
    fTemp3);
26  dsp->fRec6[0] = dsp->fConst7 * dsp->fRec7[1] + dsp->fConst6 * dsp->fRec6[1];
27  dsp->fRec7[0] = (float)(1 - dsp->iVec0[1]) + dsp->fConst6 * dsp->fRec7[1] - dsp->
    fConst7 * dsp->fRec6[1];
28  dsp->fRec2[0] = fControl[6] * dsp->fRec7[0] * (2.0f * fRec5 + -1.0f) + fControl[2] *
    (float)(dsp->iRec3[0]) - (dsp->fRec2[2] * ((fTemp1 + -0.2f) / fTemp0 + 1.0f) + 2.0
    f * dsp->fRec2[1] * (1.0f - 1.0f / mydsp_faustpower2_f(fTemp0))) / fTemp2;
29  float fTemp6 = dsp->fRec2[2] + dsp->fRec2[0] + 2.0f * dsp->fRec2[1];
30  dsp->fRec8[0] = fControl[7] + dsp->fConst3 * dsp->fRec8[1];
31  outputs[0] = (FAUSTFLOAT)(fControl[8] * ((1.0f - dsp->fRec8[0]) * fTemp6 / fTemp2));
32  outputs[1] = (FAUSTFLOAT)(fControl[8] * (dsp->fRec8[0] * fTemp6 / fTemp2));
33  dsp->iVec0[1] = dsp->iVec0[0];
34  dsp->fRec0[1] = dsp->fRec0[0];
35  dsp->fRec1[1] = dsp->fRec1[0];
36  dsp->iRec3[1] = dsp->iRec3[0];
37  dsp->fRec4[1] = dsp->fRec4[0];
38  dsp->fRec6[1] = dsp->fRec6[0];
39  dsp->fRec7[1] = dsp->fRec7[0];
40  dsp->fRec2[2] = dsp->fRec2[1];
41  dsp->fRec2[1] = dsp->fRec2[0];
42  dsp->fRec8[1] = dsp->fRec8[0];
43 }

```

Listing 6.2: Excerpt of a FAUST compiled program, taken from `audioDSP.cpp` [The].

The `controlmysp` function (line 1) is executed on the CPU (`control.elf` in Fig. 6.1). The 8 controllers from the FAUST code can be identified along with dependent computations.

The `computemydsp` function (line 13) correspond to the sample computations of the compiled C++ code from the source FAUST program and is synthesized on the FPGA. Caching of duplicated memory accesses can be seen via the `ftemp<N>` variables and exploited by the HLS compiler. In this particular example, there are no large arrays, so all variables are stored in BRAMs. Note that the same `-os2` option is used on both the FPGA and CPU sides to preserve the structure of the generated code, only the architecture files change (`hardware.cpp` and `software.cpp` on Fig. 6.1).

6.1.5 Hardware and Software Integration

The `audioCtl.cpp` file is cross-compiled, and the resulting `.elf` file is executed on the CPU in a standalone implementation for this example. The `audioDSP.cpp` file is synthesized and implemented with *Xilinx Vitis* HLS to generates a hardware IP. Listing 6.3 shows an excerpt of the `hardware.cpp` architecture file used to generate `audioDSP.cpp`. It describes the IP top-level function arguments, which define the interface of the IP block.

Lang: C

```

1 void syfala (
2     sy_ap_int* audio_out_0,
3     sy_ap_int* audio_out_1,
4     float arm_control_f[FAUST_REAL_CONTROLS],
5     int arm_control_i[FAUST_INT_CONTROLS],
6     int* control_block,
7     int arm_ok,
8     bool enable_i2s,
9     bool* i2s_rst,
10    float* mem_zone_f,
11    int* mem_zone_i,
12    [...] ) {
13 #pragma HLS INTERFACE s_axilite port=arm_control_f
14 #pragma HLS INTERFACE s_axilite port=arm_control_i
15 #pragma HLS INTERFACE s_axilite port=arm_ok
16 #pragma HLS INTERFACE s_axilite port=enable_i2s
17 #pragma HLS INTERFACE m_axi port=mem_zone_f latency=30 bundle=ram
18 #pragma HLS INTERFACE m_axi port=mem_zone_i latency=30 bundle=ram

```

Listing 6.3: IP Top Level Function, excerpt of `hardware.cpp` [The].

The arguments of the *Syfala* function are set up as explained in Chapter 5.5.1. This is automatically done by scripts of the *Syfala* compiler, which generate only the necessary ports (in this case, only the audio output stream is generated, as the example code is output-only).

In addition to generating a design for this function, HLS also generates C++ drivers that allow the CPU to interface with these ports. This is illustrated in the excerpt of the generated code driver in Listing 6.4, showing functions to access and write `arm_ok` and `enable_i2s` variables.

Lang: C

```

1 [...]
2 void XSyfala_Set_arm_ok(XSyfala *InstancePtr, u32 Data);
3 u32 XSyfala_Get_arm_ok(XSyfala *InstancePtr);
4 void XSyfala_Set_enable_i2s(XSyfala *InstancePtr, u32 Data);
5 u32 XSyfala_Get_enable_i2s(XSyfala *InstancePtr);
6 [...]

```

Listing 6.4: Excerpt of HLS generated IP drivers from `HLSdrivers.h` [The].

The audio IP is then integrated into the complete SoC architecture, which includes the custom I²S transceiver, clock management, AXI interconnection, reset management, memory mapping, I/O

mapping, and other essential components, resulting in the block design shown in Fig. E.2. These IP blocks are configured by the compiler to align with the source DSP specifications. The final design is synthesized and implemented using *Vivado*.

6.1.6 Control Binding

The FAUST IP can be controlled using the physical controller board described in Chapter 4.4.1, or a software (i.e., running on a computer connected to the FPGA board) interface system. The two interfaces can be selected using a switch on the FPGA, with the control mechanisms illustrated in Fig. 6.2.

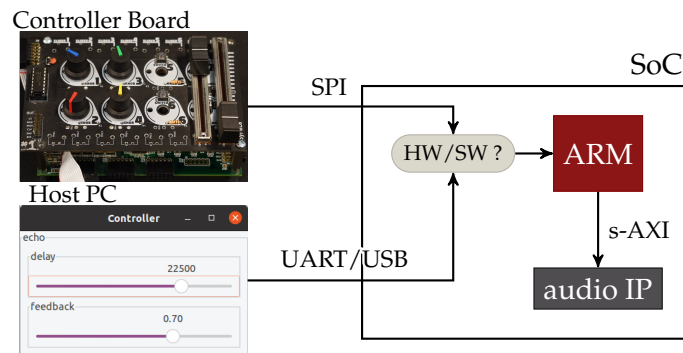


Figure 6.2: Hardware (knobs) and software (GTK app) interfaces for the audio IP (only one of them can be used at a time).

The physical controllers of the board (See Fig.6.3) can be bound to DSP parameters directly in the FAUST program using metadata such as `[knob:1]` or `[switch:1]` (see Listing 6.1). For instance, the `oscFreq` slider of the program in Listing 6.1 is bound to the first knob of the hardware interface board, etc. For processing these controllers on the CPU (including reading, conditioning, and processing), a portion of the code from the FAUST compiler is reused in the `software.cpp` architecture file. The generated functions enable the `audioCtl.cpp` file to manage the control interface effectively.

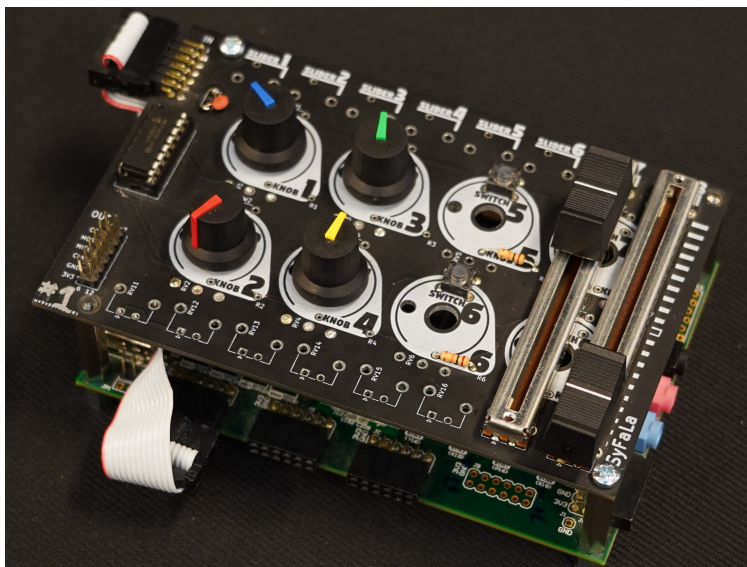


Figure 6.3: Custom controller board stacked on a Zybo Z7 presenting a possible combination of controllers.

If a software interface is used, FAUST generates a GUI based on this binding (the `UI.cpp` file equiv-

alent shown in Fig. 4.22 p.71) during compilation. This software interface is a GTK-based GUI directly adapted from the FAUST architectures environment and is meant to be executed on a host computer connected to the *Zybo* board through a serial link. Once launched, the host computer communicates with the ARM processor of the *Zybo* using the UART serial protocol. The corresponding communication classes `UartReceiverUI` and `UartSenderUI` are automatically integrated to the `audioCtl.cpp` file via the `software.cpp` architecture file and can be used for that purpose. Unlike the hardware controller, the generated software interface contains all the parameters declared in the FAUST program.

6.1.7 Experimental Results and Contributions

Compiler Improvements and Related Publications

This workflow from FAUST to FPGA led to the first publications of this thesis at the Sound and Music Computing (SMC) conference in 2022 [PMR⁺22] and the International Faust Conference (IFC) 2022 [ALGP22], where this work was demonstrated and published. It was later refined and presented at the 2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors (ASAP) [PMR⁺23a], as well as in an Inria research report [PMR⁺23b]. Pierre Cochard developed an API to simplify writing C++ programs for HLS optimizations and streamline the interface with the toolchain. This work was presented at the SMC-24 conference [CPMR24].

The effort invested in this compiler is substantial, and reflecting on the preliminary work conducted prior to this thesis [RMO⁺20], many technical obstacles have now been overcome. The maximum latency we achieved was 820 μ s, compared to just 11 μ s now. At that time, we did not use the CPU, and FAUST was not optimized, which resulted in significant limitations for the DSPs. Furthermore, automation was lacking; there were no command line interfaces or makefiles, and each step had to be performed manually.

The two main contributions brought by the modifications to the FAUST compiler are demonstrated through the experimental results presented in the next section.

Impact of Memory Usage

As mentioned in Chapter 4.2.4, the use of external memory access is often needed for audio programs because FPGA Block Rams do not offer sufficient space for storing delay lines. However, external memory accesses are slower than Block RAM accesses. Our modification of the FAUST compiler includes control over the use of external DDR, which was used exclusively by default.

Table 6.1 shows the impact on latency of a systematic external memory use compared to a systematic use of Block Rams on a set of simple FAUST programs taken from the *Amstramgrame* website [Graa].

These experiments show that using only Block RAM makes the design generated by FAUST not only faster (by a factor of up to 10, as demonstrated in the *comb* example in Table 6.1), but also lighter in LUT resource usage. However, using exclusively BRAM limits available memory space and may not be suitable for all programs.

Impact of Initialization

Another major modification to the FAUST compiler is the initialization of constants on the CPU rather than on the FPGA, including all associated processing of these constants. Fig. 6.4 shows the impact on the FAUST IP size of reporting constant initialization on the ARM CPU using *bellN.dsp* as a reference (see Listing 6.5). The *bellN.dsp* program simulates a bell sound using a biquad filter. This program allows for increased complexity (and realism of the generated sound) by varying

Prog.		LUT (% use)	1 sample comp. time (cycles)	1 sample comp. time (μ s)
Synth	BRAM	66 %	458	3.66 μ s
	DDR	73 %	1 477	11.81 μ s
Guit. demo	BRAM	77 %	1 922	15.58 μ s
	DDR	84 %	3 956	31.64 μ s
Lms	BRAM	64 %	846	7.22 μ s
	DDR	191 %	784	6.27 μ s
Tictac	BRAM	92 %	500	4.00 μ s
	DDR	95 %	1 564	12.51 μ s
Comb	BRAM	34 %	77	0.61 μ s
	DDR	47 %	738	5.90 μ s
Violin	BRAM	109 %	741	6.00 μ s
	DDR	110 %	1 955	15.64 μ s

Table 6.1: Memory storage vs. Block Ram storage for various FAUST programs. Red numbers indicate that the design does not fit on Zybo Z7-10 FPGA. Using external DDR often requires more LUTs, as many multiplexers are needed in these non-optimized implementations.

the number of biquad filters used. Fig. 6.4 instantiates N for integer values between 1 and 30 and compares resources usage (LUT and DSP Blocks) for scenarios where initialization is carried on the ARM processor or on the FPGA. It can be seen that no more than 3 resonant filters can be used if FPGA initialization is used, while up to 25 resonant filters can be used if initialization is carried out on the ARM.

These results show that the current version of the -os2 mode of the FAUST compiler provides adequate optimization of memory accesses for basic FPGA designs.

Lang: FAUST

```

1  import("stdfaust.lib");
2
3  t60 = 30;
4  pulse = button("gate") : ba.impulsify;
5
6  process = pulse : pm.frenchBellModel(N,0,t60,1,2.5);

```

Listing 6.5: The bellN.dsp program. N should be replaced by an integer value indicating the number of resonant filters used to provide the bell sound.

6.2 Ultra Low Latency Audio Path

The main motivation for this thesis, and the primary objective of using an FPGA for audio processing, was to achieve the lowest possible latency. In our one sample strategy, the audio IP latency is limited to one sample. Such a low latency cannot be reached on CPU based audio programs which usually have at least 8 samples latency to hide the memory access time. Therefore, an FPGA-based audio system can help achieve the lowest possible latency, leading us to further explore and optimize this capability. Thanks to the audio IP maintaining a constant latency of one sample, the two primary sources of latency that can be optimized are the audio codec itself and the serial audio

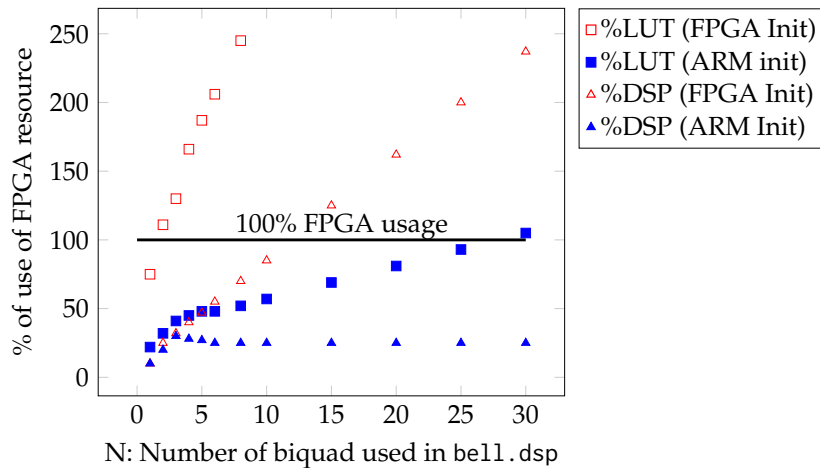


Figure 6.4: Percentage of Zybo FPGA LUTs and DSP Blocks usage for various sizes (i.e., number of biquads) of the `bell.dsp` FAUST program.

interface (I²S) connecting the audio codec to the audio IP.

6.2.1 I²S Transceiver

To ensure that the latency of the I²S serial audio interface is as low as possible, we use a hardware I²S transceiver IP. Prior works [ES21] use the AXI4-Lite interface but our custom VHDL IP provides us a better control over latency.

Regular I²S transceivers can handle stereo input/output audio interfaces. To achieve greater flexibility in terms of channel count, bit width, sample rate, and other parameters, we have designed our own custom transceiver IP in VHDL, supporting both TDM8 and standard I²S interfaces. Our implementation is based on an IP from the *Digikay* forum [Diga] which was extended to enable arbitrary sample bit depths and better control over sample rate.

As discussed in Chapter 4.3.2, for standard I²S with more than 2 channels, the interface and the associated hardware can be duplicated. An easy way to achieve this is by duplicating the VHDL IP in the block design. While this approach works, it is not efficient, as it generates redundant hardware, such as duplicated clocks that do not need to be managed separately for each transceiver since they are shared. Instead, it is more efficient to duplicate the relevant parts of the program within the VHDL code itself. We rewrote the IP so that a script can interpret the program content and duplicate the critical sections accordingly. An excerpt of the I²S VHDL code is shown in Listing 6.6, illustrating the port declaration of the IP, which should be adapted depending on the number of channels. Listing 6.7 shows the processed code by our script for 4 channels, which is then integrated as an IP into the block design.

Lang: VHDL

```

1 [...]
2 sd_#L_#R_rx : in  std_logic; -- serial data receive
3 sd_#L_#R_tx : out std_logic; -- serial data transmit
4 from_faust_#L : in std_logic_vector(d_width-1 downto 0); -- left ch data
5 from_faust_#R : in std_logic_vector(d_width-1 downto 0); -- right ch data
6 [...]

```

Listing 6.6: Excerpt of I²S transceiver VHDL code, before multichannel script processing.

Lang: C

```

1 sd_ch0_ch1_rx : in std_logic;
2 sd_ch2_ch3_rx : in std_logic;
3 sd_ch0_ch1_tx : out std_logic;
4 sd_ch2_ch3_tx : out std_logic;
5 from_faust_ch0 : in std_logic_vector(d_width-1 downto 0);
6 from_faust_ch2 : in std_logic_vector(d_width-1 downto 0);
7 from_faust_ch1 : in std_logic_vector(d_width-1 downto 0);
8 from_faust_ch3 : in std_logic_vector(d_width-1 downto 0);
9

```

Listing 6.7: Excerpt of I²S transceiver VHDL code, after multichannel script processing (autogenerated).

We can therefore produce as many I²S channels as needed, all sharing the same bit clock (BCLK) and Word Select signal.

The development of this IP was a long and complex process. The IP was almost entirely rewritten, as the base model presented several bugs. We spent significant time testing and debugging the hardware design to ensure flawless execution of the interface across all configurations. Testbenches were written to ensure the correct behavior of the IP, with strict adherence to the non-trivial I²S protocol and its timing requirements. All of this results in a fully functional, easy-to-integrate, versatile, and scalable I²S transceiver IP, which represents a significant contribution to this thesis.

Initially, the audio codec configuration through the I²C bus was managed by the IP in hardware. This meant that if a codec needed to be reconfigured (for example, to change the volume of its amplifier), the entire design, including *Vivado* implementation, had to be resynthesized. Additionally, the bugs encountered with the *SSM2603* codec (detailed in Section 6.2.4) and the highly complex configuration of *ADAU* codecs detailed in next section, led us to move this configuration to the ARM processor and manage it through software. With this approach, a change in the configuration only requires recompiling the ARM application. This allowed us to extensively test the *SSM2603* and ultimately resolve persistent issues.

6.2.2 Audio Codecs Configuration

As discussed in Chapter 4.3.3, audio codecs can vary significantly in their design and purpose. Some audio codecs are engineered with accessibility and user-friendliness as primary goals, while others are optimized for low-latency performance. Low-latency audio codecs, which are typically employed in more advanced applications, necessitate intricate configuration. Key elements such as the internal Digital Signal Processor, optional filters, buffer size, audio path, and sample rates are configurable and require meticulous management. When configured properly, these audio codecs can fully exploit their low-latency capabilities.

Different Rates. Depending on the audio codec, various sample rates can be configured in the chain. The I²S sample rate defines the sampling rate from the FPGA view, influencing the IP execution time. This rate is selectable on the audio codec, typically ranges from 8 kHz to 96 kHz, with some reaching 768 kHz. The DAC and ADC can each have distinct sample rates that do not need to match the I²S, sometimes operating at higher speeds than the serial interface. In the following paragraphs, the I²S clock rate (which is also the sample rate of our system) is noted as $f_s^{I^2S}$, and the ADC/DAC internal clock rate as Ck^{ADC} (here, we assume that the ADC and DAC operate at the same sample rate).

Audio Path. By default, the audio signal is processed through the embedded Digital Signal Processor of the audio codec. This processing can introduce significant delay, depending on the processor clock speed and the presence of filters or converters. For our application, the processor is unnecessary since we only need to route the signal from the ADC to the I²S interface, and from the I²S interface to the DAC. If the DSP can be avoided, it should be. If not, it should be used in bypass mode and operate at the highest possible clock speed to minimize latency.

Filters. Audio codecs typically include ADC high-pass and DAC de-emphasis digital filters. Disabling these filters, or at least reducing their order as much as possible, can help to minimize sample latency. On the other hand, it is important to evaluate the impact of disabling or reducing these filters on the output signal quality.

Depending on the audio codec, users may have control over these parameters. This is a key distinction between simpler audio codecs, such as the *SSM2603*, which has 14 configurable registers, and high-end ones like the *ADAU1787*, which offers more than 200 configurable registers.

6.2.3 External Audio Codec Latency Measurements

Three different audio codecs were used for our experiments. Only two low-latency codecs were tested, as very few codecs available on the market are optimized for latency at the time this thesis was written.

- The *Analog Devices ADAU1777*, which is optimized for low latency ($\max f_s^{I^2S} = 192 \text{ kHz}$ & $\max Ck^{ADC} = 768 \text{ kHz}$).
- The *Analog Devices ADAU1787*, which is the fastest audio codec we used and that provides the best performances in terms of latency ($\max f_s^{I^2S} = \max Ck^{ADC} = 768 \text{ kHz}$).
- The *Analog Devices SSM2603* (the built-in codec of the *Zybo* board), which is not optimized for low latency ($\max f_s^{I^2S} = \max Ck^{ADC} = 96 \text{ kHz}$).

The *Syfal* toolchain is also compatible and has been tested with two additional audio codecs: the *MAX98357A* and the *ADAU1761*. The *MAX98357A* is used on the frugal board detailed in Section 6.3.1, while the *ADAU1761* is the default onboard audio codec for the *Genesys* development board. These codecs have not been tested for latency as they are not designed for low-latency applications and are therefore considered irrelevant for this purpose.

Note that the *ADAU1787* works at voltage levels up to 1.8V, whereas the *Zybo* operates at 3.3v. It is therefore necessary to use a level shifter if we want to use them together. The level shifter has to be fast enough to handle the I²S clock (up to 24.576 MHz). We used a *LSF0204* [Texc] level shifter which does not induce any significant latency. When used with the *Genesys* board, 1.8V can be output on the GPIO, eliminating the need for a level shifter.

The ADC/DAC, due to the filters used by these converters, introduces at least a 1 sample delay, which is directly connected to the sampling rate: the higher the sampling rate, the lower the sample latency. For instance, a sampling rate of 48 kHz provides a latency of 20.833 μs whereas a sampling rate of 768 kHz introduces only 1.3 μs of sample latency. In practice, however, the codec introduces more than just one sample of delay. Most of the latency in sigma-delta audio codecs comes from the group delay added by the ADC/DAC. For the *ADAU1787*, the best announced group delay is 5 μs (analog input to analog output), obtained at $Ck^{ADC} = f_s^{I^2S} = 768 \text{ kHz}$. We validated this figure by measuring a 5.9 μs latency when bypassing the I²S, which approximately corresponds to a 4 samples delay.

The audio serial interface (I²S) also introduces a one-sample delay that depends on the sample rate. The DAC and ADC sample rates do not need to match the serial interface rate, so operating

them at higher frequencies minimizes sample latency. Codecs embed interpolators and decimators to achieve synchronization between different rates. However, not all rate configurations guarantee an acceptable SNR, particularly if $Ck^{ADC} \gg f_s^{I^2S}$. On the other hand, increasing the audio interface sampling rate $f_s^{I^2S}$ decreases the available time for the FAUST IP to perform its computation.

Testing Method

Our experimental setup for measuring latency is described in Fig. 6.5. A 440 Hz square wave signal is sent to the system, implementing three loopbacks at different levels: within the audio codec, on the I²S bus, and inside the FPGA. The latency between the analog input and output of the system is measured by looking at the time difference between zero crossings of the source and of the processed signal. We performed a large number of measurements of this kind. Their standard deviation is always less than one sample.

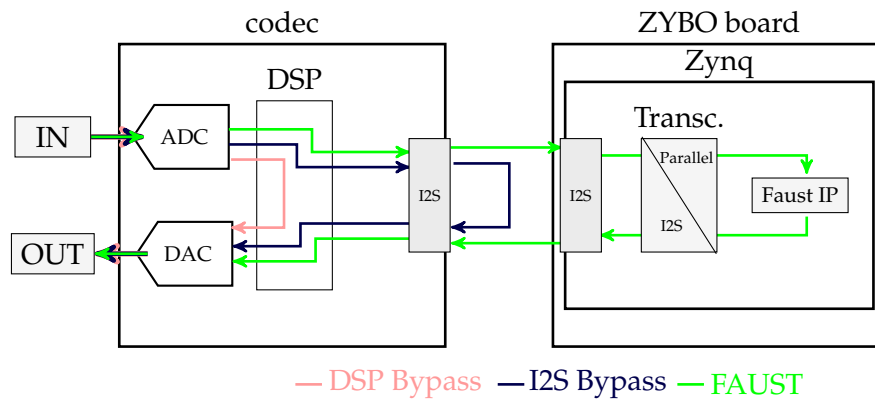


Figure 6.5: Latency measurements testbench used for the results presented in Table D.1. Latency was measured from analog input to analog output using square signals in three different configurations.

All these measurements are analog-to-analog, providing the total latency of the audio codec rather than separating the ADC and DAC latency. Wang’s PhD thesis [Wan18] aimed to measure the input and output paths separately. To achieve this, they probed the digital signal and correlated a clear binary pattern with changes in the analog signal, and measured the time difference. As noted, this approach is challenging because the test signal must be carefully chosen to ensure that binary pattern changes are clearly identifiable.

We choose to avoid relying on digital signals and instead use bypass modes at various levels. The internal Digital Signal Processor of advanced codec like the ADAUs can be configured to bypass the ADC into the DAC. This creates a digital loopback that exclusively highlights the latency of the ADC/DAC and the processor. This method is referred to as “DSP bypass” in the Fig. 6.5. The SSM2603 does not have a digital bypass mode because its bypass is purely analog, and its processor is not configurable.

Another method for measuring latency involves physically bypassing the I²S output to the I²S input by short-circuiting the serial input and output. This creates a digital bypass at the I²S level. Depending on the audio codec configuration (slave/master), clocks such as MCLK and BCLK are still needed, but this does not interfere with the bypass. This method, referred to as “I²S Bypass” in Fig. 6.5 allows for measuring the latency of the I²S interface only (by subtracting the previously measured ADC/DAC/processor latency).

Finally, with a complete system (I²S communication with the FPGA), we implemented a bypass using FAUST on the FPGA. This approach reveals the true latency of the entire system. Since all

latencies are measured using a one-sample approach, this ensures that, regardless of the complexity of the algorithm, the measured latency remains consistent with any DSP that employs a one-sample approach. This is referred to as “FAUST Bypass” in Fig. 6.5.

This “black box” approach does not allow us to precisely identify which part of the audio codec induces latency. However, this is not the goal here, as other studies (especially Wang’s PhD thesis [Wan18]) have already extensively covered the subject. The main goal of this section is to present the audio codecs’s capabilities within a complete system, disregarding architectural specificities. For the same reason, we do not discuss group delay or analog-to-analog latency sometimes provided in datasheets, as these do not represent the full latency of the system with serial communication.

Results

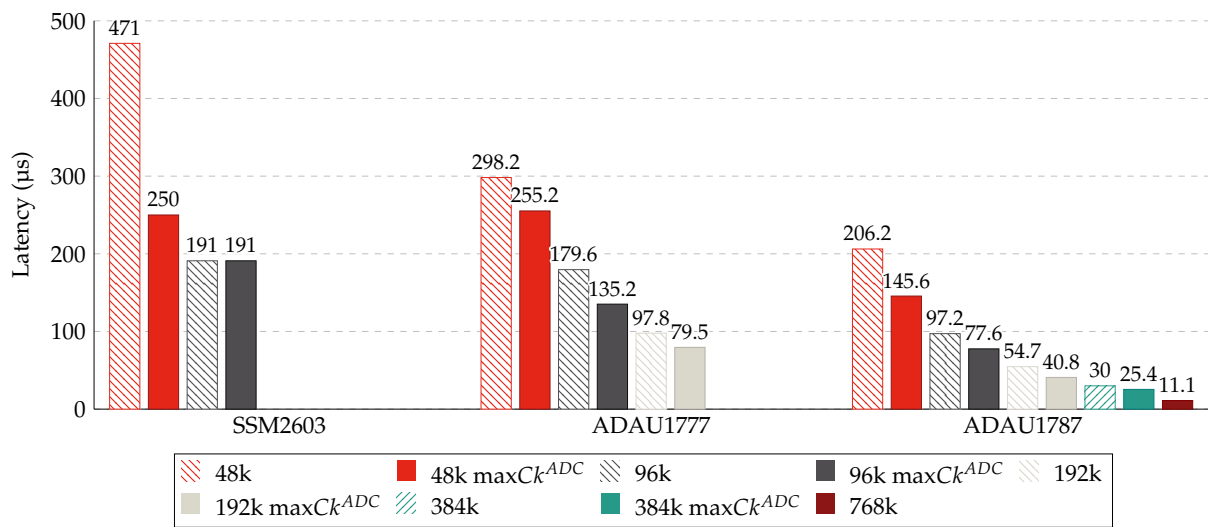


Figure 6.6: Latency report for different audio codecs for the “Faust Bypass” path, with a full analog-to-analog signal path including IP processing in a one-sample approach. Except for the case at 768 kHz, all measurements were made on 24 bits samples.

Fig.6.6 shows the latency measurements obtained on various audio codecs with different sampling rate configurations. The sample rate indicated in the legend corresponds to the I^2S sample rate (f_s^{I2S}), which is the sample rate for the audio DSP. For each sample rate of each codec, two values are provided: one for $Ck^{ADC} = f_s^{I2S}$ (hatched), ensuring a better SNR, and another for $Ck^{ADC} = \max Ck^{ADC}$ (solid color), which results in the lowest latency achievable at this rate (see $\max Ck^{ADC}$ in Section 6.2.3). Note that the ADAU1777 cannot use an Ck^{ADC} lower than 96 kHz, so this value is selected for the 48 kHz case.

As expected, the best latency (11 µs) is obtained with the fastest internal ADC/DAC with an I^2S rate of 768 kHz and using 16 bits samples.

It is interesting to note that for the same configuration (e.g., $Ck^{ADC}=96\text{ kHz}/f_s^{I2S}=96\text{ kHz}$), the ADAU1777 is not significantly faster than the SSM2603 (191 µs latency for the SSM2603 versus 179.6 µs for the ADAU1777). However, it outperforms the SSM2603 due to its ability to use a 768 kHz sampling rate. The low-latency ADAU1787 audio codec performs better under the same configuration: (97.2 µs latency for $Ck^{ADC}=96\text{ kHz}/f_s^{I2S}=96\text{ kHz}$ or 206.2 µs latency at $Ck^{ADC}=48\text{ kHz}/f_s^{I2S}=48\text{ kHz}$ versus 848 µs for the SSM2603). Its ability to achieve higher I^2S sampling rates results in exceptionally low latency in these configurations.

A comprehensive table with the three previously detailed measurements for various sampling

rates and ADC/DAC rates is provided in Appendix D Table D.1. This table displays the sample latency in number of samples, calculated based on the sampling rate. It provides valuable insights into the scheduling of each part of the system. By subtracting the I²S latency from the FAUST bypass latency, we can determine the latency of the entire FPGA. The results indicate that, in all cases, the whole SoC (transceiver+FAUST, back and forth) adds 2 samples of delay.

The FAUST IP always adds a 1 sample delay in one-sample mode. To our knowledge, none of the previous works could achieve such a low latency (i.e., 180 μ s was announced in [VDD⁺19] with a audio codec clock at 192 kHz). Our custom I²S transceiver also adds an additional sample of delay for the full conversion (input and output), demonstrating its efficiency. All these results were published at SMC 2022 [PMR⁺22].

A final consideration regarding the SNR can be addressed. All results presented here are based on the fastest possible configuration for the codec, meaning that all filters have been minimized to their simplest form (i.e., lowest filter order, etc.). Despite this, all configurations still include the essential filters, such as reconstruction filters, but they impose trade-offs in terms of noise performance and frequency response. While minimal interpolation and decimation filters help reduce latency, they are not without drawbacks and cannot guarantee the best SNR. These trade-offs are documented in the respective codec datasheets but are not further analyzed in this study.

6.2.4 Troubleshooting SSM2603: Audio Distortion Due to Faulty Initialization

The Zybo Z10 and Z20 are relatively common development boards for FPGA. They are often used in academic settings and, due to their accessibility, are among the most popular *Xilinx* development boards. Both boards are equipped with an SSM2603 audio codec. This codec contains a bug that causes its operation to be highly unpredictable. For months, we encountered an issue that appeared randomly: occasionally, the right output signal would be distorted as shown in Fig. 6.7, where the positive peak of the signal would wrap to the opposite side, and the VMID pin of the audio codec would measure 3.3 V instead of the expected 1.65 V. The left output signal was correct.

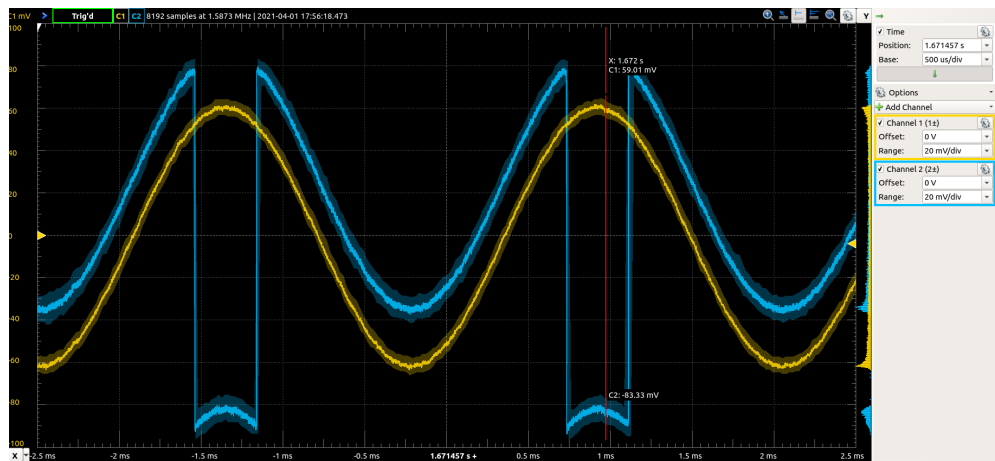


Figure 6.7: Oscilloscope visualization of the SSM2603 bug due to R06 initialization: Right output in blue, left output in yellow.

Although this might initially seem like a binary representation problem, our tests confirmed that the I²S transactions were correct, and the issue originated from the SSM2603. Surprisingly, this problem occurred across all our Zybo boards (both Z10 and Z20) and was present in all examples found online, whether using the I²S IP (with embedded SSM2603 I²C configuration) provided by *Digikey* or using *Xilinx*'s example code, which manages the I²C configuration through the ARM processor.

After identifying that the problem only occurred when the SSM2603 was configured for the first time after booting, and following numerous empirical tests, the issue was attributed to a faulty initialization of register R06 (power management). Register R06 controls the power-on settings for different blocks of the SSM2603 (DAC, ADC, crystal, clock, etc.). The datasheet specifies that the necessary power management bits in Register R06 should be enabled before any other configurations are made:

"Enable all of the necessary power management bits of Register R06 with the exception of the out bit (Bit D4)."

It turns out that the opposite approach should be taken: no modules (except the crystal and clock) should be powered up before the end of the initialization process, leaving the default value of the register (0b10011111: everything powered down except the OSC and CLKOUT bits). Despite being counterintuitive, this resolution, discovered two years ago, has prevented the problem from reoccurring. The problem was discussed on the technical support website of *Analog Devices* [Anah] and confirmed by another user, which demonstrates that it is indeed a common issue. It's a bit surprising that this issue has not been widely mentioned, despite the many example programs using the audio codec. This might suggest, as the state of the art indicated, that audio processing on FPGAs, particularly with the *Zybo* development board, is not very common.

The resolution of this bug was further complicated by the fact that the SSM2603 never returned valid signals on the I²C bus, regardless of what was attempted. Although we did not delve deeper into this particular issue, forum responses confirmed that this is also a common problem with this chip. *Analog Devices* did not respond directly on their technical support site or to the feedback we provided on the datasheet.

6.3 Multichannel Interfaces

In the fields of active acoustic control and audio spatialization, using a large number of speakers is often necessary to achieve optimal results. As discussed in Chapter 3.4.2, some applications and sound spatialization techniques require thousands of speakers [BdV93, Ahr12].

FPGAs are particularly well-suited for multichannel audio processing due to their high parallel processing capabilities. They can reduce system costs by enabling all audio channels to be managed by a single embedded system while providing sufficient resources to compute audio samples at a high sampling rate. However, to connect multiple audio inputs and outputs to an FPGA, it is necessary to perform analog-to-digital and digital-to-analog conversion with an audio codec, possibly amplify the signal, and wire each channel.

Currently, there is no easy and efficient way to interface more than a few channels to an FPGA. FPGA-based systems typically rely on three options:

- Using "classic" audio interfaces designed for CPUs: these interfaces, often employing standard protocols like USB, introduce significant latency and may necessitate the use of a CPU to handle audio data, which undermines the goal of maintaining a low-latency system.
- Using Ethernet-based audio sound cards: while Ethernet communication is fast, it still introduces latency and adds complexity to the design. Additionally, these solutions are expensive.
- Using specialized ADC/DAC modules (such as NI-9220 [NIa] ADC/NI-9264 [NIb] DAC 16-bit modules): these solutions can cost more than ten thousand euros, making them a less attractive option.

These constraints make the practical, real-time implementation of massive multichannel algorithms challenging. As a result, some papers present multichannel algorithms that are not tested on more than a few channels, despite being designed for massive multichannel systems [KKTS21], due to the lack of a physical interface.

This section proposes to facilitate the use of FPGAs in multichannel systems by developing two different audio expansion boards. The first one, referred to as the *Frugal Board*, facilitates the construction of systems with a high number of outputs at a minimal cost. It focuses on affordability and can be used with a low-cost FPGA-based board due to its Peripheral Module interface (PMOD) connectors available on entry-level *Digilent* hardware. The second board, named *Ultra Low Latency Board* (or *ULL*), aims to leverage the full potential of the FPGA by employing ultra-low-latency audio codecs capable of achieving sampling frequencies up to 768 kHz and latency as low as $11\mu\text{s}$. The board is designed to operate with *Xilinx's Ultrascale+* FPGAs, which are higher-end compared to those used with the *Frugal* board.

Each platform is open-source and available on GitHub [Emeb]. The work presented here has been published at the SMC-2024 conference [PMR24].

6.3.1 Frugal Board

We demonstrated the feasibility of designing a frugal multichannel platform by employing low-cost audio codec in our work published at the NIME 2023 conference [MBPR23]. We used an affordable *Zybo Z7-20 ADM/Xilinx* FPGA development board coupled with 32 *MAX98357A* codec to make a WFS system for less than 800 USD. The *MAX98357A* [Anae] audio codecs integrate digital-to-analog converters and built-in 3 watts amplifiers. Communication with these chips occurs through the I²S TDM protocol, enabling communication with multiple codec through a single wire and significantly reducing the required number of GPIOs.

However, connecting the 32 audio codecs necessitates substantial wiring, signal buffering and shielding, and non-trivial power management. In our prior work [MBPR23], we built the interface board that embeds the audio codecs using a dot matrix PCB and hand-soldered cables, a time-consuming and non-robust process that resulted in erratic system operation and hard to debug problems.

To improve reliability, a custom-designed PCB is presented in Fig. 6.8 aiming to enable individuals and institutions with limited resources to build a multichannel platform on FPGA. Two different manufacturing techniques are possible: either using existing commercial *Adafruit* breakout boards [Ada], allowing for hand-soldering the audio codecs and a complete DIY fabrication; or using specialized equipment to solder the chip directly to the board to enhance integration.

DIY Fabrication Approach

A reproducible board implies that it should be manufacturable using basic PCB fabrication techniques, without the need for advanced industrial machinery. This implies using sufficiently wide PCB track widths to allow for high tolerance during printing, sufficiently large drill holes for manual drilling, routing tracks only on the two outer layers, and soldering all components on one side to facilitate potential hand soldering.

A major challenge of this constraint is that the *MAX98357A* audio codec has a footprint that is too small for manual soldering and requires the use of specific equipment. Fortunately, *Adafruit* breakout boards can be used. They integrate the codec as well as a few other essential components soldered onto a small PCB. This PCB can then be hand-soldered onto the board to connect the audio codec to the FPGA (see the top of Fig. 6.9). All these constraints make it possible to assemble this board entirely by hand.

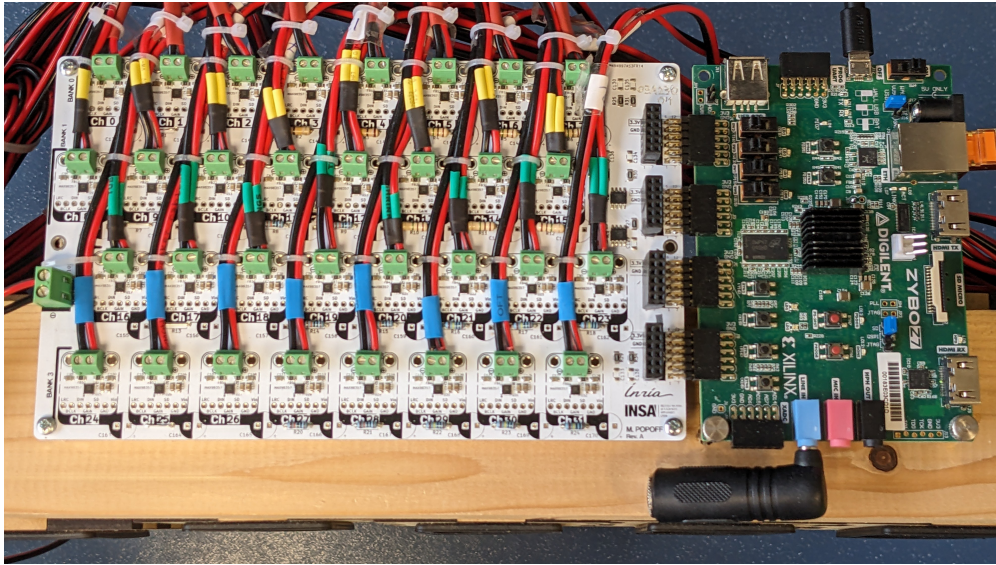


Figure 6.8: *Frugal Board (white) with 32 wired speakers, connected to a Zybo Z7-20.*

Direct Component Integration Approach

As manufacturing techniques become more accessible, it may be worthwhile to have the board manufactured and the components soldered by a specialized firm. In this case, using *Adafruit* breakout boards may be more complicated and expensive than purchasing the audio codec directly. Therefore, in addition to connectors for *Adafruit* breakout boards, the PCB also includes the footprint of the *MAX98357A* and peripheral components. This allows for the direct soldering of the audio codec and facilitates integration (see the bottom of Fig. 6.9).

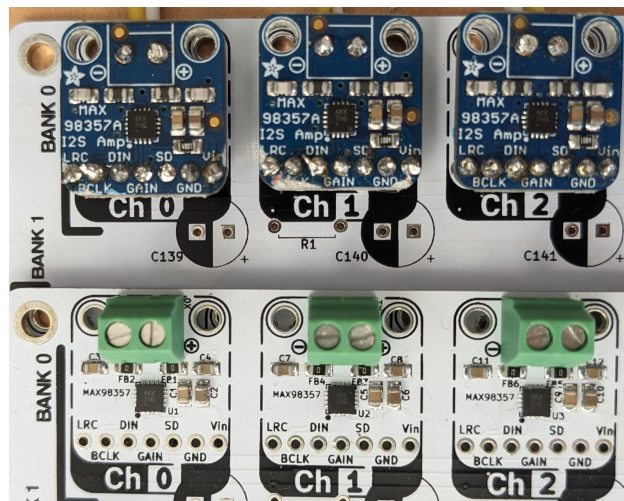


Figure 6.9: *Two Frugal Boards side by side with the two different integration: direct MAX98357A audio codec integration (bottom) or using Adafruit breakout boards (top).*

Stacking Capabilities and Clock Signals Integrity

Multiple boards can be stacked to increase the number of channels on a single FPGA. This is facilitated by the presence of stackable PMOD connectors and the possibility to select other GPIOs than the defaults for communication with the audio codecs (using jumpers). Using the TDM I²S protocol (as shown in Fig. 6.10), a 32 audio codecs board need 6 GPIOs: 4 for the data (one for 8

audio codecs) and 2 for the clocks (shared between all the amplifiers). Bit Clock and Word Select Clock can be shared between stacked boards, so each added boards only needs 4 GPIOs for data (TDM I2S wires in Fig. 6.10). Knowing that the board can access 32 GPIOs from the FPGA, up to 7 Frugal Boards can be connected to a Zybo Z7-20, allowing for 224 output channels.

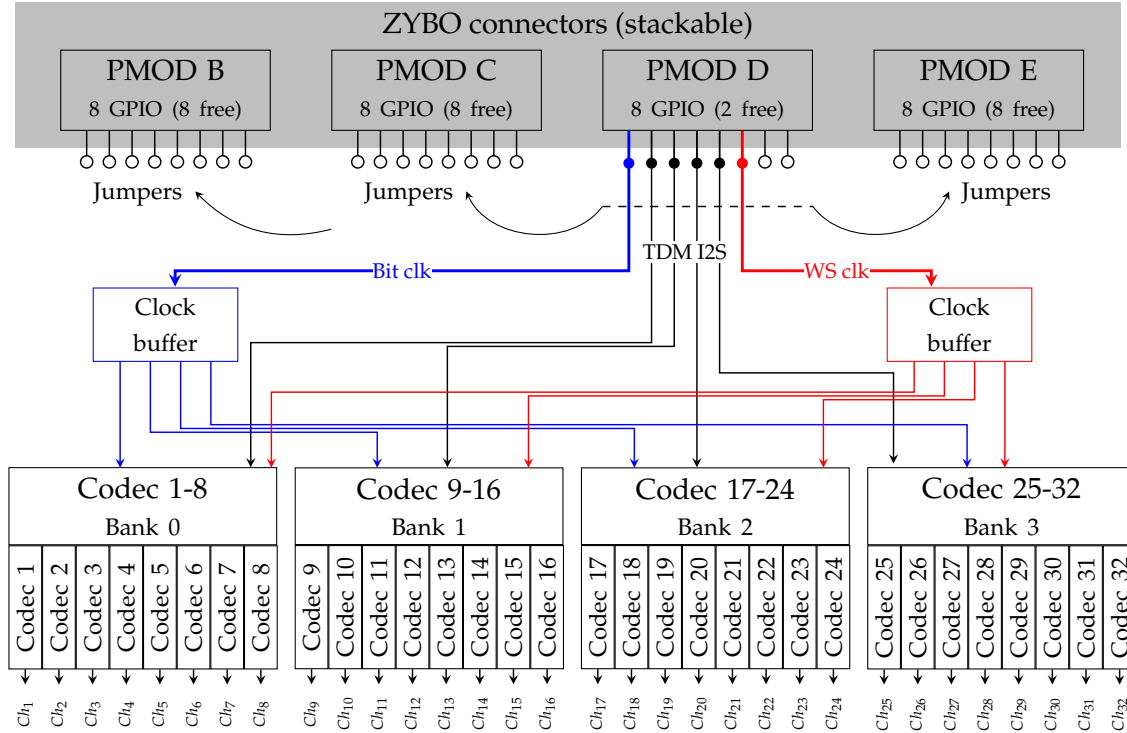


Figure 6.10: Frugal Board I²S Wiring

To ensure signal integrity and allow for the connection of multiple boards in parallel to a single FPGA, it is advisable to use a clock buffer on the clock lines. The low cost and low jitter 551S [Ren] clock buffer is used here, that takes the form of a hand-solderable surface-mount component accompanied by damping resistors.

6.3.2 Ultra Low Latency Board

The computational power and parallelization capabilities of FPGAs permit the implementation of powerful multichannel algorithms with unmatched low audio latency. While the previous section focused on low cost interfacing, the board described here enables ultra-low latency audio DSP. In Chapter 4.3, we highlighted the significant role of the audio codec and the associated antialiasing filters, as it is one of the contributors to latency.

To leverage the computational power of the FPGA, it is counterproductive to use inexpensive audio codecs that introduce excessive latency. Section 6.2 depicted the ADAU1787 [Anad] ultra-low latency audio codecs that can operate at up to 768 kHz and achieve a latency of 11 μ s from the analog input to the analog output using the Syfala compiler. Moreover, while the Frugal Board in Section 6.3.1 only featured audio codecs capable of synthesizing an audio signal (output channel), the ADAU1787 also hosts audio inputs.

For the preliminary experimental tests [PMR⁺22], we used evaluation boards [Anad] provided by Analog Devices. Building a 32-input/output audio interface can be done using 16 of these evaluation boards (each audio codec managing 2 channels). However, this approach poses several problems:

- A prohibitively high cost (~8 000 USD).
- A bulky system, due to the form factor of these boards featuring multiple connectors and peripherals unnecessary for our applications.
- Substantial wiring, leading to reliability issues.

This section presents a custom PCB in Fig. 6.11 with embedded *ADAU1787* audio codecs that addresses these three issues and facilitates the use of these codecs.

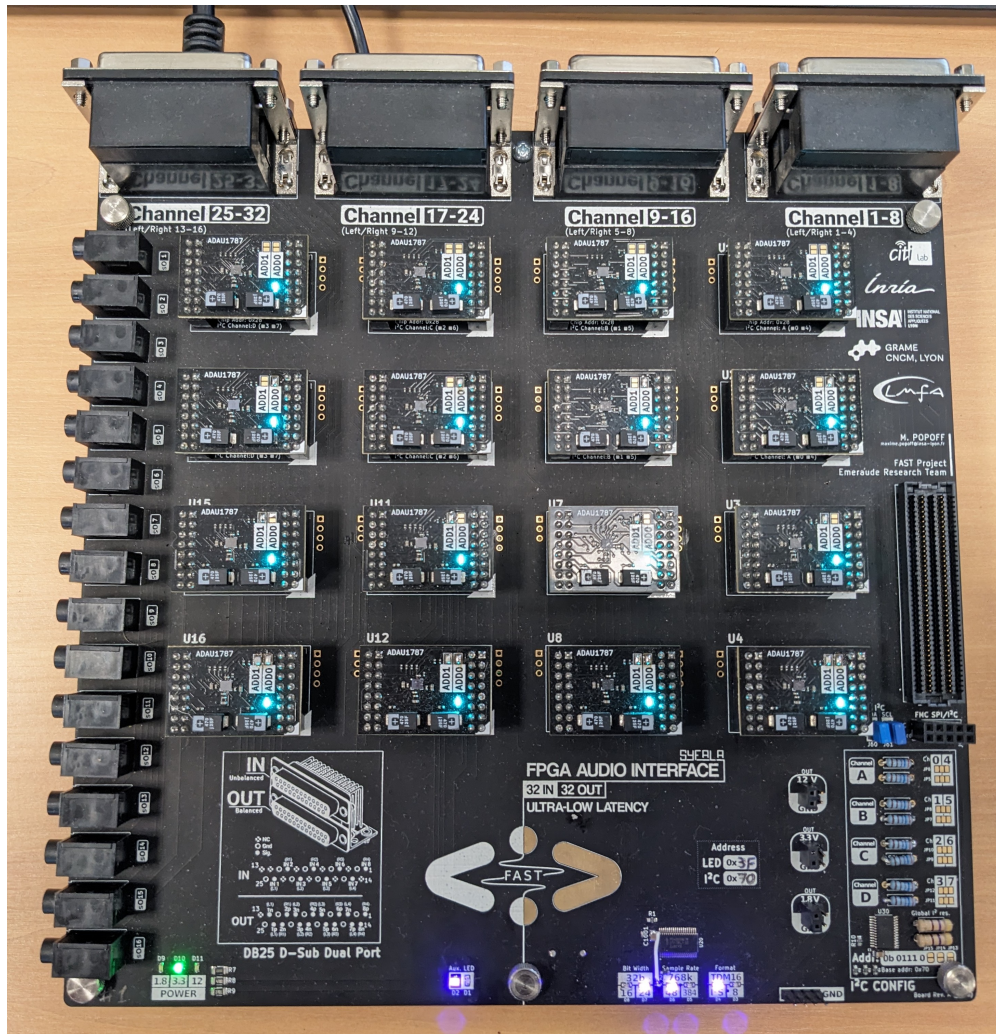


Figure 6.11: 32 Channels Ultra Low Latency FPGA interface stacked on the Genesys ZU-3EG. The motherboard embeds 16 breakout boards with *ADAU1787* audio codecs.

Modular Approach

The *ADAU1787* chips are designed for portable devices such as smartphones and digital cameras, thus they are extremely compact. Their small footprints require advanced equipment for integration, leading to costly PCB printing techniques. Designing a board with 16 codecs soldered onto it leads to high production costs and several disadvantages, including the inability to individually replace a malfunctioning codec and the impossibility of making later interface changes (e.g., audio port modifications, filter additions, etc.).

For these reasons, a two-PCB fabrication approach was chosen. The first PCB (breakout board) exclusively embeds the audio codec and its peripheral components (Fig. 6.12). It employs costly

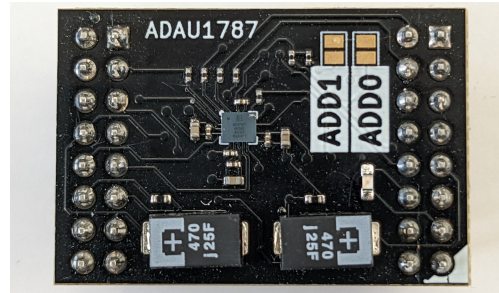


Figure 6.12: Custom ADAU1787 breakout board.

manufacturing techniques but remains minimalist to reduce costs per audio codec. The second circuit (motherboard) incorporates all the necessary connectors (audio and FPGA) and enables connection to 16 audio codec PCBs.

Audio Interface

The ADAU1787 embeds 2 single-ended input channels and 2 output channels, either differential or on an amplified and filtered headphone port. As shown in Fig. 6.13, on our board, the single-ended input and differential output are connected to DB25 connectors, with 8 channels per connector (4 input connectors and 4 output connectors for a total of 32 channels). These connectors are widely used in the field of audio and facilitate the connection to an amplifier or the use of a multicore cable with jack or XLR connectors. The headphone output channels are directly routed to a 3.5mm stereo jack port for direct use with headphones.

FPGA Interface

Communication with the FPGA is established through an FMC connector [Sam]. FMC is a standard that defines a compact electromechanical expansion interface for a daughter card to an FPGA baseboard. Most FPGA evaluation boards feature an FMC connector. Using this port ensures mechanical and electrical compatibility of the board with a large portion of commercially available FPGA boards. On the *Genesys ZU-3EG* [Digb], this port allows us to access up to 36 pairs of differential or 72 single-ended signals. Each audio codec requires 2 dedicated data wires (in and out), and 5 shared wires: 3 clocks (word select, bit clock, and master clock) and 2 wires for the I²C bus. Thus, connecting 16 audio codecs require 37 wires that can fit on the FMC connector with signals in single-ended mode. This port is configured in 1.8V voltage level to be directly compatible with the ADAU1787.

I2C Configuration

Unlike MAX98357A audio codecs in Section 6.3.1, which are very simple and do not have software configuration, the ADAU1787 audio codecs require configuration via the I²C protocol at each boot. In order to configure each codec independently, they must all have a unique I²C address. The audio codec's address is set by solderable jumpers located on the breakout board. However, the ADAU1787 can only accept four different addresses. The solution is either to use a new I²C bus for each batch of four codecs, which is costly in terms of GPIO, or to use an I²C multiplexer that can switch on multiple sub-buses from a single I²C bus. If chips have identical addresses but are on different sub-buses, they can be differentiated. With 16 audio codecs, each capable of taking four distinct addresses, a 1-to-4 multiplexer is required at a minimum. The widely used TCA9548 [Texd] 1-to-8 multiplexer was chosen.

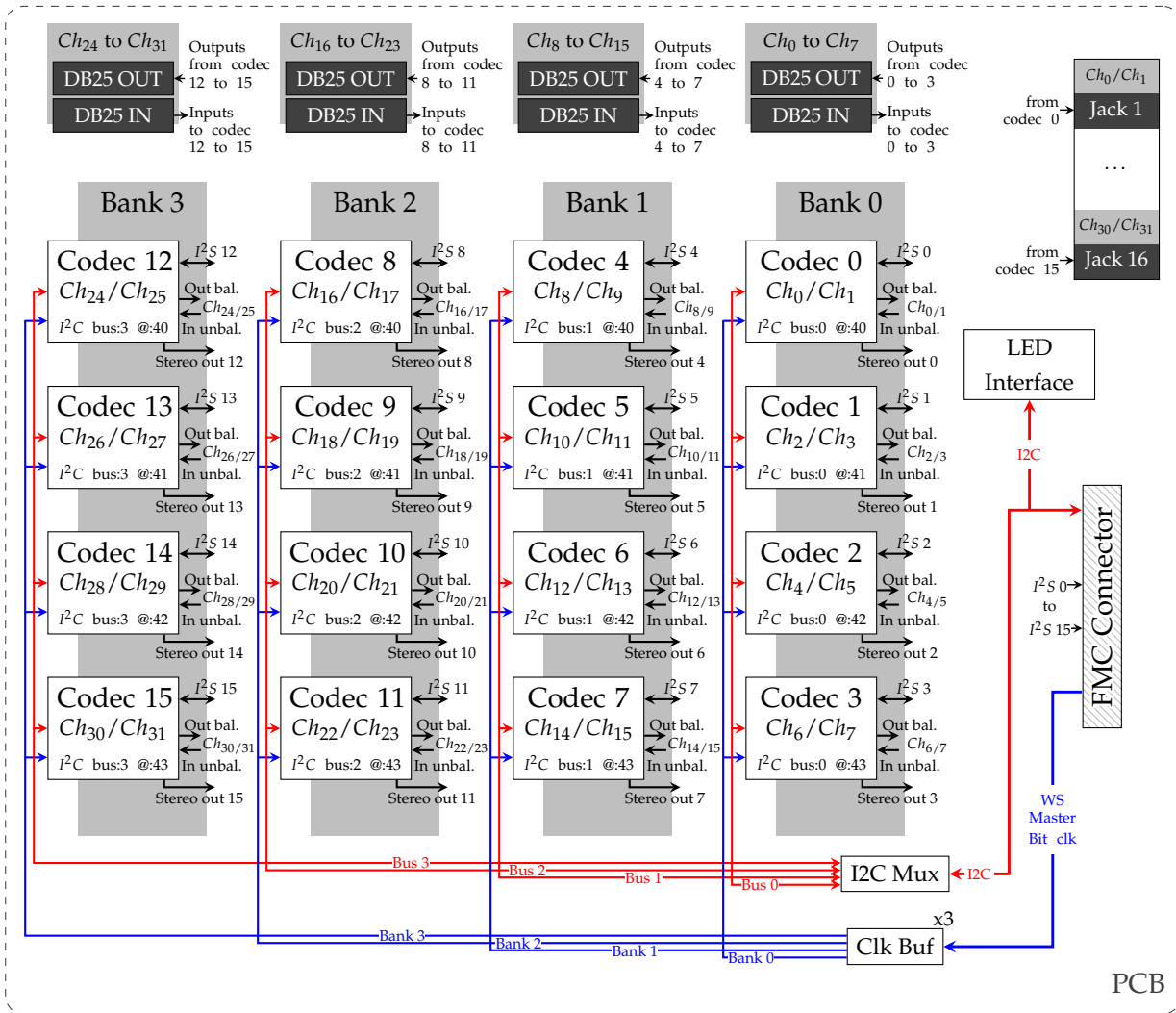


Figure 6.13: Ultra Low Latency Board I²C and I²S Wiring.

Stacking Capabilities

A single board embeds 16 codecs, providing 32 input/output channels. However, multiple boards can be parallelized on the same FMC port to increase the number of I/Os. Each audio codec communicates with the FPGA through its dedicated I²S data in and data out wires, resulting in 16 distinct I²S buses on the FMC port. By employing the I²S TDM protocol as done in Section 6.3.1 instead of the traditional I²S protocol, up to 16 audio codecs can communicate over the same data in and data out wires (using TDM16). Consequently, it becomes possible to parallelize up to 16 motherboards, each containing 16 audio codecs, resulting in a total of 256 codecs or 512 channels.

Design Considerations and Features

Given that clock signals (Word Select, Master, and Bit Clock) are shared from one GPIO to all audio codecs, it is wise to add a clock buffer to these wires to increase their strength and improve signal integrity. The buffer must have very low latency jitter, very low propagation delay and very low output skew (e.g., LMK1C1104 [Texb]) to ensure synchronicity between all audio codecs. The PCB has also been designed to potentially add damping resistors in series later to help eliminate reflection.

Particular attention has been paid to the routing of tracks on the PCB. Since the tracks are highly

parallel and can exceed 40cm, a spacing greater than $3 \times W$ (where W is the track width) is maintained in order to minimize crosstalk. The datasheet for the *ADAU1787* specifies that it is not necessary to differentiate the analog ground plane (AGND) from the digital ground plane (DGND). Therefore, they are not separated in the design. To facilitate debugging, each audio codec is paired with a status LED controlled by the PCA9956 [NXP] LED multiplexer.

All these requirements necessitated the use of a 4-layer PCB, with a minimum track width of 0.15mm and a minimum drill diameter of 0.4mm, making it accessible without highly advanced equipment. This board should be compatible with any *Ultrascale+* FPGA equipped with an FMC port and is designed to be easily stackable with the *Digilent Genesys ZU-3EG* development board by using the same dimensions.

6.3.3 Syfala Toolchain Compatibility

The use of these two interfaces is facilitated by their integration into the *Syfala* toolchain. The number of audio inputs/outputs is directly derived from the input program and the toolchain automatically generates a multichannel board-compatible FPGA design, including an I²S transceiver with the appropriate number of channels and configurable sampling rate. A dedicated option (`--shield`) added to the compilation chain allows us to specify the use of one of the two multichannel boards presented in this section (see Listing 6.8).

Lang: CMD

```
1 syfala examples/faust/wfs/wfs32.dsp -b GENESYS --shield motherboard
```

Listing 6.8: *Syfala* command-line for compiling a 32-channel FAUST program (*wfs32.dsp*) on a *Genesys* board, with the ULL board configured as the interface.

This integration allows for the seamless use of these interfaces from the user's perspective, who only needs to focus on the audio DSP algorithm implemented on the FPGA without needing to manage the hardware.

The affordability of the Frugal Board interface enables the development of large-scale multichannel systems with several hundred speakers at a relatively low cost. The second interface focuses on more professional applications for spatial audio, particularly those requiring active control. Applications for both boards are detailed in Section 6.4.

6.4 Experimental Results

This section presents the experimental results for the main applications tested. These applications vary and present different technical challenges, demonstrating various use cases of audio implementation on FPGAs as discussed in this thesis.

The results are provided for four different System on Chips, presented in Table 6.2. These include the *Zynq Z7010* and *Z7020*, which are equipped on the *Zybo Z10* and *Zybo Z20* widely accessible development boards. The *Genesys* board is equipped with an *UltraScale+ ZU3EG* SoC, which is more powerful than the *Zynq* SoCs. Despite this, the difference in resources between the *Zynq Z20* and the entry-level *UltraScale+ ZU3EG* is not substantial. While this does not significantly impact our applications, it is worth noting that the *Zynq* family features a dual-core ARM Cortex-A9 processor, whereas the *UltraScale+* family includes a dual-core ARM Cortex-A53 processor.

All three platforms are available in our laboratory, and all presented algorithms and results were tested under real conditions on these platforms, up to sound production.

The final SoC, equipped with an *UltraScale+ ZU19EG* and available at a relatively decent price from the manufacturer *Alinx*, is the most powerful SoC in the *UltraScale+* family at the time this thesis is written. Although we did not acquire this SoC, and thus did not test results in real conditions, it provides a good representation of the best computational capabilities available for purchase.

Board	SoC	LUT	FF	DSP	BRAM	Price
Z10	Zynq 7010	17 600	35k	80	270 KB	300€
Z20	Zynq 7020	53 200	106k	220	630 KB	450€
Genesys	UltraScale+ ZU3 EG	70 560	141k	360	1 MB	1 800€
Alinx Boards	UltraScale+ ZU19 EG	523k	1 045k	1 968	4.3 MB	3 600-7 200€

Table 6.2: Resources and Prices for Different SoCs on Available Development Boards.

In addition to having different amounts of resources, the *UltraScale+* and *Zynq* families exhibits different implementations with HLS tools. For the same use case with exactly the same program, the two families can yield very different results. For example, with a one-sample approach for an FIR filter with 8 000 coefficients, which will be presented in the next section, the *UltraScale* implementation uses 50% more DSP Blocks while utilizing 30% fewer FF.

6.4.1 FIR Filter

The FIR implementation is discussed in Chapter 5.4, which compares the HLS optimizations. This section focuses on the maximum FIR size that can be implemented on an FPGA using our methodology. The following rules are used to determine the maximum number of implementable coefficients:

- The design latency is considered acceptable if it is less than 2 603 cycles, which is the number of available cycles at 48 kHz for our 125 MHz FPGA clock. This is debatable because the sample rate can be decreased, and the FPGA clock can be increased. However, this value provides a reasonable baseline under fair conditions. Increasing the FPGA clock often leads to significant timing violations and is not a straightforward adjustment.
- Filter coefficient values are randomly generated to match the exact number required and do not represent real filters; therefore, the sound behavior was not tested in each case. However, the behavior of the program was validated using realistic coefficient tables.
- All results are confirmed to fit on the FPGA based on the full implementation report, not solely on the *Vitis* HLS synthesis estimation reports (except for the *UltraScale+ ZU19EG*).
- All programs (FAUST and direct C++) are compiled with the `-umo` option.

Each approach is compared against the FAUST-generated code, a C++ program without HLS optimizations, and any other relevant HLS optimizations.

One Sample

The one-sample strategy imposes very tight constraints on latency and offers limited optimization opportunities. The maximum number of implementable coefficients is presented in Fig. 6.14, for various FPGA platforms and different implementations discussed below. All coefficient numbers presented here were obtained with a sampling frequency of 48 kHz and guarantee this one-sample approach, meaning that the latency is one sample for the IP, or two samples from FPGA input to FPGA output. Using the low-latency *ADAU1787* audio codec, these results are guaranteed with a maximum latency from analog input to analog output of 206.2 μ s (see Fig. 6.6 p.145).

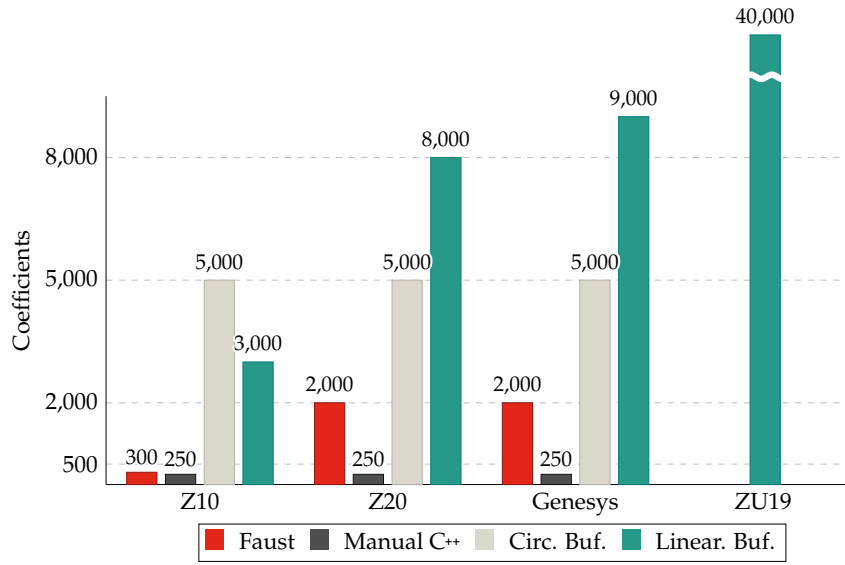


Figure 6.14: Maximum number of FIR filter coefficients implementable on different SoCs with different code implementations in a one-sample processing strategy.

Detailed resources usage and latency results are provided in Appendix D in Table D.2.

FAUST. FAUST automatically unrolls all loops, offering no control over parallelism in FAUST-generated code since HLS requires loops to manage it. Thus, the behavior of FAUST-generated programs is equivalent to fully unrolled loops in *Vitis*: consuming a lot of resources but with a very low latency. For this inherently sequential example, where latency is the limiting factor, unrolling all loops is a reasonable approach. However, we can expect a poorly optimized program, as fully unrolling loops leads to complete partitioning of the sample array, which is excessively costly. Additionally, the generated code includes other heavy operations that are not optimized for HLS, resulting in even worse resource usage and limiting the number of coefficients. For all FAUST programs, the limiting resource is the LUT. The FAUST program corresponding to the results presented is `fir.dsp` [The].

Manual C++. Without any pragmas and using a linear shifting buffer, experiments in Appendix B show that the FIR loop imposes an initiation interval of 8 clock cycles, and 1 for the linear buffer. In other words, it is not possible to fit any more coefficients than $2603/9$. On the other hand, the sequential nature makes the implementation very light on resources. This is much worse than FAUST for this example, as providing a fully rolled loop for a sequential program is very inefficient. For C++ programs with no pragmas or user optimizations, the limiting metric is the latency. The corresponding program is `fir_onesample.cpp` [The] (see Appendix F p.197) without any pragmas.

Circular buffer. With a circular buffer and carefully adapted pipeline and unroll pragmas, it is possible to support up to 5 000 coefficients. The limiting factor is latency, making this number independent of the board. The issue arises because the design requires a high unrolling factor (greater than 10) to accommodate more coefficients, which is poorly managed by circular buffers, as stated in Rule 5.5.2 p.126. With up to 5 000 coefficients, the II remains low (5 in this case), and the unrolling is efficient, with the 5 000 coefficients fitting with an unroll factor of 10. Beyond this point, the II starts to increase proportionally to the unroll factor, resulting in no further latency improvement. To increase the number of coefficients beyond 5 000, the sample array must be partitioned, which significantly increases the number of LUTs and demands hours of compi-

lation time, rendering it impractical. The file used to generate these results is `fir_onesample_circularbuffer.cpp` [The].

Linear buffer. Using a linear shifting buffer successfully reduces the latency at the cost of using resources that were previously underused. This optimization allows up to 8 000 coefficients to fit easily on a *Genesys* board. However, this approach is costly in terms of LUT and DSP Block resources, leading to rapid saturation on the *Z10*, where the circular buffer configuration provided better performance. This scenario corresponds to the program `fir_onesample.cpp` [The] (see Appendix F p.197) with the following optimizations. The methodology involved applying a pipeline forcing $II=1$ (following Rule 5.4.2 p.103) and then determining an unroll factor to approach the maximum allowable latency while minimizing resource usage. With the linear buffer configuration, the `linear_buffer` function has an II of 2, and the `compute_fir` function has an II of 5. By imposing a maximum trip count of 500 for the `linear_buffer` function and 250 for `compute_fir`, the expected latency is calculated as $250 \times 5 + 500 \times 2 = 2250$ cycles, approaching the maximum value of 2 600 cycles. Other unroll values can be chosen, but they should be perfect divisors of the number of coefficients to ensure complete partial unrolling (see Chapter 5.3.3). Here, we only tested coefficients that are multiples of one thousand for simplicity, hence the values of 250 and 500. The resulting optimizations are illustrated in Listing 6.15 and in the `fir_onesample_opti.cpp` [The] program.

The *UltraScale+ ZU19 EG* is showed only for the best approach implementation (linear buffer), as synthesizing the HLS for 40 000 coefficients already took over 12 hours. The DSP blocks are the most used resource in the resulting implementation, reaching 75% usage, leaving room for an even higher coefficient count. Generally speaking, aggressive optimization with one sample produces very long synthesis times. For example, FAUST struggles with more than 2 000 coefficients for a few hours, while HLS with more than a hundred unroll factors on tens of thousands of coefficients starts to take hours to synthesize.

Lang: C <pre> 1 void compute_fir(float* fTemp) { 2 compute_fir:for (int n = 0; n < NCOEFFS; ++n) { 3 #pragma HLS PIPELINE II=1 4 #pragma HLS unroll factor= NCOEFFS/250 5 *fTemp += samples[n]*coeffs[n]; 6 } 7 }</pre>	Lang: C <pre> 1 void linear_buffer(){ 2 linear_buffer:for (int j = NCOEFFS -1; j > 0; --j) { 3 #pragma HLS PIPELINE II=1 4 #pragma HLS unroll factor= NCOEFFS/500 5 samples[j] = samples[j-1]; 6 } 7 }</pre>
---	--

Figure 6.15: *Pragma used for better performances with linear buffer for `fir_onesample.cpp` [The] (provided in Appendix F).*

Multisample

The multisample approach is extensively covered in the case study Chapter 5.4.6, the results for the maximum number of implementable coefficients are presented in Fig. 6.16, and are discussed below.

FAUST. The same FAUST program used in the one-sample approach is tested here. The multi-sample mode is activated by adding `--multisample <N>` to the command line. In the multi-sample approach, FAUST simply encapsulates the processing function in a loop that iterates over the number of buffered samples. As a result, the handling of samples remains fully sequential, leading to

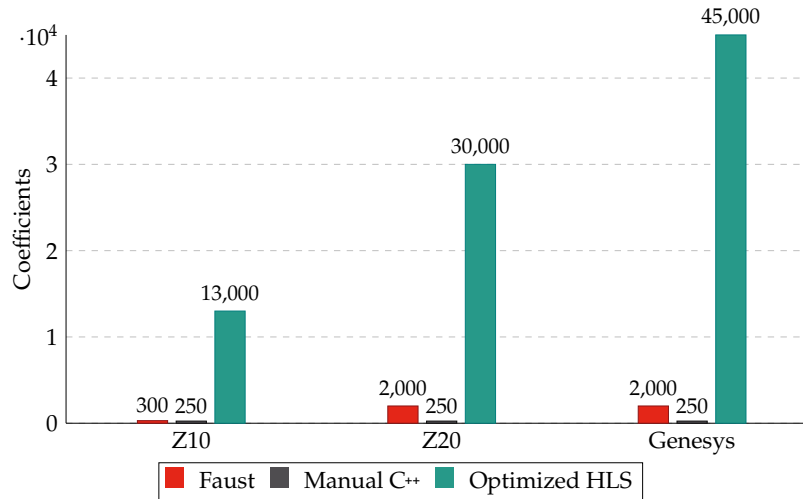


Figure 6.16: Maximum number of FIR filter coefficients implementable on different SoCs with different code implementations in a multi-sample processing strategy.

the same resource usage and a total IP latency equal to the number of buffered samples multiplied by the processing time of a single sample. This results in the same per-sample latency as in the one-sample approach, yielding identical outcomes.

Manual C++. This scenario corresponds to a multi-sample program without any pragmas or loop optimizations. It corresponds to the file `fir_multisample_base.cpp` and is presented in the multi-sample case study in Chapter 5.4.6 (Listing 5.11 p.118 and results labeled “No Opti” in Fig. 5.37 p.121). Without pragmas, the samples are processed sequentially, and buffering does not improve performance, leading to the same poor results as in the single-sample approach.

Optimized HLS. The HLS optimization corresponds to the loop interchange techniques discussed in Chapter 5.4.3 and applied in the multi-sample case study. This case uses the program `fir_multisample.cpp` [The], provided in Appendix F p.198. The methodology is as follows: First, we apply an explicit pipeline to both the `linear_buffer` loop and the `compute_fir` outer loop to ensure consistent optimization in the HLS process, independent of the number of coefficients or buffered samples. While enforcing an Π of 1 (as per Rule 5.4.2 p.103) may not necessarily yield the best latency-to-resource usage ratio, our experiments have demonstrated that HLS optimizes more effectively with this constraint. Then, the objective is to maximize resource utilization by first adjusting the number of buffered samples. Subsequently, the number of coefficients is increased (which in turn increases the latency) until the IP latency reaches $N_b \text{ buffered samples} \times \text{max available cycles}$. The goal is to achieve this with the maximum available cycles being as close as possible, but not exceeding the cycles available for processing one sample (i.e., 2 600 cycles at 48 kHz). Detailed resources usage and latency results for this scenario are provided in Appendix D in Table D.3. Results for the other two scenarios are the same as for the one-sample approach.

6.4.2 Frugal WFS System

Audio on FPGA opens the way for managing large-scale multichannel systems with a single chip. The ease of use provided by our *Syfala* toolchain facilitates the implementation of audio DSP for many channels quickly and with minimal resources. This specificity enabled the Emeraude Team to build a 32-speaker Wave Field Synthesis (WFS) system, as presented at the NIME-23 conference [MBPR23]. For less than 800 USD (including speakers), the system presented in this paper

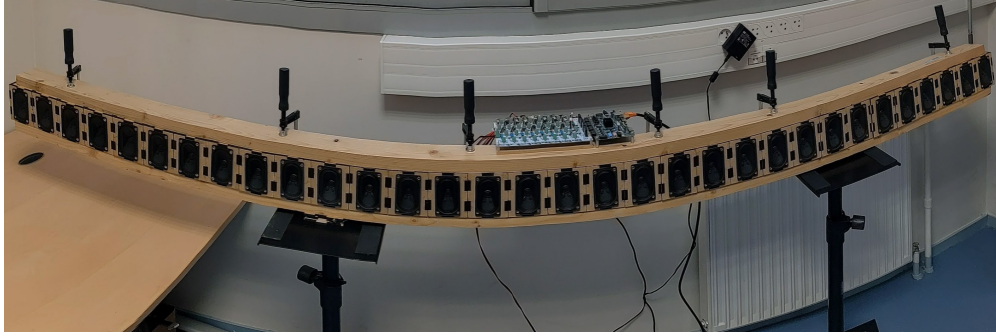


Figure 6.17: *Prototype WFS system using the Frugal Board.*

provides a 32 channels WFS system programmable in FAUST that can handle multiple sources in parallel. It is open source and easily reproducible, making advanced spatial audio techniques more accessible. If the target application in the previously cited paper is WFS, it can also be used to implement any other spatial audio technique thanks to FAUST which already provides libraries for ambisonics, VBAP, etc.

Using the open-source *Frugal Board* detailed in Section 6.3.1, this project was rebuilt with a cleaner interface to provide a new level of ease of use and accessibility for the audio community (see Fig. 6.17). The WFS algorithm that we used for our system is fairly basic and standard, but we obtained very convincing results. Accessibility comes at the expense of a lower sound quality and a higher latency compared to expensive audio codecs. Yet, this board has demonstrated sufficient sound quality to achieve effective rendering of the audio spatialization algorithms that we have tested. The MAX98357A can work at a maximum sampling frequency of 96 kHz and introduces a latency of approximately 500 μ s at 48 kHz. Since each audio codec is identical and managed by a single FPGA, all speakers are subject to the same latency and thus perfectly synchronized and suitable for audio spatialization applications. The built-in 3W amplifier and associated passive speakers demonstrated sufficient sound quality to achieve effective rendering of the audio spatialization algorithms that we tested.

This project was presented at the Tech Fest in Grenoble [Teca] and will be displayed as a permanent exhibition at the Inria Research Center.

6.4.3 Active Noise Control: LMS and FxLMS

We collaborated with École Centrale de Lyon (France) to test different Active Noise Control (ANC) algorithms implementation on FPGA.

Application to Noise Reduction in Headphones

First, we studied feedback ANC in headphones using a digital filter implemented on an FPGA, within the context of a pedagogical practice bench. We implemented a biquadratic filter using the FAUST language and compiled it on an FPGA platform with our toolchain.

Typically, the digital approach offers greater flexibility in configuring the feedback ANC filter but introduces additional latency that can impact ANC effectiveness. Our FPGA-based ultra-low latency system seeks to minimize this latency to the point where it becomes negligible. The results supported this, demonstrating similar or even improved low-frequency attenuation for some configurations of the digital biquadratic filter compared to the analog filter.

The programming workflow using the FAUST language significantly simplifies the implementation of digital filters on FPGA. This approach has proven effective in enhancing flexibility in parameterization for educational purposes. This project is a compelling use case for our system: the

team from Centrale Lyon had no prior knowledge of hardware design and limited experience in C programming. However, they were able to use FAUST to implement their algorithm effectively.

Experimental approaches and results are published in the IFC-22 [ALGP22] conference.

The promising results encouraged us to continue the experiments and explore the implementation of additional Active Noise Control algorithms.

ANC Over Extended Area

One of the promising applications of a low-latency multichannel platform is Active Noise Control over an extended area, such as a controlled zone in a room, as opposed to ANC for headphones. This application requires the use of multiple sources to generate the anti-noise field and multiple microphones to control the resulting field. The classical approach for this is to use the Multi-Point (MP) Filtered-x LMS (FxLMS) algorithm.

The objective was to implement this algorithm in the École Centrale de Lyon's listening room, which is equipped with 22 speakers and a rigid spherical microphone array of 4 microphones. This setup enables the study of large-scale algorithms (see Fig. 6.18).



Figure 6.18: *Experimental setup inside the École Centrale listening room.*

LMS. The FxLMS algorithm uses one or several adaptive filters for noise reduction applications. However, the algorithm requires an estimation of the acoustic paths between the control sources and the control microphones. The identification phase can be achieved by means of a LMS algorithm. Details and methodology regarding the implementation and optimization of LMS and FxLMS filters are provided in Appendix C. Only the results will be discussed here.

Two different LMS applications are presented:

- A single-channel LMS filter, used to ensure that the principles of the LMS filter are correctly implemented with our approach (see Fig. 6.19).
- A multichannel LMS filter, used for the FxLMS filter to identify paths. This follows the same principle as the single-channel version but introduces nested loops, allowing for loop interchanging optimization possibilities, as discussed in Chapter 5.4.3 (see Fig. 6.20).

In the first case, three results are presented: the FAUST implementation, the manually written C++ version, and the HLS optimized version using our methodology. All these experiments are for 512-coefficient filters that use a one-sample processing strategy and were implemented on a *Zybo Z20*. The results provided are from the *Vitis* HLS synthesis estimation report.

As in the case of the FIR filter, these results demonstrate that the FAUST version is currently not efficient, as it fully parallelizes everything, resulting in very low processing latency but overly high resource usage. The manually written C++ yields suboptimal results, as resources are saved at the expense of latency, limiting the size of the implemented filter in this one-sample approach. Finally, our optimized approach yields gains of around ten times.

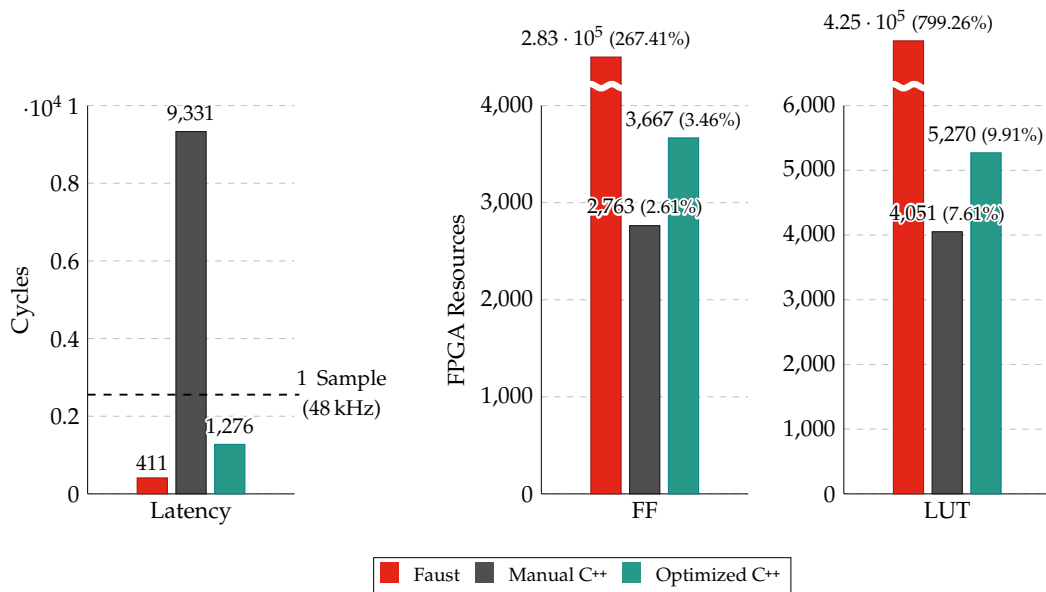


Figure 6.19: Resource usage and latency for a single-channel LMS filter algorithm with 512 coefficients (percentages for *Zybo Z20*).

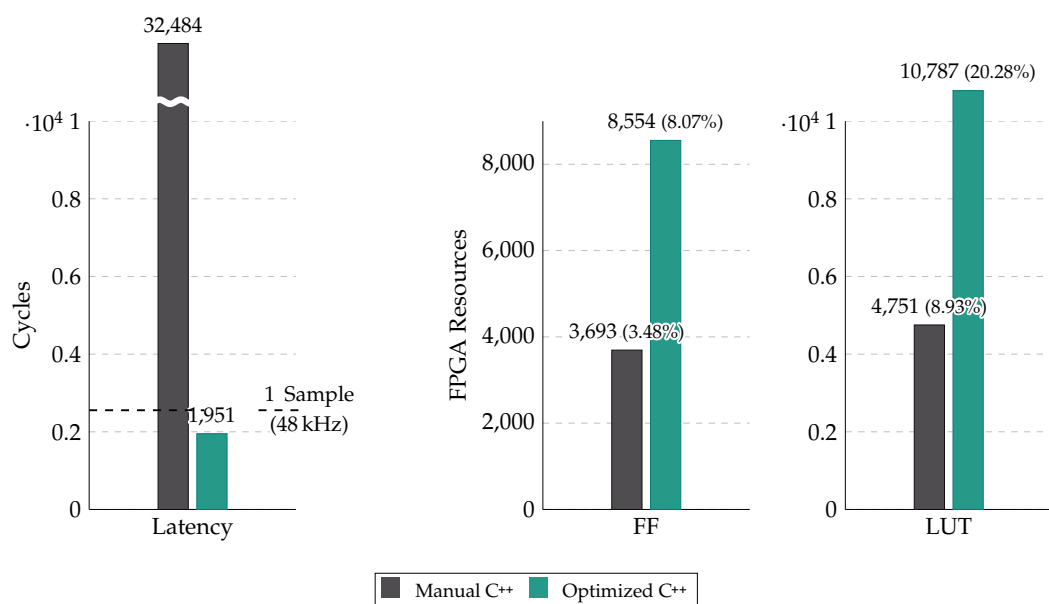


Figure 6.20: Resource usage and latency for a 6 channels LMS filter algorithm with 512 coefficients (percentages for *Zybo Z20*).

FxLMS. The Filtered-x LMS (FxLMS) algorithm is an adaptive filtering technique used for active noise control, where microphones capture the resulting sound field and speakers generate the anti-noise signals to cancel the unwanted noise emitted by a source. The algorithm iteratively updates its filter coefficients to minimize the error between the desired clean signal and the actual output.

The algorithm is extensively studied and optimized in Appendix C. Our HLS optimizations allowed us to implement a 14-speaker, 4-microphone FxLMS algorithm. Although we did not have time to test it in real conditions, *Vitis C* simulation results were compared with Python simulations and showed accurate results.

École Centrale de Lyon used a very-high-cost CPU-based (*Intel Xeon*) *Speedgoat* system [Spe] for some experiments, where they implemented a 14x14 FxLMS algorithm at 2.5 kHz sampling rate. With our optimizations, we can nearly implement a 14x14 FxLMS on the *Genesys ZU3-EG* board at 24 kHz, but the BRAM becomes saturated (see Table 6.3). Using LUT RAM is too costly, and so far, we have not been able to fit the design on the FPGA. However, future work and memory access optimizations should help address this issue. Coupled with the Ultra-Low Latency board, this setup could deliver performance comparable to the *Speedgoat* system for this application, but at a fraction of the cost.

Board	Latency (cyc.)	Latency (μ s)	BRAM	DSP	FF	LUT
GENESYS	5 031	41	593 (130%)	251 (69%)	97 779 (69%)	58 085 (82%)

Table 6.3: Resource usage estimation for a 14-microphone, 14-speaker FxLMS algorithm (percentages for *Genesys ZU3-EG*).

We have tested the FxLMS DSPs in real conditions with only 4 speakers and 4 microphones due to the resource limitations of the FPGA at that time. However, our recent optimization efforts should enable us to conduct the test using all speakers and microphones in the room. The LMS experiment has now been installed in a pedagogical practice bench at Centrale Lyon, enabling them to update their current setup from an outdated *Simulink* system, older operating systems, and a rigid FPGA-based interface.

All this work, including the case study in the appendix, has been largely conducted with the help of Loïc Alexandre, a PhD student from Centrale Lyon. A part of the work presented here as been published at the IFC-22 conference [ALGP22] and the European Acoustics Association Forum Acusticum 2023 [ALG24].

6.4.4 Modal Synthesis

Modal synthesis programs have been explored, with our implementation using an IIR filter. Unlike FIR filters, where the output depends solely on current and past inputs (no feedback), IIR filters rely on both past inputs and outputs (uses feedback), introducing different loop and memory dependencies. The program (`modal.cpp` [The]) allows for tuning the number of modes in modal synthesis, providing control over the system's complexity. The initial experimental results were primarily led by Pierre Cochard, the team engineer. For this preliminary work, the performance pragma was used, yielding good results with minimal user optimization. By fine-tuning the unroll and pipeline pragmas, I was able to slightly enhance the implementation further.

The implementation results are provided in Table 6.4, detailing the resource usage based on the final implementation report (including both synthesis and placement/routing phases). The main difficulty is that synthesis reports are often very pessimistic compared to implementation reports. This means that performing a full implementation, which can take hours, is necessary to obtain

accurate reports of resource usage. All experiments have been conducted with *Vitis* HLS 2024.1, but similar results were observed with *Vitis* 2022.2.

Board	Nb of modes	Buffer size	Latency/ Samples (cyc.)	DSP Tot Latency (cyc.)	BRAM	DSP	FF	LUT
Z10	12 000	24	2 518 (98%)	60 448 (0.492 ms)	81 (67%)	55 (68%)	18 011 (51%)	10 999 (62%)
Z20	30 000	64	2 360 (92%)	151 040 (1.229 ms)	145 (52%)	143 (65%)	42 896 (40%)	26 309 (50%)
GENESYS	45 000	80	2 264 (88%)	181 156 (1.474 ms)	197 (45%)	320 (88%)	63 364 (44%)	49 494 (70%)

Table 6.4: Modal synthesis latency and resource utilization of optimized HLS implementation, for three different FPGA SoCs using a multisample approach.

6.4.5 Comparative Insights: Evaluating Our Approach Against Existing Works

For this section, we only compare our results to the FAUST approach and the default C++ implementation. However, we do not compare them to existing HDL implementations. As explained in the introduction of the previous chapter, the reason for this is that the work presented here does not seek to provide the most optimized implementation for a specific application. This thesis aims to offer general guidelines that optimize any audio program implemented with High-Level Synthesis tools. Comparing the work of a compiler and an “automated” methodology to manually tweaked implementations does not make much sense, as the manual implementations will be thoroughly analyzed to be optimized as much as possible. The general methodology may not yield results as good as those obtained through manual implementation but provides a more straightforward way of achieving them.

One of the more advanced works in terms of computational load for low latency is that of Belloch et al. [BBS⁺14], which fits an IIR filter of approximately 300 000 equivalent order (1 256 filters composed of 128 second-order sections), on a GPU, with a latency of 0.72 ms. Even though the comparison with our experiments is not straightforward, the modal synthesis results indicate worse outcomes by a factor of approximately 10.

Now, a few points need to be put in perspective. First, Belloch’s work does not seem to take into account the latency of the audio card, thus not accounting for the whole analog-to-analog system and making it comparable only with our IP latency. Next, for our implementation, a single IIR filter of order 30 000 is not exactly the same as 1 000 filters of order 300, as loop pipelining and memory dependencies can greatly affect the outcome. For example, in the case of an FIR filter in a one-sample approach, if a *Zybo* Z20 can handle a maximum of 8 000 coefficients for a single filter, we can easily implement 300 filters with 200 coefficients each. Moreover, these results are achieved on a relatively small FPGA. On the other hand, manually writing HDL, particularly using fixed-point arithmetic instead of floating-point, does not guarantee better results. Finally, the work of Belloch is a manual implementation, which is precisely what we do not want to replicate. Our implementation follows the general guidelines provided, but we did not analyze the scheduling of the program in detail, nor did we adapt the program with specific hardware optimizations.

Conclusions and Perspectives

FPGAs are highly suitable programmable platforms for real-time audio processing. The increasing demand for performance, low latency, and higher channel counts in real-time audio processing makes them one of the most adapted solutions for cutting-edge audio systems, especially with audio spatialization techniques. However, their programming complexity has limited their widespread use in the audio industry and research.

Compiling real-time audio DSP on FPGA is inherently complex due to the nature of hardware design, but this thesis demonstrates that a general methodology can automate efficient implementations when restricted to the audio domain. Leveraging HLS tools, this work shows that high-level specifications of audio programs, even when expressed with DSLs, can be adapted for FPGA implementation, overcoming the challenges of hardware programming. The sequential nature of audio programs makes them difficult to optimize, but the narrow focus of audio applications allows for more predictable optimization strategies and better adaptation to FPGA constraints.

This work also exhibits that audio compilation on FPGA extends beyond DSP hardware implementation to include the entire system such as analog converters, control interfaces, and other necessary peripherals for external integration. An audio compiler should adapt the entire system to meet the specific needs of the audio program, such as adjusting the number of audio channels, configuring the sampling rate and bit width, generating a user interface, and more.

The main contribution of this thesis is the development of *Syfala*, a FAUST-to-FPGA audio compiler created with the Emeraude Team. *Syfala* introduces several key features for real-time audio DSP compilation on FPGA:

FAUST Automatic FPGA Compilation. *Syfala* allows for easy compilation of FAUST programs on FPGA. It automatically generates the necessary system components, such as I²S transceivers for communication with audio codecs, software and hardware controllers, and even embedded *Linux* tailored to the DSP's needs. Without any user intervention and in about 15-30 minutes, it generates, from a single FAUST DSP file, the FPGA bitstream and CPU executable ready to be flashed.

C++ Optimization. *Syfala* enables users to input C++ files directly into HLS, allowing fine-tuned optimizations like pipelining and unrolling that automatic compilers cannot infer. The guidelines provided in this thesis help accelerate program optimization based on general audio program architectures, reducing the time and knowledge usually required. By adhering to these guidelines, we achieved a tenfold increase across various applications.

User-Friendly Tools. *Syfal*a offers a range of tools to simplify the compilation and synthesis process. It provides clear reports, a robust command-line interface, verification tools, and a well-documented API for optimizing C++ code. *Syfal*a is a comprehensive audio compiler that is already being used by people outside the development team.

At the beginning of this thesis, we aimed to develop an efficient audio DSL to FPGA compiler for SoC implementation. In the process, we dedicated significant effort to optimizing the FAUST compiler and analyzing the generated code to ensure it was suited for optimized HLS synthesis. We modified the FAUST compiler to align with the FPGA design rules we established, which allowed us to increase the complexity of algorithms by almost tenfold. Seeing that the primary implementation control factor in HLS, the loops, was entirely dismissed during the FAUST compilation process, we redirected our efforts toward direct C++ optimization. This shift provides key insights for future modifications to the FAUST compiler, including the managing of loops in the generated code for better optimization.

One of our primary goals was to automate HLS optimizations for audio DSP. Given that HLS is a complex process requiring specialized hardware design expertise, we aimed to create a generic methodology by focusing specifically on audio programs. However, the task proved more intricate than anticipated. While we succeeded in developing guidelines and strategies to narrow down optimization paths for audio programmers, this thesis also revealed inconsistencies within the *Vitis* HLS tool. We demonstrated how a minor operator change could lead to substantial underoptimization throughout a program.

Manual FPGA implementations for real-time audio processing are rare, making direct comparisons of computational performance of our work difficult. However, we showed that with just a few manual C++ optimizations, we can compete with high-end CPU systems, such as the *Speedgoat*, which is promising for automating such optimizations. In terms of latency, our work surpasses existing results by a factor of 10, achieving latencies in the range of just a few dozen microseconds for full sample processing. The audio programs we successfully implemented, such as FIR filters with thousands of coefficients at such low latency, pushes beyond the capabilities of existing works.

In addition to the compiler, this work resulted in the design, construction, and testing of two multichannel interface PCB boards. These boards address a significant challenge: interfacing FPGAs with hundreds of speakers and microphones while guaranteeing ultra-low latency, an area where no platform currently provides an adequate solution. As a result, many research projects remain theoretical when it comes to testing massive multichannel algorithms. The Ultra-Low Latency board and the Frugal board serve as scalable interfaces for FPGA audio channels, each prioritizing different aspects: the Ultra-Low Latency board focuses on performance, while the Frugal board emphasizes cost efficiency. They offer a complete system with our compiler for a plug-and-play solution to deploy DSP on FPGA without additional tools.

This thesis was funded by the Agence Nationale de la Recherche (ANR – the French National Research Agency) as part of the FAST project. The goal of FAST is to design an FPGA-based platform for multichannel ultra-low-latency audio Digital Signal Processing programmable at a high-level with FAUST. It is fair to say that this objective has been successfully achieved, as we have created a functional platform.

*Syfal*a is still a relatively young project (around 5 years), but it has garnered interest from both academia and industry. It has received several external contributions and sparked significant attention from various groups:

- A *Syfal*a workshop was held at CCRMA at Stanford, with notable attendees such as Professor Julius O. Smith, Chris Chafe, and Fernando Lopez-Lezcano.

- A *Syfala* workshop was organized during the PAW (Programmable Audio Workshop) conference in Lyon, with a dozen participants at the most recent one.
- Artists, particularly those in live coding, are interested in using our toolchain.
- A researcher from the University of Chicago has recreated the controller and frugal boards to build a fully functional system, benefiting from their open-source design.
- A Swiss industrial company is exploring our platform for active noise control.
- Sveriges Radio (the public Swedish radio) has contacted us about using our system as an audio mixer in their studios.
- *Syfala* is currently used at Maynooth University [MSL⁺23].

Futur Work

Significant progress has been made since the preliminary work [RMO⁺20] conducted prior to this thesis, where the CPU was not utilized for computations, achieving a maximum latency of 820 μ s and lacking automation, with every compilation step needing to be done by hand. However, there is still work to be done.

A few specific parts of the development remain unfinished, and I aim to complete them in the coming months. This includes implementing the IP overlapping discussed in Chapter 5.4.5 and further testing our multichannel FxLMS system in the listening room at Ecole Centrale de Lyon.

On a broader scale, *Syfala* has several future development paths, led by other team members:

- First, we will try to transition from floating-point to fixed-point arithmetic, which has been a long-standing research topic within the team.
- *FloPoCo*, developed by Florent de Dinechin, member of the Emeraude Team, is a framework designed for optimized floating-point operators on FPGAs. Integrating it into the *Syfala* toolchain would enable us to continue using floating-point arithmetic while further optimizing the design for performance and resource efficiency.
- To reduce reliance on proprietary software and gain better control over the generated IP, we aim to move away from using HLS tools by developing a direct FAUST-to-VHDL compiler, leveraging LLVM. However, this is a significant task, particularly regarding the scheduling aspect.
- The *Syfala* platform, combined with the multichannel board, serves as a robust foundation for Real-Time Virtual Acoustics, Spatial Audio, and Virtual Soundscape Rendering. Future projects aim to improve virtual soundscape rendering systems and technologies to make them more accessible, efficient, and deployable, with two key applications: soundscape archaeology and live music performance.
- More generally, *Syfala* and its platform should continue to be used to explore spatial audio and auralization system designs.
- Collaborating with the person who created the language, we are working to adapt FAUST for HLS optimization by modifying the compiler to generate loops in the C++ code it produces.
- A study on Audio over Ethernet was conducted for FPGA implementation, leading to the development of a protocol for transmitting Ethernet audio streams over a LAN to an FPGA acting as an audio DSP platform [CWM⁺24]. Given the contemporary importance of real-time audio transmission over Ethernet, further work should be pursued in this direction.

Finally, I am in the process of creating a start-up company based on the Ultra-Low Latency board I developed. This will focus on improving and industrializing this ultra-low-latency, multi-channel FPGA-based platform. This is expected to allow me to finalize this promising work and further advance the development of this thesis.

A

APPENDIX A

Impact of Unsafe Math Optimization

This appendix detail the `-umo` configuration command that crucial for floating-point operations.

Let's modify the `basic_unroll.cpp` [The] program to handle floating-point operations instead of `ap_int`. The updated program is presented in Listing A.1.

Lang: C

```

1  float syfala (float A[N])
2  {
3      float out_accum = 0;
4      ap_int<16> x;
5      for (x = 0; x < N; x++) {
6          #pragma HLS unroll
7          out_accum += A[x];
8      }
9      return out_accum;
10 }
```

Listing A.1: Modification of the `basic_unroll.cpp` variable types.

By fully unrolling the loop, we would expect a fully parallelized implementation. An excerpt of the scheduler is shown in Fig. A.1. The implementation uses N distinct adders, but the schedule indicates that they are used sequentially. For example, in the scheduler, the adder line 8 is used after the adder line 4. The resource usage and latency is showed in table A.1

Latency (cyc.)	BRAM	DSP	FF	LUT
801	0	2	1 162	4 151

Table A.1: Resource usage for `basic_unroll.cpp` with `HLS unroll` pragma and no configuration commands.

This occurs because, for floating-point operations, expression balancing is off by default but can be enabled using the `config_compile -unsafe_math_optimizations` (or `-umo`) command. Enabling this option significantly decreases the latency, though it results in smalls changes in resource usage (see Table A.2).

In the schedule viewer, the multipliers are now used in parallel. In practice, the operations are still not perfectly parallelized: each adder starts with a one-cycle shift. This is due to a memory bottleneck caused by the dual-port RAM used by default, as discussed in Chapter 5.3.4. This

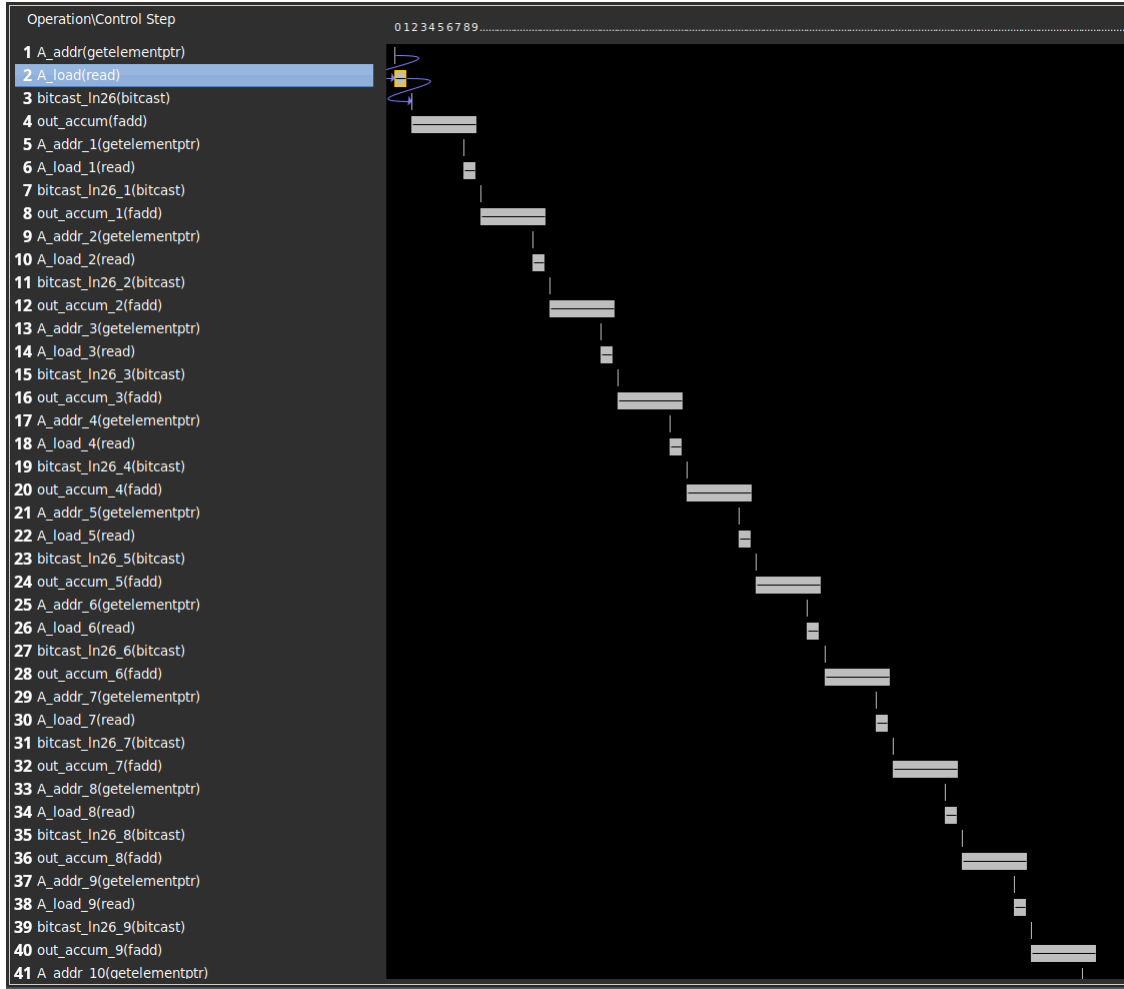


Figure A.1: Schedule viewer for `basic_unroll.cpp` with HLS `unroll` pragma and no configuration commands.

Latency (cyc.)	BRAM	DSP	FF	LUT
98	0	6	2 587	3 067

Table A.2: Resource usage for `basic_unroll.cpp` with HLS `unroll` pragma and `-umo` configuration commands.

bottleneck limits the reads to being parallelized two by two only. For example, in the scheduler, loads lines 19 and 21 are parallelized, followed by the next pair, lines 26 and 28.

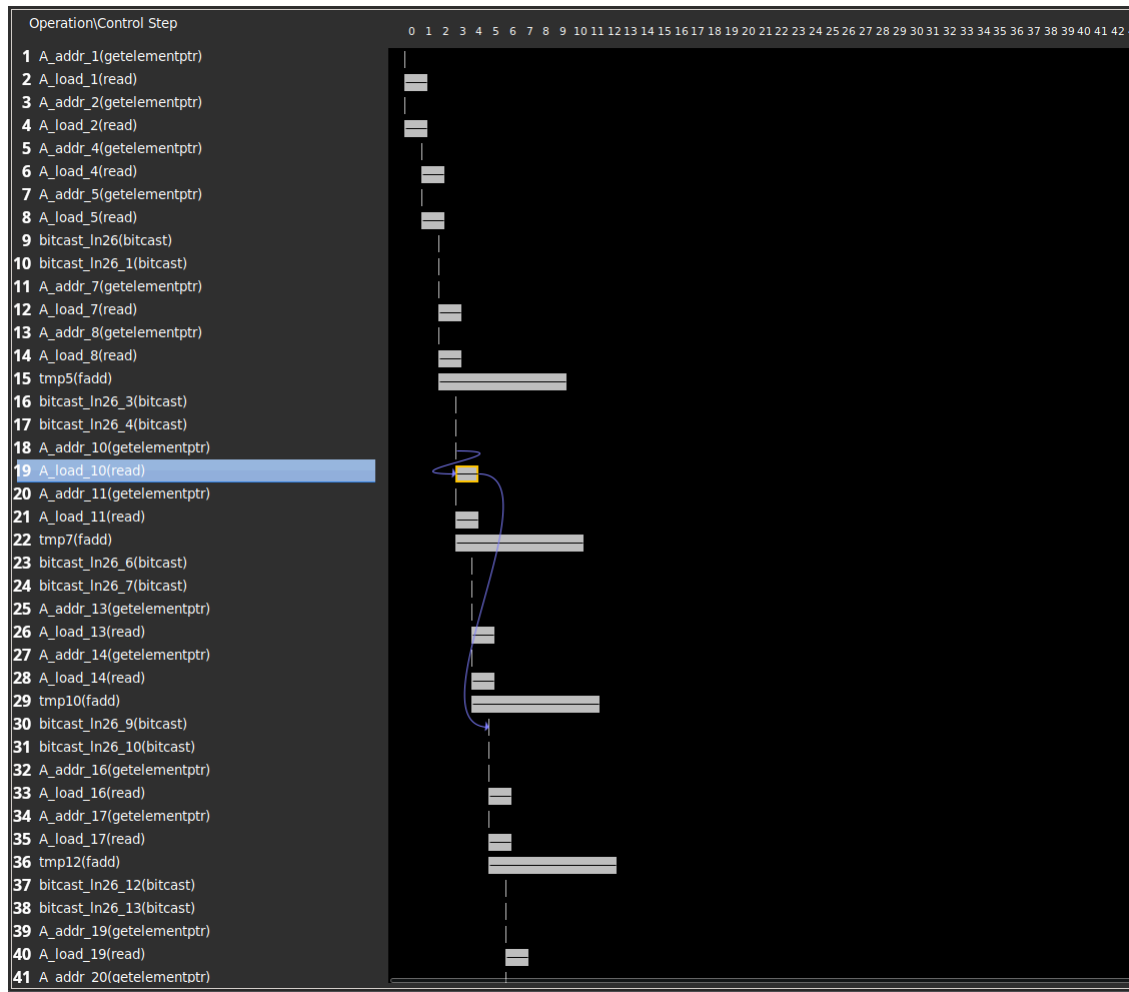


Figure A.2: Schedule viewer for `basic_unroll.cpp` with HLS `unroll pragma` and `-umo` configuration commands.

One-Sample Case Study

This appendix presents a workflow to try to optimise a sequential example, the FIR filter, with a one sample approach. If “optimization” is somewhat vague, in our case, the primary goal is to reduce latency as much as possible without consuming too many resources. A comparison of resource usage and latency resulting from different optimizations is provided at the end of the section, enabling an evaluation of the most efficient implementation. This example relies on the program `fir_onesample.cpp` [The], provided in Appendix F.

Default Behavior

All the following examples define `N_COEFFS` to be 115 and assume the following command configuration:

- `-umo` (unsafe math optimization) is enabled (see Appendix A).
- `syn.compile.pipeline_loops` threshold is set below the number of coefficients to automatically pipeline loops.

Initially, `fir_onesample.cpp` is synthesized with *Vitis* HLS without any pragmas. The resulting resource usage is provided in Table B.1.

Function	Latency	Iteration Lat.	II	Trip Count	BRAM	DSP	FF	LUT
<code>computemydsp</code>	1 064	-	1 065	-	3	7	1 270	2 246
<code>linear_buffer</code>	115	3	1	114	0	0	58	62
<code>compute_fir</code>	926	15	8	115	1	2	475	584

Table B.1: Resource usage for `fir_onesample.cpp` with no pragmas (automatic pipeline of the loops), 115 coefficients.

Analyzing Bottleneck

The II of 1 for the `linear_buffer` function indicates that achieving a better implementation is challenging (see Rule 5.4.6 p.111). The pipeline provides an execution time of approximately `N_COEFFS` cycles while reusing a single hardware instance. Unrolling this loop could potentially enhance latency, particularly if `N_COEFFS` is large, but it also requires partitioning, which can be very resource-intensive. On the other hand, the `compute_fir` function, due to its sequential nature, results in a high II. Optimization efforts should focus on this loop to improve performance.

Parallelization (Unroll)

By applying the `unroll` directive to the `compute_fir` loop with a varying factor, the parallelization ratio varies, revealing the most efficient implementation. The results are shown in Fig. B.1.

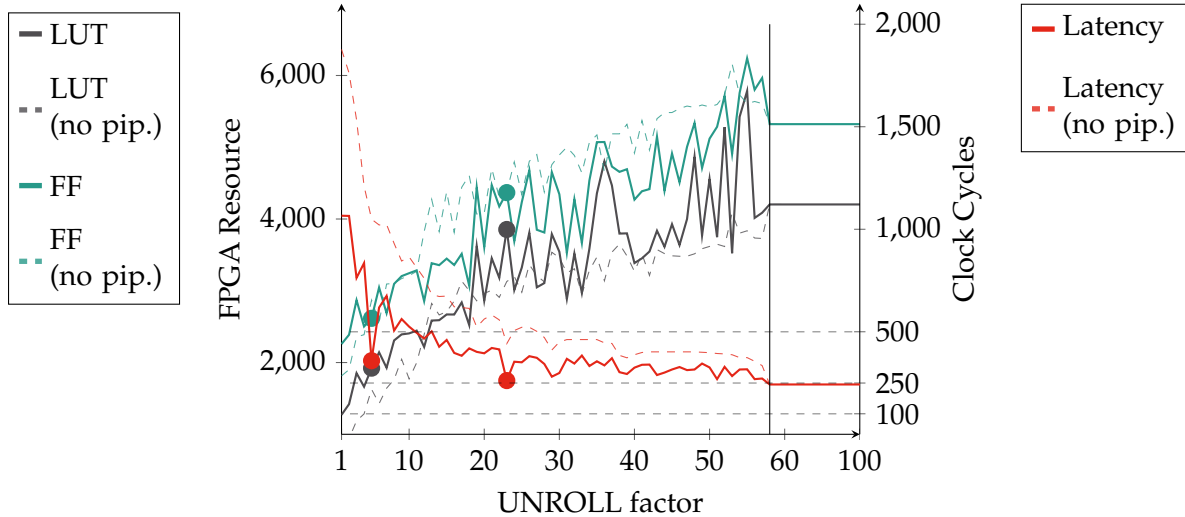


Figure B.1: Latency and resource usage for `fir_onesample.cpp` with varying `pragma` unroll factors (automatic `pragma` pipeline in all loops), 115 coefficients. Dashed plot are for no pipeline applied. Dots highlight optimal spots in the optimization process.

Please note that after applying an unroll factor of 57, the loop is fully unrolled, which causes the pipeline `pragma` to be ignored (and thus, the 2 plots to be merged).

A trend of the trade-off depicted in Fig. 5.13 p.88 emerges. However, as the FIR example is a little more complex than the trivial example from Chapter 5.3.1, the evolution is much less linear than what is depicted in Fig. 5.21 p.96.

The “noisy” plot reveals some sub-optimal or super-optimal designs that, for instance, might achieve the same latency with varying resource usage. As in the trivial example in Chapter 5.3.1, this is due to perfect partial unrolling and memory/loop dependencies, which are better optimized with some unroll factor. However, unlike the simpler example used in that section, the relative complexity of the FIR filter introduces more intricate layers of dependencies. These dependencies are harder to predict without a deep analysis of the implemented design. For example, here we only adjusted the parallelization of the `compute_fir` function without modifying others, which can lead to various bottlenecks.

Nevertheless, observations made in Chapter 5.3.2 regarding the unroll factor are still valid. The unroll factor is highly efficient for designs that are perfectly divisible by it. In this case, factor value of 5 and 23 (for 115 coefficients) are particularly noteworthy. These values are emphasized on the graph. The dashed line, which represents execution with pipelining disabled, yields results consistent with Chapter 5.3.3: pipelined implementations are always more optimized.

Minimizing the Initiation Interval

Explicitly declaring a pipeline in the `compute_fir` function, whether or not a `II=1` is specified, results in suboptimal performance, except for the case when the unroll factor is 1, which corresponds to no unrolling (see Fig. B.2).

The reason for that can be seen in Tables B.2 and B.3 which show the difference in implementation between an unroll factor of 10 (arbitrary value) with no `pragma` versus a `pragma pipeline II=1`

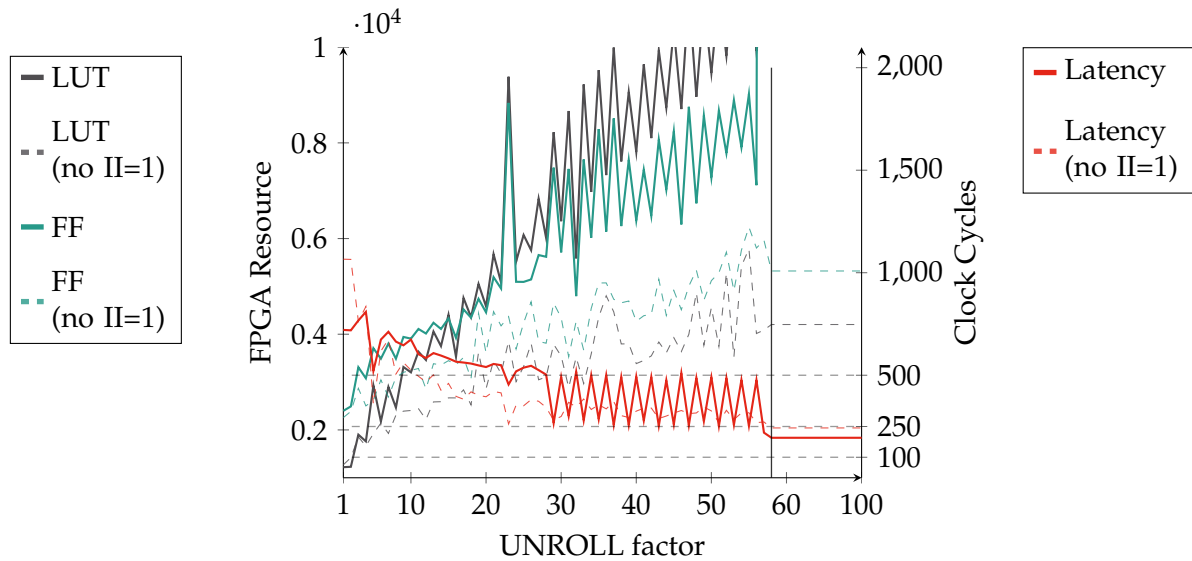


Figure B.2: Latency and resource usage for `fir_onesample.cpp` with varying pragma unroll factors and pragma pipeline `II=1` applied to `compute_fir`, 115 coefficients. Dashed plot are for no `II` specified (corresponding to Fig. B.1). Note that the legend scale has significantly changed compared to the previous graph.

applied to in the `compute_fir` function.

Function	Latency	Iteration Latency	II	Trip Count
<code>computemydsp</code>	523	-	523	-
<code>linear_buffer</code>	115	3	1	114
<code>compute_fir</code>	385	43	32	11

Table B.2: Latency report for `fir_onesample.cpp` for an unroll factor of 10 with no pragma pipeline applied (inferred pipeline in all loops), 115 coefficients.

Function	Latency	Iteration Latency	II	Trip Count
<code>computemydsp</code>	672	-	672	-
<code>linear_buffer</code>	350	12	3	114
<code>compute_fir</code>	299	35	25	11

Table B.3: Latency report for `fir_onesample.cpp` for an unroll factor of 10 with a pragma pipeline `II=1` applied in the `compute_fir` function, 115 coefficients. Green values indicate improvements, while red values indicate worsening.

The `compute_fir` function indeed performs faster with a lower `II`, though the HLS tool could not achieve an `II` of 1 clock cycle. However, this improvement comes at the cost of increasing the `II` and latency of the `linear_buffer` function, thereby raising the overall latency. The underlying issue is that to effectively pipeline the `compute_fir` function while following it with the `linear_buffer` function, the HLS tool introduced a modulo operator which adds significant delay. To address this, we can specify pipeline `II=1` in both the `compute_fir` and the `linear_buffer` function to attempt to reduce its `II`, as shown in Table B.4, which leads to a more optimized final implementation illustrated in Fig. B.3.

The modulo operator remains present, which continues to contribute to a higher iteration la-

Function	Latency	Iteration Latency	II	Trip Count
computemydsp	446	-	446	-
linear_buffer	124	12	1	114
compute_fir	299	35	25	11

Table B.4: Latency report for `fir_onesample.cpp` for an unroll factor of 10 with a `pragma pipeline II=1` applied in both the `compute_fir` and `linear_buffer` functions, 115 coefficients. Green values indicate improvements, while red values indicate worsening.

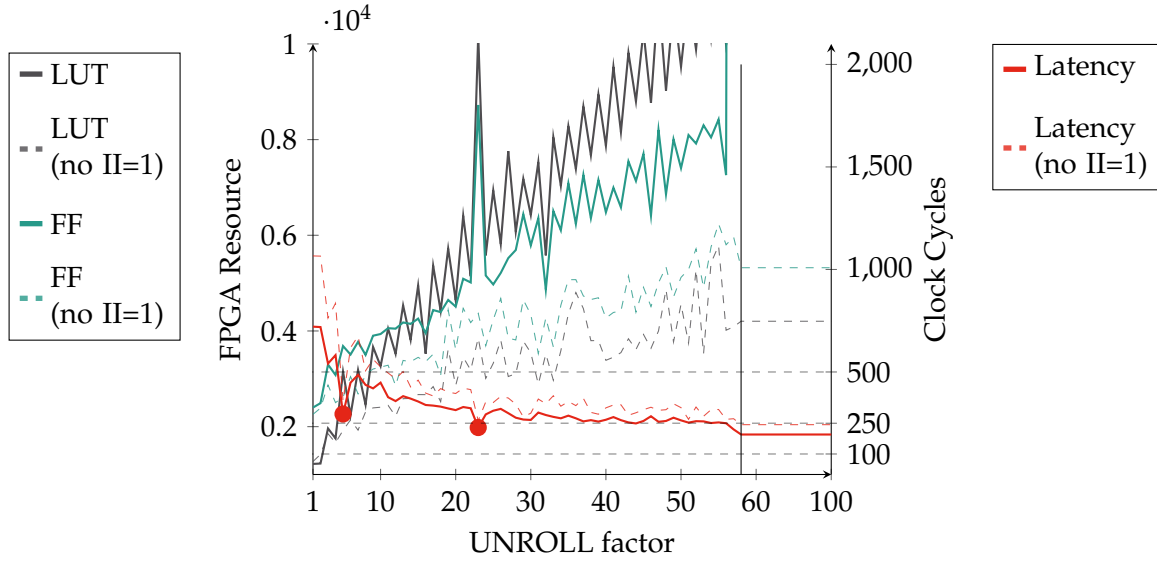


Figure B.3: Latency and resource usage for `fir_onesample.cpp` with varying `pragma` unroll factors and `pragma pipeline II=1` applied to `compute_fir` and `linear_buffer`, 115 coefficients. Dashed plot are for no `II` specified (corresponding to Fig. B.1).

tency for the `linear_buffer` loop compared to the base implementation. However, by specifying `pragma pipeline II=1`, we manage to smooth out the latency across iterations, resulting in improved performance.

The results are not as efficient as anticipated. The plot in Fig. B.3, which represents the latency performance of the modified implementation, shows that the latency for the two “sweet spot” values (where the unroll factor perfectly divides the number of coefficients) remains almost unchanged. For other values, the latency is slightly reduced and smoothed, but this comes at the expense of a substantial increase in resource usage. This increase in resource usage makes the implementation questionable and justifiable only when latency constraints are extremely tight.

Memory Partitioning

Latency can potentially be reduced further through array partitioning. However, the results, as shown in Fig. B.4, indicate that there is no significant improvement in latency compared to previous implementation but a huge use of resources. The reason is that the pipeline `pragma` already infers memory partitioning for the samples and coefficients in this case. For an unroll factor of 10, the tool infers a partition with a factor of 5 (on a dual-port memory), allowing for 10 concurrent accesses (see Listing B.1).

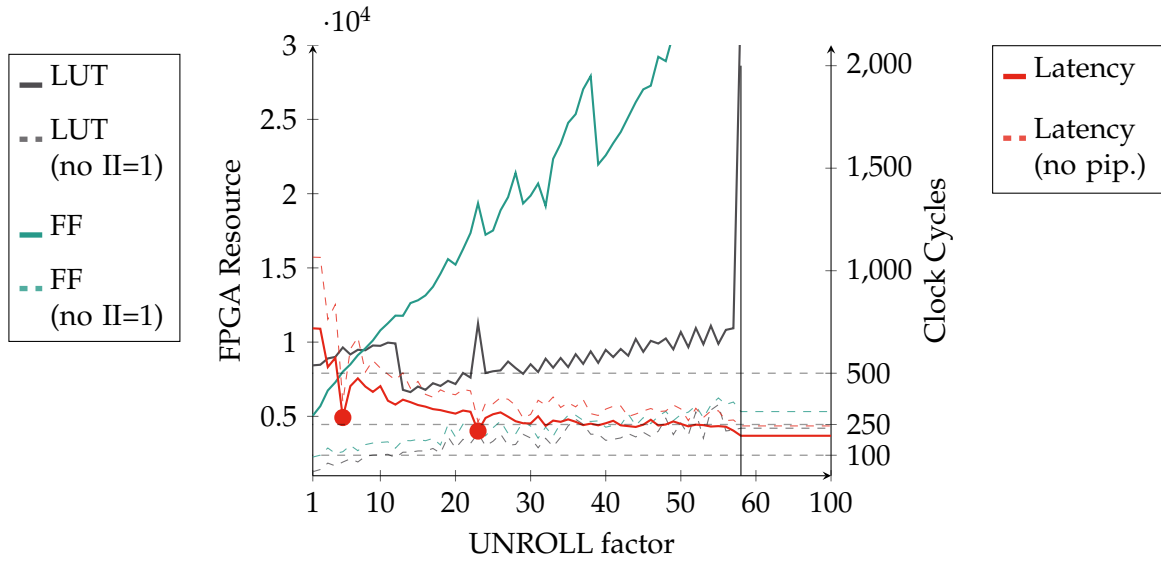


Figure B.4: Latency and resource usage for `fir_onesample.cpp` with varying pragma unroll factors and pragma `array_partition` applied to samples, 115 coefficients. Dashed plot are for no partition (corresponding to Fig. B.1). Note that the legend scale has significantly changed compared to the previous graph.

Lang: HLS

```

1 INFO: [HLS 214-270] Inferring pragma 'array_partition type=cyclic factor=5 dim=1' for
  array 'coeffs' due to pipeline pragma (syfala_ip/syfala_ip.cpp:52:9)
2 INFO: [HLS 214-270] Inferring pragma 'array_partition type=cyclic factor=5 dim=1' for
  array 'samples' due to pipeline pragma (syfala_ip/syfala_ip.cpp:52:9)

```

Listing B.1: HLS logs showing the automatic partitioning.

Adding additional partitioning does not improve latency because the optimal partitioning to guarantee the lowest II is already applied. Conversely, in the example, when complete partitioning is applied regardless of the unroll factor, the partitioning becomes excessive, using more resources without reducing latency.

The `array_partition` pragma should be used cautiously, as it can negatively impact the design with unnecessary partitioning. In most cases, the pipeline already handles array partitioning effectively if the II is forced to 1.

Minimize Resources

To minimize resource usage, the best approach is to disable both pipelining and unrolling. Disabling unrolling naturally reduces resource usage by avoiding hardware parallelization. Similarly, turning off pipelining saves additional resources as the pipelining mechanisms themselves consume hardware. While this results in a significantly higher latency compared to other implementations, it can be a viable option if resource constraints are the primary concern. Resource and latency estimations with no pipelining and unrolling are presented in Table B.5.

Results

Based on the previous study, the results are summarized in Fig. B.5.

Latency (cyc.)	BRAM	DSP	FF	LUT
2 094	2 (1%)	5 (6%)	876(2%)	1 733 (9%)

Table B.5: Resource usage for `fir_onesample.cpp` with pipeline and unroll disabled, 115 coefficients.

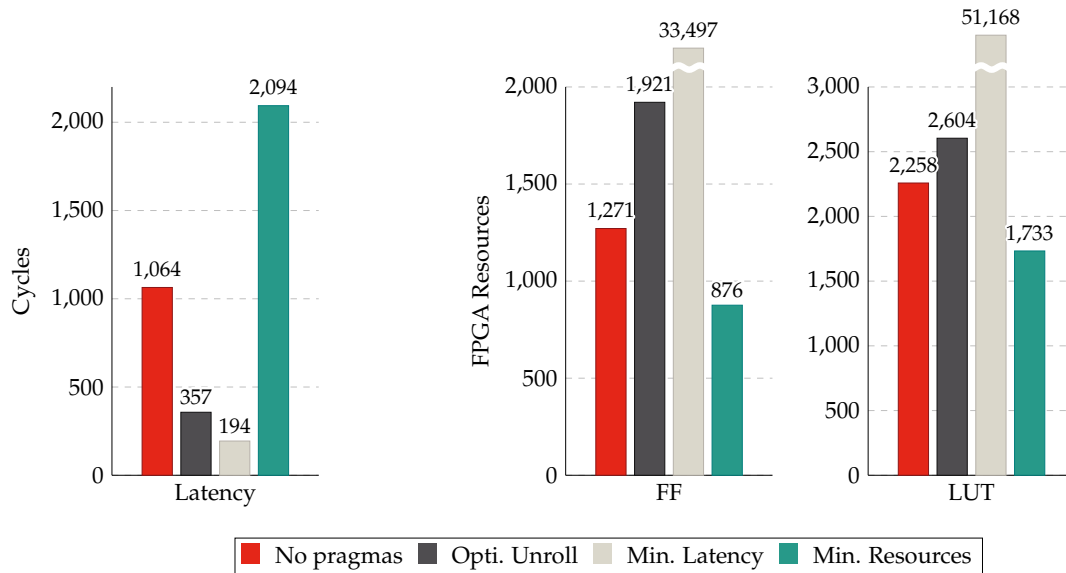


Figure B.5: Resource usage and latency results for different implementations of an FIR filter in a one-sample approach, 115 coefficients.

No Pragmas: This corresponds to Table B.1, where the program is synthesized without any directives.

Optimized Unroll: This corresponds to an unroll with a factor of 5, which is a divisor of the number of iterations. While 23 could have been chosen, 5 is more efficient in terms of resource usage. No explicit pipelines are in the code.

Minimize latency: If resource usage is not a concern, the lowest latency is achieved with a fully unrolled loop full partitioning of samples array. Fully partitioning large arrays is rarely efficient. While it may work well in this case for arrays with around a hundred coefficients, arrays with just a few thousand can result in very long synthesis times (several hours) and oversaturated resources. This example is for demonstration purposes, but the previous Optimized Unroll method is likely the most efficient for larger numbers of coefficients.

Minimize resources: This corresponds to the Table B.5 with no pipeline and no unroll.

APPENDIX C

LMS and FxLMS Case Study

This appendix details the work conducted on the implementation of two ANC algorithms: LMS and FxLMS filters.

Least Mean Squares (LMS) algorithms are adaptive filters used to model a system by iteratively adjusting the coefficients of a filter to minimize the difference between the desired signal and an estimated output, using the error to update the filter coefficients. This principle is illustrated in Fig.C.1. A detailed description of the LMS algorithm is not provided here, but the reader can refer to the *Signal processing for active control* [Ell00] for more information. The algorithm is outlined in Algorithm 3.

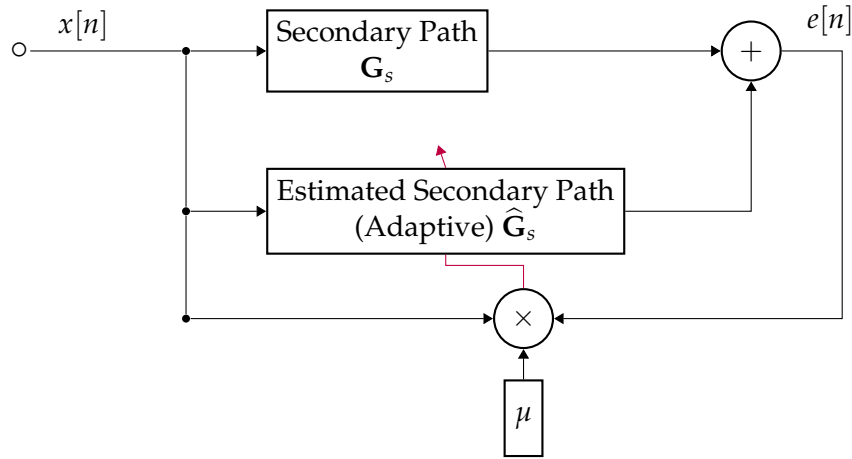


Figure C.1: Implemented LMS algorithm: G_s is the path to identify, $\hat{G}_s$ is the adaptive filter, μ is the step size.

An input signal $x[n]$ is sent to the filter to be identified (G_s) within the algorithm. A FIR filter computes the output sample of the adaptive filter ($\hat{G}_s$) based on the input sample and the coefficient array, which is updated with each new sample. The error between this output sample and the output of the filter to be identified is calculated ($e[n]$). This result is then passed in a loop to update all the coefficients based on the subtraction result.

This algorithm is studied using a one-sample approach only. In ANC applications, lower latency enhances the effectiveness and efficiency of the system. To align with the application's needs and focus our work on a single approach, all subsequent implementations adopt a one-sample methodology. However, the second part of this example uses a multichannel program, which enables the nested loops optimizations discussed in Chapter 5.4.3.

Algorithm 3 Our LMS Algorithm Implementation

```

1: Input:  $x_{iden}, C_{hat}, \mu, input\_0$ 
2: Output: Updated  $C_{hat}$ 
3: for  $k = N\_COEFS - 1$  to 1 do           ▷ Loop 1: Linear buffer (buffered samples for FIR filter)
4:    $x\_n[k] \leftarrow x\_n[k - 1]$ 
5: end for
6:  $x\_n[0] \leftarrow x_{iden}$                      ▷ New input sample of the identification signal
7:  $y\_hat \leftarrow 0$ 
8: for  $k = 0$  to  $N\_COEFS - 1$  do           ▷ Loop 2: Computation of sample through adaptive filter
9:    $y\_hat \leftarrow y\_hat + C_{hat}[k] \times x\_n[k]$ 
10: end for
11:  $error \leftarrow read(input\_0) - y\_hat$ 
12:  $x\_n\_norm \leftarrow 0$ 
13: for  $k = 0$  to  $N\_COEFS - 1$  do           ▷ Loop 3: Computation of the norm of the input buffer
14:    $x\_n\_norm \leftarrow x\_n\_norm + x\_n[k] \times x\_n[k]$ 
15: end for
16: for  $k = 0$  to  $N\_COEFS - 1$  do           ▷ Loop 4: Adapt coefficients of the adaptive filter
17:    $C_{hat}[k] \leftarrow C_{hat}[k] + \left( \frac{x\_n[k] \times \mu \times error}{x\_n\_norm + 0.001} \right)$ 
18: end for

```

Initially, the LMS was implemented using FAUST, but this approach quickly showed limitations (a maximum of 70 coefficients can be implemented on the *Genesys* board, less than 50 on *Zybo Z20*), leading us to use a direct C++ approach.

Single-Channel LMS

The resource usage and latency of the base program synthesis (`lms_mono_base.cpp` [The]) are provided in Fig. C.2 (*Base (1)*). Several initial optimizations can be applied to this program, following the guidelines in Chapter 5.4. First, loops 2 and 3 can be merged following the Rule 5.6.1 p.127 (labeled as *Divider outside (2)* in Fig. C.2). Additionally, following the Rule 5.6.2 p.128, performing the division outside the loop avoids heavy arithmetic operations during each iteration (labeled as *Merged loops (3)* in Fig. C.2). Moving the division outside the loop didn't significantly change the performance because the loop is pipelined. Nevertheless, placing the division outside the loop is beneficial if the loop is not fully sequential, as in the next example. Finally, this program uses a power of 2 as coefficients, using a circular buffer instead of a linear one, as specified on Rule 5.5.1 p.126, improve efficiency (labeled as *Circular buffer (4)* in Fig. C.2).

At this stage, we analyze the resource usage table provided in Table C.1.

Function	Latency (cyc.)	II	FF	LUT
syfala	4 712	-	2 294	3 567
y_hat	4 105	8	246	211
C_hat	527	1	240	108

Table C.1: Resources and latency report of `lms_mono.cpp` with no pragmas (512 coefficients).

Since C_{hat} has an initiation interval of 1, there is limited scope for further optimization as noted in Rule 5.4.6 p.111. However the y_{hat} loop has a high initiation interval and could likely be optimized. The first step is to apply a pragma to enforce an $II=1$ in the y_{hat} loop (labeled as *II=1 in y_{hat} (5)* in Fig. C.2).

Function	Latency (cyc.)	II	FF	LUT
syfala	3 177	-	2 540	4 135
y_hat	2 569	5	492	799
C_hat	527	1	240	108

Table C.2: Resources and latency report of `lms_mono.cpp` with pipeline II=1 in `y_hat` loop (512 coefficients).

Finally, even after setting II=1, the `y_hat` loop remains relatively high in latency (see Table C.2) due to a dependency on `y_hat`. To reduce latency further, we can unroll the loop slightly or partition `y_hat`. In this case, unrolling it four times strikes a good balance between resource usage and performance (labeled as *Unroll factor=4* (6) in Fig. C.2). Results can be tested with the program `lms_mono_opti.cpp` [The].

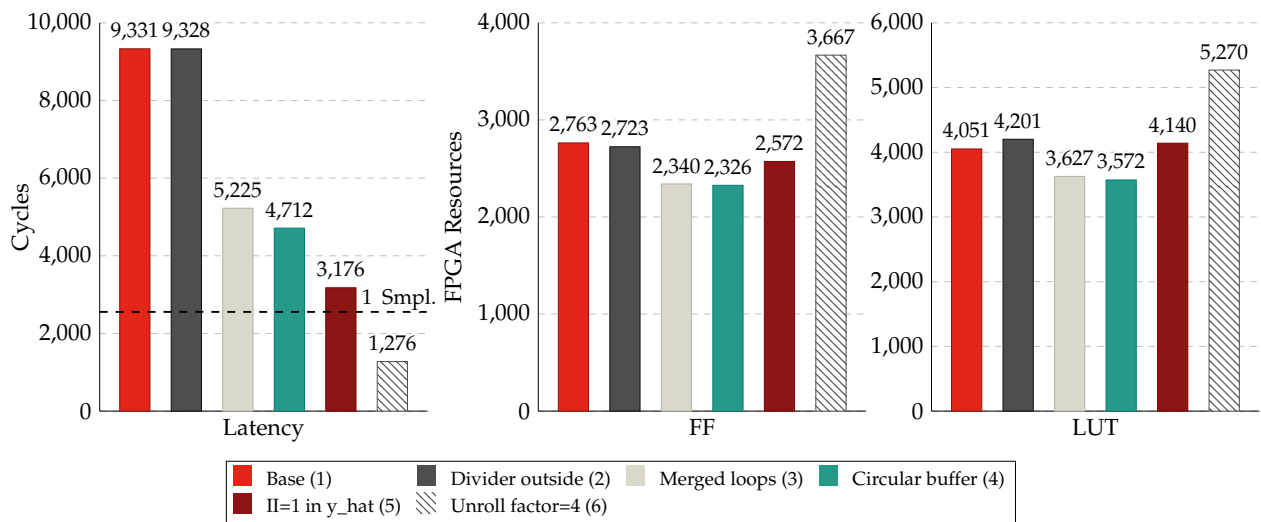


Figure C.2: Resource usage and latency for different implementation of a LMS algorithm with 512 coefficients in a one-sample processing strategy (Zybo Z20).

LMS Multichannel

For the implementation of a multichannel FxLMS algorithm, multiple acoustic paths can be identified simultaneously using a LMS algorithm with various adaptive filters. To accommodate this, we modify the LMS program to create multiple instances of the LMS algorithm that run for different sources (microphones), with each loop iterating over MIC_NB (more details in the next section).

The resource usage and latency of the base program (`lms_multichannel_base.cpp` [The]) synthesis for 6 instances are presented in Fig. C.4, labeled as *Base* (1). Similar to the previous approach, the program is initially optimized by using circular buffers and writing dividers outside the loops. Interchanging loops allows for the merging of loops that were previously non-mergeable, as shown in Listing C.3 (labeled as *Interchanged loops* (2) in Fig. C.4). Forcing the initiation interval to 1 for both loops significantly reduces the II, decreasing it from 11 and 6 for `y_hat_and_norm` and `c_hat`, respectively, to 5 and 1, as shown in Table C.3 (labeled as *Pipeline II=1* (3) in Fig. C.4).

Here, a dependency on `y_hat` limits further optimization. However, unrolling the loop 2 times allows the latency to drop below 2 603 cycles for one sample at 48 kHz (resulting in *Unroll factor=2* (4) in Fig. C.4). Results can be tested with the program `lms_multichannel_opti.cpp` [The].

Function	Latency (cyc.)	Iteration Latency	FF	LUT
syfala	3 221	-	7 422 (21%)	10 000 (56%)
y_hat_and_norm	2 571	5	1 250	1 551
c_hat	531	1	3 251	3 774

Table C.3: Resources and latency report of `lms_multichannel.cpp` with pipeline II=1 in `y_hat_and_norm` and `c_hat` loops (512 coefficients and MIC_NB=6).

Lang: C		Lang: C	
1	<code>//Non-mergeable loops:</code>	1	<code>//Merged interchanged loops:</code>
2	<code>norm:</code>	2	
3	<code>for(int k = 0; k < N_COEFS; k++){</code>	3	
4	<code> x_n_norm += x_n[k]*x_n[k];</code>	4	<code>y_hat_and_norm:</code>
5	<code>}</code>	5	<code>for(int k = 0; k < N_COEFS; k++){</code>
6	<code>y_hat:</code>	6	<code> x_n_norm += x_n[k]*x_n[k];</code>
7	<code>for (int i=0; i<MIC_NB; i++){</code>	7	<code> for (int i=0; i<MIC_NB; i++){</code>
8	<code> for(int k = 0; k < N_COEFS; k++){</code>	8	<code> y_hat[i] += C_hat[i][k]*x_n[k];</code>
9	<code> y_hat[i] += C_hat[i][k]*x_n[k];</code>	9	<code> }</code>
10	<code> }</code>	10	<code>}</code>
11	<code>}</code>	11	

Figure C.3: Excerpt of a program showing that interchanging loops allows them to be merged.

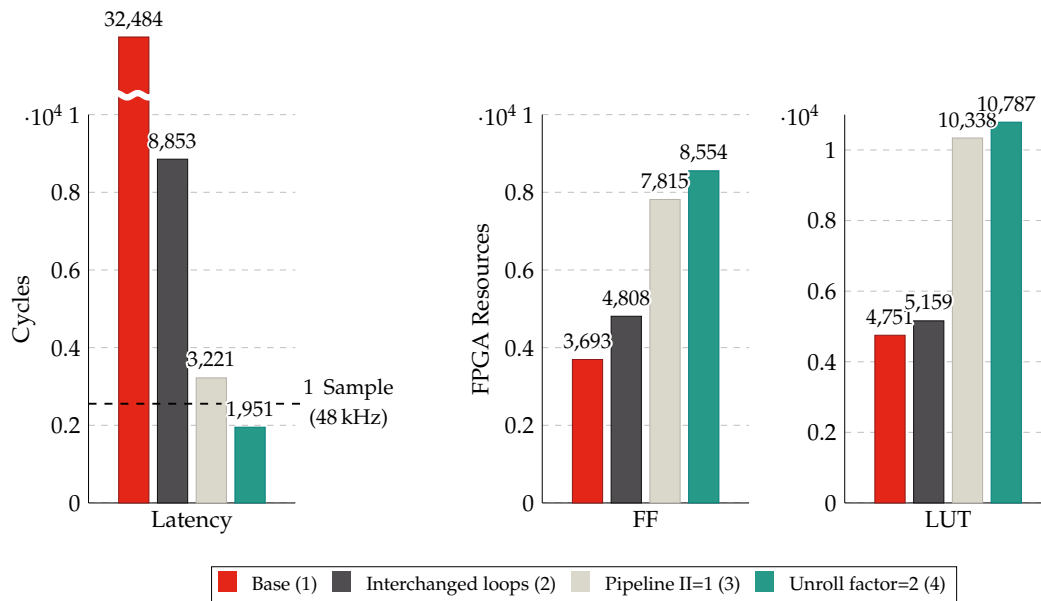


Figure C.4: Resource usage and latency for different implementation of 6 concurrent LMS algorithm with 512 coefficients in a one-sample processing strategy (Zybo Z20).

FxLMS

The FxLMS algorithm is an adaptation of the LMS algorithm designed for active noise control, where the reference signal is filtered by an estimated model of the secondary path before updating the adaptive filter. The secondary path is the transfer function between the speaker that will cancel the noise and the control microphone. This filtering accounts for the effects of the secondary path on the signal, allowing the algorithm to effectively cancel out noise.

The experiment setup involves a listening room equipped with 1 noise speaker, N speakers on the walls, and M control microphones in the middle of the room. The FxLMS algorithm is employed to cancel the noise detected by the M microphones using the N speakers. To achieve this, the secondary path (the transfer function between each of the N speakers and each of the M microphones) must first be identified using a LMS algorithm. While the LMS algorithm could be applied $N \times M$ times, since we have detailed a multichannel version (which iterates over M microphones), we only need to apply it N times: once for each speaker. The principle of FxLMS is presented in Fig. C.5

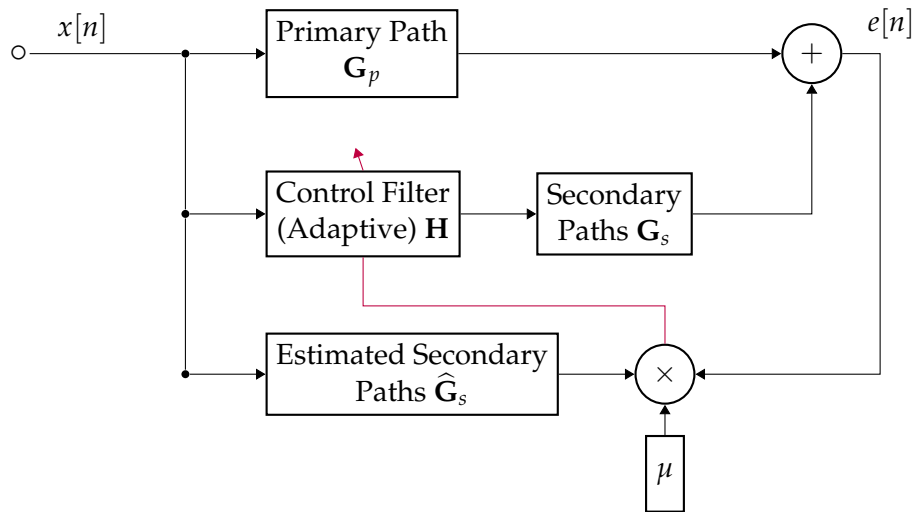


Figure C.5: Implemented FxLMS algorithm: $\hat{G}_s$ is the path previously identified with the LMS algorithm, H is the adaptive filter, μ is the step size.

All secondary paths must be computed within the same algorithm. Consequently, the filter loops, which iterate over the coefficients, are nested within a loop that iterates over the microphones, which in turn is nested inside another loop iterating over the speakers. This structure contributes to the complexity and resource intensity of the program. The full algorithm is quite complex, and the curious readers can refer to Algorithm 4 in Appendix F for more information.

The resource usage and latency of the base program synthesis (fxlms_base.cpp) are provided in Fig. C.6 (*Base (1)*). As before, loop merging and circular buffering (no dividers inside loops in this case) are applied to the program. Loops are then interchanged (*Interchanged loops (2)* in Fig. C.6) and the II forced to 1 (*Pipeline $II=1$ (3)*). An excerpt of the resulting resource usage table is provided in Table C.4 and the resulting file is fxlms_preopti.cpp [The]. The table shows that the total clock cycle latency is higher than the sample period. loop_9 cannot be optimized further, as it already has low latency but requires a significant amount of resources. loop_5_and_loop_7, as well as loop_2, have relatively high latency compared to other loops. While loop_5_and_loop_7 significantly impact resource usage and can be unrolled cautiously, loop_2 is lightweight and can be unrolled extensively. By testing different unroll factors, the optimal configuration is found with an unroll factor of 4 for loop_5_and_loop_7 and loop_2. The resulting program is provided in the file fxlms_opti.cpp.

Thanks to these optimizations, we were able to fit this 14-speaker, 4-microphone FxLMS algorithm on the Genesys ZU3-EG FPGA board.

The Testing Method

An LMS and FxLMS filter implementation is complex to verify because practical validation depends on many factors: even if the algorithm is correctly implemented, it may not work effectively

Function	Latency (cyc.)	II Latency	BRAM	DSP	FF	LUT
syfala	4 838	-	155 (35%)	283 (78%)	41 765 (29%)	40 850 (57%)
loop_2	2 054	4	1	-	149	153
loop_5_and_loop_7	2 059	4	-	-	9 135	9 740
loop_9	543	1	-	242	26 391	18 964

Table C.4: Excerpt of resources and latency estimation of `fxlms_preopti.cpp` with pipeline II=1 in loops only (no unroll) (512 coefficients, MIC_NB=4, HP_NB=14). Percentage for UltraScale+ XCZU3EG SoC. Only relevant loops with significant latency are shown.

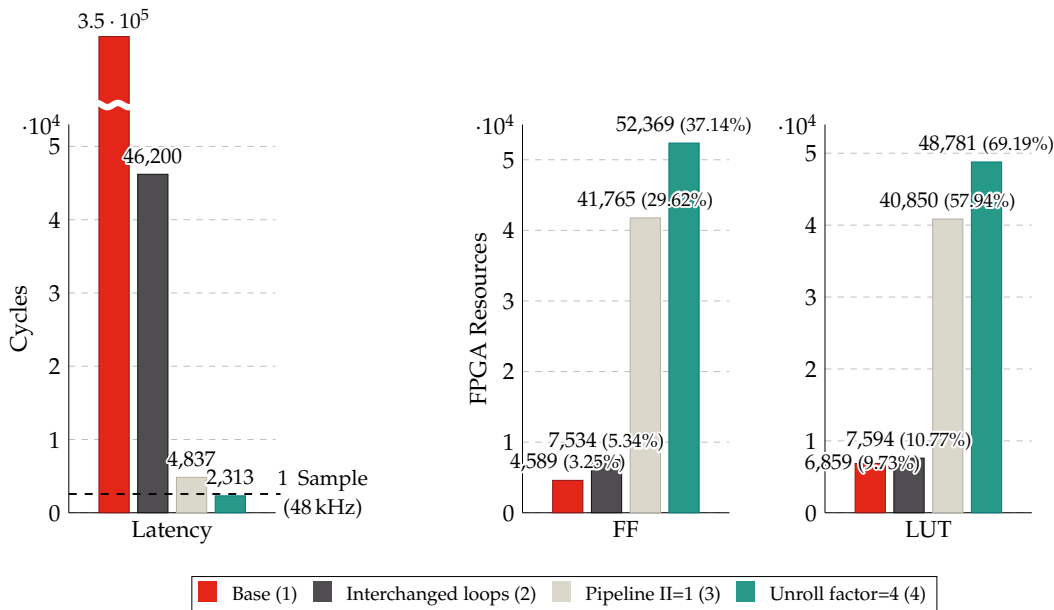


Figure C.6: Resource usage and latency for different implementation of a 4x14 FxLMS algorithm with 512 coefficients in a one-sample processing strategy (Percentage for UltraScale+ XCZU3EG).

due to poor room configuration, speaker placement, microphone positioning, etc.

To validate the functionality of the design, we first recreated the two algorithms in *Python* using the *NumPy* library and verified the results by analyzing frequency responses. This initial test was a full simulation, aiming to identify filters from a given array of coefficients. After validating the *Python* program, we used *Vitis HLS Csim* and test benches to ensure that the C++ IP produced identical results, sample by sample.

Once this was confirmed, we proceeded to test the LMS algorithm under real conditions with a speaker and a microphone. The objective was to identify the equivalent filter for the speaker/microphone path. Initial measurements were very inconsistent, leading us to hypothesize that issues with the experimental setup were the cause. We encountered several problems with faulty cables and connectors, and used a microphone amplifier with no documentation, which could alter the response or introduce additional latency. To address this, we wrote another *Python* script to identify the transfer function of the entire setup (including microphone, speaker, microphone amplifier, and sound card) and to characterize the sound card and microphone amplifier. With this validated setup, we were able to create a clean and well-defined environment, which allowed us to properly characterize the filter.

Custom Communication Between FPGA and CPU

One of the specific aspects of the two LMS programs is the substantial amount of data that needs to be transferred between the FPGA and the CPU, that are not control signals. For example, to obtain the coefficients of the identified filter, we must wait for the error to stabilize and then read the computed coefficients. This can be achieved by monitoring the error and transmitting the coefficients once a certain threshold is reached, or more conveniently, through a user command. Once stabilized, the coefficients should be sent to the ARM processor and printed.

Additionally, certain parameters, such as the μ factor or the selected speaker for LMS identification, should be controllable from the ARM processor. While these parameters can be hard-coded into the IP program, doing so would require resynthesizing the entire design just to test a new value. A more efficient approach is to hard-code them in the ARM program and transmit them via AXI, allowing for faster adjustments by only recompiling the ARM code rather than the entire FPGA program. However, this still is not very user-friendly.

To address this, we developed a Text User Interface (TUI) detailed in Chapter 4.5.4, which enables the user to control these parameters in real-time. The TUI allows the user to send a specific command when the error has stabilized, effectively “freezing” the identified coefficient values and retrieving them, or changing the selected speaker/microphone. The coefficients for the secondary path, previously computed, are also written from the ARM and sent to the FPGA during initialization. This approach is not about saving BRAM on the FPGA, as the coefficients are stored in BRAM after initialization, but rather to avoid the need to synthesize the entire program when we want to change them.



Experimental Results

This appendix presents exhaustive tables of measurements derived from our experiments.

Three tables are included:

- Table D.1 details the latency measurements of the audio codecs, which are discussed in Chapter 6.2.3, particularly in Fig. 6.6.
- Tables D.2 and D.3 display the results of HLS synthesis and implementation for the FIR filter algorithm, using both one-sample and multi-sample approaches. These tables are referenced in Chapter 6.4.1 and contribute to the synthesis of the number of implementable coefficient filters, as shown in Fig. 6.14 (one-sample) and Fig. 6.16 (multi-sample).

Audio codec	Measure	Ck^{ADC} (kHz)	$f_s^{I^2S}$ (kHz)	Latency (μ s)	Latency (sample)
SSM2603					
	Bypass I ² S	48	48	429	21
		96	96	161	15
	FAUST	48	48	471	23
		96	48	250	12
		96	96	180	17
ADAU1777					
	Bypass DSP	768	-	9.2	-
	Bypass I ² S	96	48	257	12
		96	96	157	15
		768	192	69.2	13
	FAUST	48	48	NC	NC
		96	48	298.2	14
			96	179.6	17
		192	192	97.8	19
		768	48	255.2	12
			96	135.2	13
			192	79.5	15
ADAU1787					
	Bypass DSP	768	-	5.9	-
	Bypass I ² S	48	48	164.6	8
		96	96	75.9	7
		768	48	104	5
			192	30.3	6
			384	20	8
			768	8.4	6
	FAUST	48	48	206.2	10
		96	96	97.2	9
		192	192	54.7	10
		384	384	30	12
		768	48	145.6	7
			192	40.8	8
			384	25.4	10
	FAUST 16b	768	768	11.1	8

Table D.1: Latency report for different audio codecs.

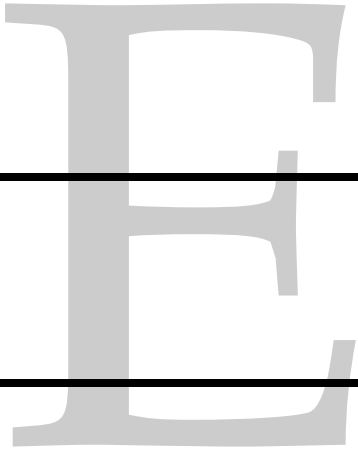
Except for the last line, all measurements were made on 24 bits samples. Latency expressed as samples is computed based on the $f_s^{I^2S}$ sampling rate and rounded to the nearest integer. The third column indicates the internal sampling rate of the audio codec ADC/DAC: Ck^{ADC} (which therefore can be different from the global sampling rate of the system). The fourth column indicates the global sampling rate of the system $f_s^{I^2S}$ (the sampling rate used by the I²S interface). Not all of these configurations guarantee an acceptable SNR, particularly if $Ck^{ADC} \gg f_s^{I^2S}$.

Scenario	Board	Nb of coeffs	HLS Stage	DSP Tot Latency (cyc.)	BRAM	DSP	FF	LUT
FAUST	Z10	300	Syn	264	2 (1%)	17 (21%)	5 807 (16%)	14 566 (82%)
			Impl	264	2 (1%)	17 (21%)	6 300 (18%)	11 364 (64%)
	Z20	2 000	Syn	1 130	4 (1%)	26 (11%)	13 146 (12%)	48 329 (90%)
			Impl	1 130	4 (1%)	26 (11%)	13 907 (13%)	45 342 (85%)
	GEN.	2 000	Syn	1 089	4 (0%)	28 (7%)	12 710 (9%)	57 716 (81%)
			Impl	1 089	4 (0%)	28 (7%)	13 014 (9%)	42 653 (60%)
Man. C++	Z10	250	Syn	2 280	3 (1%)	7 (8%)	1 274 (3%)	2 250 (12%)
			Impl	2 280	3 (1%)	7 (8%)	1 113 (3%)	904 (5%)
	Z20	250	Syn	2 280	3 (1%)	7 (3%)	1 274 (1%)	2 250 (4%)
			Impl	2 280	3 (1%)	7 (3%)	1 113 (1%)	904 (1%)
	GEN.	250	Syn	1 775	3 (0%)	7 (1%)	968 (0%)	1 608 (2%)
			Impl	1 775	3 (0%)	4 (1%)	803 (0%)	860 (1%)
Circ. Buf.	Z10	5 000	Syn	2 565	32 (26%)	15 (18%)	3 611 (10%)	4 628 (26%)
			Impl	2 565	32 (26%)	15 (18%)	3 222 (9%)	2 245 (12%)
	Z20	5 000	Syn	2 565	32 (11%)	15 (6%)	3 611 (3%)	4 628 (8%)
			Impl	2 565	32 (11%)	15 (6%)	3 222 (3%)	2 245 (7%)
	GEN.	5 000	Syn	2 549	28 (6%)	15 (4%)	3 070 (2%)	3 305 (4%)
			Impl	2 549	28 (6%)	15 (4%)	2 159 (3%)	2 693 (1%)
Lin. Buf.	Z10	3 000	Syn	2 350	20 (16%)	75 (93%)	7 148 (20%)	8 759 (49%)
			Impl	2 350	20 (16%)	72 (90%)	5 033 (14%)	5 480 (31%)
	Z20	8 000	Syn	2 367	50 (17%)	208 (94%)	19 614 (18%)	21 463 (40%)
			Impl	2 367	50 (17%)	204 (93%)	12 592 (11%)	19 476 (36%)
	GEN.	9 000	Syn	2 099	56 (12%)	342 (95%)	15 862 (11%)	21 958 (31%)
			Impl	2 099	56 (12%)	306 (85%)	9 870 (7%)	19 389 (27%)

Table D.2: FIR filter latency and resource utilization for HLS synthesis and implementation using a one-sample approach.

Scenario	Board	Nb of coeffs	Buffer size	Latency/ Samples (cyc.)	HLS Stage	DSP Tot Latency (cyc.)	BRAM	DSP	FF	LUT
Opti. HLS	Z10	13 000	32	2 457	Syn	78 644 (639 μ s)	66 (55%)	23 (28%)	8 480 (24%)	17 228 (97%)
					Impl	78 644 (639 μ s)	63 (53%)	23 (28%)	7 563 (21%)	12 777 (68%)
	Z20	30 000	70	2 591	Syn	181 390 (1.47 ms)	136 (48%)	114 (51%)	19 744 (18%)	44 539 (83%)
					Impl	181 390 (1.47 ms)	134 (47%)	114 (52%)	18 020 (17%)	45 395 (85%)
GEN.	45 000	90	2 515	Syn	226 405 (1.84 ms)	174 (40%)	161 (44%)	22 629 (16%)	65 667 (93%)	
				Impl	226 405 (1.84 ms)	216 (50%)	162 (45%)	20 134 (14%)	65 479 (92%)	

Table D.3: FIR filter latency and resource utilization for HLS synthesis and implementation using a multi-sample approach.



APPENDIX E

Figures

This appendix presents two figures included here for clarity:

- The *Syfala* Compiler Report in Fig. E.1, which is automatically generated after each *Syfala* command, is discussed in Chapter 4.5.4.
- The *Syfala* Block Design in Fig. E.2, which illustrates the complete architecture generated by the compiler and implemented on the SoC, is referenced in Chapter 4.5.2.

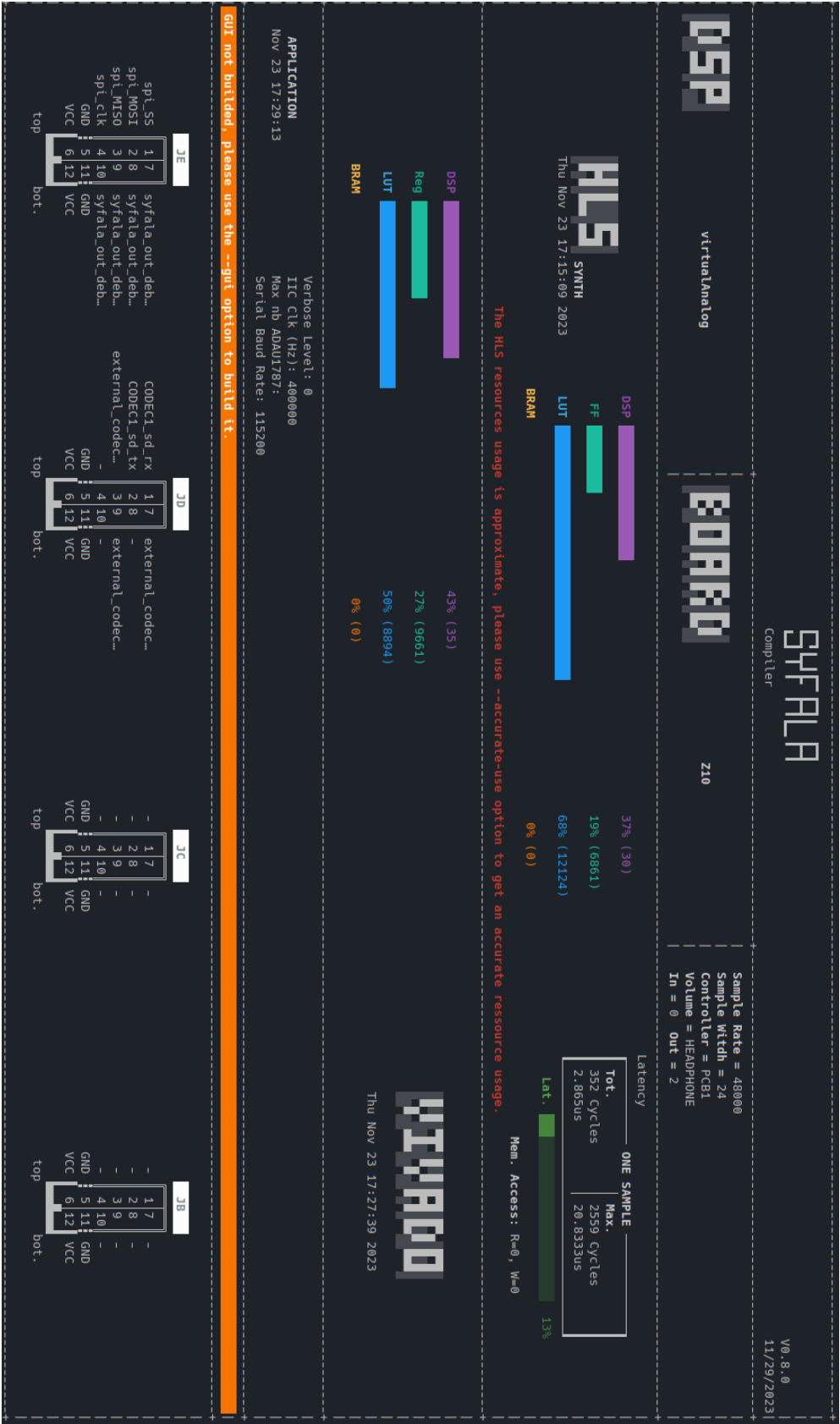


Figure E.1: Syfala Compilation Report automatically printed after a compilation with our compiler.

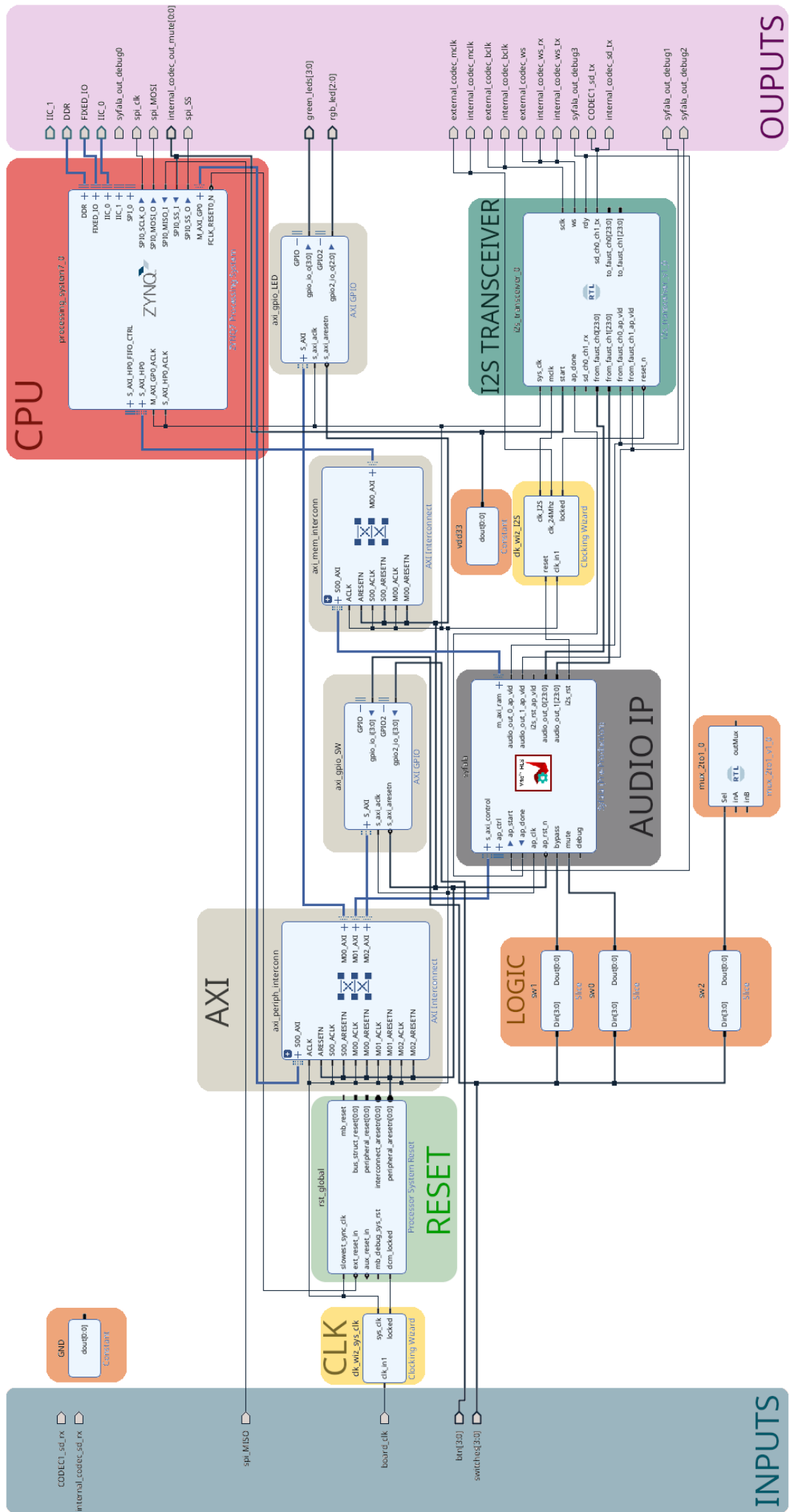
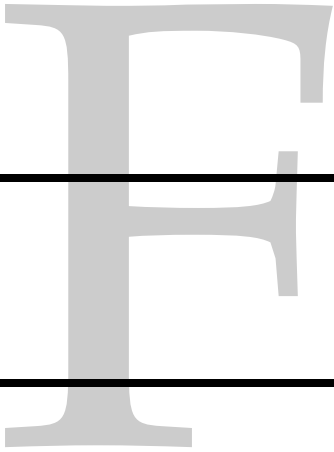


Figure E.2: Graphical view of the Syflala block design, with the following color annotations: inputs (blue), outputs (pink), reset system (green), AXI interconnect (brown), clock system (yellow), CPU (red), audio hardware IP (gray), hardware logic (orange), and I²S transceiver (teal).



APPENDIX F

Programs

This appendix contains the main programs used in this thesis, presented here for easy access by the reader to the full programs. The following programs are included:

- `audioCtl.cpp` in Listing F.1 and `audioDSP.cpp` in Listing F.2, which correspond to the hardware and software files used in our *Syfala* audio compiler, as detailed in Chapter 4.5.2.
- `fir_onesample.cpp` in Listing F.3 and `fir_multisample.cpp` in Listing F.4, which are utilized throughout Chapter 5 to illustrate the implementation of an FIR filter using both one-sample and multi-sample approaches.
- The FxLMS algorithm, presented in Algorithm 4 and used in Appendix C and Chapter 6.4.3, is included as an algorithm for clarity.

All programs are provided in the thesis repository [The].

Lang: C

```
1  #include <syfala/arm/audio.hpp>
2  #include <syfala/arm/gpio.hpp>
3  #include <syfala/arm/uart.hpp>
4  #include <syfala/arm/ip.hpp>
5  using namespace Syfala;
6  int main(int argc, char* argv[]) {
7      XSyfala x;
8      UART::data uart;
9      // UART & GPIO should be initialized first,
10     GPIO::initialize();
11     UART::initialize(uart);
12     // Wait for all peripherals to be initialized
13     Status::waiting(RN("[status] Initializing peripherals & modules"));
14     Audio::initialize();
15     IP::initialize(x);
16     IP::set_arm_ok(&x, true);
17     Status::ok(RN("[status] Application ready, now running..."));
18     // main event loop:
19     while (true) {
20         // Handle control rate computations here
21     }
22     return 0;
23 }
```

Listing F.1: `audioCtl.cpp`

Lang: C

```

1  #include <syfala/utilities.hpp>
2
3  #define INPUTS 2
4  #define OUTPUTS 2
5  static bool initialization = true;
6
7  static void compute(sy_ap_int const input_0,
8                     sy_ap_int const input_1,
9                     sy_ap_int* output_0,
10                    sy_ap_int* output_1)
11  {
12      // if you need to convert to float, use the following:
13      // (audio inputs and outputs are 24-bit integers by default)
14      float f0 = Syfala::HLS::ioreadf(input_0) * 0.5f;
15      float f1 = Syfala::HLS::ioreadf(input_1) * 0.5f;
16      Syfala::HLS::iowritef(f0, output_0);
17      Syfala::HLS::iowritef(f1, output_1);
18  }
19
20  void syfala (
21      sy_ap_int audio_in_0,
22      sy_ap_int audio_in_1,
23      sy_ap_int* audio_out_0,
24      sy_ap_int* audio_out_1,
25      int arm_ok,
26      bool* i2s_rst,
27      float* mem_zone_f,
28      int* mem_zone_i,
29      bool bypass,
30      bool mute,
31      bool debug
32  ) {
33      #pragma HLS INTERFACE s_axilite port=arm_ok
34      #pragma HLS INTERFACE m_axi port=mem_zone_f latency=30 bundle=ram
35      #pragma HLS INTERFACE m_axi port=mem_zone_i latency=30 bundle=ram
36
37      // Active high reset, this HAVE TO BE DONE FIRST (crash with *some* dsp if not
38      )
39      *i2s_rst = !arm_ok;
40      /* Initialization and computations can start after the ARM
41       * has been initialized */
42      if (arm_ok) {
43          /* First function call: initialization */
44          if (initialization) {
45              // Initialize all runtime data here.
46              // do not forget to toggle the variable off
47              initialization = false;
48          } else {
49              /* Every other iterations:
50               * either process the bypass & mute switches... */
51              if (bypass) {
52                  *audio_out_0 = audio_in_0;
53                  *audio_out_1 = audio_in_1;
54              } else if (mute) {
55                  *audio_out_0 = 0;
56                  *audio_out_1 = 0;
57              } else {
58                  /* ... or compute samples here */
59                  compute(audio_in_0, audio_in_1, audio_out_0, audio_out_1);
60              }
61          }
62      }
63  }

```

Listing F.2: audioDSP.cpp

Lang: C

```

1 #include <algorithm>
2 #include <cmath>
3 #include <inttypes.h>
4 #include <string.h>
5 // #include <syfala/utilities.hpp>
6 #include <ap_int.h>
7
8 #include "coeffs_115.hpp"
9
10 #define INPUTS 0
11 #define OUTPUTS 2
12
13 /* This bloc is for a standalone \textit{Vitis} HLS use. Everything should be
14 * declared in <syfala/utilities.hpp> if this file is used with the Syfala toolchain */
15 #define SYFALA_SAMPLE_WIDTH 24
16 float SCALE_FACTOR = (1 << (SYFALA_SAMPLE_WIDTH-1)) -1;
17 using sy_real_t = float;
18 using sy_ap_int = ap_int<SYFALA_SAMPLE_WIDTH>;
19 static inline void iowritef(float f, sy_ap_int* output) {
20     *output = sy_ap_int(f * SCALE_FACTOR);
21 }
22 /* ---- */
23
24 static bool initialization = true;
25 static float samples[NCOEFFS]={0.f};
26 static float sawtooth = 0.f;
27
28 void generate_input_samples()
29 {
30     samples[0]=sawtooth;
31     sawtooth += 0.01f;
32     sawtooth = sawtooth > 1 ? sawtooth -1 : sawtooth;
33 }
34
35 void compute_fir(float* fTemp) {
36     compute_fir:for (int n = 0; n < NCOEFFS; ++n) {
37         #pragma HLS unroll factor=5
38         #pragma HLS PIPELINE //II=1
39         *fTemp += samples[n] * coeffs[n];
40     }
41 }
42
43 void linear_buffer(){
44     linear_buffer:for (int j = NCOEFFS-1; j > 0; --j) {
45         // #pragma HLS unroll
46         // #pragma HLS PIPELINE off
47         // #pragma HLS PIPELINE II=1
48         samples[j] = samples[j-1];
49     }
50 }
51
52 void write_output_samples(
53     sy_ap_int* audio_out_0,
54     sy_ap_int* audio_out_1,
55     float* fTemp)
56 {
57     iowritef(*fTemp, audio_out_0);
58     iowritef(*fTemp, audio_out_1);
59 }
60
61 void computemydsp(sy_ap_int* audio_out_0, sy_ap_int* audio_out_1 )
62 {

```

```

63  float fTemp=0;
64  //pragma HLS ARRAY_PARTITION variable=samples dim=0 complete
65  //pragma HLS bind_storage variable=samples type=RAM_2P impl=LUTRAM
66  generate_input_samples();
67  linear_buffer();
68  compute_fir(&fTemp);
69  write_output_samples(audio_out_0, audio_out_1, &fTemp);
70  }
71
72  void syfala (
73      sy_ap_int* audio_out_0,
74      sy_ap_int* audio_out_1,
75      int arm_ok,
76      bool enable_i2s,
77      bool* i2s_rst,
78      float* mem_zone_f,
79      int* mem_zone_i,
80      bool bypass,
81      bool mute,
82      bool debug
83  ) {
84  //pragma HLS INTERFACE s_axilite port=arm_ok
85  //pragma HLS INTERFACE m_axi port=mem_zone_f latency=30 bundle=ram
86  //pragma HLS INTERFACE m_axi port=mem_zone_i latency=30 bundle=ram
87
88  /* Initialization and computations can start after the ARM
89   * has been initialized */
90  if (arm_ok) {
91      /* First function call: initialization */
92      if (initialization) {
93          // Initialize all runtime data here.
94          // do not forget to toggle the variable off
95          initialization = false;
96      } else {
97          computemydsp(audio_out_0, audio_out_1);
98      }
99  } else {
100      /* Waiting for the ARM to be ready,
101       * just output zeroes meanwhile... */
102  }
103  }

```

Listing F.3: fir_onesample.cpp

Lang: C

```

1  #include <algorithm>
2  #include <cmath>
3  #include <inttypes.h>
4  #include <string.h>
5  //include <syfala/utilities.hpp>
6  #include <ap_int.h>
7
8  #include "coeffs_115.hpp"
9
10 #define INPUTS 0
11 #define OUTPUTS 2
12 #define NCOEFFS 115
13 #define BLOCK_NSAMPLES 16
14
15 /* This bloc is for a standalone \textit{Vitis} HLS use. Everything should be
16  * declared in <syfala/utilities.hpp> if this file is used with the Syfala toolchain */
17 #define SYFALA_SAMPLE_WIDTH 24

```

```

18 float SCALE_FACTOR = (1 << (SYFALA_SAMPLE_WIDTH-1)) -1;
19 using sy_real_t = float;
20 using sy_ap_int = ap_int<SYFALA_SAMPLE_WIDTH>;
21 static inline void iowritef(float f, sy_ap_int* output) {
22     *output = sy_ap_int(f * SCALE_FACTOR);
23 }
24 /* ---- */
25
26 static bool initialization = true;
27 static float samples[NCOEFFS+BLOCK_NSAMPLES]={0.f};
28 static float sawtooth = 0.0;
29
30 void generate_input_samples()
31 {
32     generate_input_samples:
33     for (int n = 0; n < BLOCK_NSAMPLES; n++) {
34         samples[BLOCK_NSAMPLES-1-n]=sawtooth;
35         sawtooth += 0.01f;
36         sawtooth = sawtooth > 1 ? sawtooth -1 : sawtooth;
37     }
38 }
39
40
41 void compute_fir(float* fTemp) {
42     compute_fir_coeffs: for (int i = 0; i < NCOEFFS; i++) {
43         //#pragma HLS pipeline II=1
44         compute_fir_samples:
45         for (int n = 0; n < BLOCK_NSAMPLES; n++) {
46             fTemp[n] += samples[i+BLOCK_NSAMPLES-1-n] * coeffs[i];
47         }
48     }
49 }
50
51 void linear_buffer(){
52     linear_buffer:for (int j = NCOEFFS+BLOCK_NSAMPLES-1; j >= BLOCK_NSAMPLES; --j) {
53         //#pragma HLS pipeline II=1
54         samples[j] = samples[j-BLOCK_NSAMPLES];
55     }
56 }
57
58 void write_output_samples(
59     sy_ap_int* audio_out_0,
60     sy_ap_int* audio_out_1,
61     float* fTemp)
62 {
63     write_output_samples:
64     for (int n = 0; n < BLOCK_NSAMPLES; n++) {
65         iowritef(fTemp[n], &audio_out_0[n]);
66         iowritef(fTemp[n], &audio_out_1[n]);
67     }
68 }
69
70 void computemydsp(sy_ap_int* audio_out_0, sy_ap_int* audio_out_1 )
71 {
72     float fTemp[BLOCK_NSAMPLES]={0};
73     //#pragma HLS array_partition variable=fTemp type=complete
74     generate_input_samples();
75     linear_buffer();
76     compute_fir(fTemp);
77     write_output_samples(audio_out_0, audio_out_1, fTemp);
78 }
79 }
80

```

```

81 void syfala (
82     sy_ap_int* audio_out_0,
83     sy_ap_int* audio_out_1,
84     int arm_ok,
85     bool enable_i2s,
86     bool* i2s_rst,
87     float* mem_zone_f,
88     int* mem_zone_i,
89     bool bypass,
90     bool mute,
91     bool debug
92 ) {
93 #pragma HLS INTERFACE s_axilite port=arm_ok
94 #pragma HLS INTERFACE m_axi port=mem_zone_f latency=30 bundle=ram
95 #pragma HLS INTERFACE m_axi port=mem_zone_i latency=30 bundle=ram
96 /* Initialization and computations can start after the ARM
97  * has been initialized */
98 if (arm_ok) {
99     /* First function call: initialization */
100    if (initialization) {
101        // Initialize all runtime data here.
102        // do not forget to toggle the variable off
103        initialization = false;
104    } else {
105        computemydsp(audio_out_0, audio_out_1);
106    }
107 } else {
108     /* Waiting for the ARM to be ready,
109      * just output zeroes meanwhile... */
110 }
111 }

```

Listing F.4: fir_multisample.cpp

Algorithm 4 Our FxLMS Algorithm Implementation

```

1: Input:  $input, \mu, x\_iden, W$   $\triangleright$  Adaptativ filter,  $S_{hat}$   $\triangleright$  Equivalent (adapted) filter of the secondary path identified in the previous
   LMS program
2: Output: Updated  $W, output, x\_filtered$ 
3: for  $k = N\_COEFS - 1$  to 1 do  $\triangleright$  Loop 1: Linear buffer of input white noise(buffered samples for filter of loop 2)
4:    $x\_n[k] \leftarrow x\_n[k - 1]$ 
5: end for
6:  $x\_n[0] \leftarrow x\_iden$   $\triangleright$  New input sample of the identification signal
7:  $x\_filtered \leftarrow 0.0$ 
8: for  $k \leftarrow 0$  to  $N\_COEFS - 1$  do  $\triangleright$  Loop 2: Identification white noise filter
9:    $x\_filtered \leftarrow x\_filtered + WN\_filter[k] \times x\_n[k]$ 
10: end for
11: for  $k \leftarrow N\_COEFS - 1$  downto 1 do  $\triangleright$  Loop 3: Linear buffer of filtered noise (buffered samples for FIR filter)
12:    $x\_n\_filtered[k] \leftarrow x\_n\_filtered[k - 1]$ 
13: end for
14:  $x\_n\_filtered[0] \leftarrow x\_filtered$ 
15:  $error[i] \leftarrow 0.0$  for  $i = 0, \dots, HP\_NB - 1$ 
16: for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 4: Error of each adaptive filters
17:    $error[i] \leftarrow read(input[i \times 2]) + read(input[(i \times 2) + 1])$ 
18: end for
19:  $u[i] \leftarrow 0.0$  for  $i = 0, \dots, HP\_NB - 1$   $\triangleright$  Convolving samples with the adaptative filters W: filter being identified
20: for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 5 (nested): Convolution of sample with the adaptiv filter W
21:   for  $k \leftarrow 0$  to  $N\_COEFS - 1$  do
22:      $u[i] \leftarrow u[i] + W[i][k] \times x\_n\_filtered[k]$ 
23:   end for
24:    $write(u[i], \&output[i])$   $\triangleright$  Send the adapted signal on sepaers
25: end for
26:  $\triangleright$  Convolving samples with the filters  $S_{hat}$ : equivalent filter of the secondary path identified in the previous LMS program
27: for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 6 (nested): Linear buffer for samples convolved with the filters  $S_{hat}$ 
28:   for  $j \leftarrow 0$  to  $MIC\_NB - 1$  do
29:     for  $k \leftarrow N\_COEFS - 1$  downto 1 do
30:        $r\_hat\_n[i][j][k] \leftarrow r\_hat\_n[i][j][k - 1]$ 
31:     end for
32:   end for
33:    $r\_hat[i][j] \leftarrow 0.0$  for  $i = 0, \dots, HP\_NB - 1$  and  $j = 0, \dots, MIC\_NB - 1$ 
34:   for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 7 (nested): Convolution of sample with the adapted filter  $S_{hat}$ 
35:     for  $j \leftarrow 0$  to  $MIC\_NB - 1$  do
36:       for  $k \leftarrow 0$  to  $N\_COEFS - 1$  do
37:          $r\_hat[i][j] \leftarrow r\_hat[i][j] + S\_hat[(i \times 2) + j][k] \times x\_n\_filtered[k]$ 
38:       end for
39:     end for
40:   end for
41:   for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 8: Add new value of convolved sample with  $S_{hat}$  to buffer
42:     for  $j \leftarrow 0$  to  $MIC\_NB - 1$  do
43:        $r\_hat\_n[i][j][0] \leftarrow r\_hat[i][j]$ 
44:     end for
45:   end for
46:   for  $i \leftarrow 0$  to  $HP\_NB - 1$  do  $\triangleright$  Loop 9 (nested): Adapt coefficients of the adaptive filter
47:     for  $j \leftarrow 0$  to  $MIC\_NB - 1$  do
48:       for  $k \leftarrow 0$  to  $N\_COEFS - 1$  do
49:          $W[i][k] \leftarrow W[i][k] - \mu \times (r\_hat\_n[i][j][k] \times error[j])$ 
50:       end for
51:     end for
52:   end for
53:  $write(x\_filtered, \&output[output\_selected])$   $\triangleright$  Ouput the filtered noise to characterise the path

```

APPENDIX G

Syfala Command-Line Interface Reference

```
\$ syfala <command> [command-options]

\$ syfala help
\$ syfala version
\$ syfala tidy
\$ syfala clean
\$ syfala reset
\$ syfala import <path-to-file.zip>
\$ syfala export <name>
\$ syfala report <hls|any*>
\$ syfala log
\$ syfala test
\$ syfala flash
\$ syfala start-gui
\$ syfala open-project <hls|any*>

\$ syfala <file.dsp|file.cpp> [general-options]
                             [build-options]
                             [design-options]
                             [hls-options]
                             [arm-options]
                             [run-step]

# examples:
\$ syfala examples/faust/bypass.dsp
  --board Z10
  --linux --midi --osc --http

\$ syfala examples/cpp/templates/gain-control-hls.cpp
  --board Z20
  --arm-target examples/cpp/gain-control-arm.cpp
```

General Options

Name	Description	Arguments
--xilinx-root	sets XILINX_ROOT_DIR for the current build	path
--xversion	sets XILINX_VERSION for the current build	2020.2 or 2022.2*
--board, -b	Defines the target board: Digilent Zybo Z7-10/20 or Genesys ZU-3EG	Z10* or Z20 or GENESYS

Commands

Name	Description	Arguments
tidy	removes all temporary files generated by the toolchain	none
clean	deletes current build directory	none
reset	deletes current build directory as well as the syfala_log & resets the current toolchain configuration	none
import	sets previously exported .zip build as the current build	path to the .zip build
export	exports current build in a .zip file located in the 'export' directory	name of the build
report	displays HLS or global report	<hls or any*>
log	displays the current build's full log	none
test	builds & runs all toolchain tests	none
flash	flashes current build onto target device	none
start-gui	executes the Faust-generated GUI application	none
open-project	opens the generated .xpr project with Vivado	<hls or any*>
help	prints list of available commands, options, and run-time parameters	none
version	displays the current script's version	none

Build Options

Name	Description
--linux	builds the embedded linux if it doesn't already exist and exports the build in the root partition (/home/syfala/mybuild)
--midi	adds MIDI control for the Faust GUI and/or the Embedded Linux Control Application
--osc	adds OSC control for the Faust GUI and/or the Embedded Linux Control Application
--http	adds HTTP control for the Faust GUI and/or the Embedded Linux Control Application

Design Options

Name	Arguments	Description
--multisample	power of two integer (e.g., 16, 24, 32, etc.)	DSP block will compute a block of samples instead of a single one. This may improve overall throughput but will introduce audio I/O latency.
--sigma-delta	none	Builds the project with a sigma-delta DAC configuration (experimental)
--tdm	none	Builds the project with i2s TDM (experimental)
--ethernet	none	(Linux only) uses TCP/IP Ethernet to convey input/output signals from/to Faust

HLS Options

Name	Arguments	Description
--accurate-use	none	Runs HLS with the impl flow, shows more accurate resources/latency reports, but takes longer to run.
--csim	path to simulation .cpp file	Runs C simulation for the syfala DSP IP
--csim-iter	integer (1 to ...)	Sets the number of syfala calls during the C simulation
--csim-inputs	path to directory containing in0.txt, in1.txt, etc.	Set the directory containing input samples files (as .txt files). Each sample should be normalized floating point values going from -1.f to 1.f, separated by a white space or a line return.
--mcd	none	(Faust only) Max-copy-delay: threshold between copy and ring buffer implementation (default 16).
--unsafe-math-optimizations	--umo	Adds the Vitis HLS unsafe_math_optimizations directive to the syfala DSP IP.
--hls-flags	Tcl string	n/a

ARM Options

Name	Arguments	Description
--shield	adau or motherboard	Adds support for ADAU1777/ADAU1787 external codecs, or for the ADAU Motherboard
--benchmark	none	(Faust only) Enables benchmark for the ARM control-loop.
--arm-target	path to .cpp file	Selects the main (.cpp) source file for the ARM control application.
--controller-type	DEMO, PCB1*, PCB2, PCB3, PCB4	Defines the controller used to drive the controls when SW3 is UP.
--ssm-volume	FULL, HEADPHONE*, DEFAULT	(Zybo boards only) HEADPHONE: lower volume for headphone use. DEFAULT: default value +1dB, the true 0dB (0b001111001) decreases the signal slightly.
--ssm-speed	FAST, DEFAULT*	(Zybo boards only) changes SSM ADC/DAC sample rate. DEFAULT: 48kHz sample rate. FAST: 96kHz sample rate

Run Steps

Note: The `--all` is not necessary if you wish to run all steps, just run `syfala myfaustdsp.dsp`

<code>--all</code>	runs all toolchain compilation steps (from <code>--sources</code> to <code>--gui</code>)
<code>--sources</code>	uses Faust to generate ip/host cpp files for HLS and Host application compilation
<code>--hls</code>	runs Vitis HLS on generated IP cpp file
<code>--project</code>	generates Vivado project
<code>--synth</code>	synthesizes full Vivado project
<code>--host</code>	compiles Host application, exports sources and .elf output to <code>build/sw_export</code>
<code>--gui</code>	compiles Faust GUI control application
<code>--flash</code>	flashes boot files on device at the end of the run
<code>--report</code>	prints HLS report at the end of the run
<code>--export</code>	<id> exports build to <code>export/</code> directory at the end of the run

Run Parameters

Parameter	Accepted Values	Description	Default Value
<code>--memory, -m</code>	DDR or STATIC	(Faust only) Choose between DDR & static memory layout for faust delay-lines, rd/rwtables.	DDR
<code>--sample-rate</code>	48000, 96000, 192000, 384000, 768000	Changes sample rate value (Hz). Only 48kHz and 96kHz is available for SSM embedded codec. 192000 (ADAU1777 and ADAU1787 only), 384000 (ADAU1787 only), 768000 (ADAU1787 only and with <code>--sample-width 16</code> only)	48000
<code>--sample-width</code>	16, 24, 32	Defines sample bit depth (16, 24, 32)	16

Hardware Configuration (Zybo Z7-10/20)

Syfala Hardware Controller Board (SW3 UP)

If you use a Hardware Controller Board, please set the `--controller-type` command-line parameter to the proper value (see below)

Controller-Type Values Description

- **DEMO**: Popophone demo box
- **PCB1**: Emeraude PCB config 1: 4 knobs, 2 switches, 2 sliders (default)
- **PCB2**: Emeraude PCB config 2: 8 knobs
- **PCB3**: Emeraude PCB config 3: 4 knobs, 4 switches
- **PCB4**: Emeraude PCB config 4: 4 knobs above, 4 switches below
- **TEENSY**: Teensy-based controller.

You can **swap from hardware to software controller** during DSP execution by changing **SW3**.

Switch Description

Default configuration in **bold**:

SW3	SW2	SW1	SW0
Hard	ADAU	BYPASS	MUTE
GUI	SSM	USE DSP	UNMUTE

- **SW3**: Controller type select: hardware (Controller board) or software (GUI).
- **SW2**: Audio codec input select (ADAU=external or SSM=onboard). Does not affect output.
- **SW1**: Bypass audio dsp.
- **SW0**: Mute.

Status LEDs

The RGB LED indicates the program state:

- **BLUE**: waiting
- **GREEN**: all good!
- **ORANGE**: warning (bypass or mute enabled)
- **RED**: ERROR! (configuration failed or incompatible), could happen if you select the SSM codec with incompatible sample rate.

The 4 LEDs above the switches indicate the switches state. If one of them blinks, it indicates the source of the warning/error.

SD Card Files (Baremetal Configuration)

You can put the program on an SD card (if you want something reproducible and easily launchable, for the demos...). After a `make` command, you should see a `B00T.bin` file in `SW_export` (or you can build it with `make boot\file`). Put the file on the root of the SD card. And don't forget to put JP5 on 'SD' position!

Bibliography

- [Ada] Adafruit. MAX98357A Breakout Board. All URLs in this thesis were verified on October 10, 2024. <https://www.adafruit.com/product/3006>.
- [Ahr12] Jens Ahrens. *Analytic Methods of Sound Field Synthesis*. Springer Science & Business Media, January 2012.
- [ALG24] L. Alexandre, P. Lecomte, and M.-A. Galland. Experimental Active Noise Control Using Faust On FPGA: Comparison Between A Multi-Point And Spherical Harmonics Method. In *Proceedings of the 10th Convention of the European Acoustics Association Forum Acusticum 2023*, pages 2677–2684, Turin, Italy, January 2024. European Acoustics Association.
- [ALGP22] Loïc Alexandre, Pierre Lecomte, Marie-Annick Galland, and Maxime Popoff. Feedback Acoustic Noise Control with Faust on FPGA: Application to Noise Reduction in Headphones. In *International Faust Conference*, Saint-Étienne, France, June 2022. Zenodo.
- [ALRM⁺] Ghattas Akkad, Frederic Le Roy, Ali Mansour, Bachar ElHassan, and Mohamad Najem. FFT acceleration techniques using HLS and HDL for digital communication systems.
- [Alt] Altera. FPGA Architecture. https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/wp/wp-01003.pdf.
- [AMD] AMD. Zynq 7000 Product Page. <https://www.amd.com/fr/products/adaptive-socs-and-fpgas/soc/zynq-7000.html>.
- [Anaa] Analog Device. Which ADC Architecture Is Right for Your Application? <https://www.analog.com/en/resources/analog-dialogue/articles/the-right-adc-architecture.html>.
- [Anab] Analog Devices. AD4030-24 ADC. <https://www.analog.com/en/products/ad4030-24.html>.
- [Anac] Analog Devices. ADAU1787 Audio Codec. <https://www.analog.com/en/products/adau1787.html>.
- [Anad] Analog Devices. ADAU1787 Evaluation Board. <https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/eval-adau1787z.html>.
- [Anae] Analog Devices. MAX98357A Audio Codec. <https://www.analog.com/en/products/max98357a.html>.
- [Anaf] Analog Devices. SSM2603 Audio Codec. <https://www.analog.com/en/products/ssm2603.html>.
- [Anag] Analog Devices. SSM2604 Audio Codec. <https://www.analog.com/en/products/ssm2604.html>.
- [Anah] Analog Devices. Technical Forum for SSM2603 bug. <https://ez.analog.com/audio/f/q-a/543726/solved-ssm2603-right-output-offset-issue>.
- [Ant] Antelope Audio. Antelope Audio Official Website. <https://en.antelopeaudio.com/>.
- [APM⁺21] Puya Amiri, Arsène Pérard-Gayot, Richard Membarth, Philipp Slusallek, Roland Leißa, and Sebastian Hack. FLOWER: A comprehensive dataflow compiler for high-level synthesis. In *2021 International Conference on Field-Programmable Technology (ICFPT)*, pages 1–9, December 2021.
- [Arn] Arnaud Tisserand. Arithmetic for Crypto in FPGA: HDL or HLS? <https://hal.science/hal-04126074v1/file/wrac23.pdf>.
- [Aud] Antelope Audio. Synergy Core. <https://en.antelopeaudio.com/synergy-core/>.
- [BAC17] Souha Boussemlî, Nouredine Aloui, and Adnen Cherif. DSP Real-Time Implementation of an Audio Compression Algorithm by using the Fast Hartley Transform. *International Journal of Advanced Computer Science and Applications*, 8(4), 2017.
- [BB21] Andrew Boutros and Vaughn Betz. FPGA Architecture: Principles and Progression. *IEEE Circuits and Systems Magazine*, 21(2):4–29, 2021.
- [BBL⁺21] Jose A. Belloch, José M. Badía, Diego F. Larios, Enrique Personal, Miguel Ferrer, Laura Fuster, Mihaita Lupoiu, Alberto Gonzalez, Carlos León, Antonio M. Vidal, and Enrique S. Quintana-Ortí. On the performance of a GPU-based SoC in a distributed spatial audio system. *The Journal of Supercomputing*, 77(7):6920–6935, July 2021.

- [BBS⁺14] Jose A. Belloch, Balázs Bank, Lauri Savioja, Alberto Gonzalez, and Vesa Välimäki. Multi-channel IIR filtering of audio signals using a GPU. In 2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 6692–6696, May 2014.
- [BCA⁺19] Alessio Balsini, Tommaso Cucinotta, Luca Abeni, Joel Fernandes, Phil Burk, Patrick Bellasi, and Morten Rasmussen. Energy-efficient low-latency audio on android. Journal of Systems and Software, 152:182–195, June 2019.
- [BCL22] Ilias Bournias, Roselyne Chotin, and Lionel Lacassagne. Using HLS for Designing a Parametric Optical Flow Hierarchical Algorithm in FPGAs. In 2022 IEEE International Symposium on Circuits and Systems (ISCAS), pages 1600–1604, May 2022.
- [BCS⁺15] Destiny L. Babjack, Brandon Cernicky, Andrew J. Sobotka, Lee Basler, Devon Struthers, Richard Kisic, Kimberly Barone, and Anthony P. Zuccolotto. Reducing audio stimulus presentation latencies across studies, laboratories, and hardware and operating system configurations. Behavior Research Methods, 47(3):649–665, September 2015.
- [BDP⁺23] Diego Bert, Nicola Domini, Riccardo Peloso, Leonardo Severi, Matteo Sacchetto, Andrea Bianco, and Cristina Rottondi. FPGA-based Low-Latency Audio Coprocessor for Networked Music Performance. In 2023 4th International Symposium on the Internet of Sounds, pages 1–8, October 2023.
- [BdV93] A. J. Berkhout, D. de Vries, and P. Vogel. Acoustic control by wave field synthesis. The Journal of the Acoustical Society of America, 93(5):2764–2778, May 1993.
- [Bel] Bela. Bela Embedded Computing Platform Official Website. <https://bela.io/>.
- [BG92] Gérard Berry and Georges Gonthier. The Esterel synchronous programming language: Design, semantics, implementation. Science of Computer Programming, 19(2):87–152, November 1992.
- [BG98] Eric Benjamin and Benjamin Gannon. Theoretical and audible effects of jitter on digital audio quality. In Audio Engineering Society Convention 105. Audio Engineering Society, 1998.
- [BGMV11] Jose A. Belloch, Alberto Gonzalez, F. J. Martínez-Zaldívar, and Antonio M. Vidal. Real-time massive convolution for audio applications on GPU. The Journal of Supercomputing, 58(3):449–457, December 2011.
- [BGQ⁺17] Jose A. Belloch, Alberto Gonzalez, Enrique S. Quintana-Ortí, Miguel Ferrer, and Vesa Välimäki. GPU-Based Dynamic Wave Field Synthesis Using Fractional Delay Filters and Room Compensation. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 25(2):435–447, February 2017.
- [BJ10] Carlos D. Bula and Manuel Jiménez. DAC topologies for GSM $\Sigma\Delta$ modulators. In 2010 IEEE ANDESCON, pages 1–5, September 2010.
- [BJ17] Edgar Berdahl and Wendy Ju. 2011: Satellite CCRMA: A Musical Interaction and Sound Synthesis Platform. In Alexander Refsum Jensenius and Michael J. Lyons, editors, A NIME Reader, volume 3, pages 373–389. Springer International Publishing, Cham, 2017.
- [BK05] James Bryant and Walt Kester. Data converter architectures. Analog-Digital Conversion, pages 1–14, 2005.
- [BKL⁺11] Peter Brinkmann, Peter Kirn, Richard Lawler, Chris McCormick, Martin Roth, and Hans-Christoph Steiner. Embedding pure data with libpd. In Proceedings of the Pure Data Convention, volume 291. Citeseer, 2011.
- [BKPE09] Kyungjin Byun, Young-Su Kwon, Seongmo Park, and Nak-Woong Eum. Digital Audio Effect System-on-a-Chip Based on Embedded DSP Core. ETRI Journal, 31(6):732–740, 2009.
- [BL10] Richard Boulanger and Victor Lazzarini. The audio programming book. the MIT Press, 2010.
- [BLGJ91] Albert Benveniste, Paul Le Guernic, and Christian Jacquemot. Synchronous programming with events and relations: The SIGNAL language and its semantics. Science of Computer Programming, 16(2):103–149, September 1991.
- [Bou00] Richard Boulanger. The Csound book: perspectives in software synthesis, sound design, signal processing, and programming. MIT press, 2000.
- [Bou23] Ilias Bournias. Design Space Exploration of Image Processing Algorithms on FPGAs. PhD thesis, Sorbonne Université, June 2023.
- [BQ13] André Jucovsky Bianchi and Marcelo Queiroz. Real time digital audio processing using Arduino. In Proceedings of the Sound and Music Computing Conference, Stockholm, Sweden, pages 538–545, 2013.
- [BW12] Stefan Bilbao and Craig Webb. Timpani drum synthesis in 3D on gpgpus. In Proc. Of the 15th Int. Conference on Digital Audio Effects (DAFx-12), York, United Kingdom, 2012.
- [BW13] Stefan Bilbao and Craig J Webb. Physical modeling of timpani drums in 3D on GPGPUs. Journal of the Audio Engineering Society, 61(10):737–748, 2013.
- [CA22] Seung-Hun Chung and Tarek S Abdelrahman. A compilation flow for the generation of CNN inference accelerators on FPGAs. arXiv preprint arXiv:2203.04015, 2022.
- [CFS22] Dean Cannon, Tianyang Fang, and Jafar Saniie. Modular Delay Audio Effect System on FPGA. In 2022 IEEE International Conference on Electro Information Technology (eIT), pages 248–251, May 2022.
- [Chi] ChipsAlliance. Chisel Language. <https://www.chisel-lang.org/>.

- [CHZ⁺19] Yao Chen, Jiong He, Xiaofan Zhang, Cong Hao, and Deming Chen. Cloud-DNN: An Open Framework for Mapping DNN Models to Cloud FPGAs. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, FPGA '19, pages 73–82, New York, NY, USA, February 2019. Association for Computing Machinery.
- [CO19] Chris Chafe and Scott Oshiro. Jacktrip on Raspberry Pi. Stanford University, 2019.
- [Cou19] Philippe Coussy. GAUT: An open-source HLS tool. In *Free Silicon Conference (FSiC)*, Paris, France, March 2019.
- [CPF⁺23] Pierre Cochard, Maxime Popoff, Antoine Fraboulet, Tanguy Risset, Stéphane Letz, and Romain Michon. A Programmable Linux-Based FPGA Platform for Audio DSP. In *Proceedings of the 2023 Sound and Music Computing Conference (SMC-23)*, pages 110–116. Bresin, R., & Falkenberg, K., June 2023.
- [CPG⁺15] Sujit Rokka Chhetri, Bikash Poudel, Sandesh Ghimire, Shaswot Shresthamali, and Dinesh Kumar Sharma. Implementation of Audio Effect Generator in FPGA. *Nepal Journal of Science and Technology*, 15(1):89–98, February 2015.
- [CPMR24] Pierre Cochard, Maxime Popoff, Romain Michon, and Tanguy Risset. Programming FPGA Platforms for Real-Time Audio Signal Processing In C++. In *2024 Sound and Music Computing Conference (SMC-24)*, Porto, Portugal, July 2024.
- [CT98] Jing Chen and Heng-Ming Tai. Real-time implementation of the MPEG-2 audio codec on a DSP. *IEEE Transactions on Consumer Electronics*, 44(3):866–871, August 1998.
- [CVPB16] Nitin Chugh, Vinay Vasista, Suresh Purini, and Uday Bondhugula. A DSL Compiler for Accelerating Image Processing Pipelines on FPGAs. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation, PACT '16*, pages 327–338, New York, NY, USA, September 2016. Association for Computing Machinery.
- [CWM⁺24] Pierre Cochard, Jurek Weber, Romain Michon, Tanguy Risset, and Stéphane Letz. Ethernet Real-Time Audio Transmission to FPGA. In *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*, September 2024.
- [Dana] Dante. Dante For Audio, IP Core. <https://www.audinate.com/products/manufacture-products/dante-ip-core>.
- [Danb] Dante. Dante-MY16-AUD2 Board. <https://fr.yamaha.com/fr/products/proaudio/interfaces/dante-my16-aud2/index.html>.
- [DASP21] Ciprian Dragoi, Cristian Anghel, Cristian Stanciu, and Constantin Paleologu. Efficient FPGA Implementation of Classic Audio Effects. In *2021 13th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*, pages 1–6, July 2021.
- [DB01] Sofia Dahl and Roberto Bresin. Is the player more influenced by the auditory than the tactile feedback from the instrument. In *Proceedings of the Digital Audio Effects Conference (Dafx)*, pages 6–9. Citeseer, 2001.
- [DBV19] Fotios Drakopoulos, Deepak Baby, and Sarah Verhulst. Real-time audio processing on a raspberry Pi using deep neural networks. In *Proceedings of the 23rd International Congress on Acoustics : Integrating 4th EAA Euroregio 2019 : 9-13 September 2019 in Aachen, Germany*, pages 2827–2834. Deutsche Gesellschaft für Akustik, 2019.
- [Diga] Digikey. I2S Transceiver IP. <https://forum.digikey.com/t/i2s-transceiver-vhdl/12845>.
- [Digb] Digilent. Genesys ZU Product Page. <https://digilent.com/reference/programmable-logic/genesys-zu/start>.
- [DK20] Apurva S. Deulkar and Neelima R. Kolhare. FPGA implementation of audio and video processing based on Zedboard. In *2020 International Conference on Smart Innovations in Design, Environment, Management, Planning and Computing (ICSIDEMPC)*, pages 305–310, Aurangabad, India, October 2020. IEEE.
- [DL22] Raphaël Dingé and Stéphane Letz. Automatically generating eurorack hardware running faust programs using eurorack-blocks. In *IFC 22-International Faust Conference*, 2022.
- [DML⁺21] Department of Electronics, Faculty of Technology, University of Laghouat, Laghouat 03000, Algeria, L. Merah, P. Lorenz, University of Haute Alsace IUT, 68008 Colmar, France, A. Ali-Pacha, niversity of Science and Technology of Oran (USTO), Oran 31036, Algeria, N. Hadj-Said, and University of Science and Technology of Oran (USTO), Oran 31036, Algeria. A Guide on Using Xilinx System Generator to Design and Implement Real-Time Audio Effects on FPGA. *International Journal of Future Computer and Communication*, pages 38–44, September 2021.
- [DMR⁺16] Mickaël Dardaillon, Kevin Marquet, Tanguy Risset, Jérôme Martin, and Henri-Pierre Charles. A New Compilation Flow for Software-Defined Radio Applications on Heterogeneous MPSoCs. *ACM Trans. Archit. Code Optim.*, 13(2):19:1–19:25, June 2016.
- [DSPS16] Gianluca C Durelli, Fabrizio Spada, Christian Pilato, and Marco D Santambrogio. Scala-based domain-specific language for creating accelerator-based socs. In *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 225–232. IEEE, 2016.
- [DZ02] P. Dutilleux and U. Zölzer. *Nonlinear Processing*, chapter 5, pages 93–136. John Wiley & Sons, Ltd, 2002.
- [EF07] N. Epain and E. Priot. Active control of sound inside a sphere via control of the acoustic pressure at the boundary surface. *Journal of Sound and Vibration*, 299(3):587–604, January 2007.
- [Elk] Elk Audio. Elk DevKit Documentation. https://elk-audio.github.io/elk-docs/html/documents/elk_pi_datasheet.html.

- [Ell00] Stephen Elliott. Signal processing for active control. Elsevier, 2000.
- [Emea] Emeraude Team. Syfala Official Documentation. <https://inria-embede.github.io/syfala/>.
- [Emeb] Emeraude Team. Syfala Repository. <https://github.com/inria-embede/syfala>.
- [ES20] Yunus Emre ESEN and İsmail SAN. LowLAG: Low-latency hardware accelerator of a sound effect with system-on-chip design. In 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), pages 1–6, October 2020.
- [ES21] Yunus Emre Esen and İsmail San. Low-Latency SoC Design with High-Level Accelerators Specific to Sound Effects. International Journal of Advances in Engineering and Pure Sciences, 33:78–87, December 2021.
- [Fas] Faselunar. Microcosmos Board. <https://www.faselunare.com/products/instruments/microcosmos>.
- [FF15] Jeffrey Fredenburg and Michael P. Flynn. ADC trends and impact on SAR ADC architecture and analysis. In 2015 IEEE Custom Integrated Circuits Conference (CICC), pages 1–8, September 2015.
- [Fow10] Martin Fowler. Domain-specific languages. Pearson Education, 2010.
- [FS95] Anders Friberg and Johan Sundberg. Time discrimination in a monotonic, isochronous sequence. The Journal of the Acoustical Society of America, 98(5):2524–2531, November 1995.
- [fut] futur3soundz. futur3soundz DIY FPGA Synthesizer Official Website. <https://www.futur3soundz.com/>.
- [GLMC17] Petko Georgiev, Nicholas D. Lane, Cecilia Mascolo, and David Chu. Accelerating Mobile Audio Sensing Algorithms through On-Chip GPU Offloading. In Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys '17, pages 306–318, New York, NY, USA, June 2017. Association for Computing Machinery.
- [Goe19] Johannes Goebel. The EMPAC High-Resolution Modular Loudspeaker Array for Wave Field Synthesis. Universitätsbibliothek der RWTH Aachen, 2019.
- [Graa] Game-CNCM. Amstramgame Website. <https://www.amstramgame.fr/gramophone/programs/>.
- [Grab] Game-CNCM. Faust Documentation, Quick Start guide. <https://faustdoc.game.fr/manual/quick-start/>.
- [Grac] Game-CNCM. Faust IDE. <https://faustide.game.fr/>.
- [Grad] Game-CNCM. Faust Website. <https://faust.game.fr/>.
- [Gro08] Ian Grout. Chapter 1 - introduction to programmable logic. In Ian Grout, editor, Digital Systems Design with FPGAs and CPLDs, pages 1–41. Newnes, Burlington, 2008.
- [HAY⁺12] Takaaki Hori, Shoko Araki, Takuya Yoshioka, Masakiyo Fujimoto, Shinji Watanabe, Takanobu Oba, Atsunori Ogawa, Kazuhiro Otsuka, Dan Mikami, Keisuke Kinoshita, Tomohiro Nakatani, Atsushi Nakamura, and Junji Yamato. Low-Latency Real-Time Meeting Recognition and Understanding Using Distant Microphones and Omni-Directional Camera. IEEE Transactions on Audio, Speech, and Language Processing, 20(2):499–513, February 2012.
- [HBW14] Brian Hamilton, Stefan Bilbao, and Craig Webb. Improved Finite Difference Schemes for a 3-D Viscothermal Wave Equation on a GPU. In Proceedings of Forum Acusticum, September 2014.
- [HCRP91] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. The synchronous data flow programming language LUSTRE. Proceedings of the IEEE, 79(9):1305–1320, September 1991.
- [Heaa] Allen & Heath. From DSP to FPGA: a Signal Processing Revolution. <https://www.allen-heath.com/content/uploads/2023/11/XCVI.pdf>.
- [Heab] Allen & Heath. SQ Mixers. <https://www.allen-heath.com/hardware/sq/>.
- [Hol] Holoplot. X1 System. <https://holoplot.com/products/systems/x1/>.
- [HW13] Brian Hamilton and Craig J Webb. Room acoustics modelling using GPU-accelerated finite difference and finite volume methods on a face-centered cubic grid. Proc. Digital Audio Effects (DAFx), Maynooth, Ireland, pages 336–343, 2013.
- [HW14] Aristotelis Hadjakos and Simon Waloschek. SPINE: A TUI toolkit and physical computing hybrid. In NIME, pages 625–628, 2014.
- [IEE06] IEEE Standard for Verilog Hardware Description Language. IEEE Std 1364-2005 (Revision of IEEE Std 1364-2001), pages 1–590, April 2006.
- [IEE12] IEEE Standard for Standard SystemC Language Reference Manual. IEEE Std 1666-2011 (Revision of IEEE Std 1666-2005), pages 1–638, January 2012.
- [IEE19] IEEE Standard for VHDL Language Reference Manual. IEEE Std 1076-2019, pages 1–673, December 2019.
- [JAS07] Nicolas Juillerat, Stefan Müller Arisona, and Simon Schubiger-Banz. Real-time, low latency audio processing in Java. In ICMC, 2007.
- [JCFS13] Stefan Jaritz, The Previous Dafx Crew, Joseph Fourier, and Claude Shannon. Digital audio device creation by the use of a domain specific language and a hardware abstraction layer. 2013.

- [JGS15] Jingbo Zhang, Ganggang Ning, and Shufang Zhang. Design of audio signal processing and display system based on SoC. In 2015 4th International Conference on Computer Science and Network Technology (ICCSNT), pages 824–828, Harbin, China, December 2015. IEEE.
- [JSA08] Nicolas Juillerat, Simon Schubiger-Banz, and Stefan Muller Arisona. Low latency audio pitch shifting in the time domain. In 2008 International Conference on Audio, Language and Image Processing, pages 29–35, July 2008.
- [JSP20] Kundan Manohar Jadhao and Govind Singh Patel. Hardware Architecture of Digital Drum Kit Using FPGA. In 2020 IEEE International Conference on Advent Trends in Multidisciplinary Research and Innovation (ICATMRI), pages 1–4, Buldhana, India, December 2020. IEEE.
- [JVH⁺04] Wesley M. Johnston, View Profile, J. R. Paul Hanna, View Profile, Richard J. Millar, and View Profile. Advances in dataflow programming languages. ACM Computing Surveys, 36(1):1–34, March 2004.
- [KBL14] Topi Kaaresoja, Stephen Brewster, and Vuokko Lantz. Towards the Temporally Perfect Virtual Button: Touch-Feedback Simultaneity and Perceived Quality in Mobile Touchscreen Press Interactions. ACM Trans. Appl. Percept., 11(2):9:1–9:25, June 2014.
- [Kes05] Walt Kester. Data conversion handbook. Newnes, 2005.
- [KFP⁺18] David Koeplinger, Matthew Feldman, Raghu Prabhakar, Yaqi Zhang, Stefan Hadjis, Ruben Fiszal, Tian Zhao, Luigi Nardi, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. Spatial: A language and compiler for application accelerators. In Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, pages 296–311, New York, NY, USA, June 2018. Association for Computing Machinery.
- [KKTS21] Alexander Klemd, Bernd Klauer, Johannes Timmermann, and Delf Sachau. A Flexible Multi-Channel Feedback FxLMS Architecture for FPGA Platforms. In 2021 31st International Conference on Field-Programmable Logic and Applications (FPL), pages 319–326, August 2021.
- [KTR08] Ian Kuon, Russell Tessier, and Jonathan Rose. FPGA Architecture: Survey and Challenges. Foundations and Trends® in Electronic Design Automation, 2(2):135–253, April 2008.
- [LA] L-Acoustics. L-ISA Technology. <https://l-isa.l-acoustics.com/>.
- [LB07] Michael Lester and Jon Boley. The effects of latency on live sound monitoring. In Audio Engineering Society Convention 123. Audio Engineering Society, 2007.
- [LCZ⁺12] Yun Liang, Zheng Cui, Shengkui Zhao, Kyle Rupnow, Yihao Zhang, Douglas L. Jones, and Deming Chen. Real-time implementation and performance optimization of 3D sound localization on GPUs. In 2012 Design, Automation & Test in Europe Conference & Exhibition (DATE), pages 832–835, March 2012.
- [LFdG14] Jorge Lorente, Miguel Ferrer, Maria de Diego, and Alberto González. GPU Implementation of Multichannel Adaptive Algorithms for Local Active Noise Control. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 22(11):1624–1635, November 2014.
- [LK04] Nelson Posse Lago and Fabio Kon. The quest for low latency. In ICMC, 2004.
- [LM87] E.A. Lee and D.G. Messerschmitt. Synchronous data flow. Proceedings of the IEEE, 75(9):1235–1245, September 1987.
- [LMDS18] N. Lekic, Z. Mijanovic, R. Dragovic-Ivanovic, and Lj. Stankovic. Binary Weighted DAC with 2- ξ Resistor Ratio. Radioengineering, 27(2):501–509, June 2018.
- [LOF18] Stéphane Letz, Yann Orlarey, and Dominique Fober. FAUST Domain Specific Audio DSP Language Compiled to WebAssembly. In Companion Proceedings of the The Web Conference 2018, WWW '18, pages 701–709, Republic and Canton of Geneva, CHE, April 2018. International World Wide Web Conferences Steering Committee.
- [LVVN20] Pavel Lyakhov, Maria Valueva, Georgii Valuev, and Nikolai Nagornov. High-Performance Digital Filtering on Truncated Multiply-Accumulate Units in the Residue Number System. IEEE Access, 8:209181–209190, 2020.
- [LWB15] Bernhard Lehner, Gerhard Widmer, and Sebastian Bock. A low-latency, real-time-capable singing voice detection method with LSTM recurrent neural networks. In 2015 23rd European Signal Processing Conference (EUSIPCO), pages 21–25, August 2015.
- [LYH⁺16] Victor Lazzarini, Steven Yi, Joachim Heintz, Øyvind Brandtsegg, Iain McCurdy, et al. Csound: a sound and music computing system. Springer, 2016.
- [Man16] Vincent J Manzo. Max/MSP/Jitter for music: A practical guide to developing interactive music systems for education and more. Oxford University Press, 2016.
- [Mar65] James Martin. Programming Real-Time Computer Systems. -. Englewood Cliffs, N.J.: Prentice-Hall, 1965.
- [Mat] MathWorks. HDL Coder. <https://www.mathworks.com/products/hdl-coder.html>.
- [MBPR23] Romain Michon, Joseph Bizien, Maxime Popoff, and Tanguy Risset. Making Frugal Spatial Audio Systems Using Field-Programmable Gate Arrays. In Proceedings of the International Conference on New Interfaces for Musical Expression (NIME-23), pages 54–59. Zenodo, May 2023.
- [McC02] James McCartney. Rethinking the Computer Music Language: Super Collider. Computer Music Journal, 26(4):61–68, 2002.

- [McP17] Andrew McPherson. Bela: An embedded platform for low-latency feedback control of sound. *The Journal of the Acoustical Society of America*, 141(5_Supplement):3618, May 2017.
- [MJM⁺16] Andrew P McPherson, Robert H Jack, Giulio Moro, et al. Action-sound latency: Are our tools fast enough? 2016.
- [MLN22] Oliver Munroe, Stephane Letourneur, and Antonin Novak. Design of an electronic circuit for loudspeaker real-time digital signal processing. *arXiv preprint arXiv:2206.07516*, 2022.
- [MM64] Max V Mathews and Joan E Miller. *Music IV programmer's manual*. Bell Telephone Laboratories, Incorporated, 1964.
- [Mol09] Yaghoub Mollaei. Hardware implementation of adaptive filters. In *2009 IEEE Student Conference on Research and Development (SCORED)*, pages 45–48, November 2009.
- [MOL⁺20a] Romain Michon, Yann Orlarey, Stéphane Letz, Dominique Fober, and Dirk Roosenburg. Embedded real-time audio signal processing with faust. In *International Faust Conference (IFC-20)*, 2020.
- [MOL⁺20b] Romain Michon, Daniel Overholt, Stéphane Letz, Yann Orlarey, Dominique Fober, and Catinca Dumitrascu. A faust architecture for the ESP32 microcontroller. In *Sound and Music Computing Conference (SMC-20)*, 2020.
- [MOLF19] Romain Michon, Yann Orlarey, Stéphane Letz, and Dominique Fober. Real time audio digital signal processing with Faust and the Teensy. In *Sound and Music Computing Conference (SMC-19)*, 2019.
- [MOT⁺21] Keisuke Matsubara, Takuma Okamoto, Ryoichi Takashima, Tetsuya Takiguchi, Tomoki Toda, Yoshinori Shiga, and Hisashi Kawai. Full-Band LPCNet: A Real-Time Neural Vocoder for 48 kHz Audio With a CPU. *IEEE Access*, 9:94923–94933, 2021.
- [MPSD20] Tomasz Marciniak, Kacper Podbucki, Jakub Suder, and Adam Dąbrowski. Analysis of Digital Filtering with the Use of STM32 Family Microcontrollers. In Andrzej Bartoszewicz, Jacek Kabziński, and Janusz Kacprzyk, editors, *Advanced, Contemporary Control*, pages 287–295, Cham, 2020. Springer International Publishing.
- [MRW95] Gerard Mangiante, Alain Roure, and Muriel Winninger. Multiprocessor controller for active noise and vibration control. In *INTER-NOISE and NOISE-CON Congress and Conference Proceedings*, volume 1995, pages 1183–1192. Institute of Noise Control Engineering, 1995.
- [MSAA01] Mihai V Micea, Mircea Stratulat, Dan Ardelean, and Daniel Aioanei. Implementing professional audio effects with DSPs. *Trans. Autom. Control Comput. Sci.*, 60(46):55–60, 2001.
- [MSKW24] Ramon Moscatelli, Kevin Stahel, Robert Kraneis, and Christian Werner. Why Real-Time Matters: Performance Evaluation of Recent Ultra-Low Latency Audio Communication Systems. In *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*, pages 77–83, January 2024.
- [MSL⁺23] Romain Michon, Julien Sourice, Victor Lazzarini, Joseph Timoney, and Tanguy Risset. Towards High Sampling Rate Sound Synthesis On FPGA. In *Proceedings of the 2023 Digital Audio Effects Conference (DAFx23)*, September 2023.
- [MWBM07] Erdem Motuk, Roger Woods, Stefan Bilbao, and John McAllister. Design Methodology for Real-Time FPGA-Based Sound Synthesis. *IEEE Transactions on Signal Processing*, 55(12):5833–5845, December 2007.
- [MZ15] Andrew McPherson and Victor Zappi. An environment for submillisecond-latency audio and sensor processing on BeagleBone Black. In *Audio Engineering Society Convention 138*. Audio Engineering Society, 2015.
- [NCW12] Markus Noisternig, Thibaut Carpentier, and Olivier Warusfel. Dispositif de spatialisation sonore 3Da l'Espace de Projection de l'IRCAM: Un Réseau de 345 haut-parleurs pour une restitution par WFS et HOA. *Acoustique et Techniques*, 71:30–39, 2012.
- [NIa] NI. NI-9220 DAC. <https://www.ni.com/fr-fr/shop/model/ni-9220.html>.
- [NIb] NI. NI-9264 DAC. <https://www.ni.com/fr-fr/shop/model/ni-9264.html>.
- [Nov] Novation Music. Summit Keyboard. <https://novationmusic.com/en/synths/summit>.
- [NSP⁺16] Razvan Nane, Vlad-Mihai Sima, Christian Pilato, Jongsok Choi, Blair Fort, Andrew Canis, Yu Ting Chen, Hsuan Hsiao, Stephen Brown, Fabrizio Ferrandi, Jason Anderson, and Koen Bertels. A Survey and Evaluation of FPGA High-Level Synthesis Tools. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 35(10):1591–1604, October 2016.
- [NXP] NXP. 24-Channel LED Driver. <https://www.nxp.com/docs/en/data-sheet/PCA9956A.pdf>.
- [OFL04] Y. Orlarey, D. Fober, and S. Letz. Syntactical and semantical aspects of Faust. *Soft Computing*, 8(9):623–632, September 2004.
- [OK17] Jose Raul Garcia Ordaz and Dirk Koch. HLS Compilation for CPU Interlays. In *Proceedings of the 8th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies, HEART '17*, pages 1–6, New York, NY, USA, June 2017. Association for Computing Machinery.
- [P⁺96] Miller Puckette et al. Pure Data: Another integrated computer music environment. *Proceedings of the second intercollege computer music concerts*, pages 37–41, 1996.
- [PB16] Florian Pfeifle and Rolf Bader. Real-time finite-difference method physical modeling of musical instruments using field-programmable gate array hardware. *Journal of the Audio Engineering Society*, 63(12):1001–1016, 2016.

- [PBY⁺17] Jing Pu, Steven Bell, Xuan Yang, Jeff Setter, Stephen Richardson, Jonathan Ragan-Kelley, and Mark Horowitz. Programming Heterogeneous Systems from an Image Processing DSL. *ACM Trans. Archit. Code Optim.*, 14(3):26:1–26:25, August 2017.
- [PC19] Elisabeth Pongratz and Roshan Cherian. Performance Evaluation of MathWorks HDL Coder as a Vendor Independent DFE Generation. 2019.
- [PCW⁺18] Mirco Pezzoli, Luca Comanducci, Joe Waltz, Anthony Agnello, Luca Bondi, Antonio Canclini, and Augusto Sarti. A dante powered modular microphone array system. In *Audio Engineering Society Convention 145*. Audio Engineering Society, 2018.
- [pjra] pjrc. Audio Adaptor Boards for Teensy 3.x and Teensy 4.x. https://www.pjrc.com/store/teensy3_audio.html.
- [pirb] pjrc. K20P64M50SF0 Teensy 3 Datasheet. <https://www.pjrc.com/teensy/K20P64M50SF0.pdf>.
- [PMR⁺22] Maxime Popoff, Romain Michon, Tanguy Risset, Yann Orlarey, and Stéphane Letz. Towards an FPGA-Based Compilation Flow for Ultra-Low Latency Audio Signal Processing. In *Proceedings of the 2022 Sound and Music Computing Conference (SMC-22)*, pages 555–562, June 2022.
- [PMR⁺23a] Maxime Popoff, Romain Michon, Tanguy Risset, Pierre Cochard, Stephane Letz, Yann Orlarey, and Florent De Dinechin. Audio DSP to FPGA Compilation. In *Proceedings of the 2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 31–32, July 2023.
- [PMR⁺23b] Maxime Popoff, Romain Michon, Tanguy Risset, Pierre Cochard, Stephane Letz, Yann Orlarey, and Florent de Dinechin. *Audio DSP to FPGA Compilation: The Syfala Toolchain Approach*. Report, Univ Lyon, INSA Lyon, Inria, CITI, Grame, Emeraude, May 2023.
- [PMR24] Maxime Popoff, Romain Michon, and Tanguy Risset. Enabling Affordable and Scalable Audio Spatialization With Multi-channel Audio Expansion Boards for FPGA. In *2024 Sound and Music Computing Conference (SMC-24)*, Porto, Portugal, July 2024.
- [PMS⁺07] Markus Pfaff, David Malzner, Johannes Seifert, Johannes Traxler, Horst Weber, and Gerhard Wiendl. Implementing digital audio effects using a hardware/software co-design approach. In *10th International Conference on Digital Audio Effects*, pages 1–8, 2007.
- [PPC⁺07] Lorenzo Palestini, Paolo Peretti, Stefania Cecchi, Francesco Piazza, Ariano Lattanzi, and Ferruccio Bettarelli. Linear phase mixed FIR/IIR crossover networks: Design and real-time implementation. In *Audio Engineering Society Convention 123*. Audio Engineering Society, 2007.
- [PPP18] Baptiste Pauget, David J. Pearce, and Alex Potanin. Towards compilation of an imperative language for FPGAs. In *Proceedings of the 10th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, VMIL 2018*, pages 47–56, New York, NY, USA, November 2018. Association for Computing Machinery.
- [PSK⁺03] C. Pascual, Zukui Song, P.T. Krein, D.V. Sarwate, P. Midya, and W.J. Roeckner. High-fidelity PWM inverter for digital audio amplification: Spectral analysis, real-time DSP implementation, and results. *IEEE Transactions on Power Electronics*, 18(1):473–485, January 2003.
- [PW23] Thomas Pfitzinger and Hendrik Wöhrle. Embedded Real-Time Human Activity Recognition on an ESP32-S3 Microcontroller Using Ambient Audio Data. In *2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, volume 1, pages 459–464, September 2023.
- [Ras79] R. A. Rasch. Synchronization in Performed Ensemble Music. *Acta Acustica united with Acustica*, 43(2):121–131, September 1979.
- [Ren] Renesas. 551s Clock Buffer. <https://www.renesas.com/us/en/document/dst/551s-datasheet>.
- [RLO⁺20] Shihong Ren, Stephane Letz, Yann Orlarey, Romain Michon, Dominique Fober, Michel Buffa, and Jerome Lebrun. Using Faust DSL to develop custom, sample accurate DSP code and audio plugins for the Web browser. *Journal of the Audio Engineering Society*, 68(10):703–716, 2020.
- [RM90] Dean Rubine and Paul McAviney. Programmable Finger-Tracking Instrument Controllers. *Computer Music Journal*, 14(1):26–41, 1990.
- [RMO⁺20] Tanguy Risset, Romain Michon, Yann Orlarey, Stéphane Letz, Gero Müller, and Adeyemi Gbadamosi. Faust2fpga for ultra-low audio latency: Preliminary work in the syfala project. In *IFC 2020-Second International Faust Conference*. International Faust Conference, pages 1–9, 2020.
- [Roa23] Curtis Roads. *The computer music tutorial*, Second Edition. MIT press, 2023.
- [RSH⁺14] Oliver Reiche, Moritz Schmid, Frank Hannig, Richard Membarth, and Jürgen Teich. Code generation from a domain-specific language for C-based HLS of hardware accelerators. In *Proceedings of the 2014 International Conference on Hardware/Software Codesign and System Synthesis, CODES '14*, pages 1–10, New York, NY, USA, October 2014. Association for Computing Machinery.
- [RSR⁺13] Thomas Reussner, Christoph Sladeczek, Michael Rath, Sandra Brix, Karl Preidl, and Hermann Scheck. Audio network-based massive multichannel loudspeaker system for flexible use in spatial audio research. *Journal of the Audio Engineering Society*, 61(4):235–245, 2013.
- [Rus23] Thomas Albert Rushton. Networked microcontrollers for distributed audio spatialisation. 2023.

- [SA19a] Travis Skare and Jonathan Abel. Gpu-accelerated modal processors and digital waveguides. In Linux Audio Conference, 2019.
- [SA19b] Travis Skare and Jonathan Abel. Real-time modal synthesis of crash cymbals with nonlinear approximations, using a gpu. In Proc. 22nd Int. Conf. Dig. Audio Effects, 2019.
- [Sam] Samtec. FMC Standard. <https://www.samtec.com/standards/vita/fmc/>.
- [SAR⁺21] K. Ayyappa Swamy, Zachariah C. Alex, Prakash Ramachandran, T. Lazar Mathew, C. Sushma, and N. Padmaja. Real-time Implementation of Delay Efficient DCT Based Hearing Aid Algorithm Using TMS320C5505 DSP Processor. In 2021 Innovations in Power and Advanced Computing Technologies (i-PACT), pages 1–8, November 2021.
- [Sca] Scala. Scala Language Repository. <https://github.com/scala/scala3/releases/tag/3.4.0>.
- [See] Seeburg. X6 speaker. <https://www.seeburg.net/en/speakers/x-6/152>.
- [SGHL20] Dongyuan Shi, Woon-Seng Gan, Jianjun He, and Bhan Lam. Practical Implementation of Multichannel Filtered-x Least Mean Square Algorithm Based on the Multiple-Parallel-Branch With Folding Architecture for Large-Scale Active Noise Control. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 28(4):940–953, April 2020.
- [Sha49] C.E. Shannon. Communication in the Presence of Noise. Proceedings of the IRE, 37(1):10–21, January 1949.
- [Ska22] Travis Skare. GPU-accelerated drum kit synthesis plugin design. In Audio Engineering Society Convention 153. Audio Engineering Society, 2022.
- [SM20] Anish Singhani and Alexander Morrow. Real-Time Spatial 3D Audio Synthesis on FPGAs for Blind Sailing. In Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA '20, pages 104–110, New York, NY, USA, February 2020. Association for Computing Machinery.
- [Smi03] Steven Smith. Digital signal processing: a practical guide for engineers and scientists. Newnes, 2003.
- [Sni23] Ross K. Snider. Advanced Digital System Design using SoC FPGAs. Springer Cham, 01 2023.
- [Son] Sonic Emotion. The Wave I Sound Processor. <https://www.theatrical.com/sonic-emotion.html>.
- [SPA06] Andreas Spanias, Ted Painter, and Venkatraman Atti. Audio signal processing and coding. John Wiley & Sons, 2006.
- [Spe] Speedgoat. Speedgoat Official Website. <https://www.speedgoat.com/>.
- [SSK01] Zdenek Smekal, Jiri Schimmel, and Petr Krkavec. Optimizing digital musical effect implementation for harvard dsp architecture. In Proceedings of COST-G6 Conference on Digital Audio Effects Dafx, volume 1, pages 6–8. Citeseer, 2001.
- [SSK02] Jiri Schimmel, Zdenek Smekal, and Petr Krkavec. Optimizing digital musical effect implementation for multiple processor DSP systems. In Proceedings of the 5th International Conference on Digital Audio Effects Dafx, volume 2, pages 81–84, 2002.
- [SSTS20] Takaaki Saeki, Yuki Saito, Shinnosuke Takamichi, and Hiroshi Saruwatari. Real-time, full-band, online DNN-based voice conversion system using a single CPU. In INTERSPEECH, pages 1021–1022, 2020.
- [ST⁺05] Richard Schreier, Gabor C. Temes, et al. Understanding delta-sigma data converters, volume 74. IEEE press Piscataway, NJ, 2005.
- [Sta24] Evert Start. Loudspeaker Matrix Arrays: Challenging the way we create and control sound. In Audio Engineering Society Conference: AES 2024 International Acoustics & Sound Reinforcement Conference. Audio Engineering Society, 2024.
- [Stu97] J Robert Stuart. Coding high quality digital audio. Japan Audio Society (1998 June), 1997.
- [SUL18] Yasmeen SULTANA. Open-Source Audio Platform for Embedded Systems. PhD thesis, UNIVERSITY OF APPLIED SCIENCES KIEL, 2018.
- [SVS11] Lauri Savioja, Vesa Välimäki, and Julius O Smith. Audio signal processing using graphics processing units. Journal of the Audio Engineering Society, 59(1/2):3–19, 2011.
- [SW20] Benjamin Carrion Schafer and Zi Wang. High-Level Synthesis Design Space Exploration: Past, Present, and Future. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 39(10):2628–2639, October 2020.
- [TB] TU-Berlin. Wave Field Synthesis-System. <https://abartetzki.users.ak.tu-berlin.de/wfsIntroduction.pdf>.
- [TCHM09] Farhat Thabet, Philippe Coussy, Dominique Heller, and Eric Martin. Exploration and Rapid Prototyping of DSP Applications using SystemC Behavioral Simulation and High-level Synthesis. Journal of Signal Processing Systems, 56(2):167–186, September 2009.
- [TDO06] Robert Trausmuth, Christian Dusek, and Yann Orlarey. Using faust for FPGA programming. In International Conference on Digital Audio Effects, pages 287–290, 2006.
- [Teca] Tech-Fest. Tech-Fest Official Website. <https://www.tech-fest.fr/>.
- [Tecb] Linear Technologies. LTC2380-24 ADC. <https://www.analog.com/en/products/ltc2380-24.html>.
- [Texa] Texas Instruments. ADS1675 ADC. <https://www.ti.com/product/ADS1675>.

- [Texb] Texas Instruments. LMK1C1104 4-Channel Buffer. <https://www.ti.com/product/LMK1C1104>.
- [Texc] Texas Instruments. LSF0204 4-Bit Bidirectional Multi-Voltage Level Translator. <https://www.ti.com/product/LSF0204>.
- [Texd] Texas Instruments. TCA9548A I2C Multiplexer. <https://www.ti.com/product/TCA9548A>.
- [TF21] Luca Turchet and Carlo Fischione. Elk Audio OS: An Open Source Operating System for the Internet of Musical Things. *ACM Transactions on Internet of Things*, 2(2):12:1–12:18, March 2021.
- [The] Thesis Programs Repository. <https://gitlab.inria.fr/mpopoff/thesis-programs>.
- [TI19] Yiyu Tan and Toshiyuki Imamura. An fpga-based accelerator for sound field rendering. In *Proceedings of the 22nd International Conference on Digital Audio Effects (Dafx-19)*, Birmingham, UK, 2019.
- [TIO13] Takao Tsuchiya, Yukio Iwaya, and Makoto Otani. Large-scale sound field rendering with graphics processing unit cluster for three-dimensional audio with loudspeaker array. *Proceedings of Meetings on Acoustics*, 19(1):040099, May 2013.
- [TKG11] Dimitris Theodoropoulos, Georgi Kuzmanov, and Georgi Gaydadjiev. Multi-Core Platforms for Beamforming and Wave Field Synthesis. *IEEE Transactions on Multimedia*, 13(2):235–245, April 2011.
- [TO09] Fernando Trebien and Manuel M. Oliveira. Realistic real-time sound re-synthesis and processing for interactive virtual worlds. *The Visual Computer*, 25(5):469–477, May 2009.
- [Tot] Total Theatre Audio. Stagetracker FX Documentation. <https://www.tta-sound.com/files/TTA%20Stagetracker%20FX.pdf>.
- [TZM⁺14] James William Topliss, Victor Zappi, Andrew McPherson, et al. Latency performance for real-time audio on BeagleBone Black. 2014.
- [UDO] UDO Audio. UDO Synthesizer Official Website. <https://www.udo-audio.com/#introduction>.
- [Van20] Trevor Charles Vannoy. *Enabling Rapid Prototyping of Audio Signal Processing Systems Using System-on-Chip Field Programmable Gate Arrays*. PhD thesis, Montana State University-Bozeman, Norm Asbjornson College of Engineering, 2020.
- [VB16] Stylianos I. Venieris and Christos-Savvas Bouganis. fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 40–47, May 2016.
- [VDD⁺19] Trevor Vannoy, Tyler Davis, Connor Dack, Dustin Sobrero, and Ross Snider. An open audio processing platform using soc fpgas and model-based development. In *Audio Engineering Society Convention 147*. Audio Engineering Society, 2019.
- [Ver82] Barry Vercoe. Computer systems and languages for audio research. In *Audio Engineering Society Conference: 1st International Conference: Digital Audio*. Audio Engineering Society, 1982.
- [VJY19] Kevin Vaca, Mitchell M. Jefferies, and Xiaokun Yang. An Open Audio Processing Platform with Zync FPGA. In *2019 IEEE International Symposium on Measurement and Control in Robotics (ISMCR)*, pages D1–2–1–D1–2–6, September 2019.
- [VKS14] Math Verstraelen, Jan Kuper, and Gerard J. M. Smit. Declaratively programmable ultra-low latency audio effects processing on FPGA. In *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx-14)*, pages 263–270. Fraunhofer Institut, September 2014.
- [VZT22] Luca Vignati, Stefano Zambon, and Luca Turchet. A comparison of real-time linux-based architectures for embedded musical applications. *Journal of the Audio Engineering Society*, 70(1/2):83–93, 2022.
- [Wan11] Yonghao Wang. Latency measurements of audio sigma delta analog to digital and digital to analog converts. In *Audio Engineering Society Convention 131*. Audio Engineering Society, 2011.
- [Wan18] Yonghao Wang. *Low Latency Audio Processing*. Thesis, Queen Mary University of London, September 2018.
- [WB15] Craig J Webb and Stefan Bilbao. On the limits of real-time physical modelling synthesis with a modular environment. In *Proceedings of the International Conference on Digital Audio Effects*, volume 65, 2015.
- [WC03] Ge Wang and Perry R Cook. *Chuck: A Concurrent, On-the-fly, Audio Programming Language*. 2003.
- [WLK14] Thomas Webster, Guillaume LeNost, and Martin Klang. The OWL programmable stage effects pedal: Revising the concept of the on-stage computer for live music performance. In *NIME*, pages 621–624, 2014.
- [WSN22] Clemens Wegener, Sebastian Stang, and Max Neupert. Fpga-accelerated real-time audio in pure data. In *Proc. Int. Conf. in Sound and Music Computing, SMC-22*, 2022.
- [WSR10] Yonghao Wang, Ryan Stables, and Joshua Reiss. Audio latency measurement for desktop operating systems with onboard soundcards. In *Audio Engineering Society Convention 128*. Audio Engineering Society, 2010.
- [WW02] David Wessel and Matthew Wright. Problems and Prospects for Intimate Musical Control of Computers. *Computer Music Journal*, 26(3):11–22, 2002.
- [Xila] Xilinx. Vitis HLS. <https://www.xilinx.com/products/design-tools/vitis/vitis-hls.html>.

- [Xilb] Xilinx. Vitis HLS Documentation: Automatic Loop Pipelining (2022.2). <https://docs.amd.com/r/2022.2-English/ug1399-vitis-hls/Automatic-Loop-Pipelining>.
- [Xilc] Xilinx. Vitis HLS Documentation: AXI Interface. https://docs.amd.com/v/u/en-US/ug761_axi_reference_guide.
- [Xild] Xilinx. Vitis HLS Documentation: Interfaces. <https://docs.amd.com/r/2022.2-English/ug1399-vitis-hls/Interfaces-for-Vivado-IP-Flow>.
- [Xile] Xilinx. Vitis HLS Documentation: Port Level Protocols. <https://docs.amd.com/r/en-US/ug1399-vitis-hls/Port-Level-Protocols-for-Vivado-IP-Flow>.
- [Xilf] Xilinx. Vitis HLS Documentation: Target Flow. <https://docs.amd.com/r/en-US/ug1399-vitis-hls/Default-Settings-of-Vivado/Vitis-Flows>.
- [Xilg] Xilinx. Vitis Model Composer. <https://www.xilinx.com/products/design-tools/vitis/vitis-model-composer.html>.
- [ZCDC15] Zhiru Zhang, Deming Chen, Steve Dai, and Keith Campbell. High-level Synthesis for Low-power Design. IPSJ Transactions on System and LSI Design Methodology, 8:12–25, 2015.
- [ZHA⁺13] M. Sami Zitouni, M. Luai Hammadih, Abdulla AlShehhi, Saif AlKindi, Nazar Ali, and Luis Weruaga. Simulated analysis and hardware implementation of voiceband circular microphone array. In 2013 IEEE 20th International Conference on Electronics, Circuits, and Systems (ICECS), pages 508–511, December 2013.
- [ZRV⁺19] Yaqi Zhang, Alexander Rucker, Matthew Vilim, Raghu Prabhakar, William Hwang, and Kunle Olukotun. Scalable interconnects for reconfigurable spatial architectures. In Proceedings of the 46th International Symposium on Computer Architecture, ISCA '19, pages 615–628, New York, NY, USA, June 2019. Association for Computing Machinery.
- [ZRZ⁺17] Xiaofan Zhang, Anand Ramachandran, Chuanhao Zhuge, Di He, Wei Zuo, Zuofu Cheng, Kyle Rupnow, and Deming Chen. Machine learning on FPGAs to face the IoT revolution. In 2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pages 894–901, November 2017.
- [ZWZ14] Jing Bo Zhang, Xiao Feng Wang, and Shu Fang Zhang. Audio Signal Processing Based on FPGA. Advanced Materials Research, 1049–1050:1759–1764, 2014.
- [ZZC] Hongrun Zhou, Jiuling Zhou, and Michelle Chang. conFFT: An FPGA Music Synthesizer.

Glossary

audio codec In this thesis, “audio codec” always refer to a hardware component providing analog outputs and inputs (audio ADC/-DAC): not an audio compression algorithm. 3, 4, 14, 26, 27, 28, 31, 33, 38, 53, 54, 56, 57, 59, 60, 61, 62, 63, 68, 71, 73, 133, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 159, 165, 187, 188, 225

CCRMA CCRMA (Center for Computer Research in Music and Acoustics) at Stanford University is a leading research center focused on advancing music, audio, and acoustics technologies through interdisciplinary study. It provides state-of-the-art facilities for research, education, and collaboration in these fields. 27, 166

Digital Signal Processor Digital Signal Processing and Digital Signal Processor share the same acronym. To avoid confusion, in this thesis, a Digital Signal Processor is always referred to by its full name (without using the acronym), while Digital Signal Processing is referred to as DSP algorithms or simply DSP. 26, 27, 29, 30, 57, 142, 143, 144

DSP Block DSP blocks (or slices) represent a third meaning of the acronym DSP. To avoid confusion, these will be referred to as DSP blocks throughout this thesis, except in resource usage tables and figures, where the context should be sufficient for clarity. 77, 78, 80, 90, 108, 128, 140, 141, 155, 157, 224

Emeraude Team The Emeraude Team, where I conducted this thesis, is a collaboration between CITI laboratory (INSA-Lyon, Inria team) and GRAME-CNCM. It specializes in the development of new programming tools and signal processing techniques for embedded audio systems. We explore various forms of embedded systems such as small microcontrollers, DSPs, GPUs, Linux-based embedded systems, etc., with a special focus on FPGAs. 20, 74, 128, 158, 165, 167

Grame-CNCM Grame is a “Centre National de Création Musicale” (CNCM) organized in three departments: music production, transmission/mediation, and computer music research. Grame’s research department has been leading the development of the Faust programming language since its creation in 2004. 19, 20

HLS High-Level Synthesis (HLS) is a design methodology that allows developers to write hardware descriptions using software languages like C or C++, which are then automatically translated into RTL (Register Transfer Level) for FPGA implementation. 2, 3, 4, 31, 32, 36, 41, 42, 43, 66, 67, 68, 70, 72, 73, 74, 76, 79, 80, 81, 82, 83, 84, 85, 86, 89, 90, 91, 93, 95, 96, 99, 101, 102, 104, 105, 107, 112, 114, 117, 118, 122, 123, 124, 127, 129, 130, 132, 137, 155, 157, 165, 173, 175, 187

IP An IP (Intellectual Property) core is a pre-designed, reusable module that implements specific functions, such as signal processing, communication protocols, or memory controllers, to accelerate development. These IP cores can be provided by FPGA vendors or third parties and are integrated into FPGA designs to reduce development time and improve performance. 3, 4, 40, 43, 44, 45, 50, 56, 62, 66, 67, 68, 71, 72, 73, 77, 78, 79, 80, 82, 89, 96, 97, 98, 99, 100, 101, 109, 111, 112, 113, 114, 116, 117, 122, 123, 124, 129, 130, 132, 134, 135, 137, 138, 139, 140, 141, 142, 144, 146, 155, 167, 184, 185, 193, 224

LLVM LLVM is a collection of modular and reusable compiler and toolchain technologies designed to optimize and transform code across various programming languages. It provides a powerful framework for building compilers, runtime environments, and other software development tools. 25

pragma In the context of FPGA and High-Level Synthesis, a pragma is a directive used within the programming language (i.e., C/C++ code) to guide the synthesis tool on optimization strategies, such as loop unrolling, pipelining, or resource allocation. It takes the form of a line of code and helps control hardware generation without altering the functional behavior of the code, allowing for fine-tuned performance and area trade-offs. 2, 3, 41

TCL Tcl (Tool Command Language) is a high-level scripting language used for automation, configuration, and rapid prototyping. Its simplicity and flexibility make it suitable for various applications and system management tasks. 70, 72, 86

Acronyms

ADAT Alesis Digital Audio Tape. 34

ADC Analog-to-Digital Converter. 2, 3, 6, 7, 8, 10, 12, 14, 26, 27, 33, 35, 38, 39, 53, 54, 57, 58, 59, 60, 61, 63, 142, 143, 144, 145, 146, 147, 188, 223

ALSA Advanced Linux Sound Architecture. 12, 32

AMBA Advanced Microcontroller Bus Architecture. 40, 45, 65, 66, 67, 68, 71, 122, 124, 135

ANC Active Noise Control. 28, 34, 35, 159, 160, 179

ASIC Application-Specific Integrated Circuit. 1, 26, 29, 30, 31

AXI Advanced eXtensible Interface. 43, 65, 67, 68, 80, 97, 99, 100, 122, 123, 137, 185, 193, 224

BRAM Block Random-Access Memory. 50, 51, 52, 68, 77, 78, 80, 81, 83, 90, 95, 97, 105, 107, 108, 118, 119, 120, 121, 126, 135, 137, 139, 140, 155, 162, 163, 169, 170, 173, 178, 184, 185, 189, 190

CLB Configurable Logic Block. 77, 78

CLI Command Line Interface. 64, 69

CNN Convolutional Neural Network. 31, 32

CPU Central Processing Unit. 2, 3, 11, 12, 18, 19, 21, 26, 27, 28, 29, 30, 32, 33, 34, 35, 36, 38, 39, 40, 44, 45, 46, 47, 48, 49, 62, 63, 64, 65, 66, 67, 68, 71, 72, 76, 101, 121, 122, 123, 124, 125, 133, 135, 136, 137, 138, 139, 140, 147, 166, 167, 185, 193, 223

DAC Digital-to-Analog Converter. 2, 3, 6, 7, 8, 9, 12, 27, 31, 33, 35, 38, 53, 54, 57, 58, 59, 60, 61, 65, 142, 143, 144, 145, 146, 147, 188, 223

DAW Digital Audio Workstations. 12, 26

DNL differential nonlinearity. 59

DSL Domain-Specific Language. 2, 7, 19, 20, 21, 24, 25, 31, 32, 35, 43, 67, 69, 132, 165

DSP Digital Signal Processing. 1, 2, 3, 4, 6, 7, 13, 17, 18, 19, 20, 21, 24, 27, 28, 29, 30, 32, 33, 35, 38, 40, 41, 43, 44, 47, 49, 53, 57, 62, 63, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 76, 78, 80, 86, 100, 117, 121, 122, 124, 128, 130, 132, 133, 134, 135, 136, 138, 139, 143, 144, 145, 150, 154, 158, 162, 165, 166, 223

eDSP embedded Digital Signal Processor. 26

FF Flip-Flop. 77, 78, 80, 155

FFT Fast Fourier Transform. 28, 29, 41

FIR Finite Impulse Response. 19, 28, 30, 76, 79, 80, 86, 87, 89, 95, 100, 101, 103, 104, 105, 106, 108, 109, 114, 115, 116, 117, 119, 120, 121, 125, 126, 155, 156, 158, 161, 163, 173, 174, 179, 180, 187, 223, 224

FPGA Field-Programmable Gate Array. 1, 2, 3, 4, 6, 12, 14, 20, 21, 24, 26, 27, 29, 30, 31, 32, 33, 35, 36, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 56, 61, 62, 63, 65, 66, 67, 69, 70, 71, 72, 73, 74, 76, 77, 78, 79, 80, 82, 100, 101, 102, 103, 109, 111, 116, 121, 122, 123, 124, 125, 126, 128, 129, 132, 133, 135, 137, 138, 139, 140, 141, 142, 144, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 158, 159, 162, 163, 165, 166, 167, 168, 185, 223, 224

FxLMS Filtered-x LMS. 160, 162, 179, 181, 182, 183, 184, 195, 201, 224

GPIO General-Purpose Input/Output. 35, 56, 57, 63, 68, 72, 124, 148, 149, 150, 152, 153

GPOS General Purpose Operating System. 27, 62

GPP General-Purpose Processor. 52

- GPU** Graphics Processing Unit. 19, 26, 28, 30, 34, 35, 38, 76, 163
- GUI** Graphical User Interface. 20, 21, 64, 65, 69, 70, 72, 138, 139, 223
- HDL** Hardware Description Language. 2, 31, 32, 33, 39, 40, 41, 42, 43, 67, 68, 73, 76, 220
- HLT** High Level Tools. 41, 42
- I²C** Inter-Integrated Circuit. 40, 57, 63, 67, 71, 72, 142, 146, 147, 152, 153, 224
- I²S** Inter-IC Sound. 3, 4, 33, 53, 54, 55, 56, 57, 60, 68, 71, 123, 132, 137, 141, 142, 143, 144, 145, 146, 148, 149, 150, 153, 154, 165, 188, 193, 223, 224, 225
- IC** Integrated Circuit. 220, 221
- IDE** Integrated Development Environment. 20, 27, 134
- IIR** Infinite Impulse Response. 28, 30, 100, 162, 163
- LMS** Least Mean Squares. 35, 160, 161, 162, 179, 180, 181, 182, 183, 184, 185, 201, 224
- LUT** look-up table. 50, 51, 52, 56, 77, 78, 80, 81, 83, 105, 107, 108, 118, 119, 120, 121, 126, 128, 139, 140, 141, 155, 156, 157, 162, 163, 169, 170, 180, 181, 182, 184, 189, 190, 224
- MAC** Multiply-Accumulate. 26, 104
- MADI** Multichannel Audio Digital Interface. 34, 35
- MBD** Model-Based Design. 41, 42
- MIDI** Musical Instrument Digital Interface. 18, 62, 66
- MoC** Model of Computation. 7
- MSB** Most Significant Bit. 54
- OS** Operating System. 11, 12, 27, 32, 33, 38, 62, 66, 67
- OSC** Open Sound Control. 18, 62, 66
- PCB** Printed Circuit Board. 3, 132
- PCM** Pulse-Code Modulation. 53, 57
- PLL** Phase-Locked Loop. 14, 26, 77
- PMOD** Peripheral Module interface. 148, 149
- RAM** Random-Access Memory. 40, 50, 51, 52, 93, 95, 124, 135, 139, 169
- RTL** Register-Transfer Level. 41
- SAR** Successive Approximation Register. 58, 60, 61
- SDF** Synchronous Dataflow. 7
- SNR** Signal-to-Noise Ratio. 9, 144, 145, 146, 188
- SoC** System on Chip. 2, 3, 26, 29, 30, 32, 33, 34, 38, 39, 40, 53, 65, 66, 71, 72, 73, 76, 124, 133, 137, 146, 154, 155, 156, 158, 163, 184, 223, 224, 225
- SPI** Serial Peripheral Interface. 40
- SRAM** Static Random-Access Memory. 51, 77
- TDM** Time Division Multiplexing. 55, 56, 57, 68, 148, 149, 153
- TUI** Terminal User Interface. 64, 69, 71, 72, 185, 223
- UART** Universal Asynchronous Receiver Transmitter. 40, 63, 64, 67, 69, 71, 72, 139
- VHDL** Very-High-Speed Integrated Circuit Hardware Description Language. 31, 40, 123, 141, 142

VHSIC Very-High-Speed Integrated Circuit. 220

WFS Wave Field Synthesis. 1, 28, 30, 34, 35, 124, 148, 158, 159, 224

List of Figures

2.1	Basic audio DSP functional overview.	7
2.2	Sampling principle (ADC).	8
2.3	Reconstruction principle (DAC)	9
2.4	Quantization error difference for 2 and 3 bit depth signal.	9
2.5	Effect of sampling frequency.	10
2.6	Effect of aliasing.	11
2.7	Difference of digital signal resolution (time and amplitude).	11
2.8	Sample processing: Latency < 1 Period; Throughput = 1 Sample/Period (real-time).	13
2.9	Sample processing: Latency > 1 Period; Throughput < 1 Sample/Period (not real-time).	13
2.10	Sample processing parallelized: Latency > 1 Period; Throughput = 1 Sample/Period (real-time).	13
2.11	Clock Jitter	14
2.12	Software Jitter	14
2.13	Control rate and sample rate.	17
2.14	Auto-generated interfaces by the compiler.	21
4.1	Zynq7000 SoC architecture.	39
4.2	Example of an FPGA accelerated function on a CPU execution.	44
4.3	CPU-centric architecture.	45
4.4	Example of FPGA offloading function on a CPU.	46
4.5	FPGA-centric architecture.	46
4.6	Computations dispatching guideline.	49
4.7	Memory dispatching guideline.	52
4.8	I ² S (standard) timing diagram.	54
4.9	Left Justified timing diagram.	55
4.10	Right Justified timing diagram.	55
4.11	8-Channel TDM (TDM8) timing diagram.	56
4.12	SSM2604 audio codec diagram.	58
4.13	Resolution and bandwidth characteristics of different ADC architectures.	58
4.14	Sigma-delta ADC Architecture.	60
4.15	Sigma-delta DAC Architecture.	60
4.16	Custom controller board PCB.	63
4.17	Custom controller board with different combinations.	64
4.18	CPU bare-metal control implementation.	65
4.19	Audio to FPGA compiler global view.	67
4.20	Software (audioCtl.cpp), hardware (audioDSP.cpp) and block design (BlockDesign.tcl) file content.	69
4.21	Example of an automatically generated GUI with FAUST.	70
4.22	Syfal FAUST to FPGA compiler complete architecture.	71
4.23	Example of our TUI running on the SoC.	72
5.1	Example of synthesis summary report form Vitis HLS.	79
5.2	Utilization estimates for the syfala function.	81
5.3	Instance detail for the syfala function.	81
5.4	Utilization estimates and instance detail for the computemydsp function.	81
5.5	Utilization estimates and instance detail for the module inside the compute_fir function.	82
5.6	Memory detail for the module inside the compute_fir function.	82
5.7	Expression, multiplexer and register detail for the module inside the compute_fir function.	82
5.8	Function Call Graph.	83
5.9	Schedule Viewer of computemydsp.	84
5.10	Schedule Viewer of generate_input_samples.	84
5.11	Simplified fully sequential implementation of an FIR filter.	87
5.12	Simplified fully parallel implementation of an FIR filter.	87
5.13	Theoretical trade-off between resource usage and latency in hardware implementations	88
5.14	Pipeline principle.	88
5.15	Schedule viewer for basic_unroll.cpp with no pragmas or configuration commands.	90
5.16	Resource usage and latency of basic_unroll.cpp (no pipeline) for varying unroll factors.	91
5.17	Resources usage per expressions for an unroll factor of 15 and 16.	92
5.18	Schedule viewer for the program basic_unroll.cpp with no pipeline and an unroll factor of 5.	93
5.19	Resource usage and latency of basic_unroll.cpp (no pipeline) with a complete partitioning of A for varying unroll factors.	94
5.20	Schedule viewer for the program basic_unroll.cpp with a complete partitioning of A and a full unroll.	94

5.21	Resource usage and latency of <code>basic_unroll.cpp</code> pipelined, for varying unroll factors.	96
5.22	IP synthesized from <code>basic_unroll.cpp</code> with no pragmas.	97
5.23	IP synthesized from <code>basic_unroll.cpp</code> with pointers (no pragmas).	98
5.24	IP synthesized from <code>basic_unroll.cpp</code> with AXI-Lite interface and handshake protocol on A.	100
5.25	Latency, FF and LUT estimation usage for different nested loop implementations.	108
5.26	One-sample strategy real-time compatible.	109
5.27	One-sample strategy not real-time compatible.	109
5.28	Unrolling loops.	110
5.29	Multi-sample: simple buffering implementation.	112
5.30	Latency and resource usage for <code>fir_multisample.cpp</code> for 115 coefficients with varying number of buffered samples and complete partitioning of <code>fTemp</code>	112
5.31	Latency and resource usage for <code>fir_multisample.cpp</code> for 115 coefficients with a varying number of buffered samples on a fully parallelized implementation forcing <code>II=1</code>	113
5.32	Multi-sample: Optimized buffer size.	114
5.33	Classic linear buffer operation for one sample.	115
5.34	Global shift of the linear buffer for <code>BLOCK_NSAMPLES=4</code> samples.	115
5.35	Example of <code>dataflow</code> pragma behavior.	116
5.36	Example of overlapping IP execution mode.	117
5.37	Resource usage and latency results for different implementations of an FIR filter in a multisample approach.	121
5.38	Estimated vs. measured latency of the IP for different values of the <i>memory latency pragma</i>	124
5.39	Excerpt of a program showing two loops that can be merged.	127
5.40	Excerpt of a program (LMS) showing the offloading of computations from a loop.	127
5.41	Effect of lower sample rate.	129
6.1	FAUST to FPGA compilation example.	135
6.2	Hardware (knobs) and software (GTK app) interfaces for the audio IP.	138
6.3	Custom controller board stacked on a <i>Zybo Z7</i> presenting a possible combination of controllers.	138
6.4	Percentage of <i>Zybo</i> FPGA LUTs and DSP Blocks usage for various sizes of the <code>bell.dsp</code> FAUST program.	141
6.5	Latency measurements testbench.	144
6.6	Latency report for different audio codecs.	145
6.7	Oscilloscope visualization of the <i>SSM2603</i> bug.	146
6.8	Frugal Board (white) with 32 wired speakers, connected to a <i>Zybo Z7-20</i>	149
6.9	Two Frugal Boards side by side with the two different integration.	149
6.10	Frugal Board I ² S Wiring	150
6.11	32 Channels Ultra Low Latency FPGA interface stacked on the <i>Genesys ZU-3EG</i>	151
6.12	Custom <i>ADAUI787</i> breakout board.	152
6.13	Ultra Low Latency Board I ² C and I ² S Wiring.	153
6.14	Maximum number of FIR filter coefficients implementable on different SoCs in a one-sample processing strategy.	156
6.15	Pragma used for better performances with linear buffer.	157
6.16	Maximum number of FIR filter coefficients implementable on different SoCs in a multi-sample processing strategy.	158
6.17	Prototype WFS system using the Frugal Board.	159
6.18	Experimental setup inside the École Centrale listening room.	160
6.19	Resource usage and latency for a single-channel LMS filter algorithm with 512 coefficients.	161
6.20	Resource usage and latency for a 6 channels LMS filter algorithm with 512 coefficients.	161
A.1	Schedule viewer for <code>basic_unroll.cpp</code> with HLS <code>unroll</code> pragma and no configuration commands.	170
A.2	Schedule viewer for <code>basic_unroll.cpp</code> with HLS <code>unroll</code> pragma and <code>-umo</code> configuration commands.	171
B.1	Latency and resource usage for <code>fir_onesample.cpp</code> with varying pragma unroll factors.	174
B.2	Latency and resource usage for <code>fir_onesample.cpp</code> with varying pragma unroll factors and pragma pipeline <code>II=1</code> applied to <code>compute_fir</code>	175
B.3	Latency and resource usage for <code>fir_onesample.cpp</code> with varying pragma unroll factors and pragma pipeline <code>II=1</code> applied to <code>compute_fir</code> and <code>linear_buffer</code>	176
B.4	Latency and resource usage for <code>fir_onesample.cpp</code> with varying pragma unroll factors and pragma <code>array_partition</code> applied to <code>samples</code>	177
B.5	Resource usage and latency results for different implementations of an FIR filter in a one-sample approach.	178
C.1	Implemented LMS algorithm.	179
C.2	Resource usage and latency for different implementation of a LMS algorithm with 512 coefficients in a one-sample processing strategy (<i>Zybo Z20</i>).	181
C.3	Excerpt of a program showing that interchanging loops allows them to be merged.	182
C.4	Resource usage and latency for different implementation of 6 concurrent LMS algorithm with 512 coefficients in a one-sample processing strategy (<i>Zybo Z20</i>).	182
C.5	Implemented FxLMS algorithm.	183
C.6	Resource usage and latency for different implementation of a 4x14 FxLMS algorithm with 512 coefficients in a one-sample processing strategy.	184
E.1	<i>Syfala</i> Compilation Report.	192
E.2	Graphical view of the <i>Syfala</i> block design.	193

List of Tables

2.1	Proposed latency designations for interactive audio system.	17
4.1	Size and pins usage for unidirectional standard I ² S and TDM8 implementation.	56
4.2	Overview of audio codecs capabilities with minimum measured latency analog to analog.	61
5.1	Difference in <i>Vitis</i> HLS (synthesis and implementation) and <i>Vivado</i> resource usage estimation.	83
5.2	Latency and trip count report of an unroll factor of 40 for the program <code>basic_unroll.cpp</code>	92
5.3	Default paradigms and interfaces inferred by <i>Vitis</i> HLS.	97
5.4	Resource usage for <code>fir_multiinstances.cpp</code> with pipelined inner loop.	105
5.5	Resource usage for <code>fir_multiinstances.cpp</code> with pipelined outer loop.	105
5.6	Resource usage for <code>fir_multiinstances.cpp</code> with interchanged loops.	107
5.7	Resource usage for <code>fir_multiinstances.cpp</code> with flattened loops.	107
5.8	Resource usage for <code>fir_multisample_base.cpp</code> with no pragmas.	118
5.9	Resource usage for <code>fir_multisample_subloops.cpp</code> with no pragmas.	119
5.10	Resource usage for <code>fir_multisample.cpp</code> with no pragmas.	119
5.11	Resource usage for <code>fir_multisample.cpp</code> with pragma pipeline in the outer loop of <code>compute_fir</code> function.	119
5.12	Resource usage for <code>fir_multisample.cpp</code> with pragma pipeline <code>II=1</code> in the outer loop of <code>compute_fir</code> function.	119
5.13	Resource usage for <code>fir_multisample.cpp</code> with different pipeline implementations in the outer loop of the <code>compute_fir</code> function and a complete partitioning of <code>fTemp</code>	120
5.14	Resource usage for <code>fir_multisample.cpp</code> with pragma <code>loop_flatten</code> in <code>compute_fir</code>	120
5.15	Resource usage for <code>fir_onesample.cpp</code> and <code>fir_onesample_circularbuffer.cpp</code> for 115 coefficients.	126
5.16	Resource usage for <code>fir_onesample.cpp</code> and <code>fir_onesample_circularbuffer.cpp</code> for 128 coefficients.	126
6.1	Memory storage vs. Block Ram storage for various FAUST programs.	140
6.2	Resources and Prices for Different SoCs on Available Development Boards.	155
6.3	Resource usage estimation for a 14-microphone, 14-speaker FxLMS algorithm.	162
6.4	Modal synthesis latency and resource utilization of optimized HLS implementation.	163
A.1	Resource usage for <code>basic_unroll.cpp</code> with HLS <code>unroll</code> pragma and no configuration commands.	169
A.2	Resource usage for <code>basic_unroll.cpp</code> with HLS <code>unroll</code> pragma and <code>-umo</code> configuration commands.	170
B.1	Resource usage for <code>fir_onesample.cpp</code> with no pragmas (automatic pipeline of the loops).	173
B.2	Latency report for <code>fir_onesample.cpp</code> for an unroll factor of 10 with no pragma pipeline applied (inferred pipeline in all loops).	175
B.3	Latency report for <code>fir_onesample.cpp</code> for an unroll factor of 10 with a pragma pipeline <code>II=1</code> applied in the <code>compute_fir</code> function.	175
B.4	Latency report for <code>fir_onesample.cpp</code> for an unroll factor of 10 with a pragma pipeline <code>II=1</code> applied in both the <code>compute_fir</code> and <code>linear_buffer</code> functions.	176
B.5	Resource usage for <code>fir_onesample.cpp</code> with pipeline and unroll disabled.	178
C.1	Resources and latency report of <code>lms_mono.cpp</code> with no pragmas.	180
C.2	Resources and latency report of <code>lms_mono.cpp</code> with pipeline <code>II=1</code> in <code>y_hat</code> loop.	181
C.3	Resources and latency report of <code>lms_multichannel.cpp</code> with pipeline <code>II=1</code> in <code>y_hat_and_norm</code> and <code>c_hat</code> loops.	182
C.4	Excerpt of resources and latency estimation of <code>fxlms_preopti.cpp</code> with pipeline <code>II=1</code> in loops only (no unroll).	184
D.1	Latency report for different audio codecs.	188
D.2	FIR filter latency and resource utilization for HLS synthesis and implementation using a one-sample approach.	189
D.3	FIR filter latency and resource utilization for HLS synthesis and implementation using a multi-sample approach.	190

FOLIO ADMINISTRATIF
THESE DE L'INSA LYON, MEMBRE DE L'UNIVERSITE DE LYON

NOM : **POPOFF**

DATE de SOUTENANCE : **13/12/2024**

Prénoms : **Maxime, Louis**

TITRE : **Compilation of Real-Time Audio DSP on FPGA**

NATURE : **Doctorat**

Numéro d'ordre : **2024ISAL0128**

École Doctorale : **Informatique et Mathématiques**

Spécialité : **Informatique**

RÉSUMÉ :

Les FPGA (Field-Programmable Gate Arrays) sont des plateformes programmables offrant des performances uniques en termes de débit, de nombre d'entrées/sorties et de puissance de calcul, les rendant particulièrement intéressantes pour des applications de traitement du signal audio numérique en temps réel. Cependant, leur programmation constitue un défi majeur. La synthèse de haut niveau (HLS) permet de s'affranchir des langages de description matérielle complexes traditionnellement utilisés pour la programmation des FPGA. Cela ouvre la voie à une véritable chaîne de compilation allant des spécifications DSP audio de haut niveau jusqu'au design d'architectures matérielles. Toutefois, cette approche reste difficile à généraliser pour des programmes audio variés. Cette thèse étudie la compilation de DSP audio en temps réel pour FPGA. Elle propose une méthodologie générale pour générer du code C++ optimisé pour la synthèse de haut niveau et introduit le premier « compilateur DSP audio » dédié aux FPGA. Cette chaîne de compilation permet d'utiliser FAUST, un langage de programmation fonctionnel de haut niveau, spécifiquement conçu pour la synthèse sonore et le traitement audio, afin de programmer les FPGA. Deux avancées technologiques majeures sont présentées : la réalisation d'une latence ultra-faible pour les DSP audio en temps réel (à l'échelle de la microseconde) et la capacité à déployer facilement et efficacement des systèmes multicanaux de grande capacité, ouvrant ainsi de nouvelles perspectives de recherche, notamment dans les domaines de la spatialisation audio et des applications de contrôle actif de l'acoustique.

MOTS-CLÉS : **FPGA, DSP, Audio, Temps-réel, HLS, Xilinx, Faust, C++, Basse latence, Multicanal, Compilation**

Laboratoire(s) de recherche : **CITI**

Directeur de thèse : **Tanguy RISSET**

Président du Jury : **Olivier SENTIEYS**

Composition du Jury :

Roselyne CHOTIN, Andrew McPHERSON, Thomas HÉLIE, Olivier SENTIEYS, Christine SOLNON, Tanguy RISSET, Romain MICHON

